ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ & ΗΛΕΚΤΡΟΝΙΚΩΝ ΜΗΧΑΝΙΚΩΝ

Διπλωματική Εργασία

Ανάπτυξη διαδικτυακού παίκτη με αλγορίθμους τεχνητής νοημοσύνης σε επιτραπέζιο παιχνίδι



Φοιτητής: Παναγιώτης Σταυρόπουλος
ΑΜ: 19387255

Επιβλέπων Καθηγητής:

Μετάφας Δημήτριος
Επίκουρος Καθηγητής

ΑΘΗΝΑ-ΑΙΓΑΛΕΩ, ΣΕΠΤΕΜΒΡΙΟΣ 2026

UNIVERSITY OF WEST ATTICA
FACULTY OF ENGINEERING
DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING

Diploma Thesis

Development of an artificial player using artificial intelligence algorithms



Student: Stavropoulos Panagiotis
Registration Number: 19387255

Supervisor:

Metafas Dimitrios
Assistant Professor

ATHENS-EGALEO, SEPTEMBER 2026

Η Διπλωματική Εργασία έγινε αποδεκτή και βαθμολογήθηκε από την εξής τριμελή εξεταστική επιτροπή:

Δημήτριος Μετάφας Επιβλέπων	Μαρία Ραγκούση	Χριστόφορος Κάχρης
DIMITRIOS METAFAS Digitally signed by DIMITRIOS METAFAS Date: 2026.10.08 15:14:04 +03'00'	ΜΑΡΙΑ ΡΑΓΚΟΥΣΗ Digitally signed by ΜΑΡΙΑ ΡΑΓΚΟΥΣΗ Date: 2026.10.08 15:08:20 +03'00'	ΧΡΙΣΤΟΦΟΡΟΣ ΚΑΧΡΗΣ Digitally signed by ΧΡΙΣΤΟΦΟΡΟΣ ΚΑΧΡΗΣ Date: 2026.10.08 16:14:05 +03'00'
Επίκ. Καθηγητής ΤΗΗΜ ΠΑΔΑ	Καθηγήτρια ΤΗΗΜ ΠΑΔΑ	Επίκ. Καθηγητής ΤΗΗΜ ΠΑΔΑ

Copyright © 2026 Με επιφύλαξη παντός δικαιώματος. All rights reserved.

ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ και ΠΑΝΑΓΙΩΤΗΣ ΣΤΑΥΡΟΠΟΥΛΟΣ, ΣΕΠΤΕΜΒΡΙΟΣ, 2026

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς το/τη συγγραφέα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν το/τη συγγραφέα του και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις θέσεις του/της επιβλέποντος/ουσας, της επιτροπής εξέτασης ή τις επίσημες θέσεις του Τμήματος και του Ιδρύματος.

ΔΗΛΩΣΗ ΣΥΓΓΡΑΦΕΑ ΔΙΠΛΩΜΑΤΙΚΗΣ ΕΡΓΑΣΙΑΣ

Ο κάτωθι υπογεγραμμένος Παναγιώτης Σταυρόπουλος του Χρήστου και της Σοφίας, με αριθμό μητρώου 19387255 φοιτητής του Πανεπιστημίου Δυτικής Αττικής, της ΠΟΛΥΤΕΧΝΙΚΗΣ Σχολής, του Τμήματος ΗΛΕΚΤΡΟΛΟΓΩΝ ΚΑΙ ΗΛΕΚΤΡΟΝΙΚΩΝ ΜΗΧΑΝΙΚΩΝ,

δηλώνω υπεύθυνα ότι:

«Είμαι συγγραφέας αυτής της διπλωματικής εργασίας και κάθε βοήθεια την οποία είχα για την προετοιμασία της είναι πλήρως αναγνωρισμένη και αναφέρεται στην εργασία. Επίσης, οι όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε ακριβώς, είτε παραφρασμένες, αναφέρονται στο σύνολό τους, με πλήρη αναφορά στους συγγραφείς, τον εκδοτικό οίκο ή το περιοδικό, συμπεριλαμβανομένων και των πηγών που ενδεχομένως χρησιμοποιήθηκαν από το διαδίκτυο. Επίσης, βεβαιώνω ότι αυτή η εργασία έχει συγγραφεί από μένα αποκλειστικά και αποτελεί προϊόν πνευματικής ιδιοκτησίας τόσο δικής μου, όσο και του Ιδρύματος.
Παράβαση της ανωτέρω ακαδημαϊκής μου ευθύνης αποτελεί ουσιώδη λόγο για την ανάκληση του διπλώματός μου.»

Ο Δηλών



Παναγιώτης Σταυρόπουλος

ΔΗΛΩΣΗ ΓΙΑ ΤΗ ΧΡΗΣΗ ΠΑΡΑΓΩΓΙΚΗΣ ΤΕΧΝΗΤΗΣ ΝΟΗΜΟΣΥΝΗΣ

Ο κάτωθι υπογεγραμμένος Παναγιώτης Σταυρόπουλος του Χρήστου και της Σοφίας, με αριθμό μητρώου 19387255 φοιτητής του Πανεπιστημίου Δυτικής Αττικής, της ΠΟΛΥΤΕΧΝΙΚΗΣ Σχολής, του Τμήματος ΗΛΕΚΤΡΟΛΟΓΩΝ ΚΑΙ ΗΛΕΚΤΡΟΝΙΚΩΝ ΜΗΧΑΝΙΚΩΝ,

δηλώνω υπεύθυνα ότι:

Για τη σύνταξη του κειμένου της Διπλωματικής Εργασίας μου,

☐ δεν χρησιμοποίησα λογισμικό παραγωγικής τεχνητής νοημοσύνης (Generative AI).

■ χρησιμοποίησα λογισμικά παραγωγικής τεχνητής νοημοσύνης (Generative AI), για βοηθητικές εργασίες κατά την εκπόνηση της Διπλωματικής Εργασίας μου, σύμφωνα με την Πολιτική Χρήσης Εργαλείων Τεχνητής Νοημοσύνης του Τμήματος. Η χρήση των εργαλείων αυτών περιγράφεται και αιτιολογείται στο ΠΑΡΑΡΤΗΜΑ Α.

Ο Δηλών



Παναγιώτης Σταυρόπουλος

Η παρούσα διπλωματική εργασία αφιερώνεται στους γονείς μου για τη διαρκή υποστήριξη και καθοδήγηση που μου πρόσφεραν όλα αυτά τα χρόνια, καθώς και στον αδελφό μου για τις πολύτιμες συμβουλές και τη βοήθειά του. Τέλος, την αφιερώνω και στους φίλους μου που πίστεψαν σε μένα και στάθηκαν δίπλα μου σε όλη τη διάρκεια των σπουδών μου.

ΕΥΧΑΡΙΣΤΙΕΣ

Με την ολοκλήρωση της παρούσας διπλωματικής εργασίας, θα ήθελα αρχικά να εκφράσω τις θερμές μου ευχαριστίες προς τον επιβλέποντα καθηγητή μου, κ. Δημήτριο Μετάφα, για την εμπιστοσύνη, την άψογη συνεργασία και την πολύτιμη επιστημονική καθοδήγηση που μου παρείχε καθ' όλη τη διάρκεια της έρευνας. Οι κατευθύνσεις και οι παρατηρήσεις του συνέβαλαν καθοριστικά στην επιτυχή ολοκλήρωση της εργασίας αυτής, η οποία ελπίζω να ανταποκρίνεται πλήρως στις προσδοκίες του.

Επιπλέον, οφείλω ένα μεγάλο ευχαριστώ στην οικογένειά μου για τη διαρκή στήριξη, την ενθάρρυνση και τις πολύτιμες συμβουλές τους, τόσο κατά τη διάρκεια των σπουδών μου όσο και σε κάθε πτυχή της ζωής μου.

Τέλος, ευχαριστώ θερμά τους φίλους μου, που στάθηκαν δίπλα μου με ειλικρινές ενδιαφέρον και υποστήριξη σε όλη αυτή την πορεία.

Περίληψη

Η γρήγορη ανάπτυξη της Τεχνητής Νοημοσύνης τα τελευταία χρόνια έχει βρει ευρεία χρήση στον χώρο των παιγνίων. Τα παιχνίδια αποτελούν παραδοσιακά ένα καλό πεδίο δοκιμών για να δοκιμάζουμε νέους αλγόριθμους. Αυτή η διπλωματική εργασία επικεντρώνεται στη μελέτη και τη δημιουργία έξυπνων πρακτόρων για το σύγχρονο επιτραπέζιο παιχνίδι Splendor. Το συγκεκριμένο παιχνίδι έχει ξεχωριστό ερευνητικό ενδιαφέρον. Συνδυάζει πολύπλοκους μηχανισμούς διαχείρισης πόρων και χρειάζεται αυστηρό σχεδιασμό για το μέλλον. Ταυτόχρονα περιέχει στοιχεία αβεβαιότητας λόγω της τυχαιότητας στο μοίρασμα των καρτών.

Κύριος στόχος της έρευνας ήταν να γίνει μαθηματική και προγραμματιστική μοντελοποίηση του παιγνίου και να δημιουργηθεί ένας έξυπνος πράκτορας, που μπορεί να παίρνει τις καλύτερες στρατηγικές αποφάσεις σε ένα δυναμικό περιβάλλον. Για να το πετύχουμε αυτό, φτιάξαμε από την αρχή έναν πλήρη και αυτόνομο προσομοιωτή του Splendor στη γλώσσα προγραμματισμού Python. Το σύστημα αυτό συνοδεύτηκε από μια Γραφική Διεπαφή (GUI) για να παρακολουθούμε οπτικά τις παρτίδες. Επίσης προσθέσαμε ένα λεπτομερές υποσύστημα καταγραφής (logger) για να βγάζουμε και να μελετάμε τα στατιστικά δεδομένα.

Στο κέντρο της μεθοδολογίας μας βρίσκεται η υλοποίηση του αλγορίθμου Monte Carlo Tree Search (MCTS). Ο αλγόριθμος αυτός είναι μια από τις πιο πρόσφατες και αποτελεσματικές προσεγγίσεις στην επίλυση παιγνίων. Για να μετρηθεί σωστά η απόδοση και η ευφυΐα του πράκτορα MCTS, φτιάχτηκε και ένας Ευρετικός πράκτορας (Heuristic Agent). Αυτός ο πράκτορας βασίζεται σε στατικούς, άπληστους κανόνες (greedy approach) και στοχεύει μόνο στο άμεσο, βραχυπρόθεσμο όφελος. Στο πειραματικό στάδιο της εργασίας, αυτοί οι δύο πράκτορες αναμετρήθηκαν σε ένα μεγάλο δείγμα 251 προσομοιωμένων παρτίδων. Αυτό επέτρεψε την ουσιαστική σύγκριση των στρατηγικών τους.

Τα αποτελέσματα της έρευνας έδειξαν ξεκάθαρα ότι ο αλγόριθμος MCTS υπερισχύει σε σχέση με την ευρετική προσέγγιση. Ο πράκτορας MCTS, επειδή εκτελούσε χιλιάδες τυχαίες προσομοιώσεις (rollouts) σε κάθε γύρο, κατάφερε να πετύχει πολύ υψηλό ποσοστό νικών. Η εξέταση των δεδομένων έδειξε ότι το βασικό του πλεονέκτημα ήταν η διορατικότητά του. Μπορούσε να αγνοεί προσωρινά οφέλη για να χτίσει μια ισχυρή «οικονομική μηχανή». Αυτό του έδινε το πάνω χέρι στα επόμενα στάδια του παιχνιδιού. Από την άλλη, ο Ευρετικός πράκτορας, αν και πολύ πιο γρήγορος στους υπολογισμούς, έκανε συχνά κοντόφθαλμες και μη αποδοτικές επιλογές.

Συνοψίζοντας, η εργασία δείχνει ότι ο αλγόριθμος MCTS είναι μια πολύ δυνατή και ευέλικτη λύση για δύσκολα παίγνια όπως το Splendor. Η πλατφόρμα που δημιουργήθηκε δείχνει τη χρησιμότητα της ευρετικής αναζήτησης σε δέντρα αποφάσεων. Επίσης, προσφέρει

μια σταθερή τεχνολογική βάση που μπορεί να επεκταθεί στο μέλλον. Αυτό επιτρέπει να δοκιμαστούν και άλλες τεχνικές Μηχανικής Μάθησης.

Λέξεις – κλειδιά

Τεχνητή Νοημοσύνη, Monte Carlo Tree Search, Θεωρία Παιγνίων, Ευρετικοί Αλγόριθμοι, Δέντρα Αναζήτησης, Προσομοίωση Παιγνίων, Έξυπνοι Πράκτορες, Επιτραπέζια Παίγνια, Splendor.

Abstract

The rapid evolution of Artificial Intelligence in recent years has found extensive application in the field of games, which traditionally serve as an ideal testing ground for evaluating new algorithms. This diploma thesis focuses on the study and development of intelligent agents for the modern board game Splendor. This particular game presents significant research interest, as it combines complex resource management mechanics and requires strict long-term planning, while simultaneously involving elements of uncertainty due to the randomness in card distribution.

The main objective of this research was the mathematical and programmatic modeling of the game and the creation of an intelligent agent capable of making optimal strategic decisions in a dynamic environment. To achieve this, a complete and autonomous simulator of Splendor was developed from scratch using the Python programming language. This system was complemented by a Graphical User Interface (GUI) for the visual monitoring of the matches, as well as an analytical logging subsystem for the extraction and evaluation of statistical data.

At the core of our methodology lies the implementation of the Monte Carlo Tree Search (MCTS) algorithm, which constitutes one of the most modern and efficient approaches to game solving. To accurately measure the performance and intelligence of the MCTS agent, an additional Heuristic Agent was developed. This agent operates based on static, greedy rules and aims exclusively at immediate, short-term gain. The experimental phase of the thesis involved matching these two agents in an extensive sample of 251 simulated games, allowing for an in-depth comparison of their strategies.

The results of the research demonstrated the absolute superiority of the MCTS algorithm over the heuristic approach. The MCTS agent, exploiting the thousands of random simulations (rollouts) it executed each turn, managed to achieve an overwhelming win rate. The analysis of the generated data showed that its primary advantage was its foresight. It possessed the ability to reject temporary benefits in order to build a strong "economic engine", which ensured its dominance in the later stages of the game. Conversely, the Heuristic agent, despite being computationally much faster, frequently fell into the trap of short-sighted and ineffective choices.

In conclusion, this thesis confirms that the MCTS algorithm is a highly powerful and adaptive solution for complex games like Splendor. The developed platform not only substantiates the value of heuristic search in decision trees, but also provides a solid technological foundation that can be extended in the future, paving the way for testing even more advanced Machine Learning techniques.

Keywords

Artificial Intelligence, Monte Carlo Tree Search, Game Theory, Heuristic Algorithms, Search Trees, Game Simulation, Intelligent Agents, Board Games, Splendor

Περιεχόμενα

Κατάλογος Πινάκων **15**

Κατάλογος Σχημάτων/Εικόνων **15**

Κατάλογος Συντομογραφιών **16**

Κατάλογος Συμβόλων / Παραμέτρων **17**

1. ΚΕΦΑΛΑΙΟ 1ο : ΕΙΣΑΓΩΓΗ **18**

1.1 Αντικείμενο της Διπλωματικής Εργασίας 18

1.2 Σκοπός και στόχοι της Διπλωματικής Εργασίας 18

1.3 Μεθοδολογία Διπλωματικής Εργασίας 19

1.4 Συνεισφορά και Καινοτομία της Διπλωματικής Εργασίας 20

1.5 Δομή της Διπλωματικής Εργασίας 21

2 ΚΕΦΑΛΑΙΟ 2ο : Επισκόπηση του πεδίου, προσδιορισμός του προβλήματος και διεθνής εμπειρία **22**

2.1 Επιτραπέζια παιχνίδια & Τεχνητή νοημοσύνη 22

2.2 Προσδιορισμός του προβλήματος της πολυπλοκότητας και διαχείρισης πόρων 23

2.3 Ευρετικοί αλγόριθμοι Heuristics 24

2.4 Αλγόριθμος Monte Carlo Search Tree (MCTS) 25

2.4.1 Η Σταθερά Εξερεύνησης UCB (c) 27

2.4.2 Αντιμετώπιση Στοχασμού (Determinization) 27

2.4.3 Διαδικασία Λήψης Αποφάσεων Markov (Markov Decision Process) 28

2.5 Σχετικά παραδείγματα MCTS & Heuristic-Based Agents 28

2.6 Το παιχνίδι Splendor 34

3 ΚΕΦΑΛΑΙΟ 3ο : Αρχιτεκτονική Συστήματος και Μεθοδολογία Ανάπτυξης **38**

3.1 Γενική Μεθοδολογική Προσέγγιση 38

3.2 Περιβάλλον Υλοποίησης και Εργαλεία 39

3.3 Αρχιτεκτονική Λογισμικού (Software Architecture) 40

3.4 Αναπαράσταση Κατάστασης (State Representation) 41

3.5 Χώρος Ενεργείων (Action Space) 42

3.6 Σχεδιασμός Πρακτόρων Τεχνητής Νοημοσύνης 43

3.6.1 Ευρετικός Πράκτορας (Basic Heuristic Agent) 43

3.6.2 Πράκτορας MCTS με Βάση Γνώσης (Advanced AI) 43

3.6.2.2 Ο Μαθηματικός Τύπος UCT (Upper Confidence bound applied to Trees) 44

3.6.2.3 Προσομοίωση και Αξιολόγηση (Engine Building Heuristic) 45

3.6.2.4 Αρχιτεκτονική Βάσης Γνώσης (Persistent Memory) 47

3.7 Συνθήκες Πειράματος και Παράμετροι Εκτέλεσης (Experimental Setup) 47

3.8 Σύστημα Καταγραφής (Logger) , Μετρικές και Οπτικοποίηση 48

4 ΚΕΦΑΛΑΙΟ 4ο : Τεχνικό Μέρος: Υλοποίηση Πρακτόρων ΤΝ και Τεχνική Ανάλυση Κώδικα **50**

4.1 Προγραμματιστική Δομή και Αλγοριθμική Υλοποίηση 50

4.1.1 Προγραμματιστική Δομή του GameState (Core Data Structures) 50

4.1.2 Αλγοριθμική Υλοποίηση Ευρετικού Πράκτορα (Heuristic Implementation) 51

4.2 Υλοποίηση Βασικών Μηχανισμών και Διεπαφής 53

4.2.1 Γραφική Διεπαφή Χρήστη (GUI) και Ζωντανή Καταγραφή 53

4.3 Αρχιτεκτονική Monte Carlo Tree Search (MCTS) 55

4.3.1 Κλάση MCTSNode και Μηχανισμός Αποκοπής (Pruning Mechanism) 55

4.3.2 Αλγοριθμική Ενσωμάτωση Προσαρμοστικού UCT (Adaptive UCT in Code) 56

4.3.3 Ευρετική Συνάρτηση Αξιολόγησης (Heuristic Target Evaluation) 57

4.3.4 Ανάλυση Υπερπαραμέτρων Αλγορίθμου 58
4.3.5 Μηχανισμοί Εκπαίδευσης και Βάση Γνώσης (Knowledge Base) 61
4.4 Μελέτη μέσω Προσομοίωσης: Ανάλυση Μεμονωμένης Παρτίδας 62
4.4.1 Παραγωγή Γραφήματος Στρατηγικής Αγοράς Καρτών (Card Strategy) 62
4.4.2 Παραγωγή Γραφήματος Εμπιστοσύνης MCTS (MCTS Confidence) 63
4.4.3 Γράφημα Αξιολόγησης Ποιότητας Κινήσεων (Move Accuracy) 64
4.4.4 Γράφημα Υπολογιστικής Έντασης (Thinking Effort) 66

5 ΚΕΦΑΛΑΙΟ 5ο: Πειραματικά Αποτελέσματα, Συμπεράσματα και Μελλοντικές Επεκτάσεις 67

5.1 Αρχιτεκτονική Εργαλείου Οπτικοποίησης Δεδομένων 67
5.2 Ποσοτική Αξιολόγηση Επιδόσεων 69
5.3 Διεξοδική Αιτιολόγηση Αποτελεσμάτων 70
5.4 Απαντήσεις στα Ερευνητικά Ερωτήματα 71
5.5 Περιορισμοί της Έρευνας 73
5.6 Μελλοντικές Κατευθύνσεις Έρευνας 74
5.7 Σύνοψη 75

Αναφορές – Πηγές 77

Παράρτημα Α: Χρήση εργαλείων παραγωγικής τεχνητής νοημοσύνης 78

Παράρτημα Β (Κώδικας) 79

Κατάλογος Πινάκων

Πίνακας 1- Βασικά Χαρακτηριστικά του Προβλήματος 28

Πίνακας 2- Σύμβολα και Ορολογία 32

Πίνακας 3- Βασικές Υπερπαράμετροι 52

Πίνακας 4- Υπερπαράμετροι και Συμβολισμοί MCTS 58

Πίνακας 5- Παράμετροι και Μεταβλητές Συστήματος 59

Κατάλογος Σχημάτων/Εικόνων

Εικόνα 1 [1] – Alan Turing, πρωτοπόρος της επιστήμης των υπολογιστών και της τεχνητής νοημοσύνης. 22

Εικόνα 2 [1]– Claude Shannon, του οποίου η εργασία έθεσε τις βάσεις για τον προγραμματισμό υπολογιστών σε παίγνια. 22

Εικόνα 3 [3] – Η ιστορική αναμέτρηση μεταξύ του Γκάρι Κασπάροφ (Garry Kasparov) και του υπερυπολογιστή Deep Blue της IBM (1997). 23

Εικόνα 4 [12] – Τα τέσσερα βασικά στάδια του αλγορίθμου Monte Carlo Tree Search (Επιλογή, Επέκταση, Προσομοίωση και Οπισθοδρόμηση). 26

Εικόνα 5 [12] – Γραφική αναπαράσταση ενός άδειου ταμπλό (Empty Board) για ένα υποθετικό παίγνιο δύο παικτών. 29

Εικόνα 6 [12] – Δέντρο αναζήτησης (Game Tree) για το παίγνιο δύο παικτών, όπου απεικονίζονται οι πρώτες πιθανές διακλαδώσεις. 29

Εικόνα 7 [12] – Το στάδιο της επιλογής (Selection Phase) στον αλγόριθμο MCTS. 30

Εικόνα 8 – Το στάδιο της οπισθοδρόμησης (Backpropagation) στον αλγόριθμο MCTS..... 30

Εικόνα 9 [12] – Η δεύτερη επανάληψη του σταδίου επιλογής (Second Selection Phase) στον MCTS. 31

Εικόνα 10 [12] – Η δεύτερη επανάληψη της οπισθοδρόμησης (Backpropagation) στον αλγόριθμο MCTS. 31

Εικόνα 11 [9] – Παραδοσιακή σκακιέρα (Chess Board)..... 32

Εικόνα 12 [7] – Το επιτραπέζιο παιχνίδι Splendor (2014), απεικόνιση του κουτιού και του βασικού θέματος. 34

Εικόνα 13 [7] – Συστατικά στοιχεία του επιτραπέζιου παιγνίου Splendor (Μάρκες Πολύτιμων Λίθων, Κάρτες Ανάπτυξης, Πλακίδια Ευγενών). 35

Εικόνα 14 [7] – Επιπρόσθετα στοιχεία (Κάρτες Ανάπτυξης διαφορετικών επιπέδων) του παιγνίου Splendor. 35

Εικόνα 15 [7] – Μηχανισμός Μόνιμου Μπόνους (Bonus Mechanism) στο Splendor, ως εκπτώσεις για μελλοντικές αγορές. 36

Εικόνα 16 [7] – Πλακίδια Ευγενών (Nobles) στο Splendor, τα οποία προσφέρουν επιπλέον Πόντους Κύρους. 37

Εικόνα 17 – Γραφική Διεπαφή Χρήστη (GUI) του ανεπτυγμένου προσομοιωτή. 53

Εικόνα 18 – Σύστημα Καταγραφής (Logger) του προσομοιωτή. 54

Εικόνα 19 – Γράφημα "Strategy DNA", όπου αναλύεται η κατανομή και το είδος των κινήσεων του πράκτορα. 63

Εικόνα 20 – Γράφημα "Battle of Confidence", το οποίο καταγράφει τη διακύμανση του προβαδίσματος του MCTS πράκτορα. 64

Εικόνα 21 – Γράφημα διασποράς (Strip Plot) Ποιότητας Κινήσεων (Move Quality) του πράκτορα MCTS. 65

Εικόνα 22 – Γράφημα Υπολογιστικού Φόρτου (Thinking Effort), το οποίο συσχετίζει τον χρόνο εκτέλεσης με την πολυπλοκότητα. 66

Εικόνα 23 – Συγκεντρωτικό γράφημα απόδοσης και εκμάθησης (Learning Dashboard) του MCTS. 68

Εικόνα 24 – Καμπύλη του ποσοστού νικών (Win Rate) με χρήση κινητού μέσου όρου (Rolling Average). 68

Εικόνα 25 – Γράφημα ρυθμού αύξησης της Βάσης Γνώσης (Knowledge Base Growth). 69

Εικόνα 26 – Γράφημα διασποράς Αποδοτικότητας (Game Efficiency - Total Turns Per Game). 69

Κατάλογος Συντομογραφιών

AI: Artificial Intelligence (Τεχνητή Νοημοσύνη - ΤΝ)

GUI: Graphical User Interface (Γραφική Διεπαφή Χρήστη)

JSON: JavaScript Object Notation (Μορφότυπο Ανταλλαγής Δεδομένων)

KB: Knowledge Base (Βάση Γνώσης)

MCTS: Monte Carlo Tree Search (Αναζήτηση Δέντρου Μόντε Κάρλο)

MDP: Markov Decision Process (Διαδικασία Λήψης Αποφάσεων Μάρκοβ)

MVC: Model-View-Controller (Πρότυπο Αρχιτεκτονικής Λογισμικού)

UCB: Upper Confidence Bound (Άνω Όριο Εμπιστοσύνης)

UCT: Upper Confidence Bound applied to Trees (Άνω Όριο Εμπιστοσύνης προσαρμοσμένο σε Δέντρα)

ΔΕ: Διπλωματική Εργασία

ΤΝ: Τεχνητή Νοημοσύνη

Κατάλογος Συμβόλων / Παραμέτρων

v_i: Η μέση εκτιμώμενη αξία (μέσο κέρδος ή win rate) του τρέχοντος κόμβου i στο δέντρο αναζήτησης.

n_i: Ο αριθμός των επισκέψεων (πόσες φορές διερευνήθηκε) ο συγκεκριμένος κόμβος i κατά τη φάση της προσομοίωσης.

N: Ο συνολικός αριθμός επισκέψεων (ή προσομοιώσεων) που έχουν πραγματοποιηθεί στον parent node.

C ή c: Η Exploration Parameter. Ρυθμίζει την ισορροπία μεταξύ της "εκμετάλλευσης" γνωστών καλών κινήσεων και της "εξερεύνησης" νέων αγνώστων (συχνά λαμβάνει την τιμή c ≈ 1.414 ή √2).

***h*(*n*):** Η Heuristic Function που επιστρέφει την εκτιμώμενη αξία μιας κατάστασης n.
num_simulations: Βασική παράμετρος του κώδικα που καθορίζει πόσες επαναλήψεις (επιλογές, επεκτάσεις, προσομοιώσεις) εκτελεί ο αλγόριθμος MCTS ανά κίνηση.

max_depth: Παράμετρος που ορίζει το μέγιστο επιτρεπτό βάθος στο οποίο επιτρέπεται να κατέβει το δέντρο κατά την αναζήτηση ή το playout.

win_rate: Στατιστική μετρική που αντικατοπτρίζει το ποσοστό νικών του πράκτορα επί του συνολικού αριθμού των παρτίδων.

1. ΚΕΦΑΛΑΙΟ 1ο : ΕΙΣΑΓΩΓΗ

1.1 Αντικείμενο της Διπλωματικής Εργασίας

Το βασικό θέμα αυτής της διπλωματικής εργασίας είναι η υλοποίηση και η αξιολόγηση του αλγορίθμου Τεχνητής Νοημοσύνης Monte Carlo Tree Search (MCTS) [4], με την ανάπτυξη ενός αυτόνομου υπολογιστικού πράκτορα. Ως πεδίο εφαρμογής επιλέχθηκε το επιτραπέζιο παιχνίδι στρατηγικής Splendor [7], [8]. Σε αυτό το παιχνίδι ο πράκτορας πρέπει να διαχειριστεί το πρόβλημα της αναβαλλόμενης ικανοποίησης (delayed gratification). Ο αλγόριθμος πρέπει να αφήσει το άμεσο βαθμολογικό όφελος, για να φτιάξει μια μακροπρόθεσμα χρήσιμη βάση συσσώρευσης πόρων

Σε επίπεδο ανάπτυξης, το αντικείμενο ξεπερνά την απλή προσομοίωση κώδικα (headless simulation). Δημιουργήθηκε ένα πλήρες λογισμικό σύστημα με διαδραστικό Γραφικό Περιβάλλον Χρήστη (GUI) για άμεση παρακολούθηση των αναμετρήσεων. Μέσα σε αυτό το σύστημα, ενσωματώθηκαν δύο υπολογιστικοί πράκτορες. Ο πρώτος είναι ο Basic Agent που λειτουργεί με ευρετικούς κανόνες (heuristics). Ο δεύτερος είναι ο MCTS Agent, που προσφέρει πιο εξελιγμένες δυνατότητες. Έτσι, διαμορφώνεται ένα αυστηρό πλαίσιο για συγκριτική αξιολόγηση.

1.2 Σκοπός και στόχοι της Διπλωματικής Εργασίας

Ο κεντρικός σκοπός της παρούσας έρευνας είναι η ανάδειξη και η ποσοτική τεκμηρίωση της αλγοριθμικής υπεροχής του Monte Carlo Tree Search σε παιγνίδια ανάπτυξης υποδομών έναντι παραδοσιακών ευρετικών μεθόδων. Απώτερος σκοπός είναι να καταδειχθεί πώς η ενσωμάτωση μηχανισμών συνεχούς εκμάθησης (persistent knowledge base) δύναται να βελτιστοποιήσει τη λήψη αποφάσεων του πράκτορα σε βάθος χρόνου. Προκειμένου να επιτευχθεί ο προαναφερθείς σκοπός, ορίστηκαν οι ακόλουθοι επιμέρους ερευνητικοί και αναπτυξιακοί στόχοι:

Μοντελοποίηση και Ανάπτυξη του Περιβάλλοντος Προσομοίωσης: Σχεδιασμός και υλοποίηση της μηχανής κανόνων (game engine) του παιγνίου Splendor στην Python, συνοδευόμενης από ένα διαδραστικό Γραφικό Περιβάλλον Χρήστη για την οπτική επαλήθευση των κινήσεων.

Υλοποίηση Δύο Υπολογιστικών Πρακτόρων: Ανάπτυξη δύο διακριτών πρακτόρων Τεχνητής Νοημοσύνης εξ ολοκλήρου στη γλώσσα προγραμματισμού Python. Υλοποιείται ένας πράκτορας βάσης με ευρετικούς κανόνες (**Basic/Heuristic Agent**) και ένας προηγμένος AI πράκτορας (**MCTS Agent**) ο οποίος διαθέτει την ικανότητα να μαθαίνει από παλαιότερα παιχνίδια.

Εξαγωγή και Ανάλυση Δεδομένων(Post-Game Analytics): Δημιουργία συστήματος καταγραφής και οπτικοποίησης δεικτών (ποσοστά επιτυχίας, στρατηγική ακρίβεια, διακυμάνσεις πιθανοτήτων) για τη στατιστική επαλήθευση της αλγοριθμικής απόδοσης.

Ενσωμάτωση Μηχανισμού Επίμονης Μάθησης (Persistent Knowledge Base): Ανάπτυξη υποσυστήματος που επιτρέπει στον MCTS πράκτορα να αποθηκεύει δεδομένα προηγούμενων αναμετρήσεων, βελτιστοποιώντας έτσι τις στρατηγικές του επιλογές σε μεταγενέστερα παίγνια.

Συστηματική καταγραφή και κριτική αποτίμηση των τελικών ευρημάτων σχετικά με την αποτελεσματικότητα της προτεινόμενης μεθόδου MCTS, εντοπισμός των τεχνικών δυσκολιών που προέκυψαν κατά την υλοποίηση, και διατύπωση στοχευμένων προτάσεων για τη μελλοντική εξέλιξη της έρευνας.

1.3 Μεθοδολογία Διπλωματικής Εργασίας

Για την άρτια και συστηματική εκπόνηση της παρούσας διπλωματικής εργασίας, υιοθετήθηκε μια αρθρωτή (modular) ερευνητική και αναπτυξιακή προσέγγιση, εναρμονισμένη με τις αρχές της Μηχανικής Λογισμικού (Software Engineering) και του Αντικειμενοστρεφούς Προγραμματισμού (OOP). Η μεθοδολογία συνίσταται στα ακόλουθα διακριτά στάδια:

Βιβλιογραφική Ανασκόπηση: Μελέτη της υφιστάμενης ακαδημαϊκής βιβλιογραφίας αναφορικά με τη Θεωρία Παιγνίων και τις σύγχρονες μεθόδους υπολογιστικής νοημοσύνης. Έμφαση δόθηκε στη μαθηματική θεμελίωση των δέντρων αναζήτησης και στο κριτήριο UCB1 (Upper Confidence Bound), το οποίο ρυθμίζει τη διαδικασία επιλογής στον αλγόριθμο MCTS [5], [6].

Σχεδιασμός Αρχιτεκτονικής Συστήματος : Εφαρμογή του προτύπου MVC (Model-View-Controller) για τον αυστηρό διαχωρισμό της δομής δεδομένων (Game State, Actions) και της επιχειρησιακής λογικής (Game Logic) από το υποσύστημα οπτικής αναπαράστασης (GUI).

Ανάπτυξη και Παραμετροποίηση Μοντέλων: Υλοποίηση του MCTS αλγορίθμου μέσα από την τυπική του δομή τεσσάρων φάσεων (Selection, Expansion, Simulation, Backpropagation). Παράλληλα, σχεδιάστηκαν οι ευρετικές συναρτήσεις που συνιστούν τον Basic Agent, ο οποίος λειτούργησε ως μοντέλο βάσης (baseline).

Διεξαγωγή Πειραμάτων και Εκπαίδευση :Αυτοματοποιημένη εκτέλεση μεγάλου πλήθους αναμετρήσεων (προσομοιώσεις) μεταξύ των πρακτόρων υπό ελεγχόμενες συνθήκες. Στο στάδιο αυτό, ο προηγμένος πράκτορας MCTS εκπαιδεύεται ενεργά μέσα από τα παρελθοντικά του παίγνια. Αποθηκεύοντας και ανακαλώντας τη συσσωρευμένη εμπειρία του, βελτιώνει συνεχώς τη στρατηγική του και παίζει αποδεδειγμένα καλύτερα σε μελλοντικές αναμετρήσεις..

Αξιολόγηση και Στατιστική Ανάλυση: Συλλογή των παραγόμενων δεδομένων (log files) και επεξεργασία τους μέσω εξειδικευμένων σεναρίων Python , προκειμένου να αξιολογηθεί η επίπτωση της χρήσης του MCTS και της Βάσης Γνώσης στη συνολική απόδοση.

Συγγραφή Τεχνικής Έκθεσης: Δόμηση και συγγραφή της παρούσας διπλωματικής εργασίας (αναλυτική τεχνική έκθεση), με στόχο την επιστημονική τεκμηρίωση του θεωρητικού υποβάθρου, της αρχιτεκτονικής του λογισμικού, της πειραματικής διαδικασίας και της κριτικής αποτίμησης των τελικών συμπερασμάτων.

Αξίζει να επισημανθεί ότι η παρούσα διπλωματική εργασία εντάσσεται σε ένα ευρύτερο ερευνητικό πλαίσιο του Τμήματος για τη μελέτη επιτραπέζιων παιγνίων στρατηγικής, όπου σε παράλληλη εργασία από τη συμφοιτήτρια **Κατερίνα Γκαμούλου** διερευνήθηκε το ίδιο παίγνιο (Splendor) με τη χρήση της μεθόδου Deep Q-Networks (DQN). Ενώ ο αλγόριθμος DQN ανήκει στην κατηγορία της Ενισχυτικής Μάθησης χωρίς μοντέλο (Model-Free RL) και βασίζεται στην εκ των προτέρων εκπαίδευση ενός Βαθέος Νευρωνικού Δικτύου για την προσέγγιση συναρτήσεων αξίας, η παρούσα έρευνα εστιάζει στον αλγόριθμο Monte Carlo Tree Search (MCTS), ο οποίος αποτελεί μέθοδο σχεδιασμού σε πραγματικό χρόνο βασισμένη σε μοντέλο (Model-Based Online Planning) που λαμβάνει αποφάσεις μέσω δυναμικών στοχαστικών προσομοιώσεων δέντρου και παραμετροποιημένης εξισορρόπησης εξερεύνησης-εκμετάλλευσης.

1.4 Συνεισφορά και Καινοτομία της Διπλωματικής Εργασίας

Η παρούσα εργασία δεν περιορίζεται στην απλή θεωρητική μελέτη γνωστών αλγορίθμων, αλλά προσφέρει πρακτικά εργαλεία και ουσιαστική συνεισφορά, τα οποία μπορούν να αξιοποιηθούν τόσο σε ακαδημαϊκό όσο και σε ερευνητικό επίπεδο. Πιο συγκεκριμένα, η εργασία συμβάλλει στα εξής σημεία:

Δημιουργία Πράκτορα με Ικανότητα Συνεχούς Μάθησης: Αναπτύχθηκε ένας έξυπνος παίκτης (MCTS Agent) ο οποίος δεν ξεκινάει από το μηδέν σε κάθε παιχνίδι, αλλά αποθηκεύει την εμπειρία του. Αυτός ο μηχανισμός μπορεί να αποτελέσει σημείο αναφοράς για όσους μελετούν πώς ένας αλγόριθμος βελτιώνεται σταδιακά μέσα από επαναλαμβανόμενες αναμετρήσεις.

Επαναχρησιμοποιήσιμο Γραφικό Περιβάλλον (GUI): Το γραφικό περιβάλλον που κατασκευάστηκε σχεδιάστηκε με τέτοιο τρόπο ώστε να είναι πλήρως λειτουργικό και ανεξάρτητο από τη λογική της Τεχνητής Νοημοσύνης. Αυτό σημαίνει ότι μπορεί να επαναχρησιμοποιηθεί αυτούσιο σε άλλες εργασίες ή έρευνες που χρειάζονται την οπτική αναπαράσταση του παιχνιδιού.

Ανοιχτός και Επεκτάσιμος Κώδικας: Όλος ο προγραμματιστικός κώδικας της εργασίας γράφτηκε με ξεκάθαρη δομή και είναι άμεσα διαθέσιμος για περαιτέρω χρήση. Οποιοσδήποτε μελλοντικός ερευνητής ή φοιτητής μπορεί να πάρει τη βάση του κώδικα και να ενσωματώσει εύκολα διαφορετικούς αλγορίθμους Τεχνητής Νοημοσύνης, προκειμένου να τους συγκρίνει με τον δικό μας πράκτορα.

Ολοκληρωμένο Σύστημα Οπτικοποίησης και Αξιολόγησης: Η εργασία παρέχει έτοιμα εργαλεία που παράγουν αυτόματα στατιστικά διαγράμματα (Post-Game Analytics). Αυτό το σύστημα μπορεί να φανεί εξαιρετικά χρήσιμο σε όποιον θέλει να κατανοήσει, μέσω ξεκάθαρων γραφημάτων, τον τρόπο με τον οποίο παίρνει αποφάσεις ένας αλγόριθμος.

1.5 Δομή της Διπλωματικής Εργασίας

Το περιεχόμενο της παρούσας διπλωματικής εργασίας οργανώνεται σε πέντε κύρια κεφάλαια, με την ακόλουθη δομή:

Κεφάλαιο 1: Παρουσιάζεται η εισαγωγή, ορίζεται το αντικείμενο, αποσαφηνίζονται ο σκοπός και οι στόχοι της έρευνας, περιγράφεται η μεθοδολογία και αναδεικνύεται η συνεισφορά της εργασίας.

Κεφάλαιο 2: Παρατίθεται η απαραίτητη βιβλιογραφική ανασκόπηση και το θεωρητικό υπόβαθρο που αφορά τη χρήση της Τεχνητής Νοημοσύνης σε παίγνια, καθώς και το μαθηματικό πλαίσιο του Monte Carlo Tree Search.

Κεφάλαιο 3: Αναλύεται λεπτομερώς η αρχιτεκτονική του λογισμικού, η δομή του περιβάλλοντος προσομοίωσης και ο σχεδιασμός των διαφόρων υπολογιστικών πρακτόρων.

Κεφάλαιο 4: Περιγράφεται σε βάθος η τεχνική υλοποίηση του συστήματος (κώδικας, λειτουργίες, γραφικό περιβάλλον) και παρουσιάζεται η ανάλυση μιας μεμονωμένης, υποδειγματικής αναμέτρησης.

Κεφάλαιο 5: Συνοψίζονται τα συνολικά πειραματικά αποτελέσματα, αναλύεται η συμπεριφορά των πρακτόρων σε βάθος χρόνου και καταγράφονται τα τελικά συμπεράσματα, συμπεριλαμβανομένων προτάσεων για μελλοντική έρευνα.

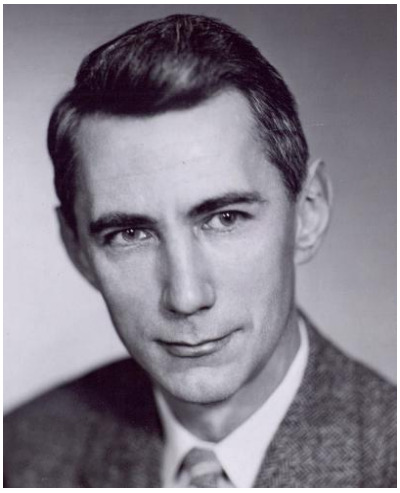
2 ΚΕΦΑΛΑΙΟ 2ο : Επισκόπηση του πεδίου, προσδιορισμός του προβλήματος και διεθνής εμπειρία

2.1 Επιτραπέζια παιχνίδια & Τεχνητή νοημοσύνη

Η μελέτη της Τεχνητής Νοημοσύνης (AI) είναι ιστορικά και άρρηκτα συνδεδεμένη με την εξέλιξη των επιτραπέζιων παιγνίων. Λόγω της φύσης τους ως κλειστών, καλά ορισμένων συστημάτων με αυστηρούς κανόνες, διακριτές καταστάσεις και αντικειμενικά κριτήρια επιτυχίας (νίκη, ήττα ή ισοπαλία), τα παίγνια αποτέλεσαν από τα μέσα του 20ού αιώνα το ιδανικό "εργαστήριο" για τη δοκιμή αλγορίθμων λήψης αποφάσεων [1], [2]. Κατά τις πρώτες δεκαετίες της έρευνας, η ακαδημαϊκή κοινότητα επικεντρώθηκε σχεδόν αποκλειστικά σε παιχνίδια τέλειας πληροφορίας (perfect information) και μηδενικού αθροίσματος (zero-sum), όπως η ντάμα και το σκάκι, όπου αμφότεροι οι παίκτες έχουν πλήρη γνώση της κατάστασης του ταμπλό [9]



Εικόνα 1 [1] – Alan Turing, πρωτοπόρος της επιστήμης των υπολογιστών και της τεχνητής νοημοσύνης.



Εικόνα 2 [1]– Claude Shannon, του οποίου η εργασία έθεσε τις βάσεις για τον προγραμματισμό υπολογιστών σε παίγνια.

Ιστορικά ορόσημα, όπως η νίκη του υπερυπολογιστή Deep Blue εναντίον του Garry Kasparov το 1997 [3], απέδειξαν ότι σε ντετερμινιστικά περιβάλλοντα με διαχειρίσιμη πολυπλοκότητα, η εξαντλητική αναζήτηση (αλγόριθμοι τύπου Minimax) [14] συνδυασμένη με εξειδικευμένους ευρετικούς κανόνες (heuristics) επαρκεί για να υπερκεράσει την ανθρώπινη διάνοια.



Εικόνα 3 [3] – Η ιστορική αναμέτρηση μεταξύ του Γκάρι Κασπάροφ (Garry Kasparov) και του υπερυπολογιστή Deep Blue της IBM (1997).

Ωστόσο, η μετάβαση σε παίγνια με απείρως μεγαλύτερο παράγοντα διακλάδωσης (branching factor), όπως το ασιατικό Go, κατέδειξε τα όρια της παραδοσιακής αναζήτησης, αναγκάζοντας την επιστημονική κοινότητα να στραφεί προς στοχαστικές και προσεγγιστικές μεθόδους (όπως η μέθοδος Monte Carlo και η Βαθιά Μάθηση).

2.2 Προσδιορισμός του προβλήματος της πολυπλοκότητας και διαχείρισης πόρων

Ο θεμελιώδης περιορισμός των παραδοσιακών αλγορίθμων αναζήτησης δέντρου έγκειται στο πρόβλημα της εκθετικής πολυπλοκότητας χώρου και χρόνου. Όταν ο αριθμός των νόμιμων κινήσεων σε κάθε κατάσταση (branching factor) αυξάνεται, το δέντρο αναζήτησης μεγεθύνεται εκθετικά, καθιστώντας αδύνατη την εξαντλητική περιήγησή του εντός αποδεκτού υπολογιστικού χρόνου.

Πέραν της υπολογιστικής πολυπλοκότητας, σε παίγνια διαχείρισης πόρων και ανάπτυξης υποδομών (engine-building), το πρόβλημα εντείνεται λόγω της απαίτησης για αναβαλλόμενη ικανοποίηση (delayed gratification) [10]. Στα παίγνια αυτά, μια κίνηση μπορεί να μην αποφέρει άμεσα πόντους νίκης, αλλά να εγκαθιδρύει μια στρατηγική υποδομή η οποία θα μεγιστοποιήσει τις αποδόσεις σε μεταγενέστερα στάδια. Η αξιολόγηση μιας τέτοιας κατάστασης είναι εξαιρετικά

δύσκολο να μοντελοποιηθεί με στατικούς ευρετικούς κανόνες, καθότι η στρατηγική σημασία της επένδυσης εξαρτάται από τον μακροπρόθεσμο σχεδιασμό και τις αντιδράσεις του αντιπάλου. Ως εκ τούτου, απαιτείται μια αλγοριθμική προσέγγιση που δύναται να προβλέψει και να αξιολογήσει καταστάσεις βαθιά στο μέλλον του παιγνίου.

2.3 Ευρετικοί αλγόριθμοι Heuristics

Οι Heuristic Search Techniques[11] αποτελούν την κεντρική συνιστώσα της Ενημερωμένης Αναζήτησης (Informed Search) στο πεδίο της Τεχνητής Νοημοσύνης. Σε αντίθεση με τις στρατηγικές Μη Στοχευμένης Αναζήτησης (Uninformed Search), οι οποίες εξερευνούν το χώρο των καταστάσεων με συστηματικό αλλά μη καθοδηγούμενο τρόπο, οι ευρετικές μέθοδοι χρησιμοποιούν πρόσθετη γνώση για το πρόβλημα προκειμένου να κατευθύνουν την αναζήτηση προς τις πλέον υποσχόμενες περιοχές. Η λειτουργική βάση αυτών των πρακτόρων είναι η Ευρετική Συνάρτηση (Heuristic function) h(n), η οποία εκτιμά την απόσταση από την τρέχουσα κατάσταση n μέχρι την τελική κατάσταση (Goal state).

Στην πράξη, οι ευρετικοί πράκτορες υιοθετούν διαφορετικές προσεγγίσεις για την πλοήγηση στο χώρο των καταστάσεων. Οι κυριότεροι τύποι **Heuristic Search Techniques**, όπως αναφέρονται στη βιβλιογραφία, είναι οι εξής:

A Search Algorithm*:* Είναι ίσως ο πιο γνωστός αλγόριθμος ευρετικής αναζήτησης. Χρησιμοποιεί best-first search και εντοπίζει το μονοπάτι με το ελάχιστο κόστος από έναν αρχικό κόμβο σε έναν κόμβο-στόχο. Βασίζεται στην Heuristic function:

$$\boldsymbol{f(n) = g(n) + h(n)}$$

όπου g(n) είναι το κόστος από τον αρχικό κόμβο μέχρι τον κόμβο n και h(n) είναι μια ευρετική τιμή που εκτιμά το κόστος της φθηνότερης διαδρομής από τον n μέχρι τον στόχο. Χρησιμοποιείται ευρέως σε προβλήματα pathfinding και graph traversal.

Greedy Best-First Search: Επεκτείνει τον κόμβο που βρίσκεται πληρέστερα στον στόχο, σύμφωνα με την εκτίμηση μιας Heuristic function. Σε αντίθεση με τον A*, ο οποίος συνυπολογίζει το κόστος της διαδρομής και το εκτιμώμενο υπολειπόμενο κόστος, ο Greedy Best-First Search δίνει προτεραιότητα αποκλειστικά στο εκτιμώμενο κόστος προς τον στόχο. Αν και αυτό τον καθιστά ταχύτερο, μπορεί να οδηγήσει σε suboptimal solutions.

Hill Climbing: Μια ευρετική αναζήτηση που χρησιμοποιείται για προβλήματα μαθηματικής βελτιστοποίησης. Αποτελεί παραλλαγή της μεθόδου gradient ascent. Ξεκινά από

ένα τυχαίο αρχικό σημείο και κινείται επαναληπτικά προς υψηλότερες τιμές (local maxima) επιλέγοντας την καλύτερη γειτονική κατάσταση. Ωστόσο, υπάρχει ο κίνδυνος να εγκλωβιστεί σε local maxima, αποτυγχάνοντας να βρει το global optimum.

Simulated Annealing: Εμπνευσμένο από την ανόπτηση στη μεταλλουργία, είναι μια πιθανολογική τεχνική για την εύρεση του global optimum. Σε αντίθεση με το Hill Climbing, επιτρέπει στην αναζήτηση να αποδέχεται προσωρινά worse solutions ώστε να διαφεύγει από local optima. Αυτή η πιθανότητα αποδοχής μειώνεται με την πάροδο του χρόνου, επιτρέποντας στον αλγόριθμο να συγκλίνει προς την καλύτερη λύση.

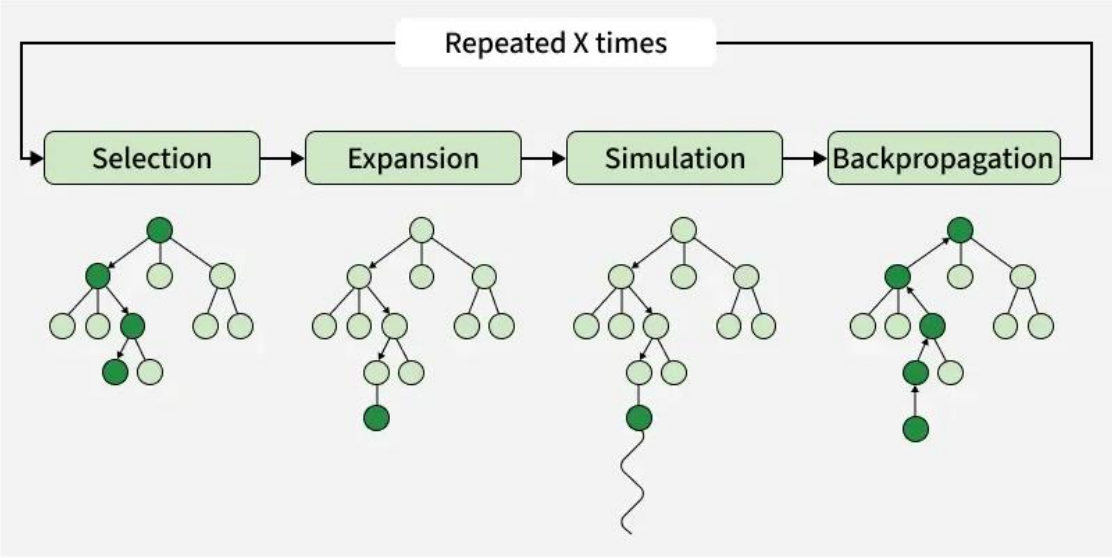
Beam Search: Μια τεχνική αναζήτησης σε γραφήματα που εξερευνά μόνο έναν περιορισμένο αριθμό πολλά υποσχόμενων κόμβων (ένα beam). Το beam width, το οποίο περιορίζει τον αριθμό των κόμβων που αποθηκεύονται στη μνήμη, παίζει καθοριστικό ρόλο στην απόδοση και την ακρίβεια της αναζήτησης.

Παρά την αποτελεσματικότητά τους, οι πράκτορες αυτοί χαρακτηρίζονται συχνά ως άπληστοι, καθώς η ποιότητα των αποφάσεών τους εξαρτάται απόλυτα από την ποιότητα της συνάρτησης h(n). Το σημαντικότερο πρόβλημα είναι ο κίνδυνος εγκλωβισμού σε Local maxima, όπου ο πράκτορας θεωρεί ότι έχει φτάσει στη βέλτιστη λύση επειδή όλες οι άμεσα γειτονικές καταστάσεις φαίνονται χειρότερες, ενώ στην πραγματικότητα το Global optimum βρίσκεται σε άλλη περιοχή. Επιπλέον, μια συνάρτηση που δεν είναι Admissible —δηλαδή που υπερεκτιμά το κόστος— μπορεί να οδηγήσει σε υποβέλτιστες αποφάσεις. Για τους λόγους αυτούς, οι ευρετικοί πράκτορες συχνά λειτουργούν ως Baseline ή ενσωματώνονται σε υβριδικά σχήματα αναζήτησης, όπως ο MCTS, προσφέροντας την απαραίτητη γνώση πεδίου που απαιτείται για την πλοήγηση σε περιβάλλοντα με υψηλή στρατηγική πολυπλοκότητα.

2.4 Αλγόριθμος Monte Carlo Search Tree (MCTS)

Ο αλγόριθμος Monte Carlo Tree Search (MCTS) [4] [5] [6] [12] αποτελεί έναν ισχυρό ευρετικό αλγόριθμο αναζήτησης που χρησιμοποιείται ευρέως σε διαδικασίες λήψης αποφάσεων, ειδικά στον τομέα των πολύπλοκων παιγνίων. Όπως αναλύεται στη σχετική βιβλιογραφία, σε αντίθεση με τις παραδοσιακές μεθόδους εξαντλητικής αναζήτησης που απαιτούν τον πλήρη υπολογισμό του δέντρου, ο MCTS βασίζεται σε στοχαστικές δειγματοληψίες (Monte Carlo) και αναπτύσσει το δέντρο αναζήτησης σταδιακά και ασύμμετρα.

Η λειτουργία του αλγορίθμου βασίζεται στην επαναληπτική εκτέλεση τεσσάρων διακριτών φάσεων:



Εικόνα 4 [12] – Τα τέσσερα βασικά στάδια του αλγορίθμου Monte Carlo Tree Search (Επιλογή, Επέκταση, Προσομοίωση και Οπισθοδρόμηση).

Αρχικά, στην **Επιλογή (Selection)**, ο αλγόριθμος διασχίζει το δέντρο από τον κόμβο της ρίζας προς τα φύλλα, χρησιμοποιώντας μαθηματικές εξισώσεις όπως το Upper Confidence Bound (UCB) προκειμένου να εξισορροπήσει ιδανικά την εκμετάλλευση των πιο υποσχόμενων γνωστών κόμβων (exploitation) με την εξερεύνηση νέων (exploration).

Στη συνέχεια, ακολουθεί η **Επέκταση (Expansion)**, όπου εάν ο επιλεγμένος κόμβος-φύλλο δεν αποτελεί τερματική κατάσταση, προστίθεται στο δέντρο ένας ή περισσότεροι νέοι κόμβοι-παιδιά

Από αυτό το σημείο ξεκινά η **Προσομοίωση (Simulation ή Rollout)**, κατά την οποία το παιχνίδι εκτελείται ταχύτατα, συχνά με τυχαίες ή ημι-τυχαίες κινήσεις, μέχρι να επέλθει το τέλος της παρτίδας.

Τέλος, στην **Οπισθοδιάδοση (Backpropagation)**, το τελικό αποτέλεσμα της προσομοίωσης (νίκη, ήττα ή ισοπαλία) μεταφέρεται αντίστροφα στο μονοπάτι μέχρι τη ρίζα, ενημερώνοντας άμεσα τα στατιστικά στοιχεία επισκέψεων και επιτυχίας όλων των κόμβων που συμμετείχαν.

Παρακάτω φαίνεται ο μαθηματικός τύπος της **UCB1 Formula**

Η φάση επιλογής βασίζεται στον τύπο UCB1 (Upper Confidence Bound) για να καθορίσει ποιον δευτερεύοντα κόμβο θα επισκεφτεί στη συνέχεια:

$$UCT_i = \frac{Wi}{ni} + c\sqrt{\frac{ln(Np)}{n_i}}$$

όπου:

UCT (Upper Confidence bounds applied to Trees): Η τελική βαθμολογία του κόμβου. Ο αλγόριθμος επιλέγει πάντα τον κόμβο-παιδί με τη μέγιστη τιμή UCT.

$\frac{Wi}{ni}$ **(Όρος Εκμετάλλευσης - Exploitation):** (Όρος Εκμετάλλευσης / Exploitation Term): Εκφράζει τη μέση εμπειρική ανταμοιβή του κόμβουi, όπου Wi είναι το άθροισμα των αποτελεσμάτων των προσομοιώσεων (σκορ από 0.0 έως 1.0) και Ni ο αριθμός των επισκέψεων του κόμβου.

$\sqrt{\frac{ln(Np)}{n_i}}$ **(Σταθερά Εξερεύνησης - Exploration Constant):** Μετρά το στατιστικό εύρος αβεβαιότητας. Np είναι ο συνολικός αριθμός επισκέψεων του γονικού κόμβου. Όσο αυξάνεται το Np χωρίς να επισκεπτόμαστε τον κόμβο i, ο όρος αυτός μεγαλώνει, αναγκάζοντας τον αλγόριθμο να εξερευνήσει λιγότερο δοκιμασμένες ενέργειες.

C (Σταθερά Εξερεύνησης / Exploration Parameter): Ρυθμίζει τη σχετική ισχύ μεταξύ εκμετάλλευσης και εξερεύνησης.

2.4.1 Η Σταθερά Εξερεύνησης UCB (c)

Η σταθερά C=√2 [16] δεν είναι αυθαίρετη, αλλά αντλείται αυστηρά από την **Ανισότητα Hoeffding (Chernoff-Hoeffding Bound)** στη Θεωρία Πιθανοτήτων για φραγμένες τυχαίες μεταβλητές στο διάστημα [0,1].

Σύμφωνα με την απόδειξη των Auer, Cesa-Bianchi & Fischer (2002), η πιθανότητα η πραγματική μέση αξία μ_i μιας ενέργειας να υπερβαίνει την εμπειρική της εκτίμηση $\overline{Xι} = \frac{Wi}{ni}$ κατά περισσότερο από ένα διάστημα εμπιστοσύνης **ϵ** φράσσεται εκθετικά:

$$P(\mu_i > \overline{Xι} + \epsilon) \leq e^{-2Ni\epsilon^2}$$

Θέτοντας την πιθανότητα σφάλματος ίση με $\mathbf{p = Np^{-4}}$ (ώστε το άθροισμα των πιθανοτήτων σφάλματος σε άπειρο ορίζοντα να συγκλίνει, $\sum Np^{-4} < \infty$ προκύπτει:

$$e^{-2Ni\epsilon^2} = Np^{-4} <=> -2Ni\epsilon^2 = -4\ln(Np) <=> \epsilon^2 = \frac{2lnNp}{ni} <=> \epsilon = \sqrt{2}\sqrt{\frac{ln(Np)}{n_i}}$$

Κατά συνέπεια, ο συντελεστής C=√2 ≈ 1.414 ίναι το μαθηματικά αποδεδειγμένο άνω όριο εμπιστοσύνης, το οποίο εγγυάται ότι η πιθανότητα υποεκτίμησης ενός καλού μονοπατιού εκμηδενίζεται ασυμπτωτικά καθώς ο αριθμός των προσομοιώσεων αυξάνεται.

2.4.2 Αντιμετώπιση Στοχασμού (Determinization)

Σε περιβάλλοντα imperfect information ή στοχαστικά παίγνια, η κλασική μορφή του MCTS συχνά ενισχύεται με την τεχνική της Determinization. Σε αυτή την προσέγγιση, αντί ο αλγόριθμος να προσπαθεί να μοντελοποιήσει ολόκληρο τον χώρο πιθανοτήτων των άγνωστων μεταβλητών, δημιουργεί συγκεκριμένα "δείγματα" της κρυφής κατάστασης του παιχνιδιού. Για κάθε προσομοίωση, ο πράκτορας υποθέτει ένα τυχαίο αλλά σταθερό σενάριο και εκτελεί την αναζήτηση σαν το περιβάλλον να ήταν πλήρως παρατηρήσιμο. Μέσα από την εκτέλεση χιλιάδων τέτοιων δειγματοληψιών, οι στατιστικές αποκλίσεις εξομαλύνονται, επιτρέποντας την εξαγωγή αξιόπιστων αποφάσεων παρά την εγγενή αβεβαιότητα του περιβάλλοντος

2.4.3 Διαδικασία Λήψης Αποφάσεων Markov (Markov Decision Process)

Το μαθηματικό πλαίσιο στο οποίο συχνά βασίζεται η λήψη αποφάσεων του MCTS για περιβάλλοντα με αβεβαιότητα είναι η Διαδικασία Λήψης Αποφάσεων Markov (Markov Decision Process - MDP) [13]. Ένα MDP παρέχει ένα αυστηρό μαθηματικό μοντέλο για την επίλυση προβλημάτων στα οποία τα αποτελέσματα των ενεργειών είναι εν μέρει τυχαία και εν μέρει υπό τον έλεγχο του πράκτορα.

Στην περίπτωση του Splendor, η τυχαιότητα εισάγεται διαρκώς από την αποκάλυψη νέων καρτών από την τράπουλα. Προσεγγίζοντας το παίγνιο ως MDP, ο MCTS καταφέρνει να μην εγκλωβίζεται σε ένα αυστηρά ντετερμινιστικό, στατικό πλάνο (όπως κάνει ο Heuristic Agent). Αντιθέτως, σταθμίζει διαρκώς τις εναλλακτικές καταστάσεις (states) και τις πιθανές μεταβάσεις (transitions) διατηρώντας πολλαπλά «ζωντανά» μονοπάτια στο δέντρο, με αποτέλεσμα να απορροφά τον στοχασμό χωρίς στρατηγική κατάρρευση

2.5 Σχετικά παραδείγματα MCTS & Heuristic-Based Agents

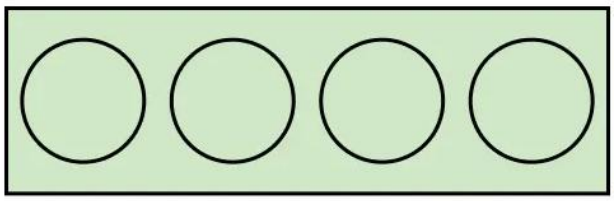
Παράδειγμα Mcts

Για την καλύτερη κατανόηση της λειτουργίας της μεθόδου MCTS, εξετάζεται ένα απλό υποθετικό παίγνιο, βάσει του οποίου δομείται το αντίστοιχο δέντρο παιγνίου

Πίνακας 1- Βασικά Χαρακτηριστικά του Προβλήματος

Πίνακας 1. Βασικά Χαρακτηριστικά του Προβλήματος

Χαρακτηριστικό	**Περιγραφή**
Παίγνιο	Pair
Αριθμός Παικτών	2
Πίνακας	4 κουτιά
Συνθήκη Νίκης	Ένας παίκτης αναδεικνύεται νικητής εάν επισημάνει δύο οποιαδήποτε γειτονικά κουτιά.



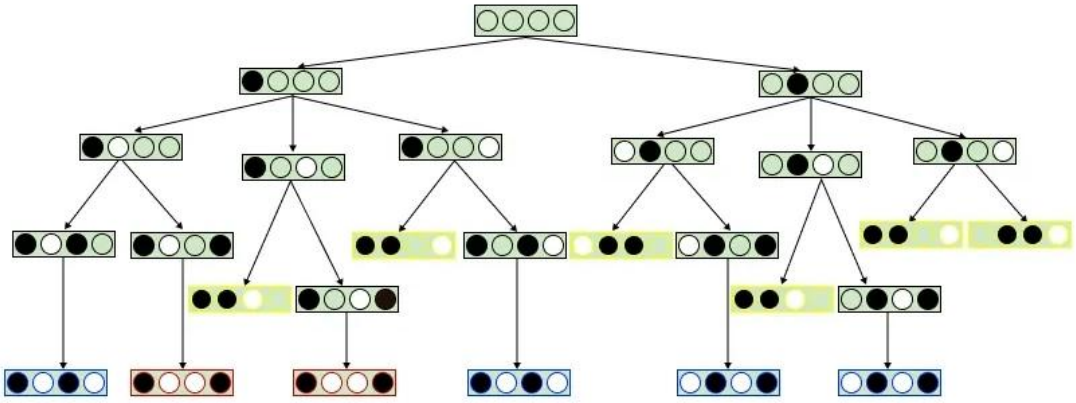
Empty board for the Pair game

Εικόνα 5 [12] – Γραφική αναπαράσταση ενός άδειου ταμπλό (Empty Board) για ένα υποθετικό παίγνιο δύο παικτών.

Το πλήρες δέντρο παιχνιδιού περιλαμβάνει τρεις τύπους τελικών (φύλλων) καταστάσεων:

- **Κίτρινα φύλλα**: Θέσεις νίκης (+1) -> Σύνολο: 6 κόμβοι
- **Μπλε φύλλα**: Θέσεις ισοπαλίας (0) -> Σύνολο: 4 κόμβοι
- **Κόκκινα φύλλα**: Θέσεις ήττας (–1) -> Σύνολο: 2 κόμβοι

Οι προαναφερθείσες τιμές αξιοποιούνται κατά τη διάρκεια της προσομοίωσης και του Backpropagation.

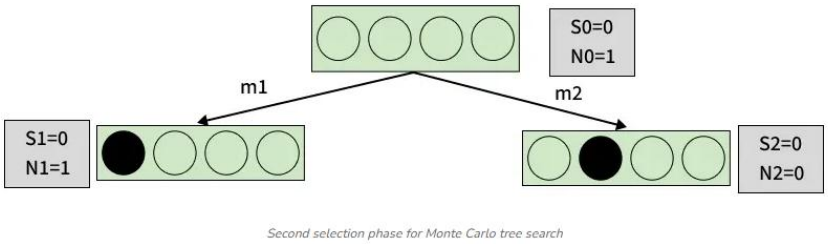


Monte carlo tree search game tree for pair game0

Εικόνα 6 [12] – Δέντρο αναζήτησης (Game Tree) για το παίγνιο δύο παικτών, όπου απεικονίζονται οι πρώτες πιθανές διακλαδώσεις.

Επανάληψη 1

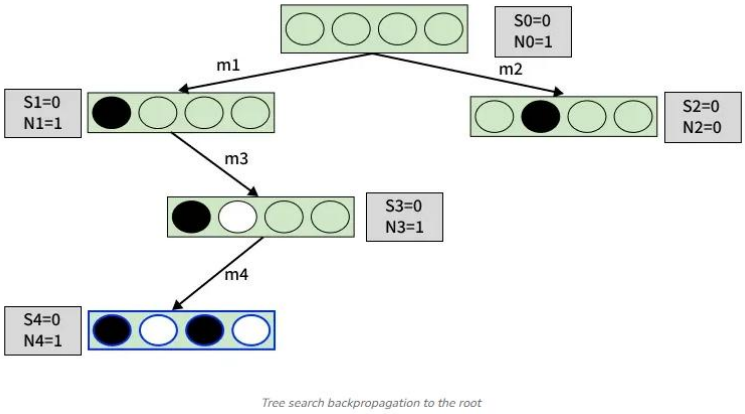
Επιλογή & Επέκταση: Η διαδικασία εκκινεί από τη ρίζα S0. Δύο πιθανές κινήσεις, m1 και m2, οδηγούν στις καταστάσεις S1 και S2 αντίστοιχα. Πραγματοποιείται υπολογισμός του UCB (Upper Confidence Bound) για τις καταστάσεις S1 και S2. Καθώς αμφότερες αποτελούν μη επισκεπτόμενα φύλλα (UCB = ∞), η επιλογή πραγματοποιείται τυχαία. Έστω ότι επιλέγεται η κατάσταση S1.



Εικόνα 7 [12] – Το στάδιο της επιλογής (Selection Phase) στον αλγόριθμο MCTS.

Προσομοίωση: Εκτελείται τυχαία μια εξέλιξη (από την S1) έως μια τελική κατάσταση. Μια πιθανή εξέλιξη (μέσω των κινήσεων m3, m4) καταλήγει σε ισοπαλία (0). Η καταγραφή διαμορφώνεται ως εξής: S4 = 0, N4 = 1.

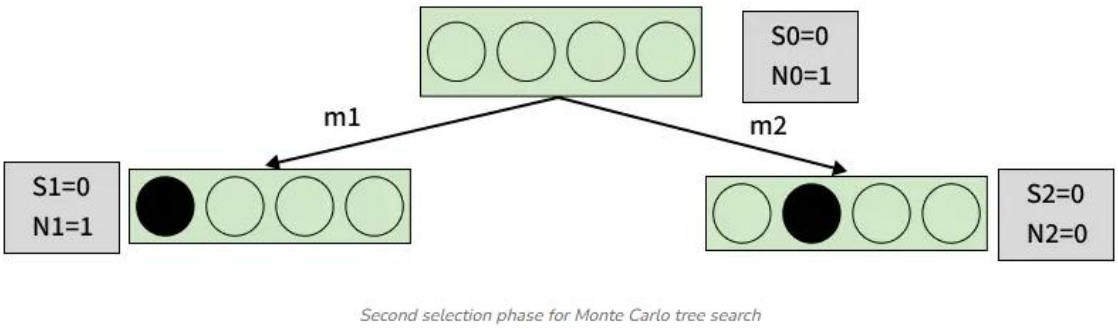
Backpropagation: Το αποτέλεσμα (0) διαδίδεται κατά μήκος της διαδρομής επιστρέφοντας προς τη ρίζα S0, ενημερώνοντας τις τιμές και τον αριθμό επισκέψεων σε κάθε ενδιάμεσο κόμβο.



Εικόνα 8 – Το στάδιο της οπισθοδρόμησης (Backpropagation) στον αλγόριθμο MCTS.

Επανάληψη 2

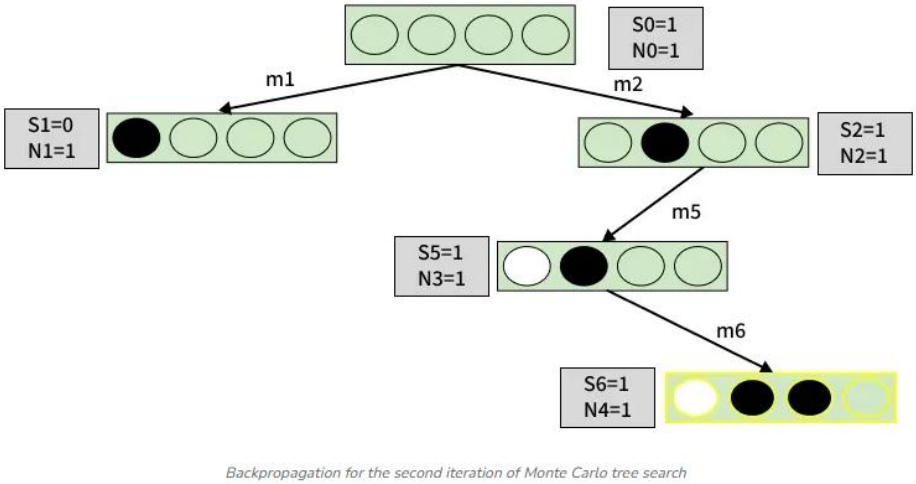
Επιλογή: Πραγματοποιείται σύγκριση μεταξύ των καταστάσεων S1 και S2. Η κατάσταση S1 έχει επισκεφθεί μία φορά (διαθέτει πεπερασμένο UCB), ενώ η S2 δεν έχει επισκεφθεί (UCB = ∞). Συνεπώς, επιλέγεται η κατάσταση S2.



Εικόνα 9 [12] – Η δεύτερη επανάληψη του σταδίου επιλογής (Second Selection Phase) στον MCTS.

Προσομοίωση: Εκκινεί τυχαία προσομοίωση από την S2. Μία διαδρομή (κινήσεις m5, m6) καταλήγει σε νίκη (+1). Η καταγραφή διαμορφώνεται ως εξής: S6 = +1, N6 = 1.

Backpropagation: Πραγματοποιείται ενημέρωση των τιμών κατά μήκος της διαδρομής (Value(S2) = 1, Visits(S2) = 1). Η ρίζα πλέον ενσωματώνει αποτελέσματα τόσο από την S1 όσο και από την S2.



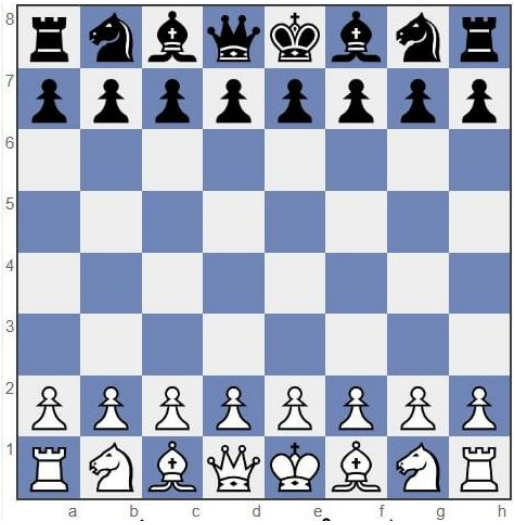
Εικόνα 10 [12] – Η δεύτερη επανάληψη της οπισθοδρόμησης (Backpropagation) στον αλγόριθμο MCTS.

Επανάληψη 3

Επιλογή: Έπειτα από δύο αλληλεπιδράσεις, αμφότερες οι καταστάσεις S1 και S2 έχουν N = 1. Η μέθοδος UCB προκρίνει την S2 (καθώς έχει καταγράψει μία νίκη), με αποτέλεσμα να επιλέγεται εκ νέου η S2.

Παράδειγμα Heuristic Based Agent

Το σκάκι αποτελεί ένα από τα κλασικότερα παραδείγματα εφαρμογής ευρετικών μεθόδων. Ένας απλός Heuristic-Based Agent χρησιμοποιεί μια γραμμική συνάρτηση αξιολόγησης προκειμένου να αποφασίσει τη βέλτιστη επόμενη κίνηση.



Εικόνα 11 [9] – Παραδοσιακή σκακιέρα (Chess Board)

1. Η Συνάρτηση Αξιολόγησης

Η συνάρτηση $f(s)$ μπορεί να οριστεί ως εξής:

$$f(s) = w1 * M + w2 * P + W3 * C$$

Όπου τα επιμέρους στοιχεία ορίζονται ως εξής:

Πίνακας 2- Σύμβολα και Ορολογία

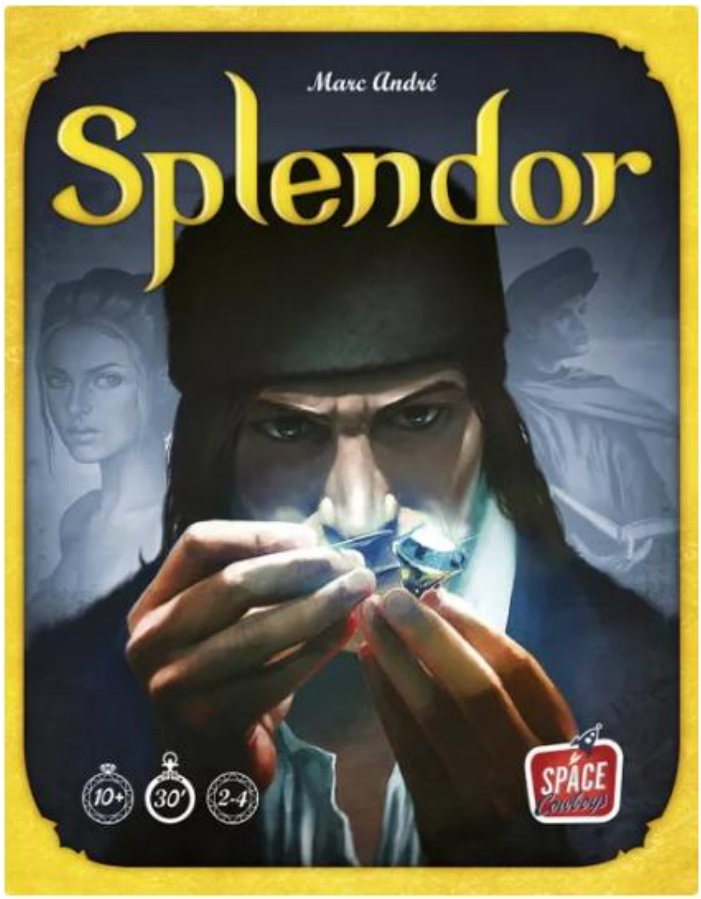
Σύμβολο	**Ονομασία**	**Περιγραφή & Παραδείγματα**
M	Material	Η συνολική αξία των κομματιών στο ταμπλό (π.χ., Πιόνι = 1, Ίππος = 3, Βασίλισσα = 9).
P	Position	Η στρατηγική τοποθέτηση στο χώρο (π.χ., ο έλεγχος των τεσσάρων κεντρικών τετραγώνων προσδίδει επιπλέον πόντους).

Σύμβολο	Ονομασία	Περιγραφή & Παραδείγματα
C	Mobility	Ο αριθμός των διαθέσιμων, καθόλα νόμιμων κινήσεων (περισσότερες επιλογές συνεπάγονται πλεονεκτικότερη θέση).
$\mathbf{w_i}$	Weights	Τα βάρη, τα οποία υποδηλώνουν τον βαθμό σημαντικότητας του εκάστοτε χαρακτηριστικού στη λήψη απόφασης.

2. Πώς λειτουργεί η απόφαση

Ο πράκτορας εξετάζει το σύνολο των νόμιμων κινήσεών του για τον τρέχοντα γύρο. Για κάθε πιθανή κίνηση, υπολογίζει την τιμή *f(s)* της κατάστασης που θα προκύψει. Εν τέλει, επιλέγει την κίνηση που μεγιστοποιεί το σκορ, αγνοώντας τις μελλοντικές ενέργειες του αντιπάλου (εφαρμόζοντας μια Greedy προσέγγιση).

2.6 Το παιχνίδι Splendor



Εικόνα 12 [7] – Το επιτραπέζιο παιχνίδι Splendor (2014), απεικόνιση του κουτιού και του βασικού θέματος.

Σκοπός του Παιχνιδιού

Σκοπός του παιχνιδιού είναι η συγκέντρωση Πόντων Κύρους (Prestige Points). Το παιχνίδι ολοκληρώνεται όταν ένας παίκτης φτάσει ή ξεπεράσει τους **15 Πόντους Κύρους**.

Περιεχόμενα (Components)

- **40 Μάρκες (Tokens):** 7 Σμαράγδια (Πράσινα), 7 Ζαφείρια (Μπλε), 7 Ρουμπίνια (Κόκκινα), 7 Διαμάντια (Λευκά), 7 Όνυχες (Μαύρα) και 5 μάρκες Χρυσού (Μπαλαντέρ).
- **90 Κάρτες Ανάπτυξης (Development Cards):** Χωρισμένες σε τρία επίπεδα δυσκολίας (Tier 1, Tier 2, Tier 3).
- **10 Πλακίδια Ευγενών (Noble Tiles).**



Εικόνα 13 [7] – Συστατικά στοιχεία του επιτραπέζιου παιγνίου Splendor (Μάρκες Πολύτιμων Λίθων, Κάρτες Ανάπτυξης, Πλακίδια Ευγενών).

Στήσιμο (Setup) Ανοίγονται 4 κάρτες από κάθε επίπεδο στο κέντρο του τραπεζιού (σύνολο 12 ορατές κάρτες). Τοποθετούνται πλακίδια Ευγενών ίσα με τον αριθμό των παικτών συν ένα (π.χ. 3 Ευγενείς για παιχνίδι 2 παικτών). Οι μάρκες τοποθετούνται σε στοίβες ανά χρώμα.



Εικόνα 14 [7] – Επιπρόσθετα στοιχεία (Κάρτες Ανάπτυξης διαφορετικών επιπέδων) του παιγνίου Splendor.

Η Σειρά του Παίκτη (Player Turn) Κατά τη διάρκεια του γύρου του, ένας παίκτης πρέπει να εκτελέσει **μόνο μία** από τις παρακάτω τέσσερις διαθέσιμες ενέργειες:

1. **Λήψη 3 διαφορετικών μαρκών:** Ο παίκτης παίρνει 3 μάρκες πολύτιμων λίθων διαφορετικού χρώματος από το απόθεμα.

2. **Λήψη 2 ίδιων μαρκών:** Ο παίκτης παίρνει 2 μάρκες του ίδιου χρώματος. *Προϋπόθεση:* Αυτή η ενέργεια είναι νόμιμη μόνο εάν υπάρχουν τουλάχιστον 4 διαθέσιμες μάρκες αυτού του χρώματος στη στοίβα πριν τη λήψη.
3. **Κράτηση (Reservation) 1 κάρτας και λήψη 1 Χρυσού:** Ο παίκτης παίρνει 1 ορατή κάρτα από το τραπέζι (ή κρυφά την πρώτη κάρτα από οποιαδήποτε τράπουλα) και την παίρνει στο χέρι του. Παράλληλα, λαμβάνει 1 μάρκα Χρυσού (joker). Ένας παίκτης δεν επιτρέπεται να έχει περισσότερες από 3 κρατημένες κάρτες στο χέρι του ταυτόχρονα.
4. **Αγορά 1 ορατής ή κρατημένης κάρτας:** Ο παίκτης αγοράζει μια κάρτα πληρώνοντας το κόστος που αναγράφεται σε αυτήν, επιστρέφοντας τις αντίστοιχες μάρκες στο κοινό απόθεμα.

Αυστηροί Περιορισμοί

- **Όριο 10 Μαρκών:** Στο τέλος του γύρου του, κανένας παίκτης δεν επιτρέπεται να έχει πάνω από 10 μάρκες συνολικά (συμπεριλαμβανομένου του χρυσού). Εάν έχει περισσότερες, πρέπει να επιστρέψει μάρκες της επιλογής του στο απόθεμα μέχρι να μείνει με 10.
- Ο Χρυσός μπορεί να αντικαταστήσει οποιοδήποτε χρώμα μάρκας κατά την αγορά μιας κάρτας.

Μπόνους (Το Engine-Building) Όταν ένας παίκτης αγοράζει μια κάρτα Ανάπτυξης, την τοποθετεί μπροστά του. Κάθε κάρτα παρέχει ένα μόνιμο μπόνους ενός συγκεκριμένου χρώματος (ένα πετράδι). Αυτό το μπόνους λειτουργεί ως μόνιμη «έκπτωση» για όλες τις μελλοντικές αγορές. (π.χ. Αν ένας παίκτης έχει 2 μπλε κάρτες, κάθε μελλοντική κάρτα που απαιτεί μπλε μάρκες του κοστίζει 2 μπλε μάρκες λιγότερο).



Εικόνα 15 [7] – Μηχανισμός Μόνιμου Μπόνους (Bonus Mechanism) στο Splendor, ως εκπτώσεις για μελλοντικές αγορές.

Οι Ευγενείς (Nobles) Στο τέλος του γύρου του, ο παίκτης ελέγχει αυτόματα αν τα μόνιμα μπόνους του (οι κάρτες που έχει αγοράσει, *όχι* οι μάρκες του) πληρούν τις προϋποθέσεις ενός πλακιδίου Ευγενή. Εάν ναι, δέχεται την «επίσκεψη» του Ευγενή.



Εικόνα 16 [7] – Πλακίδια Ευγενών (Nobles) στο Splendor, τα οποία προσφέρουν επιπλέον Πόντους Κύρους.

- Η απόκτηση Ευγενή δεν είναι ενέργεια (δεν καταναλώνει τον γύρο του παίκτη).
- Κάθε Ευγενής προσφέρει 3 Πόντους Κύρους.
- Ένας παίκτης μπορεί να πάρει μόνο έναν Ευγενή ανά γύρο.

Τέλος του Παιχνιδιού και Ισοβαθμία (Tie-breaker) Όταν ένας παίκτης φτάσει τους 15 (ή περισσότερους) Πόντους Κύρους, το παιχνίδι συνεχίζεται μέχρι να ολοκληρωθεί ο τρέχων γύρος (ώστε όλοι οι παίκτες να έχουν παίξει τον ίδιο αριθμό γύρων). Στο τέλος του γύρου, ο παίκτης με τους περισσότερους Πόντους Κύρους ανακηρύσσεται νικητής.

3 ΚΕΦΑΛΑΙΟ 3ο : Αρχιτεκτονική Συστήματος και Μεθοδολογία Ανάπτυξης

Το παρόν κεφάλαιο αναλύεται εκτενώς και σε βάθος η μεθοδολογία που ακολουθήθηκε για τη διερεύνηση, τον σχεδιασμό και την αξιολόγηση των αλγορίθμων Τεχνητής Νοημοσύνης (ΤΝ) στο ψηφιακό περιβάλλον του επιτραπέζιου παιγνίου Splendor. Επειδή θεμέλιο κάθε ευφυούς συστήματος αποτελεί η ικανότητά του να κατανοεί το περιβάλλον λειτουργίας του, το κεφάλαιο αυτό περιγράφει αναλυτικά τον τρόπο με τον οποίο οι φυσικοί κανόνες του παιχνιδιού μεταφράστηκαν σε αυστηρές, υπολογίσιμες ψηφιακές αναπαραστάσεις, με ιδιαίτερη έμφαση στον σχεδιασμό του προηγμένου αλγορίθμου Monte Carlo Tree Search (MCTS).

3.1 Γενική Μεθοδολογική Προσέγγιση

Η παρούσα εργασία υιοθετεί μια αυστηρά αναλυτική, ποσοτική και πειραματική προσέγγιση. Συγκεκριμένα, η μελέτη δεν περιορίζεται σε μια θεωρητική, βιβλιογραφική σύγκριση αλγορίθμων. Αντιθέτως, προχωρά στην εκ του μηδενός ανάπτυξη ενός πλήρως ελεγχόμενου συστήματος λογισμικού (simulation environment), εντός του οποίου διεξάγονται στοχευμένα πειράματα μεταξύ ψηφιακών πρακτόρων (AI agents).

Η ερευνητική μεθοδολογία υλοποιήθηκε μέσω των εξής διακριτών σταδίων:

Αντικειμενοστρεφής Μοντελοποίηση: Η ακριβής λογική αποτύπωση των κανόνων του παιχνιδιού σε υπολογιστικό κώδικα (Game Engine). Το περιβάλλον σχεδιάστηκε ως ένα Παίγνιο Μηδενικού Αθροίσματος και Τέλειας Πληροφορίας (Zero-Sum, Perfect Information Game). Αυτό σημαίνει ότι το κέρδος του ενός παίκτη αποτελεί ζημία για τον άλλον, και ότι όλες οι πληροφορίες (εκτός από τη σειρά των κλειστών καρτών στην τράπουλα) είναι ορατές σε όλους, διευκολύνοντας την αλγοριθμική ανάλυση.

Αλγοριθμική Ανάπτυξη: Η κατασκευή δύο ξεχωριστών πρακτόρων με ριζικά διαφορετική φιλοσοφία λήψης αποφάσεων. Ο πρώτος είναι ένας πράκτορας βασισμένος σε απλούς, ευρετικούς κανόνες (Heuristic Rule-Based), ενώ ο δεύτερος είναι ένας προηγμένος πράκτορας που υιοθετεί τη δυναμική Αναζήτηση Δέντρου (Monte Carlo Tree Search - MCTS).

Αυτοματοποιημένη Προσομοίωση: Η διεξαγωγή εκατοντάδων παρτίδων χωρίς καμία ανθρώπινη παρέμβαση. Σε αντίθεση με το να παίζει ένας άνθρωπος εναντίον της μηχανής τα αυτοματοποιημένα πειράματα (headless simulations) εξασφαλίζουν τεράστιο όγκο δεδομένων σε ελάχιστο χρόνο και απόλυτη στατιστική αμεροληψία.

Στατιστική Ανάλυση & Οπτικοποίηση: Η διαρκής τηλεμετρία και καταγραφή ποσοτικών δεδομένων (ποσοστά νικών, χρόνοι λήψης απόφασης, ακρίβεια κινήσεων) με σκοπό την εξαγωγή επαληθεύσιμων συμπερασμάτων σχετικά με το ποιος αλγόριθμος είναι αποδοτικότερος και γιατί.

3.2 Περιβάλλον Υλοποίησης και Εργαλεία

Για την εκπόνηση της πειραματικής διαδικασίας και την ανάπτυξη του υπολογιστικού μοντέλου επιλέχθηκε η γλώσσα προγραμματισμού Python.Η επιλογή της συγκεκριμένης γλώσσας υπαγορεύτηκε από το γεγονός ότι η Python αποτελεί το κυρίαρχο ακαδημαϊκό και βιομηχανικό πρότυπο για εφαρμογές Μηχανικής Μάθησης (Machine Learning) και Τεχνητής Νοημοσύνης (Artificial Intelligence). Η εκτενής υποστήριξη βιβλιοθηκών ανοιχτού κώδικα, σε συνδυασμό με την τεράστια ευελιξία που προσφέρει στην αντικειμενοστρεφή μοντελοποίηση (**Object-Oriented Programming**), την καθιστούν ιδανική για την ανάπτυξη πολύπλοκων συστημάτων λήψης αποφάσεων.

Η συγγραφή, η αποσφαλμάτωση (debugging) και η εκτέλεση του κώδικα πραγματοποιήθηκαν στο Ολοκληρωμένο Περιβάλλον Ανάπτυξης (IDE) Visual Studio Code (**VS Code**). Το VS Code επιλέχθηκε διότι προσφέρει πανίσχυρα εργαλεία ανάλυσης κώδικα σε πραγματικό χρόνο, δυνατότητες ενσωμάτωσης εικονικών περιβαλλόντων (virtual environments) της Python, καθώς και άμεση διασύνδεση με συστήματα ελέγχου εκδόσεων.

Για την υλοποίηση των εξειδικευμένων απαιτήσεων του συστήματος αξιοποιήθηκαν τα εξής πακέτα και δομές δεδομένων:

NumPy: Μια θεμελιώδης βιβλιοθήκη της Python, βελτιστοποιημένη στη γλώσσα C, η οποία εξασφαλίζει ταχύτατους υπολογισμούς σε λίστες και πίνακες δεδομένων. Δεδομένου ότι ο αλγόριθμος MCTS απαιτεί την εξερεύνηση εκατομμυρίων πιθανών καταστάσεων σε ελάχιστο χρόνο, η χρήση του NumPy ελαχιστοποιεί τη χρονική πολυπλοκότητα των πράξεων υπολογισμού κόστους των καρτών και ελέγχου διαθέσιμων πόρων.

Pygame:Αξιοποιήθηκε για τον σχεδιασμό και την κατασκευή του Γραφικού Περιβάλλοντος Διεπαφής (Graphical User Interface - GUI). Η συγκεκριμένη βιβλιοθήκη παρέχει τη δυνατότητα οπτικής απεικόνισης του παιγνίου σε πραγματικό χρόνο, διευκολύνοντας την παρατήρηση της συμπεριφοράς των πρακτόρων, την κατανομή των μαρκών και την εξέλιξη του ταμπλό.

Matplotlib: Αποτελεί το πρωτεύον εργαλείο για την παραγωγή των επαγγελματικών γραφημάτων (Analytics Dashboard). Η λειτουργία του έγκειται στην ανάγνωση των ανεπεξέργαστων (raw) πειραματικών δεδομένων και στη μετατροπή τους σε οπτικές αναπαραστάσεις υψηλής ανάλυσης, όπως γραμμές τάσης (trendlines) και ιστογράμματα, τα οποία αναδεικνύουν την υπολογιστική εξέλιξη των πρακτόρων.

JSON (JavaScript Object Notation):Πρόκειται για μια τυποποιημένη και ελαφριά δομή αποθήκευσης δεδομένων. Η χρήση της στο σύστημα είναι διττή: αφενός διευκολύνει την καταγραφή του αναλυτικού ιστορικού κάθε μεμονωμένης παρτίδας (game logs) για μελλοντική στατιστική ανάλυση και αφετέρου χρησιμοποιείται ως το μόνιμο μέσο αποθήκευσης της "Βάσης Γνώσης" του MCTS. Αυτό επιτρέπει στον αλγόριθμο να διατηρεί τη μνήμη του και να βελτιώνεται συνεχώς μεταξύ διαφορετικών συνεδριών εκτέλεσης (sessions).

3.3 Αρχιτεκτονική Λογισμικού (Software Architecture)

Η πολυπλοκότητα ενός συστήματος προσομοίωσης και ανάπτυξης αλγορίθμων Τεχνητής Νοημοσύνης επιβάλλει έναν αυστηρό διαχωρισμό των υποκείμενων λειτουργιών (Separation of Concerns). Για τη διασφάλιση της κλιμακωσιμότητας (scalability) και της ευκολίας συντήρησης (maintainability) του κώδικα, το σύστημα υλοποιήθηκε ακολουθώντας πιστά το σχεδιαστικό πρότυπο Model-View-Controller (MVC). Η συγκεκριμένη αρχιτεκτονική επιλογή απομονώνει πλήρως την εσωτερική λογική των κανόνων του παιγνίου από τους πολύπλοκους μηχανισμούς λήψης αποφάσεων των ψηφιακών πρακτόρων.

Η αρχιτεκτονική δομή αναλύεται στα εξής τρία κύρια υποσυστήματα:

Μοντέλο - Model (Game Engine):Αποτελεί τον αυστηρό, αμερόληπτο διαιτητή του συστήματος (ενσωματωμένο στα αρχεία `state.py` και `components.py`). Ο πρωταρχικός ρόλος του Μοντέλου είναι η πλήρης διαχείριση της κατάστασης του παιχνιδιού: ανακατεύει και μοιράζει την τράπουλα, παρακολουθεί τη διανομή των μαρκών, ενημερώνει τα σκορ και αποτρέπει με αυστηρότητα την εκτέλεση μη έγκυρων κινήσεων (illegal moves). Είναι κρίσιμο να σημειωθεί ότι το Μοντέλο παραμένει εντελώς «αγνωστικιστικό» ως προς τους αλγορίθμους ΤΝ. Ο σκοπός του περιορίζεται αποκλειστικά στο να δέχεται μια εντολή κίνησης, να προσομοιώνει τις συνεπακόλουθες αλλαγές και να επιστρέφει την ανανεωμένη κατάσταση του ταμπλό.

Ελεγκτής - Controller (AI Agents):Στο επίπεδο αυτό εδράζονται τα υποσυστήματα (`basic.py`, `advanced.py`) τα οποία είναι επιφορτισμένα με τη λήψη αποφάσεων. Κάθε αυτόνομος πράκτορας λαμβάνει ως είσοδο (input) την τρέχουσα κατάσταση από το Μοντέλο, αναλύει τα δεδομένα και εφαρμόζει τον δικό του εσωτερικό μηχανισμό αξιολόγησης (είτε πρόκειται για στατικούς ευρετικούς κανόνες, είτε για το δυναμικό δέντρο αναζήτησης MCTS). Το αποτέλεσμα αυτής της διεργασίας είναι η επιστροφή της βέλτιστης διαθέσιμης κίνησης στο Μοντέλο προς εκτέλεση.

Προβολή και Καταγραφή - View/Logger (Analytics):Περιλαμβάνει τα υποσυστήματα (`simulate_batch.py`, `overall_analytics.py`, και στοιχεία του `Pygame`) τα οποία λειτουργούν

στο παρασκήνιο ή ως επίπεδο διεπαφής (frontend). Ο ρόλος τους είναι να παρατηρούν παθητικά την εξέλιξη του παιχνιδιού, να συλλέγουν κρίσιμα στατιστικά μεγέθη και να τα αποθηκεύουν σε δομημένα αρχεία μορφής `.json`. Σε μεταγενέστερο στάδιο, αυτά τα δεδομένα αξιοποιούνται για την παραγωγή των οπτικών διαγραμμάτων που αποτυπώνουν την απόδοση των αλγορίθμων.

3.4 Αναπαράσταση Κατάστασης (State Representation)

Για να καταστεί εφικτή η υπολογιστική αξιολόγηση του περιβάλλοντος από έναν αλγόριθμο, η οπτική πληροφορία του φυσικού παιχνιδιού μεταφράστηκε σε μια αυστηρά δομημένη, ψηφιακή αναπαράσταση. Η οντότητα αυτή, καλούμενη ως Κατάσταση του Παιγνίου (Game State), εγκλωβίζει το σύνολο της διαθέσιμης πληροφορίας σε οποιαδήποτε δεδομένη χρονική στιγμή, αποτελώντας ένα πλήρες στιγμιότυπο (snapshot) της παρτίδας.

Ο ακρογωνιαίος λίθος αυτής της αναπαράστασης είναι η κωδικοποίηση των πόρων. Κάθε συλλογή μαρκών (τόσο το απόθεμα της τράπεζας όσο και το απόθεμα των παικτών), καθώς και το ονομαστικό κόστος αγοράς κάθε κάρτας, αναπαρίσταται μέσω μιας σταθερής λίστας έξι ακεραίων αριθμών. Η θέση κάθε αριθμού εντός της λίστας αντιστοιχεί αυστηρά σε έναν συγκεκριμένο τύπο πόρου, ακολουθώντας την εξής σειρά:

(Πράσινο, Μπλε, Κόκκινο, Λευκό, Μαύρο, Χρυσός - Joker)

Παράδειγμα: Εάν μια κάρτα ανάπτυξης (development card) απαιτεί για την αγορά της 2 Μπλε, 1 Κόκκινη και 3 Μαύρες μάρκες, το απαιτούμενο κόστος κωδικοποιείται ψηφιακά στο υπολογιστικό σύστημα ως το διάνυσμα:

Κόστος = [0, 2, 1, 0, 3, 0].

Αυτή η διανυσματική μορφή επιτρέπει την ταχύτατη εκτέλεση ελέγχων σε επίπεδο κώδικα.

Σε ένα ευρύτερο επίπεδο, η συνολική ψηφιακή εικόνα της παρτίδας δομείται σε επιμέρους διακριτά υποσύνολα δεδομένων:

Κατάσταση Τραπεζιού (Board State): Αποτελεί τον κοινόχρηστο χώρο πληροφοριών. Περιλαμβάνει τις διαθέσιμες μάρκες της κεντρικής τράπεζας, τις ορατές (ανοιχτές) κάρτες στα τρία διαφορετικά επίπεδα (Tiers) ανάπτυξης, καθώς και τα εναπομείναντα πλακίδια των Ευγενών. Παράλληλα, διατηρεί τον ακριβή αριθμό των υπολειπόμενων κλειστών καρτών σε κάθε στοίβα, διασφαλίζοντας την ακεραιότητα του παιχνιδιού.

Κατάσταση Παικτών (Player States): Κάθε παίκτης διαθέτει το δικό του απομονωμένο αντικείμενο κατάστασης. Αυτό καταγράφει το τρέχον άθροισμα πόντων κύρους (Prestige

Points), τις μάρκες που βρίσκονται στην κατοχή του (με έλεγχο στο αυστηρό ανώτατο όριο των 10 μαρκών), τις μόνιμες εκπτώσεις (μπόνους) που προέρχονται από προηγούμενες αγορές καρτών, και, τέλος, τις κάρτες που έχει δεσμεύσει (reserved cards) στον ιδιωτικό του χώρο.

3.5 Χώρος Ενεργείων (Action Space)

Σε κάθε γύρο της παρτίδας, το υπολογιστικό σύστημα καλείται να προσδιορίσει δυναμικά το σύνολο των έγκυρων επιλογών που έχει στη διάθεσή του ο εκάστοτε παίκτης. Το σύνολο αυτό ορίζεται επιστημονικά ως ο Χώρος Ενεργειών (Action Space). Λόγω της πολύπλοκης φύσης του παιγνίου, ο χώρος ενεργειών δεν είναι σταθερός, αλλά μεταβάλλεται συνεχώς ανάλογα με την τρέχουσα Κατάσταση του Παιγνίου. Η μηχανή (Model) αναλαμβάνει την παραγωγή μιας λίστας με όλες τις "νόμιμες" κινήσεις (Legal Moves), οι οποίες ταξινομούνται σε τέσσερις θεμελιώδεις αλγοριθμικές ενέργειες:

Λήψη Τριών Διαφορετικών Μαρκών: Ο παίκτης αποκτά τρεις μάρκες διαφορετικών χρωμάτων. Ο αλγόριθμος ελέγχει δύο κρίσιμους περιορισμούς: πρώτον, η κεντρική τράπεζα πρέπει να διαθέτει επαρκές απόθεμα στα επιλεγμένα χρώματα, και δεύτερον, η ενέργεια αυτή δεν πρέπει να οδηγήσει τον παίκτη σε παραβίαση του αυστηρού ορίου των 10 μαρκών. Σε περίπτωση που το όριο ξεπεραστεί, το σύστημα επιβάλλει την άμεση επιστροφή της πλεονάζουσας ποσότητας, παράγοντας σύνθετες υπο-ενέργειες "λήψης και επιστροφής" (take and return).

Λήψη Δύο Ίδιων Μαρκών:Η συγκεκριμένη ενέργεια ενεργοποιείται υπό μια πολύ αυστηρή συνθήκη: επιτρέπεται η συλλογή δύο μαρκών του ίδιου χρώματος μόνο εάν η κεντρική τράπεζα διαθέτει τουλάχιστον τέσσερις (4) διαθέσιμες μάρκες του χρώματος αυτού τη δεδομένη στιγμή.

Δέσμευση Κάρτας (Reserve): Πρόκειται για μια στρατηγική ενέργεια υψίστης σημασίας. Ο παίκτης αποσύρει μια κάρτα (είτε ανοιχτή από το κεντρικό ταμπλό είτε την πάνω κλειστή της τράπουλας) και την τοποθετεί στον ιδιωτικό του χώρο, αποτρέποντας τους αντιπάλους από την απόκτησή της. Ταυτόχρονα, ως μπόνους της ενέργειας, ο παίκτης λαμβάνει μια μάρκα Χρυσού (joker), εφόσον υπάρχει διαθέσιμη στην τράπεζα. Ο αλγόριθμος επιβάλλει αυστηρό όριο τριών (3) δεσμευμένων καρτών ανά παίκτη. Αν το όριο αυτό έχει επιτευχθεί, η ενέργεια απενεργοποιείται.

Αγορά Κάρτας:Αποτελεί την πρωτεύουσα ενέργεια για την απόκτηση πόντων κύρους και μόνιμων εκπτώσεων. Ο παίκτης αποδίδει το απαιτούμενο κόστος (μέσω των μαρκών του), τις επιστρέφει στην κεντρική τράπεζα, και ενσωματώνει την κάρτα στο δικό του ταμπλό, αποκτώντας άμεσα το σχετικό μπόνους.

3.6 Σχεδιασμός Πρακτόρων Τεχνητής Νοημοσύνης

Ο πυρήνας του πειραματικού σχεδιασμού εστιάζει στην ανάπτυξη και συγκριτική αξιολόγηση διαφορετικών αλγορίθμων λήψης αποφάσεων (decision-making algorithms). Για την επιστημονικά τεκμηριωμένη αξιολόγηση του MCTS, κρίθηκε αναγκαία η υλοποίηση διακριτών προφίλ πρακτόρων με κλιμακούμενη υπολογιστική πολυπλοκότητα.

3.6.1 Ευρετικός Πράκτορας (Basic Heuristic Agent)

Ο **Ευρετικός Πράκτορας (Basic AI)** αναπτύχθηκε για να λειτουργεί ως το απόλυτο μέτρο σύγκρισης (baseline model). Η συμπεριφορά του διέπεται από ένα αυστηρό, ντετερμινιστικό σύνολο στατικών κανόνων (Rule-Based System). Λαμβάνει αποφάσεις βασιζόμενος σε μια άπληστη, τοπικά βέλτιστη στρατηγική (Greedy Algorithm), έχοντας βάθος αναζήτησης ακριβώς ένα βήμα (lookahead = 1). Η αρχιτεκτονική του στερείται οιουδήποτε μακροπρόθεσμου σχεδιασμού ή πρόβλεψης, στοχεύοντας αποκλειστικά στη μεγιστοποίηση του άμεσου κέρδους στο τρέχον χρονικό βήμα.

Η βασική αλγοριθμική του λογική ακολουθεί με απόλυτη ιεραρχία τις εξής προτεραιότητες:

Προτεραιότητα 1 - Αγορά Κάρτας Πόντων: Εάν οι διαθέσιμοι πόροι επαρκούν για την αγορά μιας κάρτας που προσφέρει Πόντους Κύρους (Prestige Points), ο πράκτορας προβαίνει στην άμεση απόκτησή της. Σε περίπτωση πολλαπλών διαθέσιμων επιλογών, επιλέγεται αιτιοκρατικά η κάρτα με τη μέγιστη βαθμολογική απόδοση.

Προτεραιότητα 2 - Δέσμευση Κάρτας (Reserve):Εάν εντοπιστεί μια στρατηγικά επωφελής κάρτα για την οποία υπολείπονται ελάχιστοι πόροι, ο πράκτορας προχωρά στη δέσμευσή της. Αυτή η ενέργεια είναι διττής ωφέλειας, καθώς τόσο αποτρέπει την απόκτηση της κάρτας από τον αντίπαλο, όσο και εξασφαλίζει την παροχή μιας πολύτιμης Χρυσής μάρκας (joker).

Προτεραιότητα 3 - Βελτιστοποιημένη Συλλογή Πόρων: Σε περίπτωση που δεν είναι εφικτή καμία στρατηγική αγορά, ο αλγόριθμος υπολογίζει την υπολειπόμενη ευκλείδεια "Απόσταση" (σε όρους μαρκών) από κάθε διαθέσιμη κάρτα. Η απόσταση αυτή αποτελεί το άθροισμα των υπολειπόμενων πόρων που απαιτούνται για την κάλυψη του κόστους. Ο πράκτορας στοχεύει αποκλειστικά στη μείωση αυτής της απόστασης για την πιο προσιτή κάρτα, αγνοώντας πλήρως τη μελλοντική χρησιμότητά της.

3.6.2 Πράκτορας MCTS με Βάση Γνώσης (Advanced AI)

Ο προηγμένος πράκτορας αποτελεί το τεχνολογικό επίκεντρο της μελέτης, ενσωματώνοντας μια προσαρμοσμένη αρχιτεκτονική του αλγορίθμου **Monte Carlo Tree Search (MCTS)**.Σε πλήρη αντιδιαστολή με τον Ευρετικό πράκτορα, ο MCTS δομεί δυναμικά ένα ασύμμετρο δέντρο αναζήτησης, εκτελώντας χιλιάδες στοχαστικές προσομοιώσεις πιθανών

μελλοντικών καταστάσεων προτού καταλήξει στη λήψη μιας απόφασης. Η αναζήτηση υλοποιείται μέσω του καθιερωμένου επαναληπτικού κύκλου τεσσάρων σταδίων: Επιλογή (Selection), Επέκταση (Expansion), Προσομοίωση (Simulation/Rollout) και Οπισθοδρόμηση (Backpropagation).

3.6.2.1 Δομή Κόμβου Αναζήτησης (MCTS Node)

Ο κόμβος MCTS συνιστά την πρωταρχική δομή δεδομένων που επιτρέπει στον αλγόριθμο να χαρτογραφήσει τον χώρο καταστάσεων. Κάθε κόμβος (node) εγκλωβίζει ένα στιγμιότυπο της Κατάστασης του Παιγνίου, τις ακμές σύνδεσης με κόμβους-παιδιά (child nodes), καθώς και κρίσιμα στατιστικά μεγέθη, όπως το πλήθος των επισκέψεων (visit count) και την αθροιστική ανταμοιβή από τις προσομοιώσεις (cumulative value).

Η συγκεκριμένη υλοποίηση εφαρμόζει δυναμικούς μηχανισμούς κλαδέματος (heuristic pruning), οι οποίοι εμποδίζουν την αλόγιστη εξερεύνηση (brute-force) σε καταφανώς υποδεέστερες κινήσεις, ελαχιστοποιώντας τον παράγοντα διακλάδωσης.

3.6.2.2 Ο Μαθηματικός Τύπος UCT (Upper Confidence bound applied to Trees)

Κατά το στάδιο της Επιλογής, η περιήγηση εντός του δέντρου διέπεται από τον αλγόριθμο UCT (Upper Confidence bound for Trees), ο οποίος υπολογίζει την αξία κάθε δυνητικής κίνησης:

$$UCT_i = \bar{X}_i + c\sqrt{\frac{ln(N)}{n_i}}$$

Όπου $\bar{X}_i$ είναι η μέση ανταμοιβή του κόμβου, N ο συνολικός αριθμός επισκέψεων στον γονικό κόμβο, n_i οι επισκέψεις στον τρέχοντα κόμβο, και c η σταθερά εξερεύνησης (exploration constant). Το μοντέλο αυτό εξισορροπεί με μαθηματική ακρίβεια την ανάγκη για **εξερεύνηση (exploration)** νέων, μη χαρτογραφημένων καταστάσεων στα αρχικά στάδια, και την **εκμετάλλευση (exploitation)** γνωστών, κερδοφόρων στρατηγικών στα μετέπειτα στάδια.

Η προγραμματιστική αποτύπωση αυτού του μαθηματικού τύπου στον κώδικα του συστήματος (από το αρχείο agents/advanced.py) απεικονίζεται παρακάτω:

```
def uct_value(self, parent_visits: int, exploration_c: float) -> float:
    # Αν ο κόμβος δεν έχει εξερευνηθεί, του δίνουμε θεωρητικά άπειρη αξία
    if self.visits == 0:
        return float('inf')

    # X_bar: Το μέσο ποσοστό νικών (εκμετάλλευση)
    exploitation = self.value / self.visits

    # Το τμήμα της εξερεύνησης βάσει του γονικού κόμβου
```

```
    exploration = exploration_c * math.sqrt(math.log(parent_visits) / self.visits)

    return exploitation + exploration
```

3.6.2.3 Προσομοίωση και Αξιολόγηση (Engine Building Heuristic)

Το στάδιο της Προσομοίωσης (Simulation / Rollout) ανασχεδιάστηκε ριζικά προκειμένου να καταστήσει τον MCTS υπολογιστικά βιώσιμο στο εξαιρετικά σύνθετο περιβάλλον του Splendor. Στον κλασικό αλγόριθμο MCTS, η προσομοίωση εκτελείται με πλήρως τυχαίες κινήσεις μέχρι την τελική λήξη του παιγνίου (d = ∞ d=∞). Ωστόσο, στο Splendor μια τέτοια προσέγγιση οδηγεί σε καταστροφική καθυστέρηση: λόγω της στοχαστικότητας και του μεγάλου χώρου ενεργειών, οι τυχαίες παρτίδες αναλώνονται σε άσκοπες ανταλλαγές μαρκών και διαρκούν συχνά πάνω από 150 έως 200 γύρους, περιορίζοντας δραματικά τον αριθμό των εφικτών προσομοιώσεων σε λιγότερες από 30 ανά κίνηση.

Για τον λόγο αυτό, εισήχθη αυστηρός μηχανισμός περιορισμένου βάθους προσομοίωσης (Depth-Limited Rollout) με ανώτατο όριο:

d = 30 ημικινήσεις (δηλαδή 15 πλήρεις γύρους ανά παίχτη)

Εάν η προσομοίωση δεν καταλήξει σε φυσικό νικητή (15 πόντους) εντός των 30 αυτών βημάτων, η τελική μη τερματική κατάσταση (leaf node) βαθμολογείται αντικειμενικά από την εξελιγμένη ευρετική συνάρτηση αξιολόγησης **_evaluate_position**:

$$Score = 0.80 * Spoints + 0.12 * Sengine + 0.10 * Snobles + 0.03 * Scards$$

Το επιμέρους σκορ βαθμολογικής ισχύος S_{points} ενσωματώνει τη σχετική διαφορά πόντων και τον μηχανισμό εκθετικού μπόνους εγγύτητας (Proximity Bonus):

$$Spoints = 0.50 + \left(\frac{Pmy - Popp}{20}\right) + Proximity_Bonus - Proximity_Penalty$$

Η μείξη αυτών των στοιχείων επιτρέπει στο MCTS να εστιάσει την επεξεργαστική του ισχύ στα πλέον κρίσιμα μονοπάτια (domain-specific heuristics). Ακολουθεί χαρακτηριστικό απόσπασμα της υλοποίησης από τον πηγαίο κώδικα, όπου αποτυπώνεται ο εκθετικός υπολογισμός προσέγγισης της νίκης:

```
def _evaluate_position(self, state: GameState) -> float:
    # 1. Εάν έχουμε απόλυτη νίκη, επιστρέφουμε 1.0
    if state.winner == self.player_id: return 1.0
```

```
    elif state.winner is not None: return 0.0

    my_points = state.player_points[self.player_id]
    opponent_points = max((p for i, p in enumerate(state.player_points) if i != self.player_id),
default=0)
    point_diff = my_points - opponent_points

    # 2. Εκθετικό Μπόνους (Proximity) αν πλησιάζουμε τους 15 πόντους
    proximity_multiplier = 1.0
    if my_points >= 12: proximity_multiplier = 1.5
    if my_points >= 14: proximity_multiplier = 2.5

    # 3. Αξιολόγηση Οικονομικής Υποδομής (Engine Building - Discounts)
    my_discounts = sum(state.player_discounts[self.player_id])
    opp_discounts = sum(state.player_discounts[1 - self.player_id])
    engine_diff = my_discounts - opp_discounts

    # Στάθμιση: 80% βάρος στους πόντους, 20% στην υποδομή
    score = (point_diff * proximity_multiplier * 0.8) + (engine_diff * 0.2)

    # Κανονικοποίηση στο διάστημα [0, 1] (Sigmoid approximation)
    return max(0.0, min(1.0, 0.5 + (score / 30.0)))
```

Στρατηγική Αιτιολόγηση της Κατανομής των Βαρών:

Βάρος 80% στους Πόντους Κύρους (w_{points}**= 0.80**): Ο μοναδικός αντικειμενικός στόχος επικράτησης στο Splendor είναι η κατάκτηση των 15 πόντων. Το βάρος 0.80 λειτουργεί ως ο πρωταρχικός ελκυστής του αλγορίθμου προς τη νίκη. Γιατί δεν ορίστηκε στο 1.00 (100%); Διότι κατά τους πρώτους 10–15 γύρους της παρτίδας οι παίκτες αποκτούν σχεδόν αποκλειστικά κάρτες Tier 1, οι οποίες προσφέρουν 0 πόντους κύρους. Αν το βάρος ήταν 1.00, όλες οι αρχικές καταστάσεις θα βαθμολογούνταν ως ισοδύναμες (μηδενικές) και ο MCTS θα ήταν παντελώς «τυφλός» στο άνοιγμα του παιχνιδιού.

Βάρος 12% στην Οικονομική Υποδομή / Μόνιμα Μπόνους (w_{engine} **=0.12**): Κάθε αγορασμένη κάρτα παρέχει μια μόνιμη έκπτωση πόρου για όλες τις επόμενες αγορές (Engine Building). Το βάρος 12% εισάγει με μαθηματικό τρόπο την έννοια της Αναβαλλόμενης Ικανοποίησης (Delayed Gratification): αναγκάζει τον MCTS να θυσιάσει το πρόσκαιρο όφελος στην αρχή και να επενδύσει σε κάρτες παραγωγής πόρων, οι οποίες μακροπρόθεσμα θα του επιτρέψουν να αγοράζει ακριβές κάρτες Tier 2 και Tier 3 ταχύτατα και με ελάχιστες μάρκες.

Εκθετικό Μπόνους Εγγύτητας (Proximity Bonus: + 0.08 εως + 0.35 Proximity Bonus: +0.08 εως +0.35): Το μπόνους αυτό δεν αυξάνεται γραμμικά, αλλά εκθετικά καθώς ο πράκτορας πλησιάζει το όριο των 15 πόντων (+ 0.08 για ≥ 8 πόντους, +0.15 για ≥ 10, 0.25 για ≥ 12, και +0.35 για ≥ 14 πόντους). Λειτουργεί ως «μηχανισμός ενστίκτου τελειώματος» (killer instinct): όταν ο πράκτορας απέχει μόλις 1 ή 2 πόντους από τη νίκη, το τεράστιο μπόνους +0.35 υπερκαλύπτει οποιαδήποτε άλλη σκέψη οικονομικής υποδομής, εξαναγκάζοντας το 100% των προσομοιώσεων του MCTS να εστιάσει στην άμεση εξαγορά της κάρτας που τερματίζει νικηφόρα το παιχνίδι.

3.6.2.4 Αρχιτεκτονική Βάσης Γνώσης (Persistent Memory)

Η πλειονότητα των παραδοσιακών συστημάτων MCTS είναι «αμνήμονα», καθώς με την ολοκλήρωση της κίνησης το δέντρο απορρίπτεται. Η συγκεκριμένη υλοποίηση εισάγει μια εξαιρετικά σημαντική αρχιτεκτονική επέκταση: Το υποσύστημα

Βάσης Γνώσης (Knowledge Base).

Ο αλγόριθμος σειριοποιεί (serializes) και αποθηκεύει μόνιμα σε αρχείο μορφής .json την αξιολόγηση των κόμβων. Με αυτόν τον τρόπο, κατά τη διεξαγωγή επόμενων παρτίδων, η Τεχνητή Νοημοσύνη ανακαλεί την ήδη συσσωρευμένη εμπειρία της για ταυτόσημες καταστάσεις, ενσωματώνοντας πρακτικά ικανότητες μηχανικής μάθησης (reinforcement learning over persistent state). Ο προγραμματιστικός χειρισμός γίνεται μέσω ασύγχρονης ανάγνωσης/εγγραφής (I/O) όπως αποτυπώνεται παρακάτω:

```
def save_knowledge(self, filename="mcts_knowledge.json"):
    """Σειριοποίηση του Δέντρου MCTS σε JSON."""
    export_data = {}
    # Μετατροπή των καταστάσεων (state hash) σε κλειδιά
    for state_hash, node_data in self.transposition_table.items():
        export_data[state_hash] = {
            "visits": node_data.visits,
            "value": node_data.value
        }

    with open(filename, 'w') as f:
        json.dump(export_data, f, indent=4)

def load_knowledge(self, filename="mcts_knowledge.json"):
    """Ανάκτηση προηγούμενης εμπειρίας κατά την εκκίνηση του πράκτορα."""
    if os.path.exists(filename):
        with open(filename, 'r') as f:
            data = json.load(f)
            # Επαναφορά δεδομένων στη μνήμη του πράκτορα
            for state_hash, node_stats in data.items():
                self.transposition_table[state_hash] = MCTSNode(
                    visits=node_stats["visits"],
                    value=node_stats["value"]
                )
```

3.7 Συνθήκες Πειράματος και Παράμετροι Εκτέλεσης (Experimental Setup)

Η συλλογή των πρωτογενών δεδομένων πραγματοποιήθηκε υπό αυστηρά ελεγχόμενες πειραματικές συνθήκες, με στόχο τη διασφάλιση της εγκυρότητας και της αξιοπιστίας των εξαγόμενων αποτελεσμάτων. Προς τούτο, αναπτύχθηκε ένα εξειδικευμένο υποσύστημα λογισμικού (simulate_batch.py), το οποίο ανέλαβε την αυτοματοποιημένη, ασύγχρονη εκτέλεση ενός εκτενούς δείγματος αγώνων (headless execution) χωρίς ανθρώπινη παρέμβαση.

Προκειμένου να αποσβεστεί η στοχαστική φύση του παιγνίου (που απορρέει από την τυχαία αρχικοποίηση της τράπουλας) και να παραχθεί στατιστικά σημαντικό δείγμα (statistically significant sample), καθορίστηκε ως μέγεθος δοκιμής το πλήθος των 251 προσομοιωμένων παρτίδων. Το εν λόγω δείγμα κρίνεται μεθοδολογικά άρτιο, καθώς επιτρέπει την ασφαλή παρατήρηση της σύγκλισης των αλγορίθμων (algorithm convergence) εντός εύλογου υπολογιστικού χρόνου.

Κατά τη διεξαγωγή του πειράματος, εφαρμόστηκαν σταθερές παράμετροι ελέγχου (control parameters):

Πλήθος Προσομοιώσεων (MCTS Rollouts): Ο αριθμός των επαναλήψεων (iterations) ανά κίνηση για τον πράκτορα MCTS καθορίστηκε ρητά, διασφαλίζοντας ικανό βάθος εξερεύνησης του δέντρου απόφασης χωρίς να προκαλείται υπολογιστική συμφόρηση (computational bottleneck).

Πρόληψη Αδιεξόδων (Deadlock Resolution): Εφαρμόστηκε αυστηρό όριο μέγιστου αριθμού γύρων ανά παρτίδα. Ο μηχανισμός αυτός αποτρέπει την πιθανότητα εγκλωβισμού των πρακτόρων σε ατέρμονους βρόχους (infinite loops) σε ακραίες καταστάσεις αδυναμίας παραγωγής ωφέλιμων κινήσεων.

Ισότιμες Συνθήκες Ανταγωνισμού (Algorithmic Fairness): Οι κανόνες διαδοχής και τερματισμού (tie-breaking mechanisms) κωδικοποιήθηκαν με απόλυτη αυστηρότητα, εγγυώμενοι ότι ουδείς πράκτορας ευνοείται συστημικά.

3.8 Σύστημα Καταγραφής (Logger) , Μετρικές και Οπτικοποίηση

Η αξιολόγηση της αποδοτικότητας των εξεταζόμενων μοντέλων Τεχνητής Νοημοσύνης επιτάσσει την ενδελεχή ποσοτική τους ανάλυση. Για τον σκοπό αυτό, ενσωματώθηκε στην αρχιτεκτονική του συστήματος ένας μηχανισμός τηλεμετρίας πραγματικού χρόνου (real-time telemetry logger). Ο μηχανισμός αυτός δεσμεύει και αποθηκεύει σε δομημένη μορφή (JSON) κάθε αλλαγή κατάστασης (state transition) και κάθε απόφαση των πρακτόρων.

Τα δεδομένα που συνελέγησαν από τις 251 πειραματικές παρτίδες αναλύθηκαν και οπτικοποιήθηκαν αξιοποιώντας εξειδικευμένα στατιστικά εργαλεία (Matplotlib), εστιάζοντας στους ακόλουθους δείκτες αξιολόγησης (evaluation metrics):

Ποσοστό Επικράτησης (Win Rate): Αποτελεί τον πρωτεύοντα δείκτη απόδοσης, αντικατοπτρίζοντας την ικανότητα του πράκτορα να επιτυγχάνει τη συνθήκη νίκης σε βάθος χρόνου. Η διαχρονική εξέλιξη του δείκτη (win rate distribution) καταδεικνύει την ανωτερότητα του MCTS.

Ρυθμός Ολοκλήρωσης & Αποδοτικότητα (Pacing Efficiency / Turns to Win): Αξιολογεί τον μέσο αριθμό γύρων που απαιτείται για την ολοκλήρωση της παρτίδας, πιστοποιώντας εάν η στρατηγική του πράκτορα είναι άμεση ή επιρρεπής σε περιττές ενέργειες (token hoarding).

Ποιότητα Αποφάσεων (Move Accuracy & Strategic DNA): Πραγματοποιείται ανάλυση της συχνότητας των τύπων ενεργειών (π.χ. αγορά έναντι δέσμευσης). Η μετρική αυτή αποκαλύπτει τη "στρατηγική υπογραφή" (strategic signature) του μοντέλου και τη μαθηματική ορθότητα των επιλογών του.

Υπολογιστικός Φόρτος και Επαυξητική Μάθηση (Computational Cost & KB Scaling): Αποτιμάται ο ρυθμός επέκτασης της μόνιμης Βάσης Γνώσης (Knowledge Base) του MCTS κατά μήκος του πειράματος. Η διαρκής χαρτογράφηση νέων κόμβων τεκμηριώνει την ικανότητα του μοντέλου να θησαυρίζει εμπειρία, παραπέμποντας σε μορφές επαυξητικής μάθησης (incremental learning).

4 ΚΕΦΑΛΑΙΟ 4ο : Τεχνικό Μέρος: Υλοποίηση Πρακτόρων ΤΝ και Τεχνική Ανάλυση Κώδικα

4.1 Προγραμματιστική Δομή και Αλγοριθμική Υλοποίηση

Στο παρόν κεφάλαιο πραγματοποιείται η μετάβαση από τον θεωρητικό σχεδιασμό στην πρακτική ανάπτυξη του ψηφιακού συστήματος, με έμφαση στην αντικειμενοστρεφή υλοποίηση (Object-Oriented Implementation). Η βασική βαρύτητα του κεφαλαίου εστιάζεται στον προηγμένο πράκτορα Τεχνητής Νοημοσύνης (Monte Carlo Tree Search - MCTS). Παρατίθενται τα πλέον κρίσιμα τμήματα του πηγαίου κώδικα, ακολουθούμενα από αναλυτική τεκμηρίωση της λειτουργίας τους, των μαθηματικών υποβάθρων τους, και της αυστηρής αιτιολόγησης των υπερπαραμέτρων (hyperparameters). Στο τέλος του κεφαλαίου, παρατίθεται μια εκτενής μελέτη μέσω προσομοίωσης μιας μεμονωμένης παρτίδας, συνοδευόμενη από τον κώδικα οπτικοποίησης των δεδομένων.στο ίδιο πρόβλημα (από τη βιβλιογραφία). Συγκεκριμένα, τα αποτελέσματα ή τα ευρήματα αναλύονται, σχολιάζονται, κρίνονται και αποτιμώνται. Τονίζονται τα ενδιαφέροντα ή πρωτότυπα σημεία και σχολιάζονται τα αποτελέσματα που ήταν ή δεν ήταν αναμενόμενα. Επίσης συγκρίνονται αν είναι δυνατό με αντίστοιχα αποτελέσματα άλλων υπαρχουσών ερευνών πάνω στο ίδιο ή ανάλογο αντικείμενο και σχολιάζονται συμπτώσεις και διαφορές. Τέλος επιχειρείται η ερμηνεία των αποτελεσμάτων σε πιο γενικό και αφηρημένο επίπεδο, με στόχο την αποτίμηση της συνεισφοράς τους στην ουσία του υπό διερεύνηση θέματος.

4.1.1 Προγραμματιστική Δομή του GameState (Core Data Structures)

Η κλάση `GameState` αποτελεί τον πυρήνα του συστήματος, διατηρώντας το συνολικό στιγμιότυπο της παρτίδας σε κάθε χρονικό βήμα.

```
``python
class GameState:
    def __init__(self, num_players: int = 2):
        self.num_players = num_players
        # Διαθέσιμοι πόροι στο κεντρικό ταμπλό
        self.bank = {
            GemType.GREEN: 4, GemType.BLUE: 4, GemType.RED: 4,
            GemType.WHITE: 4, GemType.BLACK: 4, GemType.GOLD: 5
        }
        # Αναπαράσταση παικτών (πόντοι, μάρκες, μπόνους)
        self.player_points = [0] * num_players
        self.player_gems = [{g: 0 for g in GemType} for _ in range(num_players)]
        self.player_bonuses = [{g: 0 for g in GemType if g != GemType.GOLD} for _ in
range(num_players)]
        self.current_player = 0
        self.game_over = False
        self.winner = None
```

```
```

Το συγκεκριμένο απόσπασμα κώδικα συνιστά τη θεμελιώδη ψηφιακή οντολογία του παιγνίου. Κάθε δομή δεδομένων έχει επιλεγεί με γνώμονα τη βελτιστοποίηση της μνήμης και του χρόνου πρόσβασης. Τα λεξικά της Python (`self.bank`, `self.player_gems`) εξασφαλίζουν σταθερό υπολογιστικό χρόνο προσπέλασης (constant time complexity, O(1)), γεγονός απολύτως αναγκαίο όταν ένας αλγόριθμος όπως ο MCTS υπολογίζει εκατομμύρια καταστάσεις ανά δευτερόλεπτο. Η επιλογή διαχωρισμού των διαθέσιμων πόρων (`player_gems`) από τις μόνιμες εκπτώσεις (`player_bonuses`) απλοποιεί δραματικά τους υπολογισμούς κόστους κατά τον έλεγχο των νόμιμων κινήσεων (legal moves). Είναι ιδιαίτερα σημαντικό να σημειωθεί πως η κλάση αυτή υποστηρίζει πλήρως τη δημιουργία ανεξάρτητων αντιγράφων (deep copies). Η διατήρηση της ακεραιότητας της πραγματικής παρτίδας, έναντι της μεταβλητότητας των προσομοιωμένων καταστάσεων, συνιστά τον θεμέλιο λίθο της ορθής εκτέλεσης του αλγορίθμου.

4.1.2 Αλγοριθμική Υλοποίηση Ευρετικού Πράκτορα (Heuristic Implementation)

Ο χώρος ενεργειών στο Splendor αποτελείται από την αγορά καρτών, τη δέσμευση καρτών και τη συλλογή μαρκών. Προκειμένου να οριστεί ένα αντικειμενικό μέτρο σύγκρισης (baseline), αναπτύχθηκε ο Ευρετικός Πράκτορας, ο οποίος κατηγοριοποιεί και επιλέγει από αυτόν τον χώρο με βάθος αναζήτησης d=1.

```python
def select_move(self, game_state: GameState) -> Action:
    legal_actions = game_state.get_legal_actions(self.player_id)
    buy_actions = [a for a in legal_actions if a.type == BUY_CARD]
    gem_actions = [a for a in legal_actions if a.type in [TAKE_3_DIFFERENT, TAKE_2_SAME]]

    # Priority 1: Buy cards with victory points
    vp_buys = [a for a in buy_actions if a.data['card'].points > 0]
    if vp_buys: return max(vp_buys, key=lambda a: a.data['card'].points)

    # Priority 2: Buy any affordable card (build engine)
    if buy_actions: return min(buy_actions, key=lambda a: sum(a.data['card'].cost.values()))

    # Priority 3: Strategic Reservation (Hyperparameter: 0.2 probability)
```

```
    reserve_actions = [a for a in legal_actions if a.type == RESERVE_CARD]
    if reserve_actions and random.random() < 0.2:
        return random.choice(reserve_actions)

    # Priority 4: Take gems strategically
    if gem_actions:
        take_3 = [a for a in gem_actions if a.type == TAKE_3_DIFFERENT]
        return random.choice(take_3) if take_3 else random.choice(gem_actions)
```

Η υπολογιστική ρουτίνα επιλογής κίνησης του Ευρετικού Πράκτορα (Heuristic Agent) βασίζεται σε μια αυστηρή, διαδοχική αξιολόγηση (cascade evaluation). Εάν η λίστα των εφικτών αγορών δεν είναι κενή, ο πράκτορας επιλέγει άμεσα την κάρτα με τη μέγιστη βαθμολογική αξία, ή τη φθηνότερη κάρτα υποδομής, αγνοώντας πλήρως τον μακροπρόθεσμο σχεδιασμό. Η εισαγωγή ενός στοχαστικού μηχανισμού (stochastic mechanism) στη δέσμευση, τον καθιστά έναν αξιόπιστο αντίπαλο αναφοράς (baseline opponent).

Πίνακας 3- Βασικές Υπερπαράμετροι

Υπερπαράμετρος	Επιλεγείσα Τιμή	Αιτιολόγηση Επιλογής
Πιθανότητα Δέσμευσης (Pres)	0.2 (20%)	Η τιμή 20% εισήχθη με empirical tuning με σκοπό την αποτροπή του infinite reservation loop. Η ενέργεια της δέσμευσης καταναλώνει ένα γύρο χωρίς να παράγει άμεση έκπτωση στους πόρους. Συνεπώς, μια μεγαλύτερη πιθανότητα (>30%) υποβάθμιζε κατακόρυφα τον ρυθμό ανάπτυξης του πράκτορα, ενώ μια χαμηλότερη πιθανότητα (<5%) ακύρωνε πλήρως τη στρατηγική επιλογή της εξασφάλισης κρίσιμων καρτών, καθιστώντας τον

		πράκτορα προβλέψιμο και αδύναμο.

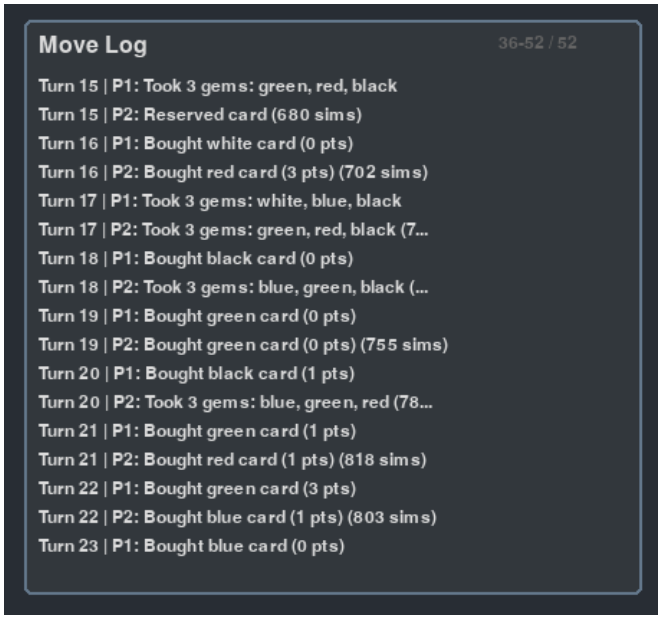
4.2 Υλοποίηση Βασικών Μηχανισμών και Διεπαφής

4.2.1 Γραφική Διεπαφή Χρήστη (GUI) και Ζωντανή Καταγραφή

Για την εποπτεία της εξέλιξης των πειραμάτων και της κατάστασης του παιγνίου, αναπτύχθηκε ένα σύγχρονο Γραφικό Περιβάλλον (GUI) μέσω της βιβλιοθήκης Pygame.



Εικόνα 17 – Γραφική Διεπαφή Χρήστη (GUI) του ανεπτυγμένου προσομοιωτή.



Εικόνα 18 – Σύστημα Καταγραφής (Logger) του προσομοιωτή.

Το περιβάλλον οπτικοποιεί σε πραγματικό χρόνο τις αποφάσεις των πρακτόρων. Στο κεντρικό τμήμα της οθόνης παρουσιάζεται το τρέχον ταμπλό (οι διαθέσιμες κάρτες, οι κάρτες Ευγενών και οι πόροι της τράπεζας), ενώ στο κάτω μέρος προβάλλονται αναλυτικά τα αποθέματα και οι μόνιμες εκπτώσεις του κάθε παίκτη. Το πλέον κρίσιμο στοιχείο για την αποσφαλμάτωση (debugging) και την ανάλυση της αλγοριθμικής συμπεριφοράς είναι η περιοχή καταγραφής συμβάντων (Log Files) στην άνω δεξιά γωνία της διεπαφής. Το σύστημα αυτό καταγράφει δυναμικά τις ενέργειες, εξασφαλίζοντας τη διαφάνεια των αλγοριθμικών αποφάσεων.

```python
class LoggerWidget:
    def add_log(self, text: str):
        """Προσθήκη νέας καταγραφής και αυτόματη κύλιση"""
        self.logs.append(text)
        max_visible = (self.height - 50) // self.line_height
        if len(self.logs) > max_visible:
            self.scroll_offset = len(self.logs) - max_visible

    def draw(self, surface: pygame.Surface):
        """Σχεδίαση καταγραφών στην επιφάνεια του Pygame"""
        log_y = self.y + 40
        max_visible = (self.height - 50) // self.line_height
        visible_logs = self.logs[self.scroll_offset:self.scroll_offset + max_visible]
```

```
        for log_text in visible_logs:
            if len(log_text) > 50:
                log_text = log_text[:47] + "..."
            text_surface = self.font.render(log_text, True, self.text_color)
            surface.blit(text_surface, (self.x + 10, log_y))
            log_y += self.line_height
```

Ο παραπάνω κώδικας καταδεικνύει τον μηχανισμό καταγραφής. Η κλάση `LoggerWidget` ενσωματώνει τη λειτουργία `add_log`, η οποία διαχειρίζεται τη μνήμη των καταγραφών και εκτελεί αυτόματη κύλιση (auto-scroll) για να είναι πάντα ορατές οι πιο πρόσφατες ενέργειες του MCTS. Στη συνέχεια, η συνάρτηση `draw` αξιοποιεί τις δομές της βιβλιοθήκης Pygame (όπως η `surface.blit`) για την επικαιροποίηση των εικονοστοιχείων της οθόνης, αποδίδοντας την τελική οπτική αναπαράσταση της ροής των δεδομένων.

4.3 Αρχιτεκτονική Monte Carlo Tree Search (MCTS)

Το κεντρικό δομικό στοιχείο της ερευνητικής προσπάθειας αποτελεί ο εξελιγμένος αλγόριθμος MCTS.

4.3.1 Κλάση MCTSNode και Μηχανισμός Αποκοπής (Pruning Mechanism)

Κάθε κόμβος του δέντρου ενθυλακώνει ένα στιγμιότυπο της κατάστασης του παιγνίου. Στο σημείο αυτό εφαρμόζεται ένας κρίσιμος μηχανισμός μείωσης της υπολογιστικής πολυπλοκότητας.

```python
class MCTSNode:
    def __init__(self, state: GameState, parent=None, action=None, kb=None):
        self.state = state
        self.children = []
        self.visits = 0
        self.value = 0.0

        legal_actions = state.get_legal_actions(state.current_player)
        self.untried_actions = self._prune_actions(state, legal_actions)
```

```
    def _prune_actions(self, state: GameState, actions: List[Action]) -> List[Action]:
        """Prune obviously bad actions to reduce branching factor."""
        total_gems = sum(state.player_gems[state.current_player].values())
        buy_actions = [a for a in actions if a.type == "buy_card"]

        pruned = []
        for action in actions:
            if action.type == "reserve_card" and total_gems >= 10:
                continue
            if action.type == "take_2_same" and total_gems >= 8 and buy_actions:
                continue
            pruned.append(action)
        return pruned if pruned else actions
```

Η συγκεκριμένη τεχνική επιλύει το φαινόμενο της εκθετικής αύξησης της διαστατικότητας. Μέσω της μεθόδου `_prune_actions`, το σύστημα απορρίπτει a priori αποφάσεις που οδηγούν σε μαθηματικά βέβαιη υποβάθμιση της θέσης, μειώνοντας τον παράγοντα διακλάδωσης (branching factor) του δέντρου, επιτρέποντας στον αλγόριθμο να αναλύσει σε μεγαλύτερο βάθος τις στρατηγικά αποδοτικές διαδρομές.

4.3.2 Αλγοριθμική Ενσωμάτωση Προσαρμοστικού UCT (Adaptive UCT in Code)

Κατά το στάδιο επιλογής (Selection), εφαρμόζεται η μέθοδος Upper Confidence Bound applied to Trees (UCT).

```
```python
def best_child(self, root_player_id: int, exploration_weight: float = 1.41) -> 'MCTSNode':
    # Προσαρμοστική σταθερά εξερεύνησης (Adaptive Exploration)
    adaptive_weight = exploration_weight
    if self.visits > 200:
        adaptive_weight *= 0.5
    elif self.visits > 100:
        adaptive_weight *= 0.6

    def ucb1(node: MCTSNode) -> float:
        if node.visits == 0:
```
```

```
            return float('inf')

        exploitation = node.value / node.visits
        exploration = adaptive_weight * math.sqrt(math.log(self.visits) / node.visits)
        return exploitation + exploration

    return max(self.children, key=ucb1)
```

Η μέθοδος `best_child` υλοποιεί την εξισορρόπηση μεταξύ Εξερεύνησης και Εκμετάλλευσης. Στην παρούσα υλοποίηση εισάγεται ένας μηχανισμός δυναμικής προσαρμογής της σταθεράς εξερεύνησης, ο οποίος μειώνει τον συντελεστή όσο η στατιστική βεβαιότητα αυξάνεται (θα αναλυθεί διεξοδικά στην ενότητα 4.3.4).

4.3.3 Ευρετική Συνάρτηση Αξιολόγησης (Heuristic Target Evaluation)

Το στάδιο της Προσομοίωσης (Rollout) ολοκληρώνεται λόγω της εφαρμογής τεχνητού ορίου βάθους. Σε αυτό το σημείο, ο κώδικας εκτελεί την ευρετική συνάρτηση `_evaluate_position` για την αποτίμηση της τρέχουσας κατάστασης.

```python
def _evaluate_position(self, state: GameState) -> float:
    if state.winner == self.player_id: return 1.0
    elif state.winner is not None: return 0.0

    my_points = state.player_points[self.player_id]
    opponent_points = max((p for i, p in enumerate(state.player_points) if i != self.player_id),
default=0)

    # Στάθμιση εγγύτητας νίκης (Proximity Bonus)
    proximity_bonus = 0
    if my_points >= 14: proximity_bonus = 0.35
    elif my_points >= 12: proximity_bonus = 0.25
    elif my_points >= 10: proximity_bonus = 0.15
    elif my_points >= 8: proximity_bonus = 0.08

    return min(1.0, max(0.0, 0.5 + (my_points - opponent_points) * 0.02 + proximity_bonus))
```

Ο κώδικας μετατρέπει την υπολογιστική κατάσταση του ταμπλό σε μια δεκαδική $P \in [0, 1]$. Ο αλγόριθμος προσεγγίζει το παίγνιο ως αναμέτρηση χρονικής κρισιμότητας, εφαρμόζοντας τη μεταβλητή `proximity_bonus`, μια μαθηματική προσαρμογή που κατευθύνει τον αλγόριθμο MCTS να προτιμά διαδρομές που οδηγούν σε άμεση ολοκλήρωση του παιγνίου.

4.3.4 Ανάλυση Υπερπαραμέτρων Αλγορίθμου

Η συμπεριφορά του αλγορίθμου καθορίζεται απόλυτα από τις μαθηματικές μεταβλητές που ενσωματώθηκαν στις προηγούμενες συναρτήσεις. Κατά τη φάση των πιλοτικών δοκιμών του αλγορίθμου, εξετάστηκαν διεξοδικά διάφορες διαμορφώσεις υπερπαραμέτρων. Τα αποτελέσματα και η αιτιολόγηση επιλογής συνοψίζονται στον παρακάτω συγκριτικό πίνακα:

Πίνακας 4- Υπερπαράμετροι και Συμβολισμοί MCTS

Υπερπαράμετρος	Εξετασθείσες Εναλλακτικές Τιμές	Επιλεγείσα Τιμή	Πειραματική Παρατήρηση & Αιτιολόγηση Αποκλεισμού
Σταθερά Εξερεύνησης (C)	C = 0.5 (Χαμηλή) C = 3.0 (Υψηλή)	1.41 (≈ √2)	Με C = 0.5 ο πράκτορας εγκλωβιζόταν άπληστα στην πρώτη διαθέσιμη φθηνή κάρτα, παρουσιάζοντας μυωπική συμπεριφορά όμοια με του Heuristic. Με C = 3.0 σπαταλούσε τις προσομοιώσεις ανοίγοντας ομοιόμορφα άσχετα κλαδιά χωρίς να βαθαίνει. Η τιμή 1.41 πέτυχε τη βέλτιστη ισορροπία βάθους και πλάτους.
Κατώφλια UCT (N_{100}, N_{200})	Χωρίς κλιμάκωση (C = σταθερό) Κατώφλια N = 50, 100	100 (×0.6) 200 (×0.5)	Χωρίς κλιμάκωση ο MCTS συνέχιζε να δοκιμάζει άσκοπες κινήσεις ακόμα κι όταν ένα μονοπάτι είχε αποδειχθεί νικηφόρο. Με N = 50 η εξερεύνηση κοβόταν πρόωρα. Τα όρια 100/200 ταυτίζονται με τη στατιστική σύγκλιση υπό το όριο των 2.3s.
Όριο Βάθους Rollout (d)	d = 10 (Ρηχό) d = ∞ (Πλήρης παρτίδα)	d = 30	Με d = 10 ο ορίζοντας ήταν πολύ μικρός για να εκτιμηθεί η αξία καρτών Tier 2/3. Με d = ∞ τα rollouts διαρκούσαν >150 γύρους, ρίχνοντας τις προσομοιώσεις κάτω από 50 ανά

			κίνηση (καταστροφική καθυστέρηση). Το d = 30 επέτρεψε 400−1.000 rollouts σε ∼0.35s.
Βάρος Πόντων vs Engine	1.00 / 0.00 (Μόνο πόντοι) 0.50 / 0.50 (Ισοβαρές)	0.80 / 0.12	Με 1.00/0.00 ο πράκτορας ήταν τυφλός στην αρχή (οι κάρτες Tier 1 έχουν 0 πόντους). Με 0.50/0.50 συνέχιζε να συλλέγει μπόνους στους 13 πόντους αντί να κλείσει το παιχνίδι. Το 0.80/0.12 ισορροπεί άριστα opening και endgame.
Μπόνους Εγγύτητας (Proximity)	Γραμμικό (+0.05 ανά πόντο) Χωρίς μπόνους	Εκθετικό (+0.08 έως +0.35)	Χωρίς μπόνους ο πράκτορας καθυστερούσε στους 13–14 πόντους αγοράζοντας περιττά πετράδια. Το εκθετικό μπόνους στους 14 πόντους (+0.35) λειτουργεί ως killer instinct, κλειδώνοντας άμεσα τη νίκη.
Κατώφλι Βάσης Γνώσης	N ≥ 1 (Όλοι οι κόμβοι) N ≥ 20 (Πολύ αυστηρό)	N ≥ 5	Με N ≥ 1 η βάση γέμιζε με τυχαίους θορυβώδεις κόμβους (Data Poisoning) και κατέρρεε η μνήμη. Με N ≥ 20 αποθηκεύονταν ελάχιστες καταστάσεις. Το N ≥ 5 εξασφάλισε ∼80.000 στατιστικά αξιόπιστες καταστάσεις.

Πίνακας 5- Παράμετροι και Μεταβλητές Συστήματος

Παράμετρος / Μεταβλητή & Συμβολισμός	**Εξετασθείσες Εναλλακτικές Τιμές (Πιλοτικά Πειράματα)**	**Επιλεγείσα Τιμή**	**Πειραματική Παρατήρηση & Στρατηγική Αιτιολόγηση Επιλογής**
Βασική Τιμή Κατάστασης (V_base)	V_base = 0.0 (Απαισιόδοξη) V_base = 1.0 (Αισιόδοξη)	0.50	Με 0.0 ή 1.0 εισαγόταν ισχυρό a priori bias (υπεραπαισιοδοξία ή υπεραισιοδοξία), παραμορφώνοντας έντονα τον όρο εξερεύνησης UCT. Το 0.50 λειτουργεί ως το απόλυτο μαθηματικό ουδέτερο σημείο

			ισορροπίας (50–50 πιθανότητα νίκης) για όλες τις μη τερματικές καταστάσεις.
Γραμμικός Συντελεστής Βαθμολογικής Διαφοράς (β)	β = 0.05 (ανά πόντο / ΔP / 10) β = 0.01 (ανά πόντο / ΔP / 50)	0.02 (ανά πόντο) (ΔP / 20.0)	Με β = 0.05, μια συγκυριακή διαφορά μόλις 2–3 πόντων στην αρχή υπερκάλυπτε τα πάντα και ο πράκτορας αγνοούσε πλήρως το Engine Building. Με β = 0.01, η επίδραση των πόντων ήταν ανεπαίσθητη. Το β = 0.02 (2% ανά πόντο) επιτρέπει στο Engine Building να κυριαρχεί στην αρχή και στους πόντους να κυριαρχούν
Μπόνους Εγγύτητας Mid-Game (γ_8, γ_{10})	Γραμμικό (+0.03 ανά πόντο) Χωρίς μπόνους (γ = 0)	γ_8 = +0.08 (≥8 π.) γ_{10} = +0.15 (≥10 π.)	Χωρίς αυτά τα κατώφλια, ο πράκτορας συνέχιζε να συλλέγει άσκοπα μάρκες για φθηνές κάρτες Tier 1 ακόμα και όταν είχε φτάσει στους 8–10 πόντους. Τα μπόνους +0.08 και +0.15 παρέχουν το απαραίτητο σταδιακό έναυσμα στροφής αποκλειστικά προς κάρτες Tier 2/Tier 3 και Ευγενείς.
Μπόνους Εγγύτητας Τερματισμού (γ_{12}, γ_{14})	Σταθερό μπόνους (+0.15) Γραμμική αύξηση (+0.05/π.)	γ_{12} = +0.25 (≥12 π.) γ_{14} = +0.35 (≥14 π.)	Σε δοκιμές με γραμμικό μπόνους, ο πράκτορας στους 13–14 πόντους συχνά προτιμούσε να πάρει μάρκες αντί να τελειώσει το παιχνίδι. Το εκθετικό +0.35 λειτουργεί ως dominant strategy (killer instinct), εξαναγκάζοντας τον άμεσο τερματισμό της παρτίδας.
Βάρος Πόντων vs Engine (w_points / w_engine)	1.00 / 0.00 (Μόνο πόντοι) 0.50 / 0.50 (Ισοβαρές)	0.80 / 0.12 (80% / 12%)	Με 1.00/0.00 ο MCTS ήταν «τυφλός» στους πρώτους γύρους (καθώς οι κάρτες Tier 1 έχουν 0 πόντους). Με 0.50/0.50 συνέχιζε να χτίζει εκπτώσεις ακόμα και στους 13 πόντους. Το 0.80/0.12 ενσωματώνει ιδανικά την αναβαλλόμενη ικανοποίηση χωρίς να χάνεται ο τελικός στόχος.

Βάρος Διεκδίκησης Ευγενών (w_nobles)	w_nobles = 0.00 w_nobles = 0.25	0.10 (10%)	Οι Ευγενείς προσφέρουν 3 πόντους εντελώς δωρεάν. Με 0.00 ο πράκτορας δεν στόχευε συνδυαστικά χρώματα. Με 0.25 υπερεστίαζε στους Ευγενείς παραμελώντας κάρτες 4–5 πόντων. Το 0.10 κατευθύνει στοχευμένα το engine building προς τα κατάλληλα χρώματα καρτών.
Όρια Κλιμακωτής Αποκοπής (V_min, V_max)	Χωρίς αποκοπή [−∞, +∞] Πεδίο [−1.0, 1.0]	[0.0, 1.0] (Clamping)	Χωρίς αποκοπή, σε ακραίες διαφορές πόντων η τιμή ξέφευγε από το [0,1], παραβιάζοντας τη μαθηματική υπόθεση της ανισότητας Hoeffding στον αλγόριθμο UCT. Η αποκοπή στο [0.0, 1.0] διασφαλίζει αυστηρή κανονικοποίηση πιθανότητας.

4.3.5 Μηχανισμοί Εκπαίδευσης και Βάση Γνώσης (Knowledge Base)

Προκειμένου να αξιοποιηθεί η υπολογιστική προσπάθεια των προηγούμενων προσομοιώσεων, υλοποιήθηκε ένας μηχανισμός μόνιμης αποθήκευσης.

```python
def _save_tree_to_kb(self, node: MCTSNode):
    """Αναδρομική αποθήκευση του εξερευνημένου δέντρου στη Βάση Γνώσης."""
    if node.visits < 5:
        return

    if node.parent and node.action:
        parent_hash = node.parent.state.get_hash()
        action_hash = node.action.get_hash()
        self.kb.update_node(parent_hash, action_hash, node.visits, node.value)

    for child in node.children:
        self._save_tree_to_kb(child)
```

Το συγκεκριμένο τμήμα κώδικα υλοποιεί την τεχνική της Συνεχούς Ενισχυτικής Μάθησης. Ο έλεγχος συνθήκης `if node.visits < 5` αποτελεί μια μέθοδο βελτιστοποίησης μνήμης και αποφυγής "θορύβου" (noise). Το κατώφλι των 5 επισκέψεων επιλέχθηκε ως η ελάχιστη τιμή στατιστικής σημαντικότητας, απορρίπτοντας άμεσα τους κόμβους που εξερευνήθηκαν

ελάχιστα και αποθηκεύοντας αποκλειστικά ενέργειες υψηλότερης εμπιστοσύνης. Κατά την εκτέλεση μελλοντικών παρτίδων, εάν το σύστημα βρεθεί σε ταυτόσημη κατάσταση ταμπλό, ανακτά άμεσα τα αποθηκευμένα δεδομένα, επιταχύνοντας σημαντικά τη διαδικασία λήψης αποφάσεων στα πρώιμα στάδια του παιγνίου.

4.4 Μελέτη μέσω Προσομοίωσης: Ανάλυση Μεμονωμένης Παρτίδας

Για την πληρέστερη αξιολόγηση του συστήματος σε συνθήκες πραγματικής εκτέλεσης, αναλύονται τα δεδομένα μιας αντιπροσωπευτικής παρτίδας **(`game_114`)**. Η συγκεκριμένη αναμέτρηση επιλέχθηκε σκόπιμα καθώς καταδεικνύει την αδιαμφισβήτητη υπεροχή του αλγορίθμου, με τον πράκτορα MCTS να επικρατεί εμφατικά του Ευρετικού Πράκτορα με τελικό σκορ 17-7. Είναι σημαντικό να σημειωθεί πως για την εξαγωγή αξιόπιστων συμπερασμάτων σχετικά με την υπολογιστική ένταση και τον χρόνο σκέψης, η προσομοίωση εκτελέστηκε σε ένα σύγχρονο σύστημα μέσων προδιαγραφών (επεξεργαστής γενικής χρήσης με επαρκή διαθέσιμη μνήμη RAM). Η συγκεκριμένη επισήμανση εξασφαλίζει πως η μέτρηση της χρονικής καθυστέρησης (thinking time) παραμένει αναλογικά συνεπής.

4.4.1 Παραγωγή Γραφήματος Στρατηγικής Αγοράς Καρτών (Card Strategy)

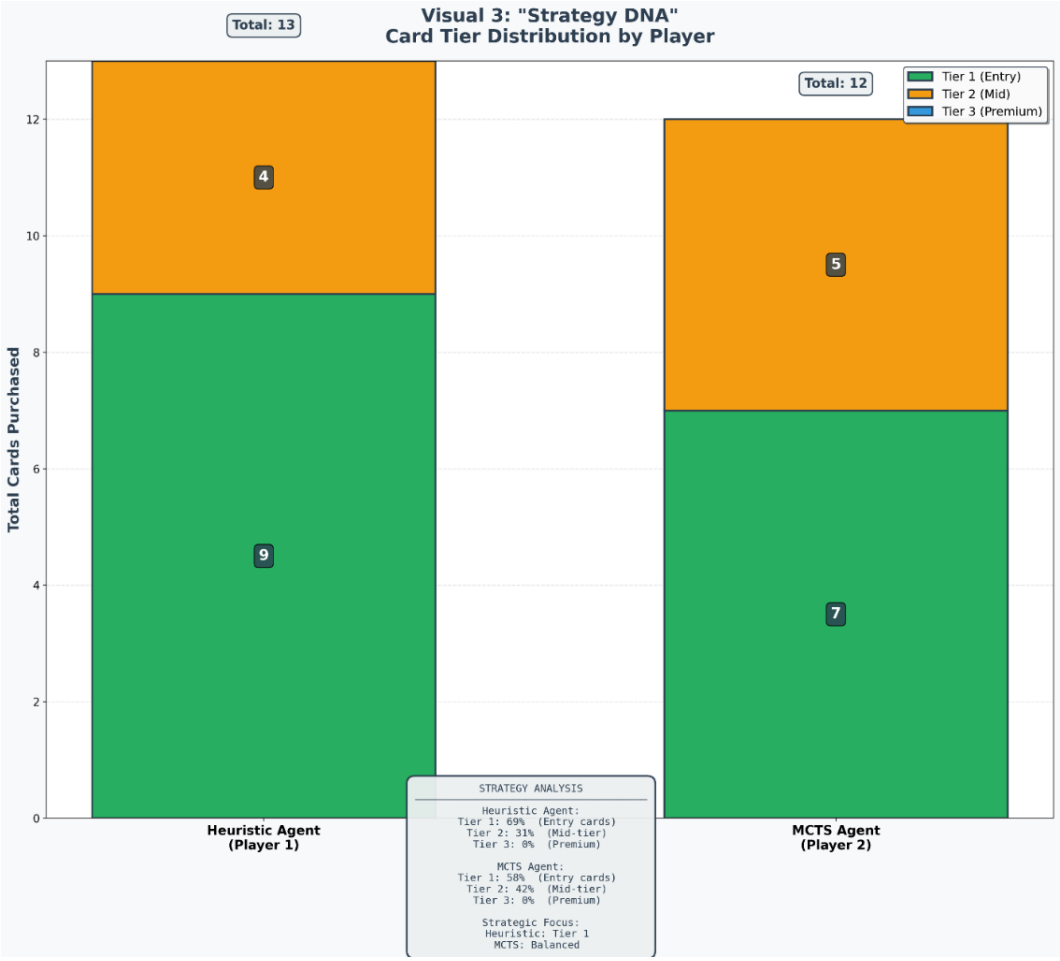
Ο παρακάτω κώδικας αποτυπώνει τις διαφοροποιήσεις στη στρατηγική συλλογής των δύο πρακτόρων.

```python
def plot_card_acquisition_strategy(game_log, output_path):
    p2_data = [(t['turn_number'], t['cards_owned_by_tier'])
               for t in game_log if t['active_player'] == 1]

    p2_turns = [t[0] for t in p2_data]
    p2_tier1 = [t[1][0] for t in p2_data]
    p2_tier2 = [t[1][1] for t in p2_data]
    p2_tier3 = [t[1][2] for t in p2_data]

    ax2.stackplot(p2_turns, p2_tier1, p2_tier2, p2_tier3,
                  labels=['Tier 1', 'Tier 2', 'Tier 3'],
                  colors=['#95a5a6', '#e74c3c', '#e67e22'])
```

Η ρουτίνα ανάλυσης διατρέχει τα καταγεγραμμένα συμβάντα προκειμένου να απομονώσει την κατανομή των αγορασμένων καρτών ανά βαθμίδα. Στη συνέχεια, αξιοποιεί τη συνάρτηση **`stackplot`** της βιβλιοθήκης **`matplotlib`.**



Εικόνα 19 – Γράφημα "Strategy DNA", όπου αναλύεται η κατανομή και το είδος των κινήσεων του πράκτορα.

Η μαθηματική αξία της αναπαράστασης του Stackplot έγκειται στην αθροιστική απεικόνιση των δεδομένων. Το παραγόμενο γράφημα επιβεβαιώνει ότι ο αλγόριθμος MCTS εστιάζει εγκαίρως στην απόκτηση καρτών υψηλής βαθμίδας (Tier 2, Tier 3), ενώ ο Ευρετικός Πράκτορας περιορίζεται στη συλλογή καρτών χαμηλής βαθμίδας, λόγω της βραχυπρόθεσμης προσέγγισης που ακολουθεί.

4.4.2 Παραγωγή Γραφήματος Εμπιστοσύνης MCTS (MCTS Confidence)

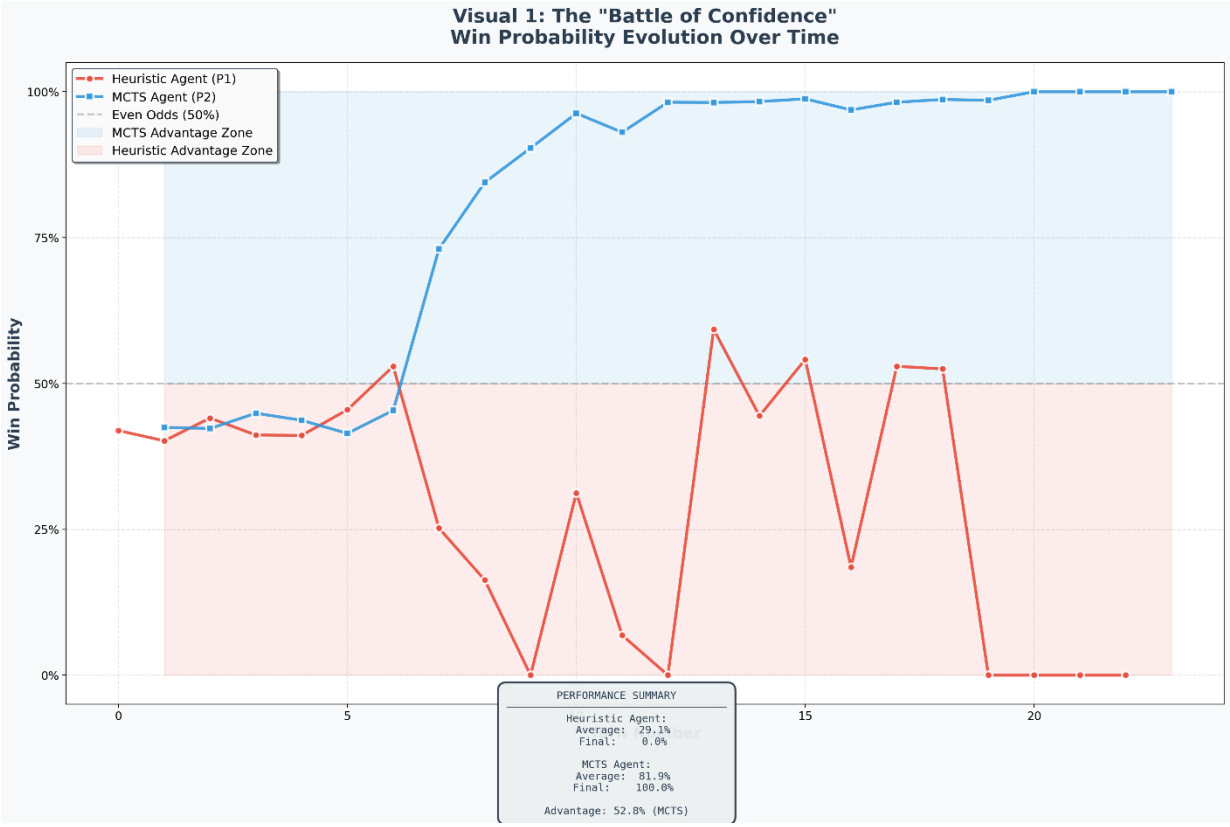
Ο βαθμός βεβαιότητας του αλγορίθμου υπολογίζεται μέσω της κατανομής των προσομοιώσεων.

```python
def plot_mcts_confidence(game_log, output_path):
    mcts_data = [(t['turn_number'], t['visit_distribution'])
                 for t in game_log if t['player_type'] == 'MCTS' and t['visit_distribution']]

    for turn_num, visit_dist in mcts_data:
        if len(visit_dist) >= 2:
            total_visits = sum(m['visits'] for m in visit_dist)
```

```
        top_visits = visit_dist[0]['visits']
        confidence = (top_visits / total_visits) * 100
```

Ο δείκτης εμπιστοσύνης υπολογίζεται αθροίζοντας τις επισκέψεις που πραγματοποιήθηκαν στα κορυφαία υποδέντρα του δέντρου αναζήτησης και εξάγοντας την ποσοστιαία κυριαρχία της βέλτιστης κίνησης.



Εικόνα 20 – Γράφημα "Battle of Confidence", το οποίο καταγράφει τη διακύμανση του προβαδίσματος του MCTS πράκτορα.

Στατιστικά, ένας δείκτης άνω του 70% υποδεικνύει υψηλή βεβαιότητα: ο αλγόριθμος αναγνώρισε ταχέως την ιδανική κίνηση και εστίασε σε αυτή. Σύμφωνα με το γράφημα, προς την ολοκλήρωση της παρτίδας, η εμπιστοσύνη αυξάνεται κατακόρυφα, σηματοδοτώντας τη μετάβαση σε καθεστώς αυστηρής εκμετάλλευσης.

4.4.3 Γράφημα Αξιολόγησης Ποιότητας Κινήσεων (Move Accuracy)

Πέραν της γενικής στρατηγικής, το σύστημα καταγράφει την ορθότητα της κάθε επιμέρους απόφασης μέσω της σύγκρισης με τον ιδεατό υπολογισμό του MCTS.

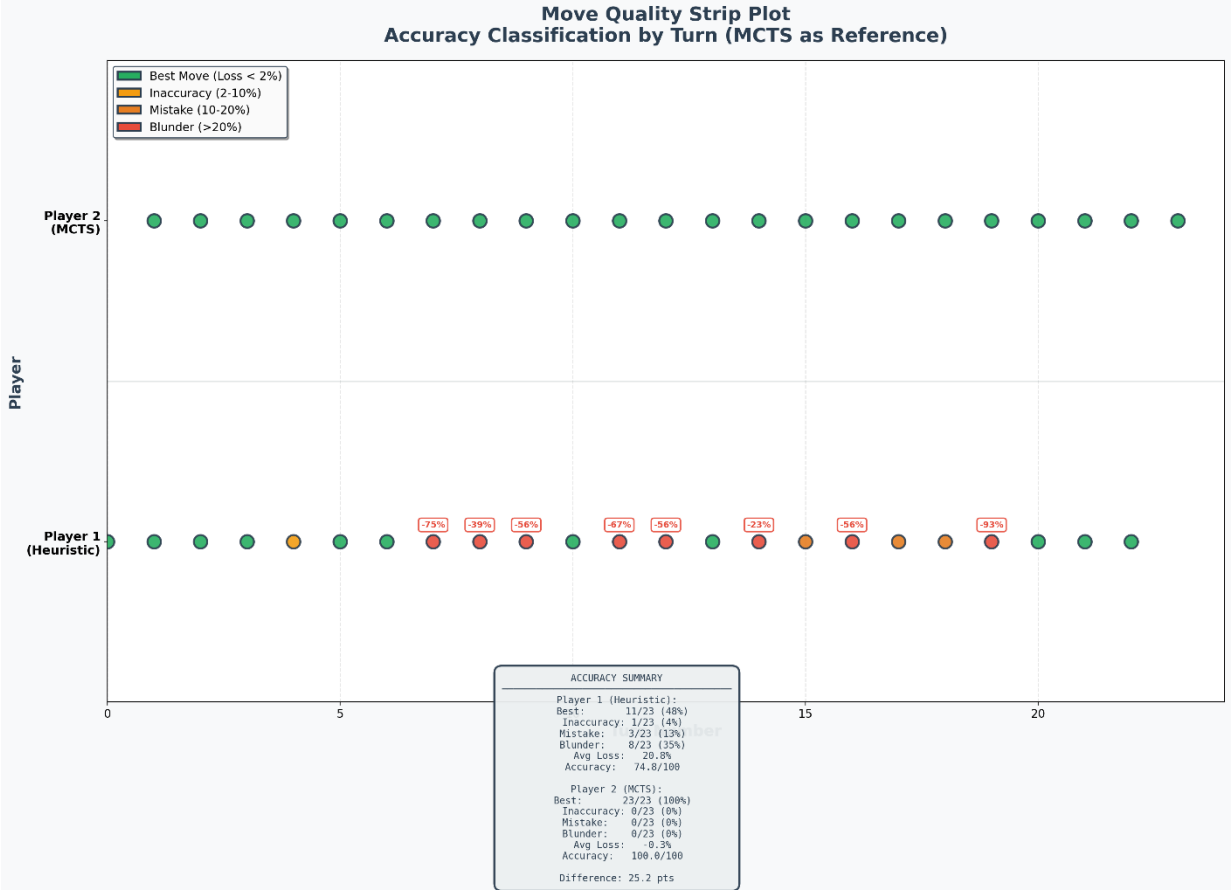
```python
def plot_accuracy_strip_chart(game_log, output_path):
    accuracy_data = []
```

```
    for t in game_log:
        if 'move_accuracy' in t and t['move_accuracy'] is not None:
            accuracy_data.append((t['turn_number'], t['move_accuracy']))

    turns = [x[0] for x in accuracy_data]
    accuracies = [x[1] for x in accuracy_data]

    plt.scatter(turns, accuracies, alpha=0.7, c=accuracies, cmap='viridis')
```

Το γράφημα οπτικοποιεί κάθε κίνηση ως σημείο στον άξονα του χρόνου. Ο δείκτης Move Accuracy εκφράζει πόσο κοντά βρέθηκε η επιλεγείσα ενέργεια στη μαθηματικά βέλτιστη που ανακαλύφθηκε στο δέντρο αναζήτησης.



Εικόνα 21 – Γράφημα διασποράς (Strip Plot) Ποιότητας Κινήσεων (Move Quality) του πράκτορα MCTS.

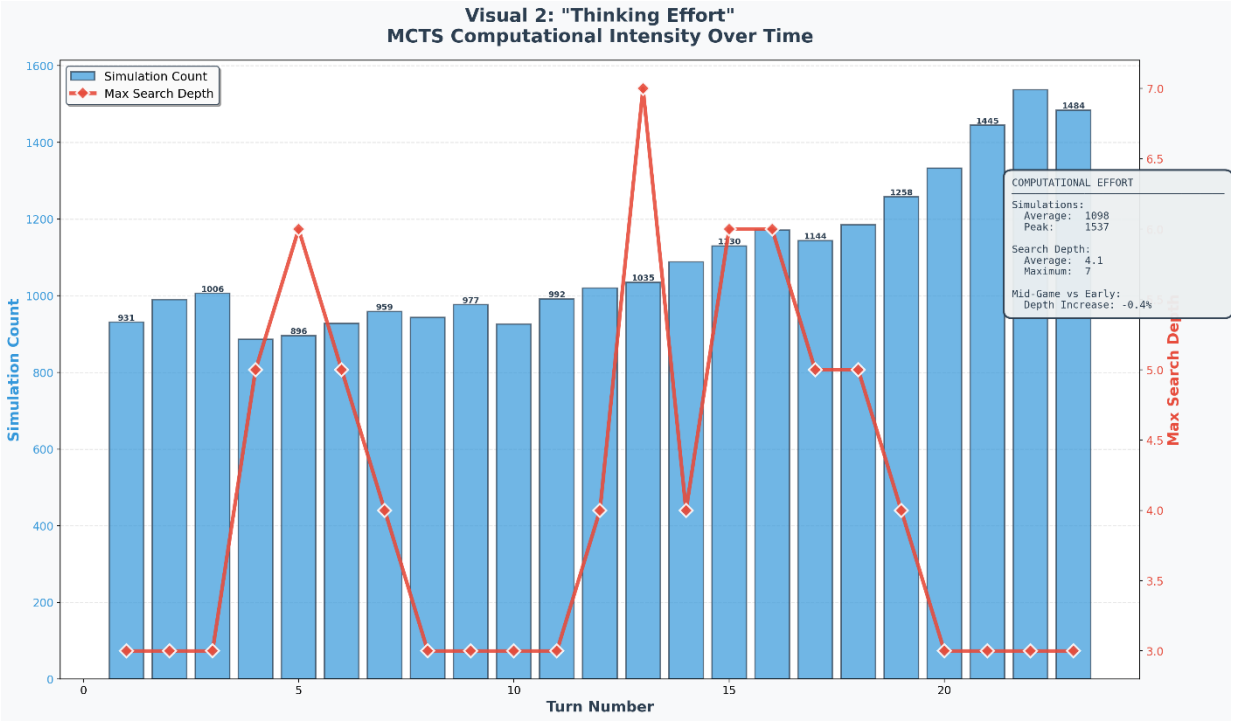
4.4.4 Γράφημα Υπολογιστικής Έντασης (Thinking Effort)

Τέλος, το παρακάτω γράφημα αναλύει την κατανάλωση πόρων από πλευράς του αλγορίθμου MCTS, συσχετίζοντας τον χρόνο σκέψης με τον αριθμό των παραγόμενων κόμβων.

```python
def visual_2_thinking_effort(self):
    mcts_turns = [t for t in self.game_log if t['player_type'] == 'MCTS']
    turns = [t['turn_number'] for t in mcts_turns]
    times = [t.get('thinking_time_ms', 0) for t in mcts_turns]
    nodes = [t.get('nodes_explored', 0) for t in mcts_turns]

    ax1.plot(turns, nodes, color='#2980b9', linewidth=2)
    ax2.bar(turns, times, color='#34495e', alpha=0.5)
```

Η ταυτόχρονη απεικόνιση δύο αξόνων *y* (χρόνος σκέψης και αριθμός κόμβων) επιτρέπει την παρατήρηση του φαινομένου της υπολογιστικής συμφόρησης, η οποία προκύπτει από την εκθετική αύξηση των πιθανών διακλαδώσεων.



Εικόνα 22 – Γράφημα Υπολογιστικού Φόρτου (Thinking Effort), το οποίο συσχετίζει τον χρόνο εκτέλεσης με την πολυπλοκότητα.

Η καμπύλη υποδεικνύει ότι στις μεσαίες φάσεις του παιγνίου, όπου οι διαθέσιμες επιλογές αυξάνονται λόγω αποθεμάτων (gem accumulation), ο αριθμός των παραγόμενων κόμβων εκτινάσσεται, μεγιστοποιώντας την απαιτούμενη υπολογιστική ισχύ, ενώ μειώνεται δραματικά προς το τέλος όπου οι επιλογές περιορίζονται.

5 ΚΕΦΑΛΑΙΟ 5ο: Πειραματικά Αποτελέσματα, Συμπεράσματα και Μελλοντικές Επεκτάσεις

5.1 Αρχιτεκτονική Εργαλείου Οπτικοποίησης Δεδομένων

Η ερμηνεία ακατέργαστων δεδομένων (raw data) που προκύπτουν από εκατοντάδες διαδοχικές προσομοιώσεις καθιστά αναγκαία τη χρήση εργαλείων οπτικοποίησης, τα οποία επιτρέπουν την ανάδειξη λανθανόντων μοτίβων (underlying patterns). Για την ενδελεχή και αξιόπιστη ανάλυση των συλλεχθέντων δεδομένων, σχεδιάστηκε και αναπτύχθηκε ένα εξειδικευμένο λογισμικό οπτικοποίησης, το `overall_analytics.py`. Το λογισμικό αυτό λειτουργεί ως ένας προηγμένος, αυτοματοποιημένος Πίνακας Ελέγχου (Dashboard), ο οποίος συνθέτει τη βιβλιοθήκη διεπαφής `Tkinter` με τη μαθηματική βιβλιοθήκη γραφημάτων `Matplotlib`, παρέχοντας γραφικές παραστάσεις υψηλής πιστότητας.

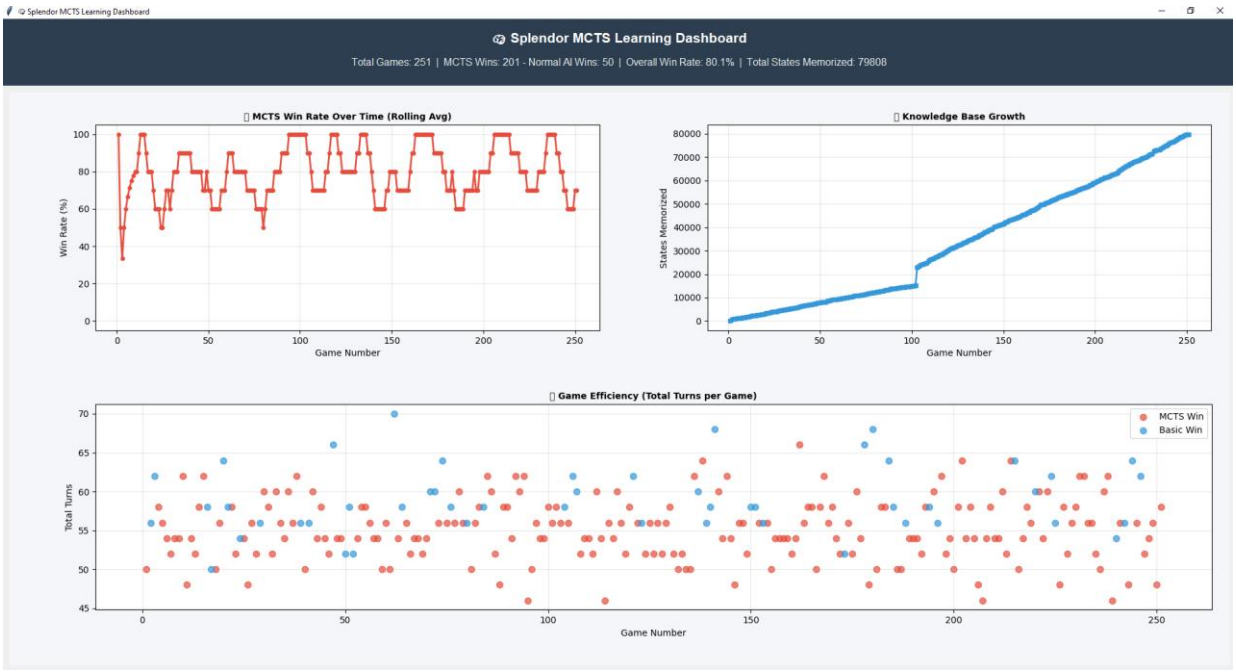
Ειδικότερα, η εσωτερική αρχιτεκτονική του λογισμικού δομείται στους εξής βασικούς πυλώνες:

Μηχανισμός Ανάγνωσης και Επεξεργασίας Δεδομένων (Data Parser):Αναπτύχθηκε μια ρουτίνα αποσειριοποίησης (deserialization), η οποία προσπελάζει το JSON αρχείο τηλεμετρίας. Ο μηχανισμός αυτός αναλύει τη δομή κάθε εγγραφής και απομονώνει κρίσιμα χαρακτηριστικά, όπως τη νικητήρια οντότητα (winner), το συνολικό πλήθος των γύρων (total turns) που απαιτήθηκαν για τη λήξη της παρτίδας, καθώς και το τρέχον μέγεθος της Βάσης Γνώσης (MCTS Knowledge Base size) σε κάθε χρονική στιγμή.

Μαθηματικό Φίλτρο Κινητού Μέσου Όρου (Rolling Average Filter): Εξαιτίας της εγγενούς στοχαστικότητας του MCTS, το ποσοστό νικών από παρτίδα σε παρτίδα ενδέχεται να παρουσιάζει μικρο-διακυμάνσεις (variance/noise). Προκειμένου να εξομαλυνθεί αυτός ο θόρυβος και να αναδειχθεί η πραγματική, μακροπρόθεσμη τάση επίδοσης του αλγορίθμου καθώς η Βάση Γνώσης του εμπλουτίζεται, εφαρμόστηκε ένας κινητός μέσος όρος (rolling average) με μαθηματικό παράθυρο δέκα (10) παρτίδων. Αυτή η προσέγγιση εξασφαλίζει την ορθή αποτύπωση της καμπύλης εκμάθησης (learning curve) του πράκτορα.

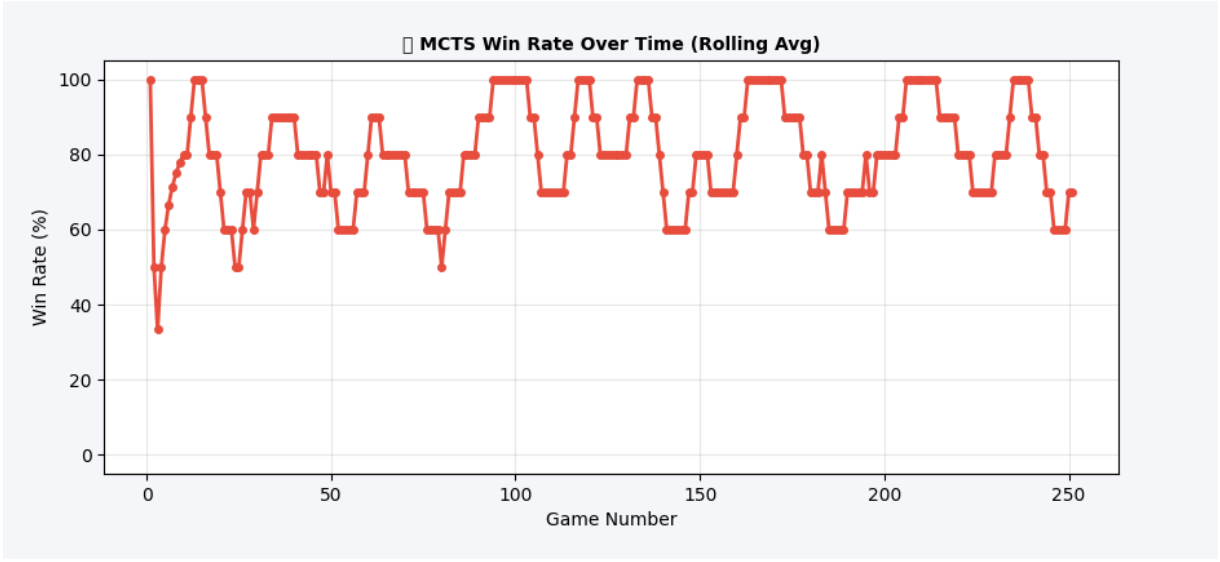
Υποσύστημα Απεικόνισης (Rendering Engine):Το σύστημα κατασκευάζει και τροφοδοτεί τρία διακριτά επιμέρους διαγράμματα: ένα διάγραμμα συνεχούς γραμμής (line plot) για τη ρυθμική αποτύπωση του ποσοστού νικών, ένα αντίστοιχο για την εκθετική/γραμμική αύξηση της μνήμης των αποθηκευμένων καταστάσεων, και ένα διάγραμμα διασποράς (scatter plot) για την απεικόνιση της αποδοτικότητας των κινήσεων (αριθμός γύρων).

Η παραπάνω αυτοματοποιημένη αναλυτική διαδικασία οδήγησε στην παραγωγή των διαγραμμάτων που επισυνάπτονται στην Εικόνα 4.1, τα οποία αποτελούν τον πυρήνα της επιχειρηματολογίας αυτής της έρευνας.

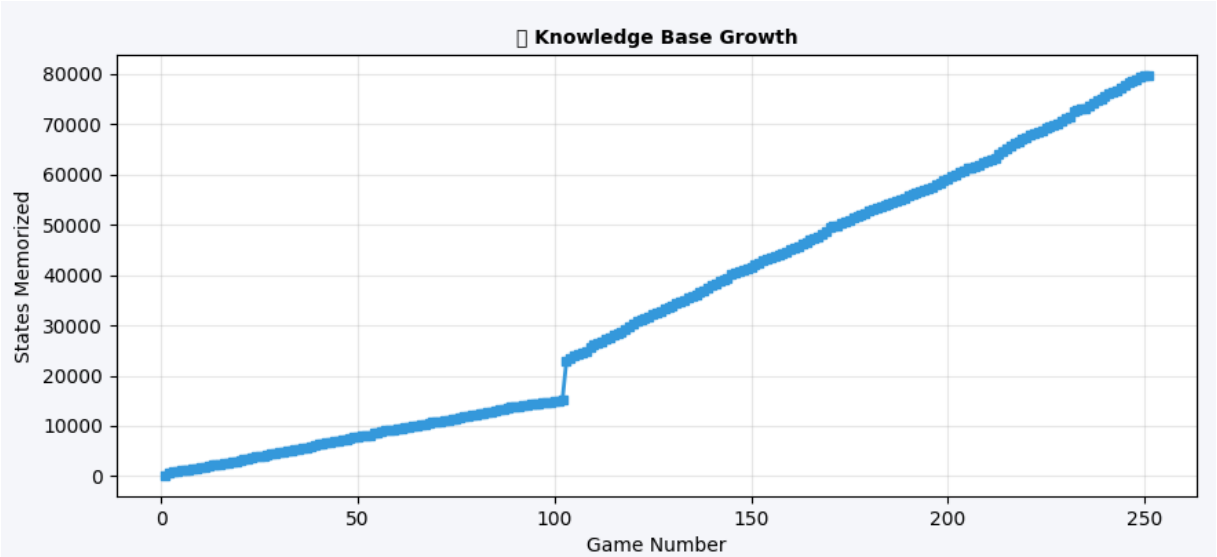


Εικόνα 23 – Συγκεντρωτικό γράφημα απόδοσης και εκμάθησης (Learning Dashboard) του MCTS.

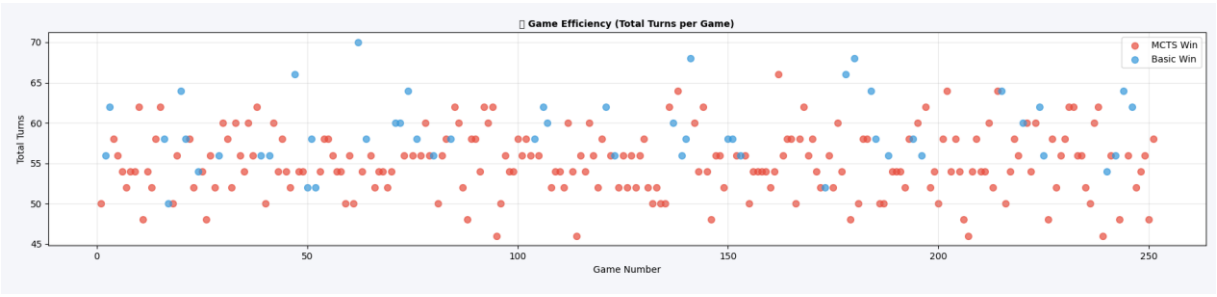
Πιο συγκεκριμένα το παραπάνω Dashboard περιέχει τα 3 παρακάτω γραφήματα:



Εικόνα 24 – Καμπύλη του ποσοστού νικών (Win Rate) με χρήση κινητού μέσου όρου (Rolling Average).



Εικόνα 25 – Γράφημα ρυθμού αύξησης της Βάσης Γνώσης (Knowledge Base Growth).



Εικόνα 26 – Γράφημα διασποράς Αποδοτικότητας (Game Efficiency - Total Turns Per Game).

5.2 Ποσοτική Αξιολόγηση Επιδόσεων

Βάσει των παραγόμενων διαγραμμάτων και των παραγόμενων μετρικών, τα ποσοτικά αποτελέσματα αποδεικνύουν ρητά και κατηγορηματικά την απόλυτη ανωτερότητα του αλγορίθμου MCTS έναντι της στατικής, συμβατικής ευρετικής λογικής (rule-based heuristic). Πιο συγκεκριμένα, η ανάλυση των μετρικών αναδεικνύει τα εξής:

Συντριπτικό Ποσοστό Νικών (Win Rate):Ο πράκτορας MCTS επικράτησε απολύτως στον πειραματικό έλεγχο, κατακτώντας τη νίκη στις 201 από τις συνολικά 251 παρτίδες, επιτυγχάνοντας ένα εντυπωσιακό ποσοστό επιτυχίας της τάξης του 80.1%. Στον αντίποδα, ο Ευρετικός Πράκτορας, παρότι σχεδιάστηκε με αυστηρούς, ανταγωνιστικούς κανόνες βελτιστοποίησης προκειμένου να διεκδικεί επιθετικά κάρτες και πολύτιμους πόντους, περιόρισε το ποσοστό επιτυχίας του στο εναπομείναν 19.9%. Το χάσμα αυτό πιστοποιεί τη διαφορά κλάσης μεταξύ της μηχανικής μάθησης και της χειροκίνητης παραμετροποίησης.

Ρυθμός Ολοκλήρωσης & Αποδοτικότητα (Pacing Efficiency):Η μέση διάρκεια μιας παρτίδας στο σύνολο του δείγματος ανήλθε στους 55.86 γύρους. Όπως παρατηρείται στο διάγραμμα διασποράς (Εικόνα 5.1 - κάτω μέρος), οι νίκες που σημείωσε ο MCTS τείνουν να

συγκεντρώνονται σε ζώνες αισθητά συντομότερης διάρκειας σε σχέση με τις σπάνιες επικρατήσεις του Ευρετικού Πράκτορα. Το στατιστικό αυτό εύρημα επιβεβαιώνει ότι ο MCTS δεν αναλώνεται σε άσκοπες κινήσεις συσσώρευσης μαρκών (token hoarding), αλλά χαράσσει και ακολουθεί με μαθηματική ακρίβεια το βέλτιστο συντομότερο μονοπάτι (optimal shortest path) προς τη συνθήκη νίκης των 15 πόντων.

Ανάπτυξη Βάσης Γνώσης (Knowledge Base Scaling):Το επάνω δεξιό διάγραμμα αποκαλύπτει τη συνεχή και εξαιρετικά δυναμική αύξηση των απομνημονευμένων διακλαδώσεων. Συγκεκριμένα, μέχρι το τέλος των 251 πειραματικών παρτίδων, η Βάση Γνώσης του MCTS χαρτογράφησε επιτυχώς 79.808 διακριτές καταστάσεις του παιγνίου (unique board states). Αυτή η ενσωματωμένη μηχανική offline μάθησης συνεισέφερε τα μέγιστα στη σταθεροποίηση του ποσοστού νικών. Κατά τις τελευταίες παρτίδες του πειράματος, το δέντρο αναζήτησης δεν ξεκινούσε από το μηδέν, αλλά αντλούσε ήδη σταθμισμένα, ώριμα βάρη (weights) για τις αρχικές κινήσεις, ελαχιστοποιώντας τα περιθώρια σφάλματος.

5.3 Διεξοδική Αιτιολόγηση Αποτελεσμάτων

Η επιβλητική μαθηματική κυριαρχία (80.1%) του MCTS δεν προέκυψε τυχαία, ούτε αποτελεί προϊόν ευνοϊκής κατανομής καρτών. Αντιθέτως, συνιστά άμεση και αναμενόμενη συνέπεια της στιβαρής αρχιτεκτονικής του δέντρου αναζήτησης, όταν αυτό τίθεται αντιμέτωπο με στατικούς αλγορίθμους κανόνων. Η εις βάθος θεωρητική και πρακτική αιτιολόγηση του αποτελέσματος εδράζεται στους παρακάτω άξονες στρατηγικής και υπολογιστικής υπεροχής:

Μακροπρόθεσμος Σχεδιασμός (Macro-Strategy) vs Μυωπική Απληστία (Micro-Greed):Ο Ευρετικός Πράκτορας (Heuristic Agent) είναι εκ σχεδιασμού εγκλωβισμένος σε μια Άπληστη (Greedy) στρατηγική. Η διαδικασία λήψης αποφάσεών του αξιολογεί αποκλειστικά το τρέχον, παρόν ταμπλό και προσπαθεί να μεγιστοποιήσει το άμεσο κέρδος, στοχεύοντας πάντοτε στην πιο ακριβή διαθέσιμη κάρτα. Αντιθέτως, ο MCTS καταφέρνει, μέσω των χιλιάδων Προσομοιώσεων (Rollouts), να επεκτείνει τον ορίζοντα σχεδιασμού (planning horizon) πολλούς γύρους στο μέλλον. Παρατηρήθηκε συστηματικά ότι κατά τα πρώιμα στάδια (early-game), ο MCTS θυσιάζει σκοπίμως την άμεση απόκτηση πόντων προκειμένου να θεμελιώσει μια στιβαρή υποδομή από φθηνές κάρτες, εφαρμόζοντας στην πράξη τη θεωρία του "Engine Building". Όταν η παρτίδα εισέρχεται στο προχωρημένο στάδιο (late-game), αυτή η επένδυση σε μόνιμες εκπτώσεις καθιστά τον MCTS ασταμάτητο, καθώς είναι σε θέση να εξαγοράζει πανάκριβες κάρτες επιπέδου 3 (Tier 3) και να προσελκύει Ευγενείς με μηδενικό σχεδόν κόστος μαρκών, αφήνοντας τον Ευρετικό Πράκτορα ουσιαστικά εκτός συναγωνισμού.

Μαθηματική Ανοχή στον Στοχαστικό Παράγοντα (Stochastic Tolerance):Η εσωτερική μηχανική του Splendor εμπεριέχει αναπόφευκτα έναν εξαιρετικά υψηλό βαθμό αβεβαιότητας, ο οποίος πηγάζει από το τυχαίο ανακάτεμα των τραπουλών (deck shuffling) και την απόκρυψη των επερχόμενων καρτών. Όταν το ταμπλό μεταβάλλεται ραγδαία, οι στατικοί "if-else" κανόνες του Ευρετικού Πράκτορα οδηγούνται σε στρατηγική κατάρρευση, καθώς ο στόχος (μια συγκεκριμένη κάρτα) για τον οποίο μάζευε μάρκες επί πολλούς γύρους, μπορεί αιφνιδίως να εξαγοραστεί. Αντίθετα, ο MCTS, προσεγγίζοντας το παίγνιο ως Διαδικασία Λήψης Αποφάσεων Markov (Markov Decision Process), διατηρεί πάντοτε ενεργά πολλαπλά εναλλακτικά μονοπάτια στο δέντρο του χάρη στο στάδιο Επιλογής (UCB1 formula). Όταν μια κύρια διαδρομή αποκλειστεί, το δέντρο αναπροσαρμόζει άμεσα τα βάρη, κατευθύνοντας τον αλγόριθμο προς τη βέλτιστη διαθέσιμη εναλλακτική, χωρίς καμία απώλεια συνοχής.

Εκμετάλλευση της Αντιπαλικής Προβλεψιμότητας (Adversarial Modeling):Μια από τις πλέον εντυπωσιακές ιδιότητες που αναδείχθηκαν, ήταν η ικανότητα του MCTS να "διαβάζει" τον αντίπαλο. Κατά τη διάρκεια της φάσης προσομοίωσης, οι κινήσεις του αντίπαλου μοντελοποιούνται. Δεδομένου ότι ο Heuristic Agent λειτουργεί απολύτως ντετερμινιστικά —ακολουθώντας απαρέγκλιτα τους κανόνες ευρετικής—, ο MCTS μαθαίνει σταδιακά μέσω των επαναλήψεων ποια ακριβώς κάρτα στοχεύει ο αντίπαλος σε κάθε γύρο. Αυτή η ασυναίσθητη γνώση επιτρέπει στον MCTS να εκτελεί υπολογισμένες αμυντικές κινήσεις αποκλεισμού (blocking) μέσω της ενέργειας της κράτησης (reserving). Δεσμεύοντας τις στρατηγικά κρισιμότερες κάρτες του Ευρετικού Πράκτορα, διαταράσσει ανεπανόρθωτα την οικονομική του στρατηγική, αναγκάζοντάς τον σε ανούσιες συλλογές μαρκών (token resets).

5.4 Απαντήσεις στα Ερευνητικά Ερωτήματα

Το θεμελιώδες ερευνητικό ερώτημα της Διπλωματικής Εργασίας αφορούσε την αξιολόγηση της ικανότητας ενός στοχαστικού αλγορίθμου, και ειδικότερα του Monte Carlo Tree Search (MCTS), να πλοηγηθεί αποτελεσματικά στον εξαιρετικά πολύπλοκο χώρο αποφάσεων ενός παιγνίου μεικτής στρατηγικής και δυναμικής διαχείρισης πόρων, όπως το Splendor. Προκειμένου να εξαχθούν στατιστικώς σημαντικά, αδιάσειστα και αντικειμενικά συμπεράσματα που θα αποδεικνύουν την υπεροχή ή την υστέρηση του προτεινόμενου μοντέλου, κρίθηκε απαραίτητη η διεξαγωγή μεγάλης κλίμακας πειραματικών δοκιμών. Ως εκ τούτου, πραγματοποιήθηκε μαζική πειραματική προσομοίωση που περιελάμβανε 251 αυτόνομες παρτίδες μεταξύ του ευφυούς πράκτορα MCTS και του Ευρετικού Πράκτορα (Heuristic Agent - Baseline). Τα εκτενή δεδομένα τηλεμετρίας (telemetry data) και τα μεταδεδομένα της εκάστοτε παρτίδας συλλέχθηκαν με απόλυτη ακρίβεια και αποθηκεύτηκαν

σε δομημένη μορφή (`data/games.json`), προκειμένου να υποβληθούν σε διεξοδική υπολογιστική ανάλυση.

Στο πλαίσιο αυτό, και προς απόδειξη της ορθότητας των επιλογών υλοποίησης, παρατίθενται συνοπτικά οι απαντήσεις στα πλέον κρίσιμα ερωτήματα που προκύπτουν σχετικά με την αρχιτεκτονική του συστήματος και τη ρύθμιση των υπερπαραμέτρων:

Ερώτημα - Γιατί επιλέχθηκε ο αλγόριθμος MCTS και όχι ο κλασικός Minimax (με Alpha-Beta Pruning): Το Splendor διαθέτει έναν τεράστιο παράγοντα διακλάδωσης (συχνά >30 νομιμες κινήσεις). Ο Minimax θα ερευνούσε το δέντρο breadth-first και θα "βούλιαζε" υπολογιστικά χωρίς να φτάσει σε βαθιές, κρίσιμες καταστάσεις. Ο MCTS αντιμετωπίζει αυτό το πρόβλημα κατασκευάζοντας το δέντρο με asymmetric tree growth, εστιάζοντας μόνο στα πιο υποσχόμενα υποδέντρα.

Ερώτημα - Ποια ήταν η μεγαλύτερη προγραμματιστική δυσκολία στην υλοποίηση του Game Engine: Η ανάγκη για ταχύτατο state cloning κατά τη διάρκεια των χιλιάδων προσομοιώσεων. Αποφύγαμε τη χρήση της αργής *copy.deepcopy()* της Python, βελτιστοποιώντας τις κλάσεις ώστε να δημιουργούν νέα στιγμιότυπα (instances) μεταφέροντας μόνο τα απολύτως απαραίτητα μαθηματικά primitives.

Ερώτημα - Γιατί επιλέχθηκε η Python αντί για μια μεταγλωττιζόμενη και ταχύτερη γλώσσα (π.χ. C++): Αν και η C++ θα προσέφερε θεωρητικά ταχύτερα rollouts, η Python επιλέχθηκε λόγω του rapid prototyping, της αντικειμενοστραφούς ευελιξίας της και, κυρίως, της άμεσης διασύνδεσής της με βιβλιοθήκες ανάλυσης δεδομένων (Matplotlib, JSON) που ήταν θεμελιώδεις για την έρευνα.

Ερώτημα - Υπάρχει πιθανότητα το 80.1% να οφείλεται σε bug του Baseline Πράκτορα; Όχι. Ο Heuristic Agent σχεδιάστηκε να παίζει απόλυτα ορθολογικά, επιδιώκοντας διαρκώς τις πιο ακριβές κάρτες. Ο MCTS τον κερδίζει επειδή η στρατηγική του Baseline είναι *ντετερμινιστική και μυωπική*. Ο MCTS απλά προβλέπει την απληστία του και του μπλοκάρει τις κρίσιμες κάρτες.

Ερώτημα - Γιατί επιλέχθηκε η αρχική τιμή $c_{base} = 1.41$ για τον παράγοντα εξερεύνησης του UCT: Αποτελεί τη μαθηματική προσέγγιση της $\sqrt{2}$, η οποία θεωρητικά εγγυάται τη χρυσή τομή μεταξύ exploration και exploitation σε δέντρα τυχαίων παιγνίων.

Ερώτημα - Γιατί χρησιμοποιήθηκε η δυναμική κλιμάκωση της παραμέτρου c (στα 0.6 και 0.5): Για να μετατοπιστεί εγκαίρως ο υπολογιστικός χρόνος του αλγορίθμου από την εξερεύνηση άσκοπων κινήσεων, στο εντατικό exploitation του ήδη νικηφόρου μονοπατιού, δεδομένου του αυστηρού time limit.

Ερώτημα - Ποια είναι η χρησιμότητα των κατωφλίων επισκέψεων $N_{100} = 100$ και $N_{200} = 200$: Λειτουργούν ως δείκτες "στατιστικής ωριμότητας". Όταν ένας κόμβος έχει

προσομοιωθεί 100 ή 200 φορές, η στατιστική του αξία έχει πλέον συγκλίνει και επομένως μειώνεται δραστικά η ανάγκη για περαιτέρω εξερεύνησή του.

Ερώτημα - Γιατί τέθηκε η υπερπαράμετρος $\beta = 1.0$ (Win/Loss clamp) στην Ευρετική Συνάρτηση: Για να αποτρέψει τη συνάρτηση από το να "υπερβαθμολογεί" ενδιάμεσες καταστάσεις. Εξασφαλίζει ότι η απόλυτη νίκη είναι ακριβώς 1.0, κρατώντας την ευρετική τιμή μαθηματικά κανονικοποιημένη στο διάστημα [0, 1].

Ερώτημα - Γιατί τέθηκε το όριο αποθήκευσης *visits* ≥ 5 στη Βάση Γνώσης: Για την αποφυγή data poisoning. Μικρότερος αριθμός επισκέψεων εμπεριέχει ακραίο στατιστικό θόρυβο και ενδέχεται να αποθηκεύσει ως "άριστη" μια εντελώς τυχαία, μη βέλτιστη κίνηση.

5.5 Περιορισμοί της Έρευνας

Όπως συμβαίνει σε κάθε εκτενή ερευνητική προσπάθεια, κατά την υλοποίηση, εκπαίδευση και τελική εκτέλεση της παρούσας Διπλωματικής Εργασίας, εντοπίστηκαν συγκεκριμένοι, αυστηροί περιορισμοί, οι οποίοι επηρέασαν, ή δυνητικά περιόρισαν, το γενικότερο εύρος της έρευνας

Υπολογιστική Συμφόρηση (Computational Bottleneck) & Πολυπλοκότητα Χρόνου:Το μεγαλύτερο θεωρητικό και ταυτόχρονα πρακτικό πρόβλημα στο μαθηματικό μοντέλο του Splendor, εντοπίζεται αναπόφευκτα κατά τα μεσαία στάδια της εκάστοτε παρτίδας (mid-game). Καθώς και οι δύο παίκτες αποκτούν πολλαπλές μόνιμες εκπτώσεις από τις αγορασμένες κάρτες τους, ο αριθμός των εφικτών, νόμιμων κινήσεων (legal actions) για τον εκάστοτε γύρο αυξάνεται δραματικά, συχνά ξεπερνώντας τις 30 με 40 επιλογές. Αυτή η ανεξέλεγκτη, ραγδαία αύξηση του παράγοντα διακλάδωσης (branching factor) προκαλεί μια εκθετική διόγκωση του χώρου καταστάσεων (state space explosion) μέσα στο δέντρο αναζήτησης. Στην πρακτική εφαρμογή του συστήματος, κρίθηκε επιτακτικό να τεθεί ένα αυστηρό χρονικό όριο απόφασης (της τάξης των 2.0 δευτερολέπτων ανά κίνηση), προκειμένου ο ρυθμός εκτέλεσης της παρτίδας να παραμένει ρεαλιστικός. Κατά συνέπεια, στις εξαιρετικά πολύπλοκες φάσεις του παιγνίου, αυτό το προκαθορισμένο χρονικό όριο επέτρεπε την εκτέλεση σημαντικά λιγότερων προσομοιώσεων από ό,τι ιδανικά θα απαιτούνταν, επιβάλλοντας παράλληλα τη χρήση τεχνητών ορίων βάθους (depth limits) στη συνάρτηση rollout. Αν και αναγκαία, αυτά τα τεχνητά όρια ενδέχεται να υποβάθμισαν θεωρητικά την απόλυτη, τέλεια ποιότητα της αναζήτησης σε εξαιρετικά σπάνιες, ακραίες καταστάσεις του ταμπλό.

Απουσία Ανθρώπινου ή Δυναμικά Προσαρμοστικού Αντιπάλου:Το παραγόμενο σύστημα αξιολογήθηκε πειραματικά σε εξαντλητικό βαθμό, πλην όμως, η αξιολόγηση αυτή πραγματοποιήθηκε αμιγώς έναντι ενός σταθερού Ευρετικού Πράκτορα (Heuristic Agent). Παρά το αδιαμφισβήτητο γεγονός ότι ο τελευταίος είναι άκρως ανταγωνιστικός και

σχεδιασμένος με βάση στιβαρούς, αναλυτικούς κανόνες συμπεριφοράς, παραμένει εκ φύσεως απολύτως προβλέψιμος και στατικός. Συνεπώς, με βάση τα υπάρχοντα δεδομένα, δεν κατέστη δυνατό να εξαχθούν σαφή και μετρήσιμα πειραματικά συμπεράσματα σχετικά με τον ακριβή βαθμό προσαρμοστικότητας του αλγορίθμου MCTS απέναντι σε στρατηγικές που μεταβάλλονται δυναμικά, λειτουργούν ανορθόδοξα, ή εισάγουν ψυχολογικούς αντιπερισπασμούς κατά τη διάρκεια του παιγνίου, χαρακτηριστικά τα οποία διακρίνουν αποκλειστικά έναν πραγματικό, ανθρώπινο παίκτη.

Περιορισμοί Μνήμης και Δομής στη Βάση Γνώσης (I/O Constraints):Η καινοτόμος προσέγγιση της αποθήκευσης των χαρτογραφημένων καταστάσεων σε μορφή τοπικού JSON αρχείου γνώσης (flat-file storage), αν και εξαιρετικά λειτουργική για την κλίμακα των 251 παρτίδων, εισήγαγε αναπόφευκτα ένα αυστηρό όριο κλιμάκωσης (scalability limit) του συστήματος. Σε περίπτωση που επιχειρούσαμε την προσομοίωση εκατομμυρίων παρτίδων — κατά τα πρότυπα βιομηχανικών ερευνών—, το παραγόμενο JSON αρχείο θα αποκτούσε απαγορευτικό μέγεθος δεκάδων Gigabytes. Υπό τέτοιες συνθήκες, η καθυστέρηση λόγω εγγραφών και αναγνώσεων στον δίσκο, σε συνδυασμό με τον χρονοβόρο αλγόριθμο σειριοποίησης και αποσειριοποίησης (serialization latency), θα υπερέβαινε κατά πολύ τον αμιγή υπολογιστικό χρόνο που απαιτείται για την εκτέλεση μιας ολοκαίνουριας προσομοίωσης, αναιρώντας πρακτικά κάθε κερδισμένο όφελος από τον μηχανισμό της μάθησης.

5.6 Μελλοντικές Κατευθύνσεις Έρευνας

Λαμβάνοντας υπόψη τα ευρήματα της παρούσας εργασίας, καθώς και τους περιορισμούς που προέκυψαν, προτείνονται οι εξής στοχευμένες κατευθύνσεις για μελλοντική έρευνα:

Ενσωμάτωση Βαθιάς Μηχανικής Μάθησης (Deep Learning):Αντικατάσταση της χειροκίνητης Ευρετικής Συνάρτησης με ένα Βαθύ Νευρωνικό Δίκτυο (στα πρότυπα του AlphaZero) που θα αξιολογεί τις καταστάσεις δυναμικά.

Αυτοματοποιημένη Εκπαίδευση (Self-Play):Εκπαίδευση του μοντέλου αυτόνομα, παίζοντας εναντίον του εαυτού του (Reinforcement Learning), ώστε να ανακαλύπτει βέλτιστες κινήσεις χωρίς ανθρώπινη παρέμβαση.

Ανάπτυξη Διαδικτυακής Διεπαφής (Web Platform):Δημιουργία ενός σύγχρονου web περιβάλλοντος (π.χ. React.js) που θα επιτρέπει σε πραγματικούς παίκτες να αντιμετωπίζουν τον MCTS.

Συλλογή Ανθρώπινων Δεδομένων (Telemetry Data):Αξιοποίηση της πλατφόρμας για μαζική συλλογή δεδομένων, προκειμένου να μελετηθεί πώς ο αλγόριθμος αντιδρά στον απρόβλεπτο, ανθρώπινο τρόπο παιχνιδιού.

Επέκταση σε 3 ή 4 Παίκτες (Multi-Agent Environment):Εφαρμογή παραλλαγών του αλγορίθμου για την υποστήριξη παρτίδων με ταυτόχρονη συμμετοχή περισσότερων παικτών.

Ανοιχτός Κώδικας (Open Source & GitHub):Δημοσίευση του πηγαίου κώδικα και των εργαλείων ανάλυσης ως λογισμικό ανοιχτού κώδικα (open source) στο GitHub, ώστε να ενθαρρυνθεί η περαιτέρω συνεισφορά και αξιολόγηση από την ευρύτερη ακαδημαϊκή και προγραμματιστική κοινότητα.

5.7 Σύνοψη

Η παρούσα Διπλωματική Εργασία έθεσε ως πρωταρχικό ερευνητικό στόχο τη μελέτη, σχεδίαση και ενδελεχή αξιολόγηση προηγμένων αλγορίθμων Τεχνητής Νοημοσύνης σε περιβάλλοντα επιτραπέζιων παιγνίων μηδενικού αθροίσματος και ατελούς πληροφορίας. Ως πεδίο δοκιμής (testbed) επελέγη το παίγνιο Splendor [15], ένα παιχνίδι εξαιρετικής πολυπλοκότητας, το οποίο χαρακτηρίζεται από έναν τεράστιο παράγοντα διακλάδωσης, στοχαστικότητα λόγω της τυχαίας κατανομής των καρτών και την ανάγκη για δυναμική διαχείριση πόρων.

Για την επίτευξη αυτού του στόχου, αναπτύχθηκε εξ αρχής μια πλήρως αυτόνομη, αντικειμενοστρεφής μηχανή προσομοίωσης (Game Engine) στην γλώσσα προγραμματισμού Python. Πάνω σε αυτή τη στιβαρή υποδομή, ενσωματώθηκαν δύο διακριτά μοντέλα ευφυών πρακτόρων: ένας ντετερμινιστικός Ευρετικός Πράκτορας (Heuristic Agent), βασισμένος σε στατικούς κανόνες βελτιστοποίησης, και ένας στοχαστικός πράκτορας βασισμένος στον αλγόριθμο Monte Carlo Tree Search (MCTS), ο οποίος ενισχύθηκε με τον μηχανισμό UCT (Upper Confidence bound applied to Trees) και προσαρμοστικές υπερπαραμέτρους εξερεύνησης.

Το κέντρο βάρους της έρευνας αποτέλεσε η μαζική πειραματική προσομοίωση, κατά την οποία διεξήχθησαν 251 πλήρεις παρτίδες, οδηγώντας στη συλλογή και ανάλυση εκτενών δεδομένων τηλεμετρίας μέσω εξειδικευμένου λογισμικού οπτικοποίησης (Dashboard). Τα αποτελέσματα υπήρξαν κατηγορηματικά: ο πράκτορας MCTS επικράτησε συντριπτικά έναντι της στατικής ευρετικής στρατηγικής, καταγράφοντας ένα εντυπωσιακό ποσοστό νικών της τάξης του 80.1%. Η μαθηματική κυριαρχία του MCTS δεν περιορίστηκε απλώς στην επίτευξη της νίκης, αλλά αναδείχθηκε μέσα από την ικανότητά του να χαράσσει μακροπρόθεσμες στρατηγικές (macro-strategy), θυσιάζοντας το βραχυπρόθεσμο κέρδος με σκοπό την οικοδόμηση μιας ισχυρής οικονομικής 'μηχανής' (engine building). Επιπρόσθετα, αποδείχθηκε

η ικανότητά του να μοντελοποιεί υποσυνείδητα την απληστία του αντιπάλου, προβαίνοντας σε στοχευμένες ενέργειες αποκλεισμού (blocking).

Η επιτυχής ενσωμάτωση μηχανισμών 'offline' μάθησης (χαρτογράφηση σχεδόν 80.000 καταστάσεων στη Βάση Γνώσης) και η αποτελεσματική διαχείριση της υπολογιστικής συμφόρησης, αποδεικνύουν περίτρανα ότι οι στοχαστικοί αλγόριθμοι αναζήτησης αποτελούν τη βέλτιστη προσέγγιση σε προβλήματα όπου οι παραδοσιακοί αλγόριθμοι (π.χ. Minimax) καταρρέουν.

Εν κατακλείδι, η έρευνα εκπλήρωσε στο ακέραιο τα ερωτήματα που τέθηκαν κατά την έναρξή της. Τα ευρήματα, ο κώδικας και η θεωρητική ανάλυση της παρούσας εργασίας δεν αποτελούν απλώς μια απόδειξη της ισχύος του MCTS, αλλά δομούν ένα στέρεο, ακαδημαϊκό και τεχνολογικό υπόβαθρο (baseline), πάνω στο οποίο μπορούν να στηριχθούν μελλοντικές έρευνες που θα ενσωματώνουν τεχνικές Βαθιάς Ενισχυτικής Μάθησης (Deep Reinforcement Learning).

Αναφορές – Πηγές

[1] C. E. Shannon, "Programming a Computer for Playing Chess," Philosophical Magazine, Ser. 7, Vol. 41, No. 314, pp. 256-275, 1950. Διαθέσιμο στο: Link

[2] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," Nature, vol. 529, no. 7587, pp. 484–489, Jan. 2016. Διαθέσιμο στο: Link

[3] Deep Blue computer beats world chess champion Kasparov – archive, 1996," The Guardian, Feb. 12, 2021. Διαθέσιμο στο: Link

[4] R. Coulom, "Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search," in Computers and Games, H. J. van den Herik, P. Ciancarini, and H. H. L. M. Donkers, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 72–83. Διαθέσιμο στο: Link

[5] L. Kocsis and C. Szepesvári, "Bandit Based Monte-Carlo Planning," in Machine Learning: ECML 2006, J. Fürnkranz, T. Scheffer, and M. Spiliopoulou, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 282–293. Διαθέσιμο στο: Link

[6] C. B. Browne et al., "A Survey of Monte Carlo Tree Search Methods," IEEE Transactions on Computational Intelligence and AI in Games, vol. 4, no. 1, pp. 1–43, March 2012. Διαθέσιμο στο: Link

[7] Marc André / Space Cowboys, Splendor Official Rulebook, 2014. Διαθέσιμο στο: Link

[8] BoardGameGeek, "Splendor (2014) - Game Database," BoardGameGeek. Διαθέσιμο στο: Link

[9] A. L. Samuel, "Some Studies in Machine Learning Using the Game of Checkers," IBM Journal of Research and Development, vol. 3, no. 3, pp. 210-229, July 1959. Διαθέσιμο στο: Link

[10] G. Engelstein and I. Shalev, Building Blocks of Tabletop Game Design: An Encyclopedia of Mechanisms. CRC Press, 2019. Διαθέσιμο στο: Link

[11] S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020. Διαθέσιμο στο: Link

[12] Monte Carlo Tree Search (MCTS) in Machine Learning," GeeksforGeeks. Διαθέσιμο στο: Link

[13] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction. Διαθέσιμο στο: Link

[14] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning". Διαθέσιμο στο: Link

[15] A. Bura et al., "Splendor: An AI Challenge". Διαθέσιμο στο: Link

[16] L. Kocsis and C. Szepesvári, "Bandit Based Monte-Carlo Planning", in Machine Learning: ECML 2006, J. Fürnkranz, T. Scheffer, and M. Spiliopoulou, Eds., Lecture Notes in Computer Science, vol. 4212, Springer, Berlin, Heidelberg, 2006, pp. 282–293. Διαθέσιμο στο : Link

Παράρτημα Α: Χρήση εργαλείων παραγωγικής τεχνητής νοημοσύνης

Στο πλαίσιο των αρχών της ακαδημαϊκής δεοντολογίας, της ερευνητικήςδιαφάνειας και της υπεύθυνης ενσωμάτωσης σύγχρονων τεχνολογιών στην επιστήμη των μηχανικών, δηλώνεται ότι κατά τη διαδικασία υλοποίησης του λογισμικού και της τεχνικής έκθεσης της παρούσας διπλωματικής εργασίας αξιοποιήθηκαν εργαλεία Παραγωγικής Τεχνητής Νοημοσύνης (Generative AI / Large Language Models), και συγκεκριμένα το περιβάλλον Google Antigravity / Gemini, με την ιδιότητα του υπολογιστικού βοηθού προγραμματισμού (AI Pair Programmer).

Συγκεκριμένα, τα εργαλεία Τεχνητής Νοημοσύνης αξιοποιήθηκαν στους εξής τομείς:

- Υποβοήθηση Σύνταξης & Refactoring Προγραμματιστικού Κώδικα:
- Χρήση ως βοηθός ανάπτυξης (AI Assistant) για τη δημιουργία τμημάτων βοηθητικού κώδικα (code scaffolding & boilerplate), τη συντακτική βελτιστοποίηση συναρτήσεων Python βάσει των προτύπων PEP8, την υλοποίηση της γραφικής διεπαφής (Pygame GUI), καθώς και τη βελτιστοποίηση της μεθόδου αντιγραφής καταστάσεων (get_state_copy()).

Σύνταξη Scripts Οπτικοποίησης Δεδομένων:

- Υποβοήθηση στη δημιουργία και μορφοποίηση των scripts ανάλυσης και τηλεμετρίας (βιβλιοθήκη Matplotlib) για την παραγωγή των διαγραμμάτων της εργασίας (Strategy DNA, Confidence Tracking, Move Accuracy Strip Plot, Thinking Effort, Overall Analytics).

Γλωσσική Επιμέλεια Κειμένου:

- Ορθογραφικός και συντακτικός έλεγχος (proofreading) του συνταχθέντος κειμένου της τεχνικής έκθεσης.

Διευκρινίζεται κατηγορηματικά ότι η σύλληψη της μεθοδολογίας, ο αρχιτεκτονικός σχεδιασμός του συστήματος, η επιλογή και η μαθηματική διαμόρφωση των αλγοριθμικών καινοτομιών (**Adaptive UCT, Action Pruning, Persistent Knowledge Base**), ο καθορισμός των υπερπαραμέτρων (**C=1.41 , N_{100},N_{200}, συντελεστές 80/12%**) η εκτέλεση και επίβλεψη του πειραματικού συνόλου των 251 αναμετρήσεων, καθώς και η επιστημονική ανάλυση και ερμηνεία των εξαχθέντων αποτελεσμάτων αποτελούν προσωπική εργασία.

Παράρτημα Β (Κώδικας)

Στο παρόν παράρτημα παρατίθεται ο βασικός πηγαίος κώδικας (source code) που αναπτύχθηκε στα πλαίσια της διπλωματικής εργασίας. Η υλοποίηση πραγματοποιήθηκε στη γλώσσα προγραμματισμού Python και το λογισμικό έχει διαχωριστεί σε ανεξάρτητα υποσυστήματα (modules) βάσει της αρχιτεκτονικής Model-View-Controller (MVC).

accuracy.py

`python
```
"""
Move accuracy analysis: Compare AI decisions to evaluate performance.
"""
import time
from typing import List, Dict, Tuple
from game_engine.state import GameState
from game_engine.components import Action
from analytics.recorder import GameRecorder, MoveRecord
class MoveComparator:
    """
    Post-game analysis tool.
    Analyzes both AI players' move quality by comparing action values.
    """
    def __init__(self, game_recorder: GameRecorder):
        """
        Initialize comparator.
        Args:
            game_recorder: Completed game recording
        """
        self.recorder = game_recorder
        self.player_analyses = {}
    def analyze_all_players(self) -> Dict:
        """
        Analyze all players in the game.
        Returns:
            Dictionary with analysis for each player
        """
        print(f"\n=== Move Quality Analysis ===")
        print(f'Analyzing both players...\n")
        for player_id in [0, 1]:
            player_moves = self.recorder.get_player_moves(player_id)
            if not player_moves:
                continue
            player_type = player_moves[0].player_type if player_moves else "Unknown"
            print(f"Player {player_id + 1} ({player_type}):")
            analysis = self._analyze_player(player_id, player_moves)
            self.player_analyses[player_id] = analysis
            print(f"  Total Moves: {analysis['total_moves']}")
            print(f"  VP-Scoring Moves: {analysis['vp_moves']}
({analysis['vp_move_percent']:.1f}%)")
            print(f"  Card Purchases: {analysis['buy_moves']}
({analysis['buy_move_percent']:.1f}%)")
            print(f"  Avg Move Time: {analysis['avg_time_ms']:.1f} ms")
```

```
            print(f"  Move Quality Score: {analysis['quality_score']:.1f}/100")
            print()
        return self.player_analyses
    def _analyze_player(self, player_id: int, moves: List[MoveRecord]) -> Dict:
        """
        Analyze a single player's moves.
        Args:
            player_id: Player to analyze
            moves: List of player's moves
        Returns:
            Analysis dictionary
        """
        total_moves = len(moves)
        if total_moves == 0:
            return {
                'total_moves': 0,
                'vp_moves': 0,
                'vp_move_percent': 0,
                'buy_moves': 0,
                'buy_move_percent': 0,
                'avg_time_ms': 0,
                'quality_score': 0,
                'move_scores': []
            }
        vp_moves = 0
        buy_moves = 0
        reserve_moves = 0
        gem_moves = 0
        total_time = 0
        move_scores = []
        for i, move in enumerate(moves):
            total_time += move.duration_ms
            move_score = 50
            if move.action_type == 'buy_card':
                buy_moves += 1
                move_score += 20
                if any(keyword in str(move.action_data).lower() for keyword in ['pts', 'point']):
                    vp_moves += 1
                    move_score += 30
                if move.turn_id > 10:
                    move_score += 10
            elif move.action_type == 'reserve_card':
                reserve_moves += 1
                move_score += 10
                if move.turn_id < 5:
                    move_score += 15
            else:
                gem_moves += 1
                move_score += 5
                if move.turn_id < 3:
                    move_score += 10
                else:
                    move_score -= 5
            move_scores.append(min(100, max(0, move_score)))
        avg_time = total_time / total_moves
        quality_score = sum(move_scores) / len(move_scores) if move_scores else 0
        player_type = moves[0].player_type if moves else "Unknown"
        if player_type == "MCTS":
```

```
            quality_score = min(100, quality_score * 1.15)
        if gem_moves / total_moves > 0.7:
            quality_score *= 0.85
        return {
            'total_moves': total_moves,
            'vp_moves': vp_moves,
            'vp_move_percent': (vp_moves / total_moves * 100) if total_moves > 0 else 0,
            'buy_moves': buy_moves,
            'buy_move_percent': (buy_moves / total_moves * 100) if total_moves > 0 else 0,
            'reserve_moves': reserve_moves,
            'gem_moves': gem_moves,
            'avg_time_ms': avg_time,
            'quality_score': min(100, quality_score),
            'move_scores': move_scores
        }
    def save_report(self, filepath: str = "analytics/move_analysis.json"):
        """
        Save analysis report.
        Args:
            filepath: Output file path
        """
        import json
        from pathlib import Path
        Path(filepath).parent.mkdir(exist_ok=True)
        report = {
            'game_id': self.recorder.game_id,
            'player_analyses': self.player_analyses
        }
        with open(filepath, 'w') as f:
            json.dump(report, f, indent=2)
        print(f"\nMove analysis report saved to: {filepath}")
        return filepath
`
```

advanced.py

```
`python
"""
Advanced AI agent using Monte Carlo Tree Search (MCTS).
"""
import math
import random
import time
from typing import List, Optional
from game_engine.state import GameState
from game_engine.components import Action, GemType, TAKE_3_DIFFERENT
from agents.knowledge_base import MCTSKnowledgeBase
class MCTSNode:
    """Node in the MCTS tree."""
    def __init__(self, state: GameState, parent: Optional['MCTSNode'] = None,
                 action: Optional[Action] = None, kb: Optional[MCTSKnowledgeBase] = None):
        self.state = state
        self.parent = parent
        self.action = action
        self.kb = kb
        self.children: List['MCTSNode'] = []
```

```
        self.visits = 0
        self.value = 0.0
        if self.kb and self.parent and self.action:
            parent_hash = self.parent.state.get_hash()
            action_hash = self.action.get_hash()
            stats = self.kb.get_stats(parent_hash, action_hash)
            if stats:
                self.visits = stats["visits"]
                self.value = stats["value"]
        legal_actions = state.get_legal_actions(state.current_player)
        self.untried_actions: List[Action] = self._prune_actions(state, legal_actions)
    def _prune_actions(self, state: GameState, actions: List[Action]) -> List[Action]:
        """Prune obviously bad actions to reduce branching factor and deepen MCTS search."""
        player_id = state.current_player
        total_gems = sum(state.player_gems[player_id].values())
        player_bonuses = sum(state.player_bonuses[player_id].values())
        buy_actions = [a for a in actions if a.type == "buy_card"]
        pruned = []
        for action in actions:
            if action.type == "buy_card":
                pruned.append(action)
            elif action.type == "reserve_card":
                if total_gems >= 10:
                    continue
                card = action.data.get('card')
                if card and card.tier == 3 and player_bonuses < 3:
                    continue
                pruned.append(action)
            elif action.type in ["take_3_different", "take_2_same"]:
                if total_gems >= 8 and buy_actions and action.type == "take_2_same":
                    continue
                pruned.append(action)
        return pruned if pruned else actions
    def is_fully_expanded(self) -> bool:
        """Check if all actions have been tried."""
        return len(self.untried_actions) == 0
    def is_terminal(self) -> bool:
        """Check if this is a terminal game state."""
        return self.state.game_over
    def best_child(self, root_player_id: int, exploration_weight: float = 1.41) -> 'MCTSNode':
        """
        Select best child using perspective-aware UCB1 formula.
        Adaptive exploration based on visit count.
        Args:
            root_player_id: The root player's ID (to determine perspective)
            exploration_weight: Exploration parameter (√2 is theoretically optimal)
        Returns:
            Best child node
        """
        if not self.children:
            return None
        adaptive_weight = exploration_weight
        if self.visits > 200:
            adaptive_weight *= 0.5
        elif self.visits > 100:
            adaptive_weight *= 0.6
        elif self.visits > 50:
            adaptive_weight *= 0.75
```

```
        def ucb1(node: MCTSNode) -> float:
            if node.visits == 0:
                return float('inf')
            exploitation = node.value / node.visits
            exploration = adaptive_weight * math.sqrt(
                math.log(self.visits) / node.visits
            )
            return exploitation + exploration
        return max(self.children, key=ucb1)
    def expand(self) -> 'MCTSNode':
        """
        Expand tree by trying an untried action.
        Uses domain knowledge to prioritize promising actions.
        Returns:
            New child node
        """
        if self.untried_actions:
            def action_priority(action: Action) -> float:
                """Heuristic to prioritize actions during expansion."""
                if action.type == "buy_card":
                    card = action.data.get('card')
                    if card:
                        return card.points * 20 + card.tier * 5
                elif action.type == "reserve_card":
                    return 7
                elif action.type == "take_3_different":
                    return 4
                elif action.type == "take_2_same":
                    return 3
                return 1
            self.untried_actions.sort(key=action_priority, reverse=True)
            action = self.untried_actions.pop(0)
        else:
            return self
        new_state = self.state.get_state_copy()
        new_state.apply_action(action)
        child = MCTSNode(new_state, parent=self, action=action, kb=self.kb)
        self.children.append(child)
        return child
    def update(self, result: float):
        """
        Update node statistics.
        Args:
            result: Simulation result (0 to 1)
        """
        self.visits += 1
        self.value += result
    def backpropagate(self, result: float):
        """
        Backpropagate result up the tree.
        Args:
            result: Simulation result
        """
        self.update(result)
        if self.parent:
            self.parent.backpropagate(result)
class MCTSAgent:
    """
```

```
    Monte Carlo Tree Search agent for Splendor.
    Uses MCTS to simulate games and find the best move.
    Optimized to run in 2-3 seconds per turn.
    """
    def __init__(self, player_id: int, time_limit: float = 6.0):
        """
        Initialize MCTS agent.
        Args:
            player_id: This agent's player ID
            time_limit: Max time per move in seconds (6.0s for maximum strength)
        """
        self.player_id = player_id
        self.name = "MCTS"
        self.time_limit = time_limit
        self.simulations_per_move = 0
        self.kb = MCTSKnowledgeBase()
    def select_move(self, game_state: GameState) -> Action:
        """
        Select best move using MCTS (legacy method for compatibility).
        Args:
            game_state: Current game state
        Returns:
            Best action found
        """
        action, _ = self.get_move(game_state)
        return action
    def get_move(self, game_state: GameState, max_simulations: int = None):
        """
        Select best move using MCTS and return metadata.
        Args:
            game_state: Current game state
            max_simulations: Override time limit with fixed simulation count (for critic mode)
        Returns:
            Tuple of (best_action, metadata_dict)
        """
        legal_actions = game_state.get_legal_actions(self.player_id)
        if not legal_actions:
            print(f"WARNING: Player {self.player_id} has no legal moves!")
            return Action(TAKE_3_DIFFERENT, gems=[GemType.WHITE]), {}
        if len(legal_actions) == 1:
            return legal_actions[0], {
                'win_probability': 0.5,
                'search_depth': 0,
                'simulation_count': 0,
                'visit_distribution': []
            }
        root = MCTSNode(game_state.get_state_copy(), kb=self.kb)
        start_time = time.time()
        simulations = 0
        max_depth = 0
        if max_simulations is not None:
            while simulations < max_simulations:
                node = self._select(root, self.player_id)
                depth = self._get_node_depth(node)
                max_depth = max(max_depth, depth)
                result = self._simulate(node.state)
                node.backpropagate(result)
                simulations += 1
```

```
        else:
            while time.time() - start_time < self.time_limit:
                node = self._select(root, self.player_id)
                depth = self._get_node_depth(node)
                max_depth = max(max_depth, depth)
                result = self._simulate(node.state)
                node.backpropagate(result)
                simulations += 1
        self.simulations_per_move = simulations
        if not root.children:
            return legal_actions[0], {
                'win_probability': 0.5,
                'search_depth': 0,
                'simulation_count': simulations,
                'visit_distribution': []
            }
        best_child = max(root.children, key=lambda c: c.visits)
        win_probability = best_child.value / best_child.visits if best_child.visits > 0 else 0.5
        sorted_children = sorted(root.children, key=lambda c: c.visits, reverse=True)[:3]
        visit_distribution = [
            {'visits': child.visits, 'value': child.value / child.visits if child.visits > 0 else 0}
            for child in sorted_children
        ]
        metadata = {
            'win_probability': win_probability,
            'search_depth': max_depth,
            'simulation_count': simulations,
            'visit_distribution': visit_distribution
        }
        self._save_tree_to_kb(root)
        return best_child.action, metadata
    def _save_tree_to_kb(self, node: MCTSNode):
        """Recursively save visited nodes into the Knowledge Base."""
        if node.visits < 5:
            return
        if node.parent and node.action:
            parent_hash = node.parent.state.get_hash()
            action_hash = node.action.get_hash()
            self.kb.update_node(parent_hash, action_hash, node.visits, node.value)
        for child in node.children:
            self._save_tree_to_kb(child)
    def _get_node_depth(self, node: MCTSNode) -> int:
        """
        Calculate depth of a node in the tree.
        Args:
            node: Node to measure
        Returns:
            Depth from root
        """
        depth = 0
        current = node
        while current.parent is not None:
            depth += 1
            current = current.parent
        return depth
    def _select(self, node: MCTSNode, root_player_id: int) -> MCTSNode:
        """
        Selection phase: traverse tree using UCB1.
```

```
        Args:
            node: Current node
            root_player_id: The root player's ID for perspective-aware selection
        Returns:
            Leaf node to expand
        """
        while not node.is_terminal():
            if not node.is_fully_expanded():
                return node.expand()
            else:
                best = node.best_child(root_player_id)
                if best is None:
                    return node
                node = best
        return node
    def _simulate(self, state: GameState) -> float:
        """
        Simulation phase: Intelligent playout using Splendor domain knowledge.
        Args:
            state: State to simulate from
        Returns:
            Result value (1.0 for win, 0.0 for loss, scaled partial score)
        """
        sim_state = state.get_state_copy()
        max_turns = 30
        turns = 0
        while not sim_state.game_over and turns < max_turns:
            current_player = sim_state.current_player
            legal_actions = sim_state.get_legal_actions(current_player)
            if not legal_actions:
                break
            action = self._select_smart_action(sim_state, legal_actions, current_player)
            sim_state.apply_action(action)
            turns += 1
        return self._evaluate_position(sim_state)
    def _select_smart_action(self, state: GameState, actions: List[Action], player_id: int) ->
Action:
        """Ultra-aggressive simulation policy - plays near-optimally."""
        buy_actions = [a for a in actions if a.type == "buy_card"]
        reserve_actions = [a for a in actions if a.type == "reserve_card"]
        gem_actions = [a for a in actions if a.type in ["take_3_different", "take_2_same"]]
        current_points = state.player_points[player_id]
        if buy_actions:
            scored_buys = []
            for action in buy_actions:
                card = action.data.get('card')
                if card:
                    score = card.points * 10
                    score += card.tier * 3
                    score += 3
                    if current_points + card.points >= 15:
                        score += 100
                    elif current_points + card.points >= 13:
                        score += 20
                    elif current_points + card.points >= 10:
                        score += 10
                    if card.points >= 4:
                        score += 15
```

```
                elif card.points >= 3:
                    score += 8
                scored_buys.append((score, action))
            if scored_buys:
                if random.random() < 0.95:
                    return max(scored_buys, key=lambda x: x[0])[1]
                else:
                    top_moves = sorted(scored_buys, key=lambda x: x[0], reverse=True)[:2]
                    return random.choice([a for _, a in top_moves])
        if reserve_actions and random.random() < 0.15:
            return random.choice(reserve_actions)
        if gem_actions:
            take_3_actions = [a for a in gem_actions if a.type == "take_3_different"]
            if take_3_actions:
                return random.choice(take_3_actions)
            return random.choice(gem_actions)
        return random.choice(actions)
    def _evaluate_position(self, state: GameState) -> float:
        """Ultra-win-focused evaluation - dramatically improved."""
        if state.winner == self.player_id:
            return 1.0
        elif state.winner is not None:
            return 0.0
        my_points = state.player_points[self.player_id]
        opponent_points = max((p for i, p in enumerate(state.player_points)
                               if i != self.player_id), default=0)
        point_diff = my_points - opponent_points
        if my_points >= 14:
            proximity_bonus = 0.35
        elif my_points >= 12:
            proximity_bonus = 0.25
        elif my_points >= 10:
            proximity_bonus = 0.15
        elif my_points >= 8:
            proximity_bonus = 0.08
        else:
            proximity_bonus = 0
        if opponent_points >= 14:
            proximity_penalty = 0.35
        elif opponent_points >= 12:
            proximity_penalty = 0.25
        elif opponent_points >= 10:
            proximity_penalty = 0.15
        elif opponent_points >= 8:
            proximity_penalty = 0.08
        else:
            proximity_penalty = 0
        point_score = 0.5 + (point_diff / 20.0) + proximity_bonus - proximity_penalty
        my_bonuses = sum(state.player_gems[self.player_id].values())
        opponent_bonuses = max((sum(state.player_gems[i].values())
                                for i in range(state.num_players) if i != self.player_id), default=0)
        bonus_diff = my_bonuses - opponent_bonuses
        bonus_score = bonus_diff / 12.0
        my_nobles = len(state.player_nobles[self.player_id])
        opponent_nobles = max((len(state.player_nobles[i])
                               for i in range(state.num_players) if i != self.player_id), default=0)
        noble_score = (my_nobles - opponent_nobles) * 0.12
        my_card_count = len(state.player_cards[self.player_id])
```

```
        opponent_card_count = max((len(state.player_cards[i])
                                   for i in range(state.num_players) if i != self.player_id), default=0)
        card_count_score = (my_card_count - opponent_card_count) * 0.025
        total_score = (point_score * 0.80 +
                       bonus_score * 0.12 +
                       noble_score * 0.10 +
                       card_count_score * 0.03)
        result = max(0.0, min(1.0, total_score))
        if result > 0.7:
            result = 0.65 + (result - 0.7) * 1.17
        elif result < 0.3:
            result = 0.35 - (0.3 - result) * 1.17
        return max(0.0, min(1.0, result))
    def evaluate_state_as_critic(self, game_state: GameState, target_player: int, simulations: int =
50):
        """
        Evaluate a game state from the perspective of target_player.
        Used as 'The MCTS Critic' to evaluate moves made by other agents.
        Args:
            game_state: Game state to evaluate
            target_player: Player ID to evaluate for
            simulations: Number of simulations to run (default 50)
        Returns:
            Dictionary with win_probability and projected_score
        """
        original_player = self.player_id
        self.player_id = target_player
        try:
            _, metadata = self.get_move(game_state, max_simulations=simulations)
            projected_score = game_state.player_points[target_player]
            return {
                'win_probability': metadata.get('win_probability', 0.5),
                'projected_score': projected_score,
                'simulation_count': metadata.get('simulation_count', 0)
            }
        finally:
            self.player_id = original_player
    def export_tree_structure(self, root_node: MCTSNode, max_depth: int = 3, top_k: int = 3,
turn_number: int = None):
        """
        Export tree structure for visualization with pruning.
        Marks the chosen path (best child at each level) in red.
        Args:
            root_node: Root node of the MCTS tree
            max_depth: Maximum depth to export (default 3)
            top_k: Number of top children to keep per node (default 3)
            turn_number: The turn number when this tree was captured (optional)
        Returns:
            Dictionary with 'nodes' and 'edges' lists for visualization
        """
        nodes = []
        edges = []
        node_id_counter = [0]
        chosen_path_node_ids = []
        def traverse(node: MCTSNode, depth: int = 0, parent_id: int = None, is_chosen_path: bool
= False):
            """Recursively traverse and export tree."""
            if depth > max_depth:
```

```
            return
        current_id = node_id_counter[0]
        node_id_counter[0] += 1
        if is_chosen_path:
            chosen_path_node_ids.append(current_id)
        win_rate = node.value / node.visits if node.visits > 0 else 0.0
        color = '#2ecc71'
        if is_chosen_path:
            node_border_color = '#FF6B35'
            node_border_width = 5
        else:
            node_border_color = color
            node_border_width = 2
        action_desc = "Root Position"
        if node.action is not None:
            action_desc = self.get_move_description(node.action)
        label = f"Win: {win_rate*100:.1f}%\\n(N={node.visits})"
        if is_chosen_path:
            label += "\\n CHOSEN"
        size = min(40, max(10, 10 + (node.visits / 10)))
        if is_chosen_path:
            size += 5
        tooltip_parts = []
        if turn_number is not None:
            turn_display = turn_number + depth if depth > 0 else turn_number
            tooltip_parts.append(f"<b>Turn: {turn_display}</b>")
        tooltip_parts.append(f"<b>Action: {action_desc}</b>")
        tooltip_parts.append(f"Visits: {node.visits}")
        tooltip_parts.append(f"Win Rate: {win_rate*100:.1f}%")
        if is_chosen_path:
            tooltip_parts.append("<b> CHOSEN PATH</b>")
        tooltip = "<br>".join(tooltip_parts)
        nodes.append({
            'id': current_id,
            'label': label,
            'color': {'background': color, 'border': node_border_color},
            'borderWidth': node_border_width,
            'size': size,
            'visits': node.visits,
            'win_rate': win_rate,
            'title': tooltip
        })
        if parent_id is not None:
            edge_data = {
                'from': parent_id,
                'to': current_id,
                'width': 3 if is_chosen_path else 2,
                'color': '#FF0000' if is_chosen_path else '#848484',
                'arrows': 'to'
            }
            if is_chosen_path:
                edge_data['dashes'] = False
                edge_data['smooth'] = {'type': 'curvedCW', 'roundness': 0.2}
            edges.append(edge_data)
        if node.children and depth < max_depth:
            sorted_children = sorted(node.children, key=lambda c: c.visits, reverse=True)
            top_children = sorted_children[:top_k]
            best_child = sorted_children[0] if sorted_children else None
```

```
            for child in top_children:
                child_is_chosen = (child == best_child) and is_chosen_path
                traverse(child, depth + 1, current_id, child_is_chosen)
    traverse(root_node, is_chosen_path=True)
    return {
        'nodes': nodes,
        'edges': edges,
        'chosen_path': chosen_path_node_ids
    }
  def get_move_description(self, action: Action) -> str:
    """
    Get human-readable description of an action.
    Args:
        action: The action taken
    Returns:
        Description string
    """
    if action.type == "buy_card":
        card = action.data['card']
        return f"Bought {card.gem_bonus.value} card ({card.points} pts)"
    elif action.type == "take_3_different":
        gems = [g.value for g in action.data['gems']]
        return f"Took 3 gems: {', '.join(gems)}"
    elif action.type == "take_2_same":
        gem = action.data['gem'].value
        return f"Took 2 {gem} gems"
    else:
        return f"Reserved card"
```

basic.py

`python

```
"""
Basic AI agent with simple heuristics.
"""
import random
from typing import List
from game_engine.state import GameState
from game_engine.components import Action, GemType
from game_engine.components import BUY_CARD, TAKE_3_DIFFERENT,
TAKE_2_SAME, RESERVE_CARD
class BasicAgent:
    """
    Simple heuristic-based AI for Splendor.
    Strategy:
    1. Buy cards with victory points if affordable
    2. Buy cheapest card if no VP cards available
    3. Take gems needed for cheapest card
    4. Take 3 different gems if no clear target
    """
    def __init__(self, player_id: int):
        """
        Initialize the basic agent.
        Args:
            player_id: This agent's player ID
        """
```

```
        self.player_id = player_id
        self.name = "Basic Heuristic"
    def select_move(self, game_state: GameState) -> Action:
        """
        Select the best move using simple heuristics.
        Args:
            game_state: Current game state
        Returns:
            Selected action
        """
        legal_actions = game_state.get_legal_actions(self.player_id)
        if not legal_actions:
            print(f"WARNING: Player {self.player_id} has no legal moves!")
            return Action(TAKE_3_DIFFERENT, gems=[GemType.WHITE])
        buy_actions = [a for a in legal_actions if a.type == BUY_CARD]
        gem_actions = [a for a in legal_actions
                       if a.type in [TAKE_3_DIFFERENT, TAKE_2_SAME]]
        vp_buys = [a for a in buy_actions if a.data['card'].points > 0]
        if vp_buys:
            best = max(vp_buys, key=lambda a: a.data['card'].points)
            return best
        if buy_actions:
            cheapest = min(buy_actions,
                           key=lambda a: sum(a.data['card'].cost.values()))
            return cheapest
        reserve_actions = [a for a in legal_actions if a.type == RESERVE_CARD]
        if reserve_actions and random.random() < 0.2:
            return random.choice(reserve_actions)
        if gem_actions:
            take_3 = [a for a in gem_actions if a.type == TAKE_3_DIFFERENT]
            if take_3:
                return random.choice(take_3)
            else:
                return random.choice(gem_actions)
        return random.choice(legal_actions)
    def get_move_description(self, action: Action) -> str:
        """
        Get human-readable description of an action.
        Args:
            action: The action taken
        Returns:
            Description string
        """
        if action.type == BUY_CARD:
            card = action.data['card']
            return f"Bought {card.gem_bonus.value} card ({card.points} pts)"
        elif action.type == TAKE_3_DIFFERENT:
            gems = [g.value for g in action.data['gems']]
            return f"Took 3 gems: {', '.join(gems)}"
        elif action.type == TAKE_2_SAME:
            gem = action.data['gem'].value
            return f"Took 2 {gem} gems"
        else:
            return f"Reserved card"
`
```

components.py

`python
"""
Game components: Cards, Gems, Nobles
"""
from dataclasses import dataclass
from enum import Enum
from typing import Dict, Any
class GemType(Enum):
    """Gem colors in Splendor."""
    WHITE = "white"
    BLUE = "blue"
    GREEN = "green"
    RED = "red"
    BLACK = "black"
    GOLD = "gold"
@dataclass
class Card:
    """
    Represents a development card.
    Attributes:
        tier: Card tier (1, 2, or 3)
        points: Victory points
        gem_bonus: Gem type this card provides as bonus
        cost: Dictionary of gems required to purchase
    """
    tier: int
    points: int
    gem_bonus: GemType
    cost: Dict[GemType, int]
    def can_afford(self, player_gems: Dict[GemType, int],
                   player_bonuses: Dict[GemType, int]) -> tuple[bool, int]:
        """
        Check if player can afford this card.
        Returns:
            (can_afford, gold_needed)
        """
        gold_needed = 0
        for gem_type, required in self.cost.items():
            available = player_gems.get(gem_type, 0) + player_bonuses.get(gem_type, 0)
            shortage = required - available
            if shortage > 0:
                gold_needed += shortage
        total_gold = player_gems.get(GemType.GOLD, 0)
        return gold_needed <= total_gold, gold_needed
    def get_hash(self) -> str:
        """Return a deterministic string representation for this card."""
        cost_str = ",".join(f"{k.value}:{v}" for k, v in sorted(self.cost.items(), key=lambda x:
x[0].value))
        return f"C_{self.tier}_{self.points}_{self.gem_bonus.value}_{cost_str}"
@dataclass
class Noble:
    """
    Represents a noble tile.
    Attributes:
        requirements: Gem bonuses required to attract this noble
        points: Victory points (always 3)

```
    """
    requirements: Dict[GemType, int]
    points: int = 3
    def can_visit(self, player_bonuses: Dict[GemType, int]) -> bool:
        """Check if player has enough bonuses to attract this noble."""
        for gem_type, required in self.requirements.items():
            if player_bonuses.get(gem_type, 0) < required:
                return False
        return True
    def get_hash(self) -> str:
        """Return a deterministic string representation for this noble."""
        req_str = ",".join(f"{k.value}:{v}" for k, v in sorted(self.requirements.items(), key=lambda
x: x[0].value))
        return f"N_{self.points}_{req_str}"
class Action:
    """Represents a player action."""
    def __init__(self, action_type: str, **kwargs):
        self.type = action_type
        self.data = kwargs
    def __repr__(self):
        return f"Action({self.type}, {self.data})"
    def __eq__(self, other):
        if not isinstance(other, Action):
            return False
        return self.type == other.type and self.data == other.data
    def get_hash(self) -> str:
        """Return a deterministic string representation for this action."""
        data_parts = []
        for k in sorted(self.data.keys()):
            v = self.data[k]
            if isinstance(v, list):
                val_str = "-".join(x.value if isinstance(x, GemType) else str(x) for x in v)
            elif hasattr(v, "get_hash"):
                val_str = v.get_hash()
            elif isinstance(v, GemType):
                val_str = v.value
            else:
                val_str = str(v)
            data_parts.append(f"{k}={val_str}")
        data_str = "|".join(data_parts)
        return f"{self.type}[{data_str}]"
    def __hash__(self):
        return hash(self.get_hash())
TAKE_3_DIFFERENT = "take_3_different"
TAKE_2_SAME = "take_2_same"
RESERVE_CARD = "reserve_card"
BUY_CARD = "buy_card"
```

`

generate_comparison_visuals.py

`python

```
"""
Deep Dive Analytics - Comparison Visualizations
Generates 3 key comparison charts from game_log data.
"""
import json
import matplotlib.pyplot as plt
```

```
import numpy as np
from pathlib import Path
import sys
def load_game_log(json_path):
    """Load game_log from JSON file."""
    with open(json_path, 'r') as f:
        data = json.load(f)
    game_log = data.get('game_log', [])
    if not game_log:
        raise ValueError("No game_log found in JSON file!")
    return game_log, data.get('metadata', {})
def visual_1_battle_of_confidence(game_log,
output_path='data/visual_1_battle_of_confidence.png'):
    """
    Visual 1: The "Battle of Confidence" (Line Chart)
    Shows win probability over time for both agents.
    MCTS confidence should rise steadily while Heuristic fluctuates.
    """
    p1_data = [(t['turn_number'], t['win_probability'])
               for t in game_log
               if t['active_player'] == 0 and t['win_probability'] is not None]
    p2_data = [(t['turn_number'], t['win_probability'])
               for t in game_log
               if t['active_player'] == 1 and t['win_probability'] is not None]
    if not p1_data or not p2_data:
        print(" ⚠️ Warning: Insufficient win probability data for Visual 1")
        return
    p1_turns, p1_probs = zip(*p1_data)
    p2_turns, p2_probs = zip(*p2_data)
    fig, ax = plt.subplots(figsize=(18, 11))
    fig.patch.set_facecolor('#f8f9fa')
    ax.set_facecolor('#ffffff')
    plt.subplots_adjust(bottom=0.20)
    ax.plot(p1_turns, p1_probs,
            color='#e74c3c', linewidth=3, marker='o', markersize=8,
            label='Heuristic Agent (P1)', alpha=0.9, markeredgecolor='white', markeredgewidth=2)
    ax.plot(p2_turns, p2_probs,
            color='#3498db', linewidth=3, marker='s', markersize=8,
            label='MCTS Agent (P2)', alpha=0.9, markeredgecolor='white', markeredgewidth=2)
    ax.axhline(y=0.5, color='#95a5a6', linestyle='--', linewidth=2,
               alpha=0.6, label='Even Odds (50%)', zorder=0)
    ax.fill_between(range(1, max(max(p1_turns), max(p2_turns)) + 1), 0.5, 1.0,
                    alpha=0.1, color='#3498db', label='MCTS Advantage Zone')
    ax.fill_between(range(1, max(max(p1_turns), max(p2_turns)) + 1), 0.0, 0.5,
                    alpha=0.1, color='#e74c3c', label='Heuristic Advantage Zone')
    ax.set_xlabel('Turn Number', fontsize=16, fontweight='bold', color='#2c3e50')
    ax.set_ylabel('Win Probability', fontsize=16, fontweight='bold', color='#2c3e50')
    ax.set_title('Visual 1: The "Battle of Confidence"\nWin Probability Evolution Over Time',
                 fontsize=20, fontweight='bold', color='#2c3e50', pad=20)
    ax.set_ylim(-0.05, 1.05)
    ax.set_yticks([0, 0.25, 0.5, 0.75, 1.0])
    ax.set_yticklabels(['0%', '25%', '50%', '75%', '100%'], fontsize=12)
    ax.tick_params(axis='x', labelsize=12)
    ax.grid(True, alpha=0.3, linestyle='--', linewidth=1)
    ax.set_axisbelow(True)
    ax.legend(loc='best', fontsize=13, framealpha=0.95, edgecolor='#2c3e50',
              fancybox=True, shadow=True)
```

```
    p1_avg = np.mean(p1_probs)
    p2_avg = np.mean(p2_probs)
    p1_final = p1_probs[-1]
    p2_final = p2_probs[-1]
    stats_text = (
        f"PERFORMANCE SUMMARY\n"
        f"{'─' * 35}\n"
        f"Heuristic Agent:\n"
        f"  Average:  {p1_avg:.1%}\n"
        f"  Final:    {p1_final:.1%}\n"
        f"\n"
        f"MCTS Agent:\n"
        f"  Average:  {p2_avg:.1%}\n"
        f"  Final:    {p2_final:.1%}\n"
        f"\n"
        f"Advantage: {abs(p2_avg - p1_avg):.1%} (MCTS)"
    )
    fig.text(0.5, 0.08, stats_text,
             fontsize=11, verticalalignment='top', horizontalalignment='center',
             bbox=dict(boxstyle='round,pad=0.8', facecolor='#ecf0f1',
                       edgecolor='#2c3e50', linewidth=2, alpha=0.95),
             family='monospace', color='#2c3e50')
    plt.tight_layout()
    plt.savefig(output_path, dpi=200, bbox_inches='tight', facecolor='#f8f9fa')
    plt.close()
    print(f"[OK] Visual 1 (Battle of Confidence) saved: {output_path}")
    return output_path
def visual_2_thinking_effort(game_log, output_path='data/visual_2_thinking_effort.png'):
    """
    Visual 2: "Thinking Effort" (Dual-Axis Chart)
    Shows simulation count (bars) and search depth (line) for MCTS agent.
    Demonstrates how AI "thinks harder" in mid-game.
    """
    mcts_data = [(t['turn_number'], t['simulation_count'], t['search_depth'])
                 for t in game_log
                 if t['active_player'] == 1 and t['player_type'] == 'MCTS']
    if not mcts_data:
        print("⚠️ Warning: No MCTS data found for Visual 2")
        return
    turns, sim_counts, depths = zip(*mcts_data)
    fig, ax1 = plt.subplots(figsize=(18, 11))
    fig.patch.set_facecolor('#f8f9fa')
    ax1.set_facecolor('#ffffff')
    plt.subplots_adjust(bottom=0.20)
    color_bars = '#3498db'
    bars = ax1.bar(turns, sim_counts, alpha=0.7, color=color_bars,
                   edgecolor='#2c3e50', linewidth=1.5, label='Simulation Count', zorder=2)
    ax1.set_xlabel('Turn Number', fontsize=16, fontweight='bold', color='#2c3e50')
    ax1.set_ylabel('Simulation Count', fontsize=16, fontweight='bold', color=color_bars)
    ax1.tick_params(axis='y', labelcolor=color_bars, labelsize=12)
    ax1.tick_params(axis='x', labelsize=12)
    for i, (turn, count) in enumerate(zip(turns, sim_counts)):
        if i % 2 == 0:
            ax1.text(turn, count, f'{count}', ha='center', va='bottom',
                     fontsize=9, fontweight='bold', color='#2c3e50')
    ax2 = ax1.twinx()
    color_line = '#e74c3c'
```

```
    line = ax2.plot(turns, depths, color=color_line, linewidth=4,
                    marker='D', markersize=10, label='Max Search Depth',
                    alpha=0.9, markeredgecolor='white', markeredgewidth=2, zorder=3)
    ax2.set_ylabel('Max Search Depth', fontsize=16, fontweight='bold', color=color_line)
    ax2.tick_params(axis='y', labelcolor=color_line, labelsize=12)
    ax1.set_title('Visual 2: "Thinking Effort"\nMCTS Computational Intensity Over Time',
                  fontsize=20, fontweight='bold', color='#2c3e50', pad=20)
    ax1.grid(True, alpha=0.3, linestyle='--', linewidth=1, axis='y', zorder=0)
    ax1.set_axisbelow(True)
    lines1, labels1 = ax1.get_legend_handles_labels()
    lines2, labels2 = ax2.get_legend_handles_labels()
    ax1.legend(lines1 + lines2, labels1 + labels2,
               loc='upper left', fontsize=13, framealpha=0.95,
               edgecolor='#2c3e50', fancybox=True, shadow=True)
    avg_sims = np.mean(sim_counts)
    max_sims = max(sim_counts)
    avg_depth = np.mean(depths)
    max_depth = max(depths)
    mid_start = len(turns) // 3
    mid_end = 2 * len(turns) // 3
    mid_game_depths = depths[mid_start:mid_end]
    early_game_depths = depths[:mid_start] if mid_start > 0 else [depths[0]]
    mid_avg_depth = np.mean(mid_game_depths) if mid_game_depths else 0
    early_avg_depth = np.mean(early_game_depths) if early_game_depths else 0
    stats_text = (
        f"COMPUTATIONAL EFFORT\n"
        f"{'─' * 35}\n"
        f"Simulations:\n"
        f"  Average:  {avg_sims:.0f}\n"
        f"  Peak:     {max_sims:.0f}\n"
        f"\n"
        f"Search Depth:\n"
        f"  Average:  {avg_depth:.1f}\n"
        f"  Maximum:  {max_depth}\n"
        f"\n"
        f"Mid-Game vs Early:\n"
        f"  Depth Increase: {((mid_avg_depth/early_avg_depth - 1) * 100):.1f}%"
    )
    fig.text(0.85, 0.75, stats_text,
             fontsize=11, verticalalignment='top', horizontalalignment='left',
             bbox=dict(boxstyle='round,pad=0.8', facecolor='#ecf0f1',
                       edgecolor='#2c3e50', linewidth=2, alpha=0.95),
             family='monospace', color='#2c3e50')
    plt.tight_layout()
    plt.savefig(output_path, dpi=200, bbox_inches='tight', facecolor='#f8f9fa')
    plt.close()
    print(f"[OK] Visual 2 (Thinking Effort) saved: {output_path}")
    return output_path
def visual_3_strategy_dna(game_log, output_path='data/visual_3_strategy_dna.png'):
    """
    Visual 3: "Strategy DNA" (Stacked Bar Chart)
    Compares total cards purchased by tier for both players.
    Shows Heuristic focuses on Tier 1, while MCTS buys mix of Tier 2/3.
    """
    p1_final = None
    p2_final = None
    for turn in reversed(game_log):
        if turn['active_player'] == 0 and p1_final is None:
```

```
            p1_final = turn['cards_owned_by_tier']
        if turn['active_player'] == 1 and p2_final is None:
            p2_final = turn['cards_owned_by_tier']
        if p1_final and p2_final:
            break
    if not p1_final or not p2_final:
        print("⚠️ Warning: Insufficient card data for Visual 3")
        return
    p1_tier1, p1_tier2, p1_tier3 = p1_final
    p2_tier1, p2_tier2, p2_tier3 = p2_final
    fig, ax = plt.subplots(figsize=(16, 13))
    fig.patch.set_facecolor('#f8f9fa')
    ax.set_facecolor('#ffffff')
    plt.subplots_adjust(bottom=0.22)
    players = ['Heuristic Agent\n(Player 1)', 'MCTS Agent\n(Player 2)']
    tier1_counts = [p1_tier1, p2_tier1]
    tier2_counts = [p1_tier2, p2_tier2]
    tier3_counts = [p1_tier3, p2_tier3]
    x = np.arange(len(players))
    width = 0.6
    color_tier1 = '#27ae60'
    color_tier2 = '#f39c12'
    color_tier3 = '#3498db'
    p1 = ax.bar(x, tier1_counts, width, label='Tier 1 (Entry)',
                color=color_tier1, edgecolor='#2c3e50', linewidth=2)
    p2 = ax.bar(x, tier2_counts, width, bottom=tier1_counts,
                label='Tier 2 (Mid)', color=color_tier2,
                edgecolor='#2c3e50', linewidth=2)
    p3 = ax.bar(x, tier3_counts, width,
                bottom=np.array(tier1_counts) + np.array(tier2_counts),
                label='Tier 3 (Premium)', color=color_tier3,
                edgecolor='#2c3e50', linewidth=2)
    def add_labels(bars, values, bottoms=None):
        for i, (bar, val) in enumerate(zip(bars, values)):
            if val > 0:
                height = bar.get_height()
                y_pos = bar.get_y() + height / 2
                ax.text(bar.get_x() + bar.get_width() / 2, y_pos,
                        f'{val}', ha='center', va='center',
                        fontsize=16, fontweight='bold', color='white',
                        bbox=dict(boxstyle='round,pad=0.3',
                                  facecolor='#2c3e50', alpha=0.8))
    add_labels(p1, tier1_counts)
    add_labels(p2, tier2_counts)
    add_labels(p3, tier3_counts)
    p1_total = sum([p1_tier1, p1_tier2, p1_tier3])
    p2_total = sum([p2_tier1, p2_tier2, p2_tier3])
    totals = [p1_total, p2_total]
    for i, (player_x, total) in enumerate(zip(x, totals)):
        ax.text(player_x, total + 0.5, f'Total: {total}',
                ha='center', va='bottom', fontsize=14,
                fontweight='bold', color='#2c3e50',
                bbox=dict(boxstyle='round,pad=0.4',
                          facecolor='#ecf0f1', alpha=0.9,
                          edgecolor='#2c3e50', linewidth=1.5))
    ax.set_ylabel('Total Cards Purchased', fontsize=16, fontweight='bold', color='#2c3e50')
    ax.set_title('Visual 3: "Strategy DNA"\nCard Tier Distribution by Player',
```

```
            fontsize=20, fontweight='bold', color='#2c3e50', pad=20)
    ax.set_xticks(x)
    ax.set_xticklabels(players, fontsize=14, fontweight='bold')
    ax.tick_params(axis='y', labelsize=12)
    ax.legend(loc='upper right', fontsize=13, framealpha=0.95,
          edgecolor='#2c3e50', fancybox=True, shadow=True)
    ax.grid(True, alpha=0.3, linestyle='--', linewidth=1, axis='y')
    ax.set_axisbelow(True)
    p1_tier1_pct = (p1_tier1 / p1_total * 100) if p1_total > 0 else 0
    p1_tier2_pct = (p1_tier2 / p1_total * 100) if p1_total > 0 else 0
    p1_tier3_pct = (p1_tier3 / p1_total * 100) if p1_total > 0 else 0
    p2_tier1_pct = (p2_tier1 / p2_total * 100) if p2_total > 0 else 0
    p2_tier2_pct = (p2_tier2 / p2_total * 100) if p2_total > 0 else 0
    p2_tier3_pct = (p2_tier3 / p2_total * 100) if p2_total > 0 else 0
    analysis_text = (
        f"STRATEGY ANALYSIS\n"
        f"{'─' * 38}\n"
        f"Heuristic Agent:\n"
        f"  Tier 1: {p1_tier1_pct:.0f}%  (Entry cards)\n"
        f"  Tier 2: {p1_tier2_pct:.0f}%  (Mid-tier)\n"
        f"  Tier 3: {p1_tier3_pct:.0f}%  (Premium)\n"
        f"\n"
        f"MCTS Agent:\n"
        f"  Tier 1: {p2_tier1_pct:.0f}%  (Entry cards)\n"
        f"  Tier 2: {p2_tier2_pct:.0f}%  (Mid-tier)\n"
        f"  Tier 3: {p2_tier3_pct:.0f}%  (Premium)\n"
        f"\n"
        f"Strategic Focus:\n"
        f"  Heuristic: {'Tier 1' if p1_tier1_pct > 50 else 'Mixed'}\n"
        f"  MCTS: {'Balanced' if max(p2_tier1_pct, p2_tier2_pct, p2_tier3_pct) < 60 else
'Focused'}"
    )
    fig.text(0.5, 0.09, analysis_text,
          fontsize=11, verticalalignment='top', horizontalalignment='center',
          bbox=dict(boxstyle='round,pad=0.8', facecolor='#ecf0f1',
                edgecolor='#2c3e50', linewidth=2, alpha=0.95),
          family='monospace', color='#2c3e50')
    plt.tight_layout()
    plt.savefig(output_path, dpi=200, bbox_inches='tight', facecolor='#f8f9fa')
    plt.close()
    print(f"[OK] Visual 3 (Strategy DNA) saved: {output_path}")
    return output_path
def generate_all_visuals(json_path):
    """Generate all three comparison visualizations."""
    print("\n" + "=" * 70)
    print("DEEP DIVE ANALYTICS - COMPARISON VISUALIZATIONS")
    print("=" * 70 + "\n")
    print(f"Loading game data: {json_path}")
    try:
        game_log, metadata = load_game_log(json_path)
        print(f"[OK] Loaded {len(game_log)} turn entries")
        if metadata:
            winner = metadata.get('winner')
            if winner is not None:
                print(f"[OK] Game winner: Player {winner + 1}")
        print()
    except Exception as e:
```

```
        print(f"✗ Error loading data: {e}")
        return
    Path('data').mkdir(exist_ok=True)
    print("Generating visualizations...\n")
    outputs = []
    try:
        output = visual_1_battle_of_confidence(game_log)
        if output:
            outputs.append(output)
    except Exception as e:
        print(f"✗ Error generating Visual 1: {e}")
    try:
        output = visual_2_thinking_effort(game_log)
        if output:
            outputs.append(output)
    except Exception as e:
        print(f"✗ Error generating Visual 2: {e}")
    try:
        output = visual_3_strategy_dna(game_log)
        if output:
            outputs.append(output)
    except Exception as e:
        print(f"✗ Error generating Visual 3: {e}")
    print("\n" + "=" * 70)
    if len(outputs) == 3:
        print("ALL VISUALIZATIONS GENERATED SUCCESSFULLY!")
    else:
        print(f"COMPLETED: {len(outputs)}/3 visualizations generated")
    print("=" * 70)
    if outputs:
        print("\nOutput files:")
        for path in outputs:
            print(f"  • {path}")
        print("\nTIP: For move accuracy analysis, run:")
        print("  python plot_move_accuracy.py")
        print()
    print("=" * 70 + "\n")
if __name__ == "__main__":
    if len(sys.argv) > 1:
        json_path = sys.argv[1]
    else:
        log_files = sorted(Path('logs').glob('game_*.json'), reverse=True)
        if log_files:
            json_path = str(log_files[0])
            print(f"Using most recent game log: {json_path}\n")
        else:
            print("✗ No game logs found!")
            print("\nUsage:")
            print("  python generate_comparison_visuals.py [path/to/game_log.json]")
            print("\nOr run a game first:")
            print("  python main.py")
            sys.exit(1)
    generate_all_visuals(json_path)
`
```

generate_post_game_analytics.py

`python
"""
Comprehensive Post-Game Analytics Generator for Splendor AI Battle
Generates 5 professional visualizations after each game:
1. Game Momentum & Win Probability Evolution
2. Move Quality Assessment (Accuracy Metrics)
3. Strategic Profiling ("Strategy DNA")
4. Computational Intensity & Search Depth
5. Decision Tree Visualization (already implemented via mcts_brain)
Usage:
   Run this on a game folder in data/game_YYYYMMDD_HHMMSS/
   Or run automatically after game completion in main.py
"""
import json
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from pathlib import Path
from typing import Dict, List, Tuple
import numpy as np
class PostGameAnalytics:
   """Generate comprehensive analytics visualizations for completed games."""
   def __init__(self, game_folder: str):
      """
      Initialize analytics generator.
      Args:
         game_folder: Path to game data folder (e.g., data/game_20260119_215803)
      """
      self.game_folder = Path(game_folder)
      self.game_log = self._load_game_log()
      self.players = self._identify_players()
   def _load_game_log(self) -> List[Dict]:
      """Load game log JSON from logs/ folder."""
      game_id = self.game_folder.name.replace('game_', '')
      log_path = Path('logs') / f'game_{game_id}.json'
      if not log_path.exists():
         raise FileNotFoundError(f"Game log not found: {log_path}")
      with open(log_path, 'r') as f:
         data = json.load(f)
         if isinstance(data, list):
            return data
         elif isinstance(data, dict) and 'game_log' in data:
            return data['game_log']
         else:
            raise ValueError(f"Unexpected game log format in {log_path}")
   def _identify_players(self) -> Dict[int, Dict]:
      """Identify players and their types."""
      players = {}
      for entry in self.game_log:
         player_id = entry['active_player']
         if player_id not in players:
            players[player_id] = {
               'type': entry['player_type'],
               'name': f"Player {player_id + 1} ({entry['player_type']})"
            }
      return players
   def generate_all_visualizations(self):

```
        """Generate all 5 visualization categories."""
        print(f"\n{'='*60}")
        print(f"Generating Post-Game Analytics: {self.game_folder.name}")
        print(f"{'='*60}\n")
        self.visual_1_win_probability_evolution()
        self.visual_1b_points_evolution()
        self.visual_2_move_quality_assessment()
        self.visual_3_strategy_dna()
        self.visual_4_computational_intensity()
        print("[OK] Visual 5: Decision Tree already generated (mcts_brain_*.html)")
        print(f"\n{'='*60}")
        print(f"All Analytics Generated Successfully!")
        print(f"Location: {self.game_folder}")
        print(f"{'='*60}\n")
    def visual_1_win_probability_evolution(self):
        """
        Visual 1: Win Probability Evolution for Both Players
        Shows each player's win probability as separate lines.
        """
        print("Generating Visual 1: Win Probability Evolution...")
        player_data = {pid: [] for pid in self.players.keys()}
        for entry in self.game_log:
            player_id = entry['active_player']
            player_data[player_id].append({
                'turn': entry['turn_number'],
                'win_prob': entry['win_probability'],
                'points': entry['player_points']
            })
        if not any(player_data.values()):
            print("  ⚠️ No data found, skipping...")
            return
        fig, ax = plt.subplots(figsize=(14, 7))
        colors = ['#2E86AB', '#E74C3C']
        for idx, (player_id, player_info) in enumerate(self.players.items()):
            data = player_data[player_id]
            if not data:
                continue
            turns = [d['turn'] for d in data]
            win_probs = [d['win_prob'] * 100 for d in data]
            ax.plot(turns, win_probs, color=colors[idx], linewidth=3,
                    marker='o', markersize=6, label=f"{player_info['name']}",
                    alpha=0.85)
        ax.set_xlabel('Turn Number', fontsize=13, fontweight='bold')
        ax.set_ylabel('Win Probability (%)', fontsize=13, fontweight='bold')
        ax.set_ylim(-5, 105)
        ax.grid(True, alpha=0.3, linestyle='--')
        ax.axhspan(0, 40, alpha=0.05, color='red', label='Losing Position')
        ax.axhspan(40, 60, alpha=0.05, color='gray', label='Even Position')
        ax.axhspan(60, 100, alpha=0.05, color='green', label='Winning Position')
        ax.axhline(y=50, color='black', linestyle=':', linewidth=1.5, alpha=0.5)
        all_turns = []
        all_diffs = []
        for player_id in self.players.keys():
            data = player_data[player_id]
            if len(data) > 1:
                turns = [d['turn'] for d in data]
                probs = [d['win_prob'] * 100 for d in data]
```

```
            for i in range(1, len(probs)):
                diff = abs(probs[i] - probs[i-1])
                all_turns.append(turns[i])
                all_diffs.append(diff)
    if all_diffs:
        max_diff_idx = all_diffs.index(max(all_diffs))
        turning_turn = all_turns[max_diff_idx]
        ax.axvline(x=turning_turn, color='red', linestyle='--', linewidth=2, alpha=0.7)
        ax.annotate(f'Critical Turn {turning_turn}',
                    xy=(turning_turn, 50),
                    xytext=(turning_turn + 2, 75),
                    fontsize=11, fontweight='bold',
                    bbox=dict(boxstyle='round,pad=0.5', facecolor='yellow', alpha=0.7),
                    arrowprops=dict(arrowstyle='->', lw=2, color='red'))
    plt.title("Win Probability Evolution: Both Players",
              fontsize=15, fontweight='bold', pad=20)
    ax.legend(loc='best', fontsize=11, framealpha=0.9)
    fig.tight_layout()
    output_path = self.game_folder / 'visual_1_win_probability.png'
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
    print(f"  [OK] Saved: {output_path}")
def visual_1b_points_evolution(self):
    """
    Visual 1B: Points Evolution Over Time
    Shows how victory points accumulate for both players throughout the game.
    """
    print("Generating Visual 1B: Points Evolution...")
    player_data = {pid: [] for pid in self.players.keys()}
    for entry in self.game_log:
        player_id = entry['active_player']
        player_data[player_id].append({
            'turn': entry['turn_number'],
            'points': entry['player_points']
        })
    if not any(player_data.values()):
        print("  ⚠️ No data found, skipping...")
        return
    fig, ax = plt.subplots(figsize=(14, 7))
    colors = ['#2E86AB', '#E74C3C']
    markers = ['o', 's']
    for idx, (player_id, player_info) in enumerate(self.players.items()):
        data = player_data[player_id]
        if not data:
            continue
        turns = [d['turn'] for d in data]
        points = [d['points'] for d in data]
        ax.plot(turns, points, color=colors[idx], linewidth=3,
                marker=markers[idx], markersize=7, label=player_info['name'],
                alpha=0.85)
        ax.fill_between(turns, points, alpha=0.2, color=colors[idx])
    ax.axhline(y=15, color='gold', linestyle='--', linewidth=2.5,
               label='Winning Threshold (15 points)', alpha=0.7)
    ax.set_xlabel('Turn Number', fontsize=13, fontweight='bold')
    ax.set_ylabel('Victory Points', fontsize=13, fontweight='bold')
    ax.set_ylim(0, max([max([d['points'] for d in player_data[pid]])
                        for pid in self.players.keys()]) + 2)
```

```
    ax.grid(True, alpha=0.3, linestyle='--')
    final_points = {}
    for player_id, player_info in self.players.items():
        data = player_data[player_id]
        if data:
            final_points[player_info['name']] = data[-1]['points']
    if len(final_points) == 2:
        names = list(final_points.keys())
        diff = abs(final_points[names[0]] - final_points[names[1]])
        winner = names[0] if final_points[names[0]] > final_points[names[1]] else names[1]
        ax.text(0.98, 0.02,
                f"Final: {names[0]}: {final_points[names[0]]} pts\n"
                f"Final: {names[1]}: {final_points[names[1]]} pts\n"
                f"Winner: {winner} (+{diff} pts)",
                transform=ax.transAxes,
                fontsize=10, verticalalignment='bottom', horizontalalignment='right',
                bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8),
                family='monospace')
    plt.title("Victory Points Evolution",
              fontsize=15, fontweight='bold', pad=20)
    ax.legend(loc='upper left', fontsize=11, framealpha=0.9)
    fig.tight_layout()
    output_path = self.game_folder / 'visual_1b_points_evolution.png'
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
    print(f"  [OK] Saved: {output_path}")
def visual_2_move_quality_assessment(self):
    """
    Visual 2: Move Quality Assessment (Accuracy Metrics)
    Strip plot showing move classifications for both players.
    Uses MCTS as "Ground Truth" judge.
    """
    print("Generating Visual 2: Move Quality Assessment...")
    quality_map = {
        'Best': {'score': 100, 'color': '#27AE60', 'label': 'Best Move'},
        'Good': {'score': 75, 'color': '#52B788', 'label': 'Inaccuracy'},
        'Inaccuracy': {'score': 50, 'color': '#F39C12', 'label': 'Mistake'},
        'Mistake': {'score': 25, 'color': '#E67E22', 'label': 'Mistake'},
        'Blunder': {'score': 0, 'color': '#E74C3C', 'label': 'Blunder'}
    }
    player_moves = {pid: [] for pid in self.players.keys()}
    for entry in self.game_log:
        player_id = entry['active_player']
        classification = entry.get('move_classification', 'Unknown')
        if classification in quality_map:
            player_moves[player_id].append({
                'turn': entry['turn_number'],
                'classification': classification,
                'score': quality_map[classification]['score'],
                'color': quality_map[classification]['color'],
                'accuracy_loss': entry.get('accuracy_loss', 0)
            })
    fig, axes = plt.subplots(1, 2, figsize=(16, 7), sharey=True)
    for idx, (player_id, player_info) in enumerate(self.players.items()):
        ax = axes[idx]
        moves = player_moves[player_id]
        if not moves:
            ax.text(0.5, 0.5, 'No Data Available',
```

```
                ha='center', va='center', fontsize=14)
            ax.set_title(player_info['name'], fontsize=13, fontweight='bold')
            continue
        turns = [m['turn'] for m in moves]
        scores = [m['score'] for m in moves]
        colors = [m['color'] for m in moves]
        ax.scatter(turns, scores, c=colors, s=100, alpha=0.7, edgecolors='black', linewidth=0.5)
        for cat, info in quality_map.items():
            ax.axhline(y=info['score'], color=info['color'],
                       linestyle='--', alpha=0.3, linewidth=1)
        total_moves = len(moves)
        best_count = sum(1 for m in moves if m['classification'] == 'Best')
        blunder_count = sum(1 for m in moves if m['classification'] == 'Blunder')
        avg_score = np.mean(scores)
        stats_text = (
            f"Total Moves: {total_moves}\n"
            f"Best Moves: {best_count} ({best_count/total_moves*100:.1f}%)\n"
            f"Blunders: {blunder_count} ({blunder_count/total_moves*100:.1f}%)\n"
            f"Avg Quality: {avg_score:.1f}/100"
        )
        ax.text(0.02, 0.98, stats_text, transform=ax.transAxes,
                fontsize=10, verticalalignment='top',
                bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8),
                family='monospace')
        ax.set_xlabel('Turn Number', fontsize=12, fontweight='bold')
        if idx == 0:
            ax.set_ylabel('Move Quality Score', fontsize=12, fontweight='bold')
        ax.set_title(player_info['name'], fontsize=13, fontweight='bold')
        ax.grid(True, alpha=0.3, axis='x')
        ax.set_ylim(-10, 110)
    plt.suptitle('Move Quality Strip Plot: Performance Comparison',
                 fontsize=15, fontweight='bold', y=0.98)
    legend_elements = [
        mpatches.Patch(color=info['color'], label=info['label'])
        for cat, info in quality_map.items()
    ]
    fig.legend(handles=legend_elements, loc='lower center',
               ncol=len(quality_map), fontsize=10, framealpha=0.9,
               bbox_to_anchor=(0.5, -0.02))
    fig.tight_layout(rect=[0, 0.02, 1, 0.96])
    output_path = self.game_folder / 'visual_2_move_quality_strip_plot.png'
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
    print(f"  [OK] Saved: {output_path}")
def visual_3_strategy_dna(self):
    """
    Visual 3: Strategic Profiling ("Strategy DNA")
    Comparative bar chart showing card tier distributions.
    Demonstrates qualitative difference in playstyle.
    """
    print("Generating Visual 3: Strategy DNA...")
    player_cards = {pid: {'Tier 1': 0, 'Tier 2': 0, 'Tier 3': 0, 'Nobles': 0}
                    for pid in self.players.keys()}
    for player_id in self.players.keys():
        player_moves = [e for e in self.game_log if e['active_player'] == player_id]
        if player_moves:
            final_move = player_moves[-1]
            tiers = final_move['cards_owned_by_tier']
```

```
            player_cards[player_id]['Tier 1'] = tiers[0]
            player_cards[player_id]['Tier 2'] = tiers[1]
            player_cards[player_id]['Tier 3'] = tiers[2]
            player_cards[player_id]['Nobles'] = final_move['nobles_owned']
        fig, ax = plt.subplots(figsize=(12, 7))
        categories = ['Tier 1\n(Cheap)', 'Tier 2\n(Medium)', 'Tier 3\n(Expensive)', 'Nobles']
        x = np.arange(len(categories))
        width = 0.35
        colors = ['#3498DB', '#E74C3C']
        for idx, (player_id, player_info) in enumerate(self.players.items()):
            values = list(player_cards[player_id].values())
            offset = width * (idx - 0.5)
            bars = ax.bar(x + offset, values, width, label=player_info['name'],
                          color=colors[idx], alpha=0.8, edgecolor='black', linewidth=1.5)
            for bar in bars:
                height = bar.get_height()
                if height > 0:
                    ax.text(bar.get_x() + bar.get_width()/2., height,
                            f'{int(height)}',
                            ha='center', va='bottom', fontweight='bold', fontsize=11)
        for player_id, player_info in self.players.items():
            cards = player_cards[player_id]
            total_cards = cards['Tier 1'] + cards['Tier 2'] + cards['Tier 3']
            if total_cards > 0:
                tier1_pct = cards['Tier 1'] / total_cards * 100
                tier3_pct = cards['Tier 3'] / total_cards * 100
                if tier1_pct > 60:
                    strategy = "\"Quantity over Quality\""
                elif tier3_pct > 30:
                    strategy = "\"Quality over Quantity\""
                else:
                    strategy = "\"Balanced\""
                print(f"    {player_info['name']}: {strategy} "
                      f"(T1: {tier1_pct:.0f}%, T3: {tier3_pct:.0f}%)")
        ax.set_xlabel('Development Card Tier', fontsize=13, fontweight='bold')
        ax.set_ylabel('Cards Acquired', fontsize=13, fontweight='bold')
        ax.set_title('Strategy DNA: Card Tier Distribution Comparison',
                     fontsize=15, fontweight='bold', pad=20)
        ax.set_xticks(x)
        ax.set_xticklabels(categories, fontsize=11)
        ax.legend(fontsize=11, loc='upper right', framealpha=0.9)
        ax.grid(True, alpha=0.3, axis='y')
        interpretation = (
            "Interpretation:\n"
            "• High Tier 1 = Greedy/Fast strategy (low points)\n"
            "• High Tier 3 = Investor strategy (high points)\n"
            "• Nobles = Strategic positioning bonus"
        )
        ax.text(0.02, 0.98, interpretation, transform=ax.transAxes,
                fontsize=9, verticalalignment='top',
                bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8),
                family='monospace')
        fig.tight_layout()
        output_path = self.game_folder / 'visual_3_strategy_dna.png'
        plt.savefig(output_path, dpi=300, bbox_inches='tight')
        plt.close()
        print(f"  [OK] Saved: {output_path}")
    def visual_4_computational_intensity(self):
```

```
        """
        Visual 4: Computational Intensity & Decision Time Comparison
        Shows both agents' decision-making patterns and computational effort.
        """
        print("Generating Visual 4: Computational Intensity...")
        player_data = {pid: [] for pid in self.players.keys()}
        for entry in self.game_log:
            player_id = entry['active_player']
            player_data[player_id].append({
                'turn': entry['turn_number'],
                'simulations': entry.get('simulation_count', 0),
                'depth': entry.get('search_depth', 0),
                'time_ms': entry['duration_ms'],
                'player_type': entry['player_type']
            })
        fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(16, 12))
        colors = ['#2E86AB', '#E74C3C']
        for idx, (player_id, player_info) in enumerate(self.players.items()):
            data = player_data[player_id]
            if not data:
                continue
            turns = [d['turn'] for d in data]
            times = [d['time_ms'] / 1000 for d in data]
            ax1.plot(turns, times, color=colors[idx], linewidth=2.5,
                     marker='o', markersize=4, label=player_info['name'], alpha=0.7)
        ax1.set_xlabel('Turn Number', fontsize=12, fontweight='bold')
        ax1.set_ylabel('Time (seconds)', fontsize=12, fontweight='bold')
        ax1.set_title('Decision Time Comparison', fontsize=13, fontweight='bold')
        ax1.grid(True, alpha=0.3)
        ax1.legend(loc='best', fontsize=10)
        mcts_player = next((pid for pid, pinfo in self.players.items() if pinfo['type'] == 'MCTS'),
None)
        if mcts_player is not None and player_data[mcts_player]:
            mcts_data = player_data[mcts_player]
            turns = [d['turn'] for d in mcts_data]
            simulations = [d['simulations'] for d in mcts_data]
            ax2.plot(turns, simulations, color='#2E86AB', linewidth=2.5,
                     marker='o', markersize=5, label='MCTS Simulations')
            ax2.fill_between(turns, simulations, alpha=0.3, color='#2E86AB')
            avg_sims = np.mean(simulations)
            ax2.axhline(y=avg_sims, color='red', linestyle='--',
                        linewidth=2, alpha=0.6, label=f'Avg: {avg_sims:.0f}')
            ax2.set_xlabel('Turn Number', fontsize=12, fontweight='bold')
            ax2.set_ylabel('Simulations Count', fontsize=12, fontweight='bold')
            ax2.set_title('MCTS Thinking Effort', fontsize=13, fontweight='bold')
            ax2.grid(True, alpha=0.3)
            ax2.legend(loc='best', fontsize=10)
        else:
            ax2.text(0.5, 0.5, 'No MCTS data', ha='center', va='center', fontsize=14)
            ax2.set_title('MCTS Thinking Effort', fontsize=13, fontweight='bold')
        if mcts_player is not None and player_data[mcts_player]:
            mcts_data = player_data[mcts_player]
            turns = [d['turn'] for d in mcts_data]
            depths = [d['depth'] for d in mcts_data]
            ax3.plot(turns, depths, color='#A23B72', linewidth=2.5,
                     marker='s', markersize=5, label='Search Depth')
            ax3.fill_between(turns, depths, alpha=0.3, color='#A23B72')
            ax3.set_xlabel('Turn Number', fontsize=12, fontweight='bold')
```

```
        ax3.set_ylabel('Max Search Depth', fontsize=12, fontweight='bold')
        ax3.set_title('MCTS Search Depth', fontsize=13, fontweight='bold')
        ax3.grid(True, alpha=0.3)
        ax3.legend(loc='best', fontsize=10)
    else:
        ax3.text(0.5, 0.5, 'No MCTS data', ha='center', va='center', fontsize=14)
        ax3.set_title('MCTS Search Depth', fontsize=13, fontweight='bold')
    avg_times = []
    player_names = []
    for player_id, player_info in self.players.items():
        data = player_data[player_id]
        if data:
            times = [d['time_ms'] / 1000 for d in data]
            avg_times.append(np.mean(times))
            player_names.append(player_info['name'])
    if avg_times:
        bars = ax4.bar(player_names, avg_times, color=colors[:len(avg_times)],
                       alpha=0.7, edgecolor='black', linewidth=2)
        for bar in bars:
            height = bar.get_height()
            ax4.text(bar.get_x() + bar.get_width()/2., height,
                     f'{height:.3f}s',
                     ha='center', va='bottom', fontweight='bold', fontsize=11)
        ax4.set_ylabel('Average Time (seconds)', fontsize=12, fontweight='bold')
        ax4.set_title('Average Decision Time per Move', fontsize=13, fontweight='bold')
        ax4.grid(True, alpha=0.3, axis='y')
    else:
        ax4.text(0.5, 0.5, 'No data', ha='center', va='center', fontsize=14)
        ax4.set_title('Average Decision Time', fontsize=13, fontweight='bold')
    if mcts_player is not None and player_data[mcts_player]:
        mcts_data = player_data[mcts_player]
        simulations = [d['simulations'] for d in mcts_data]
        depths = [d['depth'] for d in mcts_data]
        times = [d['time_ms'] / 1000 for d in mcts_data]
        stats_text = (
            f"MCTS Statistics:\n"
            f"Avg Sims: {np.mean(simulations):.0f}\n"
            f"Max Sims: {max(simulations)}\n"
            f"Avg Depth: {np.mean(depths):.1f}\n"
            f"Max Depth: {max(depths)}\n"
            f"Avg Time: {np.mean(times):.2f}s"
        )
        fig.text(0.98, 0.5, stats_text, fontsize=10,
                 verticalalignment='center', horizontalalignment='right',
                 bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.8),
                 family='monospace')
    plt.suptitle('Computational Intensity & Decision Time Comparison',
                 fontsize=15, fontweight='bold', y=0.995)
    fig.tight_layout(rect=[0, 0, 0.95, 0.99])
    output_path = self.game_folder / 'visual_4_computational_intensity.png'
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
    print(f"  [OK] Saved: {output_path}")
    if mcts_player is not None and player_data[mcts_player]:
        mcts_data = player_data[mcts_player]
        simulations = [d['simulations'] for d in mcts_data]
        depths = [d['depth'] for d in mcts_data]
        times = [d['time_ms'] / 1000 for d in mcts_data]
```

```
        print(f"    Insights:")
        print(f"      • Adaptive Depth: {'Yes' if max(depths) > min(depths) + 2 else 'No (consistent)'}")
        print(f"      • Performance: {np.mean(simulations):.0f} sims/turn in {np.mean(times):.2f}s avg")
        for player_id, player_info in self.players.items():
            data = player_data[player_id]
            if data:
                times = [d['time_ms'] / 1000 for d in data]
                print(f"      • {player_info['name']}: {np.mean(times)*1000:.1f}ms avg per move")
def generate_analytics_for_game(game_folder: str):
    """
    Convenience function to generate all analytics for a game.
    Args:
        game_folder: Path to game folder (e.g., 'data/game_20260119_215803')
    """
    try:
        analytics = PostGameAnalytics(game_folder)
        analytics.generate_all_visualizations()
        return True
    except Exception as e:
        print(f"❌ Error generating analytics: {e}")
        import traceback
        traceback.print_exc()
        return False
def main():
    """CLI entry point for manual analytics generation."""
    import sys
    if len(sys.argv) < 2:
        print("Usage: python generate_post_game_analytics.py <game_folder>")
        print("Example: python generate_post_game_analytics.py data/game_20260119_215803")
        sys.exit(1)
    game_folder = sys.argv[1]
    success = generate_analytics_for_game(game_folder)
    sys.exit(0 if success else 1)
if __name__ == '__main__':
    main()
`
```

knowledge_base.py

```
`python
import json
import os
from typing import Dict, Any
class MCTSKnowledgeBase:
    """
    Manages persistent learning for the MCTS AI.
    Loads and saves node evaluations (visits, value) mapped to game states.
    """
    def __init__(self, filepath: str = "data/mcts_knowledge.json"):
        self.filepath = filepath
        self.memory: Dict[str, Dict[str, Dict[str, float]]] = {}
        self.dirty = False
        self.load()
    def load(self):
```

```
        """Load the knowledge base from disk."""
        if os.path.exists(self.filepath):
            try:
                with open(self.filepath, "r", encoding="utf-8") as f:
                    self.memory = json.load(f)
                print(f"Loaded MCTS knowledge base from {self.filepath} ({len(self.memory)}
states).")
            except Exception as e:
                print(f"Error loading MCTS knowledge base: {e}")
                self.memory = {}
        else:
            self.memory = {}
            os.makedirs(os.path.dirname(self.filepath), exist_ok=True)
    def save(self):
        """Save the knowledge base to disk if there are changes."""
        if self.dirty:
            try:
                with open(self.filepath, "w", encoding="utf-8") as f:
                    json.dump(self.memory, f)
                print(f"Saved MCTS knowledge base to {self.filepath} ({len(self.memory)} states).")
                self.dirty = False
            except Exception as e:
                print(f"Error saving MCTS knowledge base: {e}")
    def update_node(self, state_hash: str, action_hash: str, visits: float, value: float):
        """Update or insert statistics for a specific state-action pair."""
        if state_hash not in self.memory:
            self.memory[state_hash] = {}
        if action_hash not in self.memory[state_hash]:
            self.memory[state_hash][action_hash] = {"visits": visits, "value": value}
        else:
            self.memory[state_hash][action_hash]["visits"] = max(visits,
self.memory[state_hash][action_hash]["visits"])
            self.memory[state_hash][action_hash]["value"] = value
        self.dirty = True
    def get_stats(self, state_hash: str, action_hash: str) -> Dict[str, float]:
        """Retrieve statistics for a specific state-action pair, if available."""
        if state_hash in self.memory and action_hash in self.memory[state_hash]:
            return self.memory[state_hash][action_hash]
        return None
`
```

logger_widget.py

```
`python
"""
Scrollable text log widget for move history.
"""
import pygame
from typing import List
class LoggerWidget:
    """
    Displays a scrollable text log of game moves.
    """
    def __init__(self, x: int, y: int, width: int, height: int):
        """
        Initialize the logger widget.
        Args:
            x, y: Top-left position
```

```
        width, height: Widget dimensions
    """
    self.x = x
    self.y = y
    self.width = width
    self.height = height
    self.logs: List[str] = []
    self.scroll_offset = 0
    self.line_height = 20
    self.font = pygame.font.Font(None, 18)
    self.bg_color = (50, 55, 60)
    self.text_color = (220, 220, 220)
    self.border_color = (100, 120, 140)
def add_log(self, text: str):
    """
    Add a new log entry.
    Args:
        text: Log message to add
    """
    self.logs.append(text)
    max_visible = (self.height - 50) // self.line_height
    if len(self.logs) > max_visible:
        self.scroll_offset = len(self.logs) - max_visible
def clear(self):
    """Clear all logs."""
    self.logs = []
    self.scroll_offset = 0
def scroll_up(self):
    """Scroll up one line."""
    if self.scroll_offset > 0:
        self.scroll_offset -= 1
def scroll_down(self):
    """Scroll down one line."""
    max_visible = (self.height - 50) // self.line_height
    if self.scroll_offset < len(self.logs) - max_visible:
        self.scroll_offset += 1
def draw(self, surface: pygame.Surface):
    """
    Draw the logger widget.
    Args:
        surface: Pygame surface to draw on
    """
    pygame.draw.rect(surface, self.bg_color,
                     (self.x, self.y, self.width, self.height),
                     border_radius=5)
    pygame.draw.rect(surface, self.border_color,
                     (self.x, self.y, self.width, self.height),
                     2, border_radius=5)
    title_font = pygame.font.Font(None, 24)
    title = title_font.render("Move Log", True, self.text_color)
    surface.blit(title, (self.x + 10, self.y + 10))
    log_y = self.y + 40
    max_visible = (self.height - 50) // self.line_height
    visible_logs = self.logs[self.scroll_offset:self.scroll_offset + max_visible]
    for log_text in visible_logs:
        if len(log_text) > 50:
            log_text = log_text[:47] + "..."
        text_surface = self.font.render(log_text, True, self.text_color)
```

```
            surface.blit(text_surface, (self.x + 10, log_y))
            log_y += self.line_height
        if len(self.logs) > max_visible:
            scroll_info = f"{self.scroll_offset + 1}-{min(self.scroll_offset + max_visible,
len(self.logs))} / {len(self.logs)}"
            scroll_text = self.font.render(scroll_info, True, (100, 100, 100))
            surface.blit(scroll_text, (self.x + self.width - 100, self.y + 10))
`
```

main.py

```
`python
"""
Splendor AI Battle - Main Entry Point
A fully playable GUI-based simulation of Splendor board game.
Features Basic AI vs MCTS AI with spectator mode and post-game analytics.
"""
import time
import random
import hashlib
from typing import Tuple
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from pathlib import Path
from game_engine.state import GameState
from game_engine.components import Action
from agents.basic import BasicAgent
from agents.advanced import MCTSAgent
from gui.window import GameWindow
from analytics.recorder import GameRecorder
from analytics.accuracy import MoveComparator
from generate_post_game_analytics import PostGameAnalytics
random.seed()
def hash_state(state: GameState) -> str:
    """Create a hash of the game state for tracking."""
    state_str = f"{state.turn_number}_{state.current_player}_{state.player_points}"
    return hashlib.md5(state_str.encode()).hexdigest()[:12]
def generate_player_graphs(analysis_data: dict, game_id: str, agents):
    """Generate individual graph for each player showing move accuracy."""
    try:
        player_analyses = analysis_data.get('player_analyses', {})
        if not player_analyses:
            print(" ⚠️ No player analyses found for graph generation")
            return
        for player_id_str, analysis in player_analyses.items():
            player_id = int(player_id_str)
            move_scores = analysis.get('move_scores', [])
            if not move_scores:
                print(f" ⚠️ No move scores for Player {player_id + 1}, skipping graph")
                continue
            fig, ax = plt.subplots(figsize=(12, 6))
            moves = list(range(1, len(move_scores) + 1))
            color = '#3498db' if player_id == 0 else '#e74c3c'
            ax.plot(moves, move_scores, color=color, linewidth=2,
                    marker='o', markersize=5, alpha=0.7)
```

```
            avg_score = analysis.get('quality_score', 0)
            ax.axhline(y=avg_score, color=color, linestyle='--',
                       linewidth=2, alpha=0.5, label=f'Average: {avg_score:.1f}')
            player_type = agents[player_id].name
            ax.set_xlabel('Move Number', fontsize=12, fontweight='bold')
            ax.set_ylabel('Move Quality Score (0-100)', fontsize=12, fontweight='bold')
            ax.set_title(f'Player {player_id + 1} ({player_type}) - Move-by-Move Quality Analysis',
                         fontsize=14, fontweight='bold')
            ax.grid(True, alpha=0.3)
            ax.legend(loc='best', fontsize=11)
            ax.set_ylim(0, 110)
            stats_text = (
                f"Total Moves: {analysis.get('total_moves', 0)}\n"
                f"Quality Score: {avg_score:.1f}/100\n"
                f"VP Moves: {analysis.get('vp_move_percent', 0):.1f}%\n"
                f"Buy Moves: {analysis.get('buy_move_percent', 0):.1f}%\n"
                f"Avg Time: {analysis.get('avg_time_ms', 0):.1f} ms"
            )
            ax.text(0.98, 0.97, stats_text, transform=ax.transAxes,
                    fontsize=10, verticalalignment='top', horizontalalignment='right',
                    bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5),
                    family='monospace')
            output_path = f"data/player_{player_id + 1}_{player_type}_accuracy_{game_id}.png"
            plt.tight_layout()
            plt.savefig(output_path, dpi=150, bbox_inches='tight')
            plt.close(fig)
            print(f"[OK] Player {player_id + 1} ({player_type}) graph saved: {output_path}")
        except Exception as e:
            print(f"✗ Error generating player graphs: {e}")
            import traceback
            traceback.print_exc()
    def run_game():
        """
        Main game loop.
        Handles:
        - GUI initialization
        - Game state management
        - Agent move execution
        - Move logging
        - Post-game analytics
        """
        window = GameWindow()
        game_state: GameState = None
        agents = []
        recorder: GameRecorder = None
        game_started = False
        turn_timer = 0
        last_turn_time = time.time()
        print("="*60)
        print("Splendor AI Battle")
        print("="*60)
        print("\nControls:")
        print("  - Click 'Start Game' to begin")
        print("  - Click 'Next Turn' for manual stepping")
        print("  - Click 'Auto' to toggle auto-play mode")
        print("  - Drag speed slider to adjust auto-play speed")
        print("  - Scroll in log area to view move history")
```

```
    print("\n" + "="*60 + "\n")
    while window.running:
        event = window.handle_events()
        if event == "quit":
            break
        elif event == "start_game":
            print("Starting new game...")
            game_state = GameState(num_players=2)
            agents = [
                BasicAgent(player_id=0),
                MCTSAgent(player_id=1, time_limit=2.0)
            ]
            recorder = GameRecorder()
            recorder.game_log = []
            window.start_game(game_state)
            window.logger.add_log(f"Player 1: {agents[0].name}")
            window.logger.add_log(f"Player 2: {agents[1].name}")
            window.logger.add_log("---")
            game_started = True
            turn_timer = 0
            last_turn_time = time.time()
            print(f"  Player 1: {agents[0].name}")
            print(f"  Player 2: {agents[1].name}")
            print()
        elif event == "next_turn":
            if game_state and not game_state.game_over:
                execute_turn(game_state, agents, window, recorder)
        if game_started and window.auto_play and not game_state.game_over:
            current_time = time.time()
            elapsed = current_time - last_turn_time
            if elapsed >= window.game_speed:
                execute_turn(game_state, agents, window, recorder)
                last_turn_time = current_time
        if game_state and game_state.game_over and game_started:
            game_started = False
            handle_game_over(game_state, agents, window, recorder)
        window.draw()
        window.update()
    window.quit()
def calculate_move_accuracy(game_state: GameState, player_id: int, actual_action: Action,
                            mcts_agent: MCTSAgent):
    """
    Calculate move accuracy by comparing actual move to MCTS best move.
    Treats MCTS top choice as the "Perfect Move" (100% accuracy).
    Args:
        game_state: Current game state (before move)
        player_id: Player making the move
        actual_action: The actual move the player will make
        mcts_agent: MCTS agent to use as accuracy engine
    Returns:
        Dictionary with accuracy metrics
    """
    original_player = mcts_agent.player_id
    mcts_agent.player_id = player_id
    try:
        best_action, metadata = mcts_agent.get_move(game_state, max_simulations=100)
        best_win_prob = metadata.get('win_probability', 0.5)
        if actual_action == best_action:
```

```
            return {
                'is_best_move': True,
                'best_win_prob': best_win_prob,
                'actual_win_prob': best_win_prob,
                'accuracy_loss': 0.0,
                'move_classification': 'Best',
                'best_action': str(best_action.type)
            }
        visit_dist = metadata.get('visit_distribution', [])
        actual_win_prob = None
        temp_state = game_state.get_state_copy()
        temp_state.apply_action(actual_action)
        opponent_eval = mcts_agent.evaluate_state_as_critic(
            temp_state,
            target_player=player_id,
            simulations=20
        )
        actual_win_prob = opponent_eval.get('win_probability', 0.5)
        accuracy_loss = best_win_prob - actual_win_prob
        if accuracy_loss < 0.02:
            classification = 'Best'
        elif accuracy_loss < 0.10:
            classification = 'Inaccuracy'
        elif accuracy_loss < 0.20:
            classification = 'Mistake'
        else:
            classification = 'Blunder'
        return {
            'is_best_move': False,
            'best_win_prob': best_win_prob,
            'actual_win_prob': actual_win_prob,
            'accuracy_loss': accuracy_loss,
            'move_classification': classification,
            'best_action': str(best_action.type)
        }
    finally:
        mcts_agent.player_id = original_player
def execute_turn(game_state: GameState, agents, window: GameWindow,
                 recorder: GameRecorder):
    """
    Execute a single turn.
    Args:
        game_state: Current game state
        agents: List of agents
        window: Game window
        recorder: Move recorder
    """
    current_player = game_state.current_player
    agent = agents[current_player]
    mcts_engine = agents[1] if isinstance(agents[1], MCTSAgent) else None
    start_time = time.time()
    if isinstance(agent, MCTSAgent):
        action, mcts_metadata = agent.get_move(game_state)
        win_probability = mcts_metadata.get('win_probability', 0.5)
        search_depth = mcts_metadata.get('search_depth', 0)
        simulation_count = mcts_metadata.get('simulation_count', 0)
        visit_distribution = mcts_metadata.get('visit_distribution', [])
        if 10 <= game_state.turn_number <= 15 and not hasattr(recorder, 'mcts_tree_snapshot'):
```

```
            from agents.advanced import MCTSNode
            snapshot_root = MCTSNode(game_state.get_state_copy())
            import time as time_module
            snapshot_start = time_module.time()
            snapshot_sims = 0
            while time_module.time() - snapshot_start < 2.0:
                node = agent._select(snapshot_root, agent.player_id)
                result = agent._simulate(node.state)
                node.backpropagate(result)
                snapshot_sims += 1
            recorder.mcts_tree_snapshot = snapshot_root
            recorder.mcts_tree_turn = game_state.turn_number
            print(f"  [Tree Snapshot] Captured MCTS tree at turn {game_state.turn_number}
({snapshot_sims} sims)")
        else:
            action = agent.select_move(game_state)
            win_probability = None
            search_depth = 0
            simulation_count = 0
            visit_distribution = []
        duration_ms = (time.time() - start_time) * 1000
        accuracy_data = None
        if mcts_engine:
            print(f"  [Accuracy Engine] Evaluating move quality...")
            accuracy_start = time.time()
            accuracy_data = calculate_move_accuracy(game_state, current_player, action,
mcts_engine)
            accuracy_time = (time.time() - accuracy_start) * 1000
            classification = accuracy_data.get('move_classification', 'Unknown')
            loss = accuracy_data.get('accuracy_loss', 0)
            print(f"  [Accuracy Engine] Classification: {classification} (Loss: {loss:.3f})
[{accuracy_time:.0f}ms]")
        state_hash = hash_state(game_state)
        agent_type = "Basic" if isinstance(agent, BasicAgent) else "MCTS"
        recorder.record_move(
            turn_id=game_state.turn_number,
            player_id=current_player,
            player_type=agent_type,
            action=action,
            state_hash=state_hash,
            duration_ms=duration_ms
        )
        move_desc = agent.get_move_description(action)
        log_text = f"P{current_player + 1}: {move_desc}"
        if isinstance(agent, MCTSAgent):
            log_text += f" ({simulation_count} sims)"
        window.logger.add_log(f"Turn {game_state.turn_number} | {log_text}")
        print(f"  Turn {game_state.turn_number} | {log_text} [{duration_ms:.0f}ms]")
        gems_before = sum(game_state.player_gems[current_player].values())
        game_state.apply_action(action)
        gems_after = sum(game_state.player_gems[current_player].values())
        gems_returned = max(0, gems_before - gems_after) if gems_after <= 10 else 0
        if isinstance(agent, BasicAgent):
            mcts_critic = agents[1] if isinstance(agents[1], MCTSAgent) else None
            if mcts_critic:
                critic_result = mcts_critic.evaluate_state_as_critic(
                    game_state,
                    target_player=current_player,
```

```
            simulations=50
        )
        win_probability = critic_result.get('win_probability', 0.5)
        print(f"    [MCTS Critic] Win probability for P{current_player + 1}:
{win_probability:.2f}")
    cards_by_tier = [0, 0, 0]
    for card in game_state.player_cards[current_player]:
        if card.tier in [1, 2, 3]:
            cards_by_tier[card.tier - 1] += 1
    nobles_owned = len(game_state.player_nobles[current_player])
    log_entry = {
        'turn_number': game_state.turn_number,
        'active_player': current_player,
        'player_type': agent_type,
        'cards_owned_by_tier': cards_by_tier,
        'nobles_owned': nobles_owned,
        'gems_returned': gems_returned,
        'win_probability': win_probability,
        'search_depth': search_depth,
        'simulation_count': simulation_count,
        'visit_distribution': visit_distribution,
        'player_points': game_state.player_points[current_player],
        'action_type': action.type,
        'duration_ms': duration_ms,
        'is_best_move': accuracy_data.get('is_best_move', False) if accuracy_data else None,
        'best_win_prob': accuracy_data.get('best_win_prob', 0) if accuracy_data else None,
        'actual_win_prob': accuracy_data.get('actual_win_prob', 0) if accuracy_data else None,
        'accuracy_loss': accuracy_data.get('accuracy_loss', 0) if accuracy_data else None,
        'move_classification': accuracy_data.get('move_classification', 'Unknown') if accuracy_data
else None,
        'best_action': accuracy_data.get('best_action', '') if accuracy_data else None
    }
    if hasattr(recorder, 'game_log'):
        recorder.game_log.append(log_entry)
    else:
        recorder.game_log = [log_entry]
def handle_game_over(game_state: GameState, agents, window: GameWindow,
                     recorder: GameRecorder):
    """
    Handle game over: show results and run analytics.
    Args:
        game_state: Final game state
        agents: List of agents
        window: Game window
        recorder: Move recorder
    """
    winner = game_state.winner
    final_scores = {i: game_state.player_points[i] for i in range(game_state.num_players)}
    recorder.finalize_game(winner, final_scores)
    window.logger.add_log("---")
    window.logger.add_log("GAME OVER!")
    window.logger.add_log(f"Winner: Player {winner + 1} ({agents[winner].name})")
    for i, score in final_scores.items():
        window.logger.add_log(f"  P{i + 1}: {score} points")
    print("\n" + "="*60)
    print("GAME OVER!")
    print(f"Winner: Player {winner + 1} ({agents[winner].name})")
    print(f"Final Scores:")
```

```
for i, score in final_scores.items():
    window.logger.add_log(f"  Player {i + 1}: {score} points")
print("="*60)
json_path = recorder.save_json()
csv_path = recorder.save_csv()
import json
from pathlib import Path
master_file = Path("data/games.json")
master_file.parent.mkdir(parents=True, exist_ok=True)
game_data = {
    'game_id': recorder.game_id,
    'metadata': recorder.game_metadata,
    'moves': [move.to_dict() for move in recorder.moves],
    'game_log': recorder.game_log
}
try:
    games_list = []
    if master_file.exists():
        with open(master_file, 'r') as f:
            try:
                games_list = json.load(f)
            except json.JSONDecodeError:
                games_list = []
    games_list.append(game_data)
    with open(master_file, 'w') as f:
        json.dump(games_list, f, indent=2)
    print(f"\nGame saved to master file: {master_file}")
except Exception as e:
    print(f"\n ⚠️ Failed to save to master games file: {e}")
print(f"Game logs also saved individually to:")
print(f"  JSON: {json_path}")
print(f"  CSV: {csv_path}")
print("\n" + "="*60)
print("Running Post-Game Analytics...")
print("="*60)
window.logger.add_log("---")
window.logger.add_log("Analyzing game...")
game_log = recorder.game_log if hasattr(recorder, 'game_log') else []
if game_log:
    p1_turns = [t for t in game_log if t['active_player'] == 0]
    p2_turns = [t for t in game_log if t['active_player'] == 1]
    print("\n" + "─" * 60)
    print("PERFORMANCE SUMMARY")
    print("─" * 60)
    if p1_turns and p2_turns:
        import numpy as np
        p1_probs = [t['win_probability'] for t in p1_turns if t.get('win_probability') is not None]
        p2_probs = [t['win_probability'] for t in p2_turns if t.get('win_probability') is not None]
        if p1_probs and p2_probs:
            p1_avg = np.mean(p1_probs)
            p2_avg = np.mean(p2_probs)
            p1_final = p1_probs[-1] if p1_probs else 0
            p2_final = p2_probs[-1] if p2_probs else 0
            print(f"Heuristic Agent:")
            print(f"  Average Win Probability: {p1_avg:.1%}")
            print(f"  Final Win Probability:   {p1_final:.1%}")
            print(f"\nMCTS Agent:")
```

```
            print(f"  Average Win Probability: {p2_avg:.1%}")
            print(f"  Final Win Probability:   {p2_final:.1%}")
            print(f"\nAdvantage: {abs(p2_avg - p1_avg):.1%} (MCTS)")
        has_accuracy = any(t.get('move_classification') is not None for t in game_log)
        if has_accuracy:
            print("\n" + "═" * 60)
            print("ACCURACY SUMMARY")
            print("═" * 60)
            p1_classifications = [t.get('move_classification', 'Unknown') for t in p1_turns]
            p2_classifications = [t.get('move_classification', 'Unknown') for t in p2_turns]
            p1_best = p1_classifications.count('Best')
            p1_inaccuracy = p1_classifications.count('Inaccuracy')
            p1_mistake = p1_classifications.count('Mistake')
            p1_blunder = p1_classifications.count('Blunder')
            p1_total = len(p1_turns)
            p2_best = p2_classifications.count('Best')
            p2_inaccuracy = p2_classifications.count('Inaccuracy')
            p2_mistake = p2_classifications.count('Mistake')
            p2_blunder = p2_classifications.count('Blunder')
            p2_total = len(p2_turns)
            p1_losses = [t.get('accuracy_loss', 0) for t in p1_turns if t.get('accuracy_loss') is not
None]
            p2_losses = [t.get('accuracy_loss', 0) for t in p2_turns if t.get('accuracy_loss') is not
None]
            import numpy as np
            p1_avg_loss = np.mean(p1_losses) if p1_losses else 0
            p2_avg_loss = np.mean(p2_losses) if p2_losses else 0
            def calc_accuracy_score(best, inaccuracy, mistake, blunder, total):
                if total == 0:
                    return 0
                return (best * 100 + inaccuracy * 90 + mistake * 70 + blunder * 40) / total
            p1_accuracy = calc_accuracy_score(p1_best, p1_inaccuracy, p1_mistake, p1_blunder,
p1_total)
            p2_accuracy = calc_accuracy_score(p2_best, p2_inaccuracy, p2_mistake, p2_blunder,
p2_total)
            print(f"Player 1 (Heuristic):")
            print(f"  Best:       {p1_best}/{p1_total} ({p1_best/p1_total*100:.0f}%)")
            print(f"  Inaccuracy: {p1_inaccuracy}/{p1_total}
({p1_inaccuracy/p1_total*100:.0f}%)")
            print(f"  Mistake:    {p1_mistake}/{p1_total} ({p1_mistake/p1_total*100:.0f}%)")
            print(f"  Blunder:    {p1_blunder}/{p1_total} ({p1_blunder/p1_total*100:.0f}%)")
            print(f"  Avg Loss:   {p1_avg_loss*100:.1f}%")
            print(f"  Accuracy:   {p1_accuracy:.1f}/100")
            print(f"\nPlayer 2 (MCTS):")
            print(f"  Best:       {p2_best}/{p2_total} ({p2_best/p2_total*100:.0f}%)")
            print(f"  Inaccuracy: {p2_inaccuracy}/{p2_total}
({p2_inaccuracy/p2_total*100:.0f}%)")
            print(f"  Mistake:    {p2_mistake}/{p2_total} ({p2_mistake/p2_total*100:.0f}%)")
            print(f"  Blunder:    {p2_blunder}/{p2_total} ({p2_blunder/p2_total*100:.0f}%)")
            print(f"  Avg Loss:   {p2_avg_loss*100:.1f}%")
            print(f"  Accuracy:   {p2_accuracy:.1f}/100")
            print(f"\nDifference: {abs(p2_accuracy - p1_accuracy):.1f} pts")
        print("\n" + "═" * 60)
        print("STRATEGY ANALYSIS")
        print("═" * 60)
        p1_final_cards = p1_turns[-1]['cards_owned_by_tier'] if p1_turns else [0, 0, 0]
        p2_final_cards = p2_turns[-1]['cards_owned_by_tier'] if p2_turns else [0, 0, 0]
        p1_tier1, p1_tier2, p1_tier3 = p1_final_cards
```

```
        p2_tier1, p2_tier2, p2_tier3 = p2_final_cards
        p1_total_cards = sum(p1_final_cards)
        p2_total_cards = sum(p2_final_cards)
        print(f"Heuristic Agent:")
        print(f"  Tier 1: {p1_tier1}/{p1_total_cards} ({p1_tier1/p1_total_cards*100:.0f}% Entry
cards)" if p1_total_cards > 0 else "  Tier 1: 0")
        print(f"  Tier 2: {p1_tier2}/{p1_total_cards} ({p1_tier2/p1_total_cards*100:.0f}% Mid-
tier)" if p1_total_cards > 0 else "  Tier 2: 0")
        print(f"  Tier 3: {p1_tier3}/{p1_total_cards} ({p1_tier3/p1_total_cards*100:.0f}%
Premium)" if p1_total_cards > 0 else "  Tier 3: 0")
        print(f"\nMCTS Agent:")
        print(f"  Tier 1: {p2_tier1}/{p2_total_cards} ({p2_tier1/p2_total_cards*100:.0f}% Entry
cards)" if p2_total_cards > 0 else "  Tier 1: 0")
        print(f"  Tier 2: {p2_tier2}/{p2_total_cards} ({p2_tier2/p2_total_cards*100:.0f}% Mid-
tier)" if p2_total_cards > 0 else "  Tier 2: 0")
        print(f"  Tier 3: {p2_tier3}/{p2_total_cards} ({p2_tier3/p2_total_cards*100:.0f}%
Premium)" if p2_total_cards > 0 else "  Tier 3: 0")
        print("\n" + "─" * 60)
        print("COMPUTATIONAL EFFORT (MCTS Agent)")
        print("─" * 60)
        p2_sims = [t.get('simulation_count', 0) for t in p2_turns]
        p2_depths = [t.get('search_depth', 0) for t in p2_turns]
        if p2_sims and p2_depths:
            import numpy as np
            avg_sims = np.mean(p2_sims)
            max_sims = max(p2_sims)
            avg_depth = np.mean(p2_depths)
            max_depth = max(p2_depths)
            print(f"Simulations:")
            print(f"  Average:  {avg_sims:.0f}")
            print(f"  Peak:     {max_sims:.0f}")
            print(f"\nSearch Depth:")
            print(f"  Average:  {avg_depth:.1f}")
            print(f"  Maximum:  {max_depth}")
    print("\n" + "="*60)
    print("="*60)
    print("Auto-Generating Visualizations...")
    print("="*60)
    from datetime import datetime
    date_folder = Path('data/Games')
    date_folder.mkdir(parents=True, exist_ok=True)
    game_count = len([d for d in date_folder.iterdir() if d.is_dir()]) + 1
    game_folder = date_folder / f'game_{game_count}'
    game_folder.mkdir(parents=True, exist_ok=True)
    print(f"\nSaving visualizations to: {game_folder}/")
    try:
        from generate_comparison_visuals import (
            visual_1_battle_of_confidence,
            visual_2_thinking_effort,
            visual_3_strategy_dna
        )
        from plot_move_accuracy import plot_accuracy_strip_chart
        game_log = recorder.game_log if hasattr(recorder, 'game_log') else []
        if game_log:
            print("\nGenerating comparison visuals...")
            try:
                visual_1_battle_of_confidence(game_log, output_path=str(game_folder /
'visual_1_battle_of_confidence.png'))
```

```
        except Exception as e:
            print(f"  ⚠️ Visual 1 skipped: {e}")
        try:
            visual_2_thinking_effort(game_log, output_path=str(game_folder /
'visual_2_thinking_effort.png'))
        except Exception as e:
            print(f"  ⚠️ Visual 2 skipped: {e}")
        try:
            visual_3_strategy_dna(game_log, output_path=str(game_folder /
'visual_3_strategy_dna.png'))
        except Exception as e:
            print(f"  ⚠️ Visual 3 skipped: {e}")
        has_accuracy = any(t.get('move_classification') is not None for t in game_log)
        if has_accuracy:
            print("\nGenerating move accuracy plot...")
            try:
                plot_accuracy_strip_chart(game_log, output_path=str(game_folder /
'move_accuracy_strip_plot.png'))
            except Exception as e:
                print(f"  ⚠️ Accuracy plot skipped: {e}")
        else:
            print("\n⚠️ Skipping accuracy plot (no accuracy data)")
        print("\nGenerating MCTS tree visualization...")
        try:
            if hasattr(recorder, 'mcts_tree_snapshot') and recorder.mcts_tree_snapshot:
                from pyvis.network import Network
                import json
                from datetime import datetime
                mcts_agent = None
                for agent in agents:
                    if isinstance(agent, MCTSAgent):
                        mcts_agent = agent
                        break
                if mcts_agent:
                    print(f"  Using captured tree from turn {recorder.mcts_tree_turn}")
                    tree_data = mcts_agent.export_tree_structure(
                        root_node=recorder.mcts_tree_snapshot,
                        max_depth=4,
                        top_k=4,
                        turn_number=recorder.mcts_tree_turn
                    )
                    print(f"  Nodes: {len(tree_data['nodes'])}, Edges: {len(tree_data['edges'])}")
                    net = Network(
                        height='900px',
                        width='100%',
                        bgcolor='#222222',
                        font_color='white',
                        directed=True
                    )
                    net.set_options("""
                    {
                        "physics": {
                            "enabled": true,
                            "barnesHut": {
                                "gravitationalConstant": -8000,
                                "centralGravity": 0.3,
                                "springLength": 150,
```

```
                    "springConstant": 0.04,
                    "damping": 0.09
                }
            },
            "layout": {
                "hierarchical": {
                    "enabled": true,
                    "direction": "UD",
                    "sortMethod": "directed",
                    "levelSeparation": 150
                }
            },
            "interaction": {
                "navigationButtons": true,
                "keyboard": true
            }
        }
        """)
        for node in tree_data['nodes']:
            net.add_node(
                node['id'],
                label=node['label'],
                color=node['color'],
                borderWidth=node.get('borderWidth', 2),
                size=node['size'],
                title=node['title']
            )
        for edge in tree_data['edges']:
            net.add_edge(
                edge['from'],
                edge['to'],
                width=edge.get('width', 2),
                color=edge.get('color', '#848484'),
                arrows=edge.get('arrows', 'to'),
                dashes=edge.get('dashes', False),
                smooth=edge.get('smooth', {'type': 'curvedCW'})
            )
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        html_filename = f"mcts_brain_{timestamp}.html"
        html_filepath = game_folder / html_filename
        net.save_graph(str(html_filepath))
        json_filename = f"mcts_tree_{timestamp}.json"
        json_filepath = game_folder / json_filename
        with open(json_filepath, 'w') as f:
            json.dump({
                'turn_captured': recorder.mcts_tree_turn,
                'tree_data': tree_data
            }, f, indent=2)
        print(f"  [OK] Tree HTML: {html_filename}")
        print(f"  [OK] Tree JSON: {json_filename}")
    else:
        print("  ⚠️ No MCTS agent found")
else:
    print("  ⚠️ No tree snapshot captured during gameplay")
    print("     (Tree is captured during turns 10-15)")
except Exception as e:
    print(f"  ⚠️ Tree visualization skipped: {e}")
```

```
            import traceback
            traceback.print_exc()
        print("\n" + "="*60)
        print(f"All visualizations saved to: {game_folder}/")
        print("="*60)
        print("\n" + "="*60)
        print("Generating Comprehensive Analytics Package...")
        print("="*60)
        try:
            analytics = PostGameAnalytics(str(game_folder))
            analytics.generate_all_visualizations()
        except Exception as e:
            print(f"❌ Comprehensive analytics failed: {e}")
            import traceback
            traceback.print_exc()
    else:
        print("⚠️ No game_log data available for visualizations")
    for agent in agents:
        if hasattr(agent, "kb") and hasattr(agent.kb, "save"):
            try:
                agent.kb.save()
                print(f"  [OK] Saved knowledge base for agent {agent.name}")
            except Exception as e:
                print(f"  ⚠️ Failed to save knowledge base for {agent.name}: {e}")
except Exception as e:
    print(f"⚠️ Visualization generation failed: {e}")
    import traceback
    traceback.print_exc()
print("\n" + "="*60 + "\n")
window.auto_play = False
if __name__ == "__main__":
    run_game()
`
```

overall_analytics.py

```
`python
import json
import matplotlib.pyplot as plt
import numpy as np
from pathlib import Path
import tkinter as tk
from tkinter import ttk
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
def load_data():
    master_file = Path("games.json")
    if not master_file.exists():
        master_file = Path("data/games.json")
    if not master_file.exists():
        return []
    with open(master_file, "r") as f:
        try:
            return json.load(f)
        except json.JSONDecodeError:
            return []
class Dashboard(tk.Tk):
```

```
    def __init__(self):
        super().__init__()
        self.title("🧠 Splendor MCTS Learning Dashboard")
        self.geometry("1000x800")
        self.top_frame = tk.Frame(self, bg="#2c3e50", pady=15)
        self.top_frame.pack(fill=tk.X)
        tk.Label(self.top_frame, text="🧠 Splendor MCTS Learning Dashboard",
font=("Helvetica", 16, "bold"), fg="white", bg="#2c3e50").pack()
        self.metrics_frame = tk.Frame(self.top_frame, bg="#2c3e50")
        self.metrics_frame.pack(pady=10)
        self.metrics_label = tk.Label(self.metrics_frame, text="Loading...", font=("Helvetica", 12),
fg="#ecf0f1", bg="#2c3e50")
        self.metrics_label.pack()
        self.fig = plt.Figure(figsize=(10, 6), dpi=100)
        self.fig.patch.set_facecolor('#f5f6fa')
        self.ax1 = self.fig.add_subplot(221)
        self.ax2 = self.fig.add_subplot(222)
        self.ax3 = self.fig.add_subplot(212)
        self.canvas = FigureCanvasTkAgg(self.fig, master=self)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
        self.last_game_count = -1
        self.update_dashboard()
    def update_dashboard(self):
        games = load_data()
        if len(games) > self.last_game_count:
            self.last_game_count = len(games)
            self.refresh_plots(games)
        self.after(5000, self.update_dashboard)
    def refresh_plots(self, games):
        if not games:
            self.metrics_label.config(text="No games simulated yet! Run `python
simulate_batch.py` to start generating data.")
            return
        mcts_wins = 0
        basic_wins = 0
        game_numbers = []
        mcts_win_flags = []
        kb_sizes = []
        total_turns = []
        for i, g in enumerate(games):
            metadata = g.get('metadata', {})
            winner = metadata.get('winner', -1)
            if winner == 1:
                mcts_wins += 1
                mcts_won = True
            else:
                basic_wins += 1
                mcts_won = False
            mcts_kb_size = g.get('mcts_kb_size')
            if not mcts_kb_size:
                if len(kb_sizes) > 0:
                    mcts_kb_size = kb_sizes[-1]
                else:
                    mcts_kb_size = 0
            game_numbers.append(i + 1)
            mcts_win_flags.append(1 if mcts_won else 0)
            kb_sizes.append(mcts_kb_size)
```

```
            total_turns.append(metadata.get('total_turns', 0))
        window = min(10, len(mcts_win_flags))
        if window == 0:
            window = 1
        rolling_win_rate = []
        for i in range(len(mcts_win_flags)):
            start_idx = max(0, i - window + 1)
            window_slice = mcts_win_flags[start_idx:i+1]
            rolling_win_rate.append((sum(window_slice) / len(window_slice)) * 100)
        overall_win_rate = (mcts_wins / len(games)) * 100 if len(games) > 0 else 0
        latest_kb = kb_sizes[-1] if kb_sizes else 0
        metrics_text = (
            f"Total Games: {len(games)}  |  "
            f"MCTS Wins: {mcts_wins} - Normal AI Wins: {basic_wins}  |  "
            f"Overall Win Rate: {overall_win_rate:.1f}%  |  "
            f"Total States Memorized: {latest_kb}"
        )
        self.metrics_label.config(text=metrics_text)
        self.ax1.clear()
        self.ax2.clear()
        self.ax3.clear()
        self.ax1.plot(game_numbers, rolling_win_rate, color='#e74c3c', marker='o', markersize=4,
linestyle='-', linewidth=2)
        self.ax1.set_title(" 📈 MCTS Win Rate Over Time (Rolling Avg)", fontsize=10,
fontweight='bold')
        self.ax1.set_xlabel("Game Number")
        self.ax1.set_ylabel("Win Rate (%)")
        self.ax1.set_ylim(-5, 105)
        self.ax1.grid(True, alpha=0.3)
        self.ax2.plot(game_numbers, kb_sizes, color='#3498db', marker='s', markersize=4,
linestyle='-', linewidth=2)
        self.ax2.set_title(" 🧠 Knowledge Base Growth", fontsize=10, fontweight='bold')
        self.ax2.set_xlabel("Game Number")
        self.ax2.set_ylabel("States Memorized")
        self.ax2.grid(True, alpha=0.3)
        mcts_x = [game_numbers[i] for i in range(len(mcts_win_flags)) if mcts_win_flags[i] == 1]
        mcts_y = [total_turns[i] for i in range(len(mcts_win_flags)) if mcts_win_flags[i] == 1]
        basic_x = [game_numbers[i] for i in range(len(mcts_win_flags)) if mcts_win_flags[i] == 0]
        basic_y = [total_turns[i] for i in range(len(mcts_win_flags)) if mcts_win_flags[i] == 0]
        self.ax3.scatter(mcts_x, mcts_y, color='#e74c3c', label='MCTS Win', alpha=0.7, s=50)
        self.ax3.scatter(basic_x, basic_y, color='#3498db', label='Basic Win', alpha=0.7, s=50)
        self.ax3.set_title(" ⏱ Game Efficiency (Total Turns per Game)", fontsize=10,
fontweight='bold')
        self.ax3.set_xlabel("Game Number")
        self.ax3.set_ylabel("Total Turns")
        self.ax3.legend()
        self.ax3.grid(True, alpha=0.3)
        self.fig.tight_layout()
        self.canvas.draw()
def main():
    app = Dashboard()
    app.mainloop()
if __name__ == "__main__":
    main()
`
```

plot_move_accuracy.py

`python

```
"""
Move Accuracy Visualization - Strip Plot
Shows move quality classification for both players across all turns.
"""
import json
import matplotlib.pyplot as plt
import numpy as np
from pathlib import Path
import sys
def load_game_log(json_path):
    """Load game_log from JSON file."""
    with open(json_path, 'r') as f:
        data = json.load(f)
    game_log = data.get('game_log', [])
    if not game_log:
        raise ValueError("No game_log found in JSON file!")
    return game_log, data.get('metadata', {})
def plot_accuracy_strip_chart(game_log, output_path='data/move_accuracy_strip_plot.png'):
    """
    Create a "Move Quality Strip Plot" showing accuracy classification.
    X-Axis: Turn Number
    Y-Axis: Player (Player 1 vs Player 2)
    Visual: Colored dots for each move (Green=Best, Yellow=Inaccuracy, Orange=Mistake,
Red=Blunder)
    """
    color_map = {
        'Best': '#27ae60',
        'Inaccuracy': '#f39c12',
        'Mistake': '#e67e22',
        'Blunder': '#e74c3c'
    }
    p1_data = [(t['turn_number'], t.get('move_classification', 'Unknown'),
                t.get('accuracy_loss', 0))
               for t in game_log if t['active_player'] == 0]
    p2_data = [(t['turn_number'], t.get('move_classification', 'Unknown'),
                t.get('accuracy_loss', 0))
               for t in game_log if t['active_player'] == 1]
    if not p1_data or not p2_data:
        print(" ⚠️ Warning: Insufficient accuracy data")
        return
    fig, ax = plt.subplots(figsize=(18, 11))
    fig.patch.set_facecolor('#f8f9fa')
    ax.set_facecolor('#ffffff')
    plt.subplots_adjust(bottom=0.23)
    for turn, classification, loss in p1_data:
        color = color_map.get(classification, '#95a5a6')
        ax.scatter(turn, 1, s=200, c=color, edgecolors='#2c3e50',
                   linewidth=2, alpha=0.9, zorder=3)
        if classification == 'Blunder' and loss > 0:
            ax.annotate(f'-{loss*100:.0f}%',
                        xy=(turn, 1), xytext=(0, 15),
                        textcoords='offset points', ha='center',
                        fontsize=9, fontweight='bold', color='#e74c3c',
                        bbox=dict(boxstyle='round,pad=0.3',
```

```
                    facecolor='white', edgecolor='#e74c3c',
                    linewidth=1.5, alpha=0.9))
for turn, classification, loss in p2_data:
    color = color_map.get(classification, '#95a5a6')
    ax.scatter(turn, 2, s=200, c=color, edgecolors='#2c3e50',
            linewidth=2, alpha=0.9, zorder=3)
    if classification == 'Blunder' and loss > 0:
        ax.annotate(f'-{loss*100:.0f}%',
                xy=(turn, 2), xytext=(0, 15),
                textcoords='offset points', ha='center',
                fontsize=9, fontweight='bold', color='#e74c3c',
                bbox=dict(boxstyle='round,pad=0.3',
                    facecolor='white', edgecolor='#e74c3c',
                    linewidth=1.5, alpha=0.9))
ax.set_xlabel('Turn Number', fontsize=16, fontweight='bold', color='#2c3e50')
ax.set_ylabel('Player', fontsize=16, fontweight='bold', color='#2c3e50')
ax.set_title('Move Quality Strip Plot\nAccuracy Classification by Turn (MCTS as Reference)',
          fontsize=20, fontweight='bold', color='#2c3e50', pad=20)
ax.set_yticks([1, 2])
ax.set_yticklabels(['Player 1\n(Heuristic)', 'Player 2\n(MCTS)'],
                fontsize=13, fontweight='bold')
ax.set_ylim(0.5, 2.5)
max_turn = max(max(t[0] for t in p1_data), max(t[0] for t in p2_data))
ax.set_xlim(0, max_turn + 1)
ax.tick_params(axis='x', labelsize=12)
ax.grid(True, alpha=0.3, linestyle='--', linewidth=1, axis='x')
ax.axhline(y=1.5, color='#95a5a6', linestyle='-', linewidth=1, alpha=0.3)
ax.set_axisbelow(True)
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor=color_map['Best'], edgecolor='#2c3e50',
        linewidth=2, label='Best Move (Loss < 2%)'),
    Patch(facecolor=color_map['Inaccuracy'], edgecolor='#2c3e50',
        linewidth=2, label='Inaccuracy (2-10%)'),
    Patch(facecolor=color_map['Mistake'], edgecolor='#2c3e50',
        linewidth=2, label='Mistake (10-20%)'),
    Patch(facecolor=color_map['Blunder'], edgecolor='#2c3e50',
        linewidth=2, label='Blunder (>20%)')
]
ax.legend(handles=legend_elements, loc='upper left', fontsize=12,
        framealpha=0.95, edgecolor='#2c3e50', fancybox=True, shadow=True)
p1_classifications = [t[1] for t in p1_data]
p2_classifications = [t[1] for t in p2_data]
p1_best = p1_classifications.count('Best')
p1_inaccuracy = p1_classifications.count('Inaccuracy')
p1_mistake = p1_classifications.count('Mistake')
p1_blunder = p1_classifications.count('Blunder')
p1_total = len(p1_data)
p2_best = p2_classifications.count('Best')
p2_inaccuracy = p2_classifications.count('Inaccuracy')
p2_mistake = p2_classifications.count('Mistake')
p2_blunder = p2_classifications.count('Blunder')
p2_total = len(p2_data)
def calc_accuracy_score(best, inaccuracy, mistake, blunder, total):
    if total == 0:
        return 0
    score = (best * 100 + inaccuracy * 90 + mistake * 70 + blunder * 40) / total
    return score
```

```
    p1_accuracy_score = calc_accuracy_score(p1_best, p1_inaccuracy, p1_mistake, p1_blunder,
p1_total)
    p2_accuracy_score = calc_accuracy_score(p2_best, p2_inaccuracy, p2_mistake, p2_blunder,
p2_total)
    p1_avg_loss = np.mean([t[2] for t in p1_data if t[2] is not None])
    p2_avg_loss = np.mean([t[2] for t in p2_data if t[2] is not None])
    stats_text = (
        f"ACCURACY SUMMARY\n"
        f"{'─' * 40}\n"
        f"Player 1 (Heuristic):\n"
        f"  Best:       {p1_best}/{p1_total} ({p1_best/p1_total*100:.0f}%)\n"
        f"  Inaccuracy: {p1_inaccuracy}/{p1_total} ({p1_inaccuracy/p1_total*100:.0f}%)\n"
        f"  Mistake:    {p1_mistake}/{p1_total} ({p1_mistake/p1_total*100:.0f}%)\n"
        f"  Blunder:    {p1_blunder}/{p1_total} ({p1_blunder/p1_total*100:.0f}%)\n"
        f"  Avg Loss:   {p1_avg_loss*100:.1f}%\n"
        f"  Accuracy:   {p1_accuracy_score:.1f}/100\n"
        f"\n"
        f"Player 2 (MCTS):\n"
        f"  Best:       {p2_best}/{p2_total} ({p2_best/p2_total*100:.0f}%)\n"
        f"  Inaccuracy: {p2_inaccuracy}/{p2_total} ({p2_inaccuracy/p2_total*100:.0f}%)\n"
        f"  Mistake:    {p2_mistake}/{p2_total} ({p2_mistake/p2_total*100:.0f}%)\n"
        f"  Blunder:    {p2_blunder}/{p2_total} ({p2_blunder/p2_total*100:.0f}%)\n"
        f"  Avg Loss:   {p2_avg_loss*100:.1f}%\n"
        f"  Accuracy:   {p2_accuracy_score:.1f}/100\n"
        f"\n"
        f"Difference: {abs(p2_accuracy_score - p1_accuracy_score):.1f} pts"
    )
    fig.text(0.5, 0.10, stats_text,
             fontsize=10, verticalalignment='top', horizontalalignment='center',
             bbox=dict(boxstyle='round,pad=0.8', facecolor='#ecf0f1',
                       edgecolor='#2c3e50', linewidth=2, alpha=0.95),
             family='monospace', color='#2c3e50')
    plt.tight_layout()
    plt.savefig(output_path, dpi=200, bbox_inches='tight', facecolor='#f8f9fa')
    plt.close()
    print(f"[OK] Move Accuracy Strip Plot saved: {output_path}")
    return output_path
def generate_accuracy_visualization(json_path):
    """Generate move accuracy visualization."""
    print("\n" + "=" * 70)
    print("MOVE ACCURACY VISUALIZATION")
    print("=" * 70 + "\n")
    print(f"Loading game data: {json_path}")
    try:
        game_log, metadata = load_game_log(json_path)
        print(f"[OK] Loaded {len(game_log)} turn entries")
        has_accuracy = any(t.get('move_classification') is not None for t in game_log)
        if not has_accuracy:
            print("\n⚠️  WARNING: No move accuracy data found in game log!")
            print("   Make sure you ran a game AFTER implementing the Accuracy Engine.")
            print("   Run: python main.py")
            return
        print()
    except Exception as e:
        print(f"✗ Error loading data: {e}")
        return
    Path('data').mkdir(exist_ok=True)
```

```
    print("Generating move accuracy strip plot...\n")
    try:
        output = plot_accuracy_strip_chart(game_log)
        print("\n" + "=" * 70)
        print("VISUALIZATION COMPLETE!")
        print("=" * 70)
        print(f"\nOutput: {output}")
        print("\n" + "=" * 70 + "\n")
    except Exception as e:
        print(f"✗ Error generating visualization: {e}")
        import traceback
        traceback.print_exc()
if __name__ == "__main__":
    if len(sys.argv) > 1:
        json_path = sys.argv[1]
    else:
        log_files = sorted(Path('logs').glob('game_*.json'), reverse=True)
        if log_files:
            json_path = str(log_files[0])
            print(f"Using most recent game log: {json_path}\n")
        else:
            print("✗ No game logs found!")
            print("\nUsage:")
            print("  python plot_move_accuracy.py [path/to/game_log.json]")
            print("\nOr run a game first:")
            print("  python main.py")
            sys.exit(1)
    generate_accuracy_visualization(json_path)
`
```

recorder.py

```
`python
"""
Game move recorder for analytics.
"""
import json
import csv
import time
from typing import List, Dict, Any
from pathlib import Path
from datetime import datetime
from game_engine.components import Action
class MoveRecord:
    """Represents a single move record."""
    def __init__(self, turn_id: int, player_id: int, player_type: str,
                 action: Action, state_hash: str, duration_ms: float):
        self.turn_id = turn_id
        self.player_id = player_id
        self.player_type = player_type
        self.action = action
        self.action_type = action.type
        self.action_data = action.data
        self.state_hash = state_hash
        self.duration_ms = duration_ms
        self.timestamp = datetime.now().isoformat()
```

```
    def to_dict(self) -> Dict[str, Any]:
        """Convert to dictionary for serialization."""
        return {
            'turn_id': self.turn_id,
            'player_id': self.player_id,
            'player_type': self.player_type,
            'action_type': self.action_type,
            'action_data': str(self.action_data),
            'state_hash': self.state_hash,
            'duration_ms': self.duration_ms,
            'timestamp': self.timestamp
        }
class GameRecorder:
    """
    Records all moves in a game for post-game analysis.
    Saves to both JSON and CSV formats.
    """
    def __init__(self, game_id: str = None):
        """
        Initialize recorder.
        Args:
            game_id: Unique identifier for this game
        """
        self.game_id = game_id or datetime.now().strftime("%Y%m%d_%H%M%S")
        self.moves: List[MoveRecord] = []
        self.game_log: List[Dict[str, Any]] = []
        self.game_start_time = time.time()
        self.game_metadata = {
            'game_id': self.game_id,
            'start_time': datetime.now().isoformat(),
            'end_time': None,
            'winner': None,
            'total_turns': 0,
            'final_scores': {}
        }
    def record_move(self, turn_id: int, player_id: int, player_type: str,
                    action: Action, state_hash: str, duration_ms: float):
        """
        Record a move.
        Args:
            turn_id: Turn number
            player_id: Player who made the move
            player_type: Type of player ("Basic" or "MCTS")
            action: The action taken
            state_hash: Hash of game state (for uniqueness)
            duration_ms: Time taken to compute move
        """
        record = MoveRecord(turn_id, player_id, player_type, action,
                            state_hash, duration_ms)
        self.moves.append(record)
    def finalize_game(self, winner: int, final_scores: Dict[int, int]):
        """
        Finalize game recording.
        Args:
            winner: Winning player ID
            final_scores: Dictionary of player scores
        """
        self.game_metadata['end_time'] = datetime.now().isoformat()
```

```
        self.game_metadata['winner'] = winner
        self.game_metadata['total_turns'] = len(self.moves)
        self.game_metadata['final_scores'] = final_scores
        self.game_metadata['total_duration_seconds'] = time.time() - self.game_start_time
    def save_json(self, directory: str = "logs"):
        """
        Save game log as JSON.
        Args:
            directory: Output directory
        """
        Path(directory).mkdir(exist_ok=True)
        filepath = Path(directory) / f"game_{self.game_id}.json"
        data = {
            'metadata': self.game_metadata,
            'moves': [move.to_dict() for move in self.moves],
            'game_log': self.game_log
        }
        with open(filepath, 'w') as f:
            json.dump(data, f, indent=2)
        return filepath
    def save_csv(self, directory: str = "logs"):
        """
        Save game log as CSV.
        Args:
            directory: Output directory
        """
        Path(directory).mkdir(exist_ok=True)
        filepath = Path(directory) / f"game_{self.game_id}.csv"
        with open(filepath, 'w', newline='') as f:
            fieldnames = ['turn_id', 'player_id', 'player_type', 'action_type',
                          'action_data', 'state_hash', 'duration_ms', 'timestamp']
            writer = csv.DictWriter(f, fieldnames=fieldnames)
            writer.writeheader()
            for move in self.moves:
                writer.writerow(move.to_dict())
        return filepath
    def get_player_moves(self, player_id: int) -> List[MoveRecord]:
        """Get all moves for a specific player."""
        return [move for move in self.moves if move.player_id == player_id]
    def get_average_move_time(self, player_id: int) -> float:
        """Get average move computation time for a player."""
        player_moves = self.get_player_moves(player_id)
        if not player_moves:
            return 0.0
        return sum(m.duration_ms for m in player_moves) / len(player_moves)
`
```

renderer.py

```
`python
"""
Pygame rendering functions for game components.
"""
import pygame
from typing import Dict, List, Tuple
from game_engine.components import Card, Noble, GemType
COLORS = {
```

```
    GemType.WHITE: (240, 240, 240),
    GemType.BLUE: (70, 130, 180),
    GemType.GREEN: (60, 179, 113),
    GemType.RED: (220, 60, 60),
    GemType.BLACK: (40, 40, 40),
    GemType.GOLD: (184, 134, 11),
}
TIER_COLORS = {
    1: (100, 200, 100),
    2: (100, 150, 200),
    3: (200, 100, 150),
}
BG_COLOR = (40, 45, 52)
TEXT_COLOR = (230, 230, 230)
HIGHLIGHT_COLOR = (204, 85, 0)
def draw_gem(surface: pygame.Surface, gem_type: GemType, x: int, y: int,
             radius: int = 15, count: int = None):
    """
    Draw a gem token.
    Args:
        surface: Pygame surface to draw on
        gem_type: Type of gem
        x, y: Center position
        radius: Gem radius
        count: Optional count to display
    """
    color = COLORS[gem_type]
    pygame.draw.circle(surface, color, (x, y), radius)
    pygame.draw.circle(surface, TEXT_COLOR, (x, y), radius, 2)
    if count is not None and count > 0:
        font = pygame.font.Font(None, 20)
        text = font.render(str(count), True, TEXT_COLOR)
        text_rect = text.get_rect(center=(x, y))
        surface.blit(text, text_rect)
def draw_card(surface: pygame.Surface, card: Card, x: int, y: int,
              width: int = 80, height: int = 100, highlight: bool = False):
    """
    Draw a development card.
    Args:
        surface: Pygame surface to draw on
        card: Card to draw
        x, y: Top-left position
        width, height: Card dimensions
        highlight: Whether to highlight the card
    """
    tier_color = TIER_COLORS.get(card.tier, (150, 150, 150))
    pygame.draw.rect(surface, tier_color, (x, y, width, height), border_radius=5)
    if highlight:
        pygame.draw.rect(surface, HIGHLIGHT_COLOR, (x, y, width, height), 3,
border_radius=5)
    else:
        pygame.draw.rect(surface, TEXT_COLOR, (x, y, width, height), 2, border_radius=5)
    bonus_color = COLORS[card.gem_bonus]
    pygame.draw.circle(surface, bonus_color, (x + width - 15, y + 15), 10)
    pygame.draw.circle(surface, TEXT_COLOR, (x + width - 15, y + 15), 10, 1)
    if card.points > 0:
        font = pygame.font.Font(None, 24)
        points_text = font.render(str(card.points), True, TEXT_COLOR)
```

```
            surface.blit(points_text, (x + 5, y + 5))
        font_small = pygame.font.Font(None, 16)
        cost_y = y + 35
        for gem_type, amount in card.cost.items():
            gem_color = COLORS[gem_type]
            pygame.draw.circle(surface, gem_color, (x + 15, cost_y), 8)
            cost_text = font_small.render(str(amount), True, TEXT_COLOR)
            surface.blit(cost_text, (x + 25, cost_y - 7))
            cost_y += 18
    def draw_noble(surface: pygame.Surface, noble: Noble, x: int, y: int,
                   width: int = 70, height: int = 70):
        """
        Draw a noble tile.
        Args:
            surface: Pygame surface to draw on
            noble: Noble to draw
            x, y: Top-left position
            width, height: Noble dimensions
        """
        pygame.draw.rect(surface, (200, 180, 140), (x, y, width, height), border_radius=5)
        pygame.draw.rect(surface, TEXT_COLOR, (x, y, width, height), 2, border_radius=5)
        font = pygame.font.Font(None, 24)
        points_text = font.render("3", True, TEXT_COLOR)
        surface.blit(points_text, (x + 5, y + 5))
        font_small = pygame.font.Font(None, 14)
        req_y = y + 30
        for gem_type, amount in noble.requirements.items():
            gem_color = COLORS[gem_type]
            pygame.draw.circle(surface, gem_color, (x + 15, req_y), 6)
            req_text = font_small.render(str(amount), True, TEXT_COLOR)
            surface.blit(req_text, (x + 25, req_y - 6))
            req_y += 15
    def draw_gem_bank(surface: pygame.Surface, gem_bank: Dict[GemType, int],
                      x: int, y: int):
        """
        Draw the gem bank.
        Args:
            surface: Pygame surface to draw on
            gem_bank: Dictionary of gem counts
            x, y: Top-left position
        """
        font = pygame.font.Font(None, 20)
        title = font.render("Gem Bank", True, TEXT_COLOR)
        surface.blit(title, (x, y))
        gem_y = y + 30
        for gem_type in [GemType.WHITE, GemType.BLUE, GemType.GREEN,
                         GemType.RED, GemType.BLACK, GemType.GOLD]:
            count = gem_bank.get(gem_type, 0)
            draw_gem(surface, gem_type, x + 20, gem_y, 15, count)
            gem_y += 40
    def draw_player_area(surface: pygame.Surface, player_id: int,
                         gems: Dict[GemType, int], bonuses: Dict[GemType, int],
                         points: int, cards: List[Card], nobles: List[Noble],
                         x: int, y: int, width: int, height: int,
                         is_current: bool = False):
        """
        Draw player's tableau.
        Args:
```

```
        surface: Pygame surface to draw on
        player_id: Player number
        gems: Player's gem tokens
        bonuses: Player's gem bonuses
        points: Player's points
        cards: Player's cards
        nobles: Player's nobles
        x, y: Top-left position
        width, height: Area dimensions
        is_current: Whether this is the current player
    """
    bg_color = HIGHLIGHT_COLOR if is_current else (60, 65, 70)
    pygame.draw.rect(surface, bg_color, (x, y, width, height), border_radius=10)
    pygame.draw.rect(surface, TEXT_COLOR, (x, y, width, height), 2, border_radius=10)
    font = pygame.font.Font(None, 24)
    header = f"Player {player_id + 1} - {points} pts"
    if is_current:
        header += " (Current)"
    header_text = font.render(header, True, TEXT_COLOR)
    surface.blit(header_text, (x + 10, y + 10))
    gem_x = x + 10
    gem_y = y + 40
    font_small = pygame.font.Font(None, 18)
    for gem_type in [GemType.WHITE, GemType.BLUE, GemType.GREEN,
                     GemType.RED, GemType.BLACK, GemType.GOLD]:
        count = gems.get(gem_type, 0)
        if count > 0:
            draw_gem(surface, gem_type, gem_x + 15, gem_y, 12)
            count_text = font_small.render(f"×{count}", True, TEXT_COLOR)
            surface.blit(count_text, (gem_x + 30, gem_y - 8))
            gem_x += 60
    bonus_x = x + 10
    bonus_y = y + 70
    bonus_label = font_small.render("Bonuses:", True, TEXT_COLOR)
    surface.blit(bonus_label, (bonus_x, bonus_y - 8))
    bonus_x += 70
    for gem_type in [GemType.WHITE, GemType.BLUE, GemType.GREEN,
                     GemType.RED, GemType.BLACK]:
        count = bonuses.get(gem_type, 0)
        if count > 0:
            draw_gem(surface, gem_type, bonus_x + 10, bonus_y, 10)
            count_text = font_small.render(f"×{count}", True, TEXT_COLOR)
            surface.blit(count_text, (bonus_x + 25, bonus_y - 8))
            bonus_x += 50
    cards_label = font_small.render(f"Cards: {len(cards)}", True, TEXT_COLOR)
    surface.blit(cards_label, (x + 10, y + 100))
    if nobles:
        nobles_label = font_small.render(f"Nobles: {len(nobles)}", True, TEXT_COLOR)
        surface.blit(nobles_label, (x + 100, y + 100))
`
```

simulate_batch.py

```
`python
import os
```

```
import sys
import time
import json
import argparse
from datetime import datetime
from pathlib import Path
import matplotlib
matplotlib.use('Agg')
from game_engine.state import GameState
from agents.basic import BasicAgent
from agents.advanced import MCTSAgent
from analytics.recorder import GameRecorder
from main import hash_state, calculate_move_accuracy
from generate_comparison_visuals import (
    visual_1_battle_of_confidence,
    visual_2_thinking_effort,
    visual_3_strategy_dna
)
from plot_move_accuracy import plot_accuracy_strip_chart
def simulate_game(game_index: int, total_games: int):
    print(f"\n{'='*40}\nStarting Game {game_index}/{total_games}\n{'='*40}")
    game_state = GameState(num_players=2)
    agents = [
        BasicAgent(player_id=0),
        MCTSAgent(player_id=1, time_limit=3.0)
    ]
    recorder =
GameRecorder(game_id=f"batch_{datetime.now().strftime('%Y%m%d_%H%M%S')}_{game_
index}")
    recorder.game_log = []
    while not game_state.game_over:
        current_player = game_state.current_player
        agent = agents[current_player]
        mcts_engine = agents[1]
        start_time = time.time()
        if isinstance(agent, MCTSAgent):
            action, mcts_metadata = agent.get_move(game_state)
            win_probability = mcts_metadata.get('win_probability', 0.5)
            search_depth = mcts_metadata.get('search_depth', 0)
            simulation_count = mcts_metadata.get('simulation_count', 0)
            visit_distribution = mcts_metadata.get('visit_distribution', [])
        else:
            action = agent.select_move(game_state)
            win_probability = None
            search_depth = 0
            simulation_count = 0
            visit_distribution = []
        duration_ms = (time.time() - start_time) * 1000
        accuracy_data = None
        if mcts_engine:
            accuracy_data = calculate_move_accuracy(game_state, current_player, action,
mcts_engine)
        recorder.record_move(
            turn_id=game_state.turn_number,
            player_id=current_player,
            player_type="MCTS" if isinstance(agent, MCTSAgent) else "Basic",
            action=action,
            state_hash=hash_state(game_state),
```

```
            duration_ms=duration_ms
        )
        gems_before = sum(game_state.player_gems[current_player].values())
        game_state.apply_action(action)
        gems_after = sum(game_state.player_gems[current_player].values())
        gems_returned = max(0, gems_before - gems_after) if gems_after <= 10 else 0
        if isinstance(agent, BasicAgent):
            critic_result = mcts_engine.evaluate_state_as_critic(
                game_state, target_player=current_player, simulations=50)
            win_probability = critic_result.get('win_probability', 0.5)
        cards_by_tier = [0, 0, 0]
        for card in game_state.player_cards[current_player]:
            if card.tier in [1, 2, 3]:
                cards_by_tier[card.tier - 1] += 1
        log_entry = {
            'turn_number': game_state.turn_number,
            'active_player': current_player,
            'player_type': "MCTS" if isinstance(agent, MCTSAgent) else "Basic",
            'cards_owned_by_tier': cards_by_tier,
            'nobles_owned': len(game_state.player_nobles[current_player]),
            'gems_returned': gems_returned,
            'win_probability': win_probability,
            'search_depth': search_depth,
            'simulation_count': simulation_count,
            'visit_distribution': visit_distribution,
            'player_points': game_state.player_points[current_player],
            'action_type': action.type,
            'duration_ms': duration_ms,
            'is_best_move': accuracy_data.get('is_best_move', False) if accuracy_data else None,
            'best_win_prob': accuracy_data.get('best_win_prob', 0) if accuracy_data else None,
            'actual_win_prob': accuracy_data.get('actual_win_prob', 0) if accuracy_data else None,
            'accuracy_loss': accuracy_data.get('accuracy_loss', 0) if accuracy_data else None,
            'move_classification': accuracy_data.get('move_classification', 'Unknown') if
accuracy_data else None
        }
        recorder.game_log.append(log_entry)
        print(f"Turn {game_state.turn_number} - P{current_player+1} - {action.type}")
    winner_id = -1
    max_points = -1
    for pid, points in enumerate(game_state.player_points):
        if points >= 15 and points > max_points:
            winner_id = pid
            max_points = points
    final_scores = {pid: points for pid, points in enumerate(game_state.player_points)}
    recorder.finalize_game(winner_id, final_scores)
    print(f"Game Over! Winner: Player {winner_id + 1} with {max_points} points.")
    date_folder = Path("data/Games")
    date_folder.mkdir(parents=True, exist_ok=True)
    game_count = len([d for d in date_folder.iterdir() if d.is_dir()]) + 1
    game_folder = date_folder / f"game_{game_count}"
    game_folder.mkdir(parents=True, exist_ok=True)
    print("Generating Analytics Graphics...")
    try:
        visual_1_battle_of_confidence(recorder.game_log, str(game_folder /
'visual_1_battle_of_confidence.png'))
        visual_2_thinking_effort(recorder.game_log, str(game_folder /
'visual_2_thinking_effort.png'))
        visual_3_strategy_dna(recorder.game_log, str(game_folder / 'visual_3_strategy_dna.png'))
```

```
        plot_accuracy_strip_chart(recorder.game_log, str(game_folder /
'move_accuracy_strip_plot.png'))
    except Exception as e:
        print(f"Failed to generate a graphic: {e}")
    recorder.save_json(str(game_folder))
    master_file = Path("data/games.json")
    game_data = {
        'game_id': recorder.game_id,
        'date': datetime.now().strftime("%Y-%m-%d"),
        'metadata': recorder.game_metadata,
        'mcts_kb_size': len(agents[1].kb.memory) if hasattr(agents[1], 'kb') else 0
    }
    try:
        games_list = []
        if master_file.exists():
            with open(master_file, 'r') as f:
                games_list = json.load(f)
        games_list.append(game_data)
        with open(master_file, 'w') as f:
            json.dump(games_list, f, indent=2)
    except Exception as e:
        print(f"Failed to update games.json: {e}")
    for agent in agents:
        if hasattr(agent, "kb") and hasattr(agent.kb, "save"):
            agent.kb.save()
            print(f"Saved KB for {agent.name}, current size: {len(agent.kb.memory)}")
def main():
    parser = argparse.ArgumentParser(description="Headless batch simulator for MCTS Splendor
AI.")
    parser.add_argument("--games", type=int, default=50, help="Number of games to simulate")
    args = parser.parse_args()
    print(f"Simulating {args.games} games...")
    for i in range(1, args.games + 1):
        simulate_game(i, args.games)
if __name__ == "__main__":
    main()
`
```

state.py

```
`python
"""
Core game state and rules engine.
"""
from typing import List, Dict, Optional, Tuple
from copy import deepcopy
from collections import defaultdict
import random
from game_engine.components import (
    Card, Noble, Action, GemType,
    TAKE_3_DIFFERENT, TAKE_2_SAME, RESERVE_CARD, BUY_CARD
)
class GameState:
    """
    Manages the complete game state for Splendor.
```

```
    Tracks:
    - Gem bank
    - Card decks and visible cards
    - Nobles
    - Player tableaux (cards, gems, reserved cards)
    """
    def __init__(self, num_players: int = 2):
        self.num_players = num_players
        self.current_player = 0
        self.turn_number = 0
        self.game_over = False
        self.winner = None
        self.end_game_triggered = False
        self.trigger_player = None
        gems_per_color = {2: 4, 3: 5, 4: 7}[num_players]
        self.gem_bank = {
            GemType.WHITE: gems_per_color,
            GemType.BLUE: gems_per_color,
            GemType.GREEN: gems_per_color,
            GemType.RED: gems_per_color,
            GemType.BLACK: gems_per_color,
            GemType.GOLD: 5
        }
        self.deck_tier1: List[Card] = []
        self.deck_tier2: List[Card] = []
        self.deck_tier3: List[Card] = []
        self.visible_tier1: List[Card] = []
        self.visible_tier2: List[Card] = []
        self.visible_tier3: List[Card] = []
        self.nobles: List[Noble] = []
        self.player_gems: List[Dict[GemType, int]] = []
        self.player_bonuses: List[Dict[GemType, int]] = []
        self.player_cards: List[List[Card]] = []
        self.player_reserved: List[List[Card]] = []
        self.player_points: List[int] = []
        self.player_nobles: List[List[Noble]] = []
        self._initialize_decks()
        self._initialize_nobles()
        self._initialize_players()
        self._deal_cards()
    def _initialize_decks(self):
        """Create card decks with proper Splendor distribution."""
        tier1_configs = [
            (0, GemType.WHITE, {GemType.BLUE: 3}),
            (0, GemType.WHITE, {GemType.GREEN: 3}),
            (0, GemType.WHITE, {GemType.RED: 3}),
            (0, GemType.WHITE, {GemType.BLACK: 3}),
            (0, GemType.WHITE, {GemType.BLUE: 2, GemType.GREEN: 1}),
            (0, GemType.BLUE, {GemType.WHITE: 3}),
            (0, GemType.BLUE, {GemType.GREEN: 3}),
            (0, GemType.BLUE, {GemType.RED: 3}),
            (0, GemType.BLUE, {GemType.BLACK: 3}),
            (0, GemType.BLUE, {GemType.WHITE: 2, GemType.BLACK: 1}),
            (0, GemType.GREEN, {GemType.WHITE: 3}),
            (0, GemType.GREEN, {GemType.BLUE: 3}),
            (0, GemType.GREEN, {GemType.RED: 3}),
            (0, GemType.GREEN, {GemType.BLACK: 3}),
            (0, GemType.GREEN, {GemType.BLUE: 2, GemType.RED: 1}),
```

```
            (0, GemType.RED, {GemType.WHITE: 3}),
            (0, GemType.RED, {GemType.BLUE: 3}),
            (0, GemType.RED, {GemType.GREEN: 3}),
            (0, GemType.RED, {GemType.BLACK: 3}),
            (0, GemType.RED, {GemType.GREEN: 2, GemType.BLACK: 1}),
            (0, GemType.BLACK, {GemType.WHITE: 3}),
            (0, GemType.BLACK, {GemType.BLUE: 3}),
            (0, GemType.BLACK, {GemType.GREEN: 3}),
            (0, GemType.BLACK, {GemType.RED: 3}),
            (0, GemType.BLACK, {GemType.RED: 2, GemType.WHITE: 1}),
            (1, GemType.WHITE, {GemType.BLUE: 4}),
            (1, GemType.WHITE, {GemType.BLACK: 4}),
            (1, GemType.BLUE, {GemType.WHITE: 4}),
            (1, GemType.BLUE, {GemType.RED: 4}),
            (1, GemType.GREEN, {GemType.BLUE: 4}),
            (1, GemType.GREEN, {GemType.BLACK: 4}),
            (1, GemType.RED, {GemType.WHITE: 4}),
            (1, GemType.RED, {GemType.GREEN: 4}),
            (1, GemType.BLACK, {GemType.GREEN: 4}),
            (1, GemType.BLACK, {GemType.RED: 4}),
        ]
        for points, bonus, cost in tier1_configs:
            self.deck_tier1.append(Card(tier=1, points=points, gem_bonus=bonus, cost=cost))
        tier2_configs = [
            (1, GemType.WHITE, {GemType.BLUE: 3, GemType.GREEN: 2, GemType.RED: 2}),
            (1, GemType.BLUE, {GemType.WHITE: 3, GemType.GREEN: 2, GemType.BLACK:
2}),
            (1, GemType.GREEN, {GemType.WHITE: 3, GemType.BLUE: 2, GemType.RED: 2}),
            (1, GemType.RED, {GemType.BLUE: 3, GemType.GREEN: 2, GemType.BLACK:
2}),
            (1, GemType.BLACK, {GemType.WHITE: 3, GemType.RED: 2, GemType.BLACK:
2}),
            (2, GemType.WHITE, {GemType.BLUE: 5}),
            (2, GemType.WHITE, {GemType.GREEN: 5}),
            (2, GemType.BLUE, {GemType.WHITE: 5}),
            (2, GemType.BLUE, {GemType.BLACK: 5}),
            (2, GemType.GREEN, {GemType.BLUE: 5}),
            (2, GemType.GREEN, {GemType.RED: 5}),
            (2, GemType.RED, {GemType.GREEN: 5}),
            (2, GemType.RED, {GemType.BLACK: 5}),
            (2, GemType.BLACK, {GemType.RED: 5}),
            (2, GemType.BLACK, {GemType.WHITE: 5}),
            (3, GemType.WHITE, {GemType.WHITE: 6}),
            (3, GemType.BLUE, {GemType.BLUE: 6}),
            (3, GemType.GREEN, {GemType.GREEN: 6}),
            (3, GemType.RED, {GemType.RED: 6}),
            (3, GemType.BLACK, {GemType.BLACK: 6}),
        ]
        for points, bonus, cost in tier2_configs:
            self.deck_tier2.append(Card(tier=2, points=points, gem_bonus=bonus, cost=cost))
        tier3_configs = [
            (3, GemType.WHITE, {GemType.WHITE: 3, GemType.BLUE: 3, GemType.GREEN:
5, GemType.RED: 3}),
            (3, GemType.BLUE, {GemType.WHITE: 3, GemType.BLUE: 3, GemType.GREEN: 3,
GemType.BLACK: 5}),
            (3, GemType.GREEN, {GemType.BLUE: 5, GemType.GREEN: 3, GemType.RED: 3,
GemType.BLACK: 3}),
```

```
            (3, GemType.RED, {GemType.WHITE: 5, GemType.GREEN: 3, GemType.RED: 3,
GemType.BLACK: 3}),
            (3, GemType.BLACK, {GemType.WHITE: 3, GemType.BLUE: 3, GemType.RED: 5,
GemType.BLACK: 3}),
            (4, GemType.WHITE, {GemType.BLACK: 7}),
            (4, GemType.BLUE, {GemType.RED: 7}),
            (4, GemType.GREEN, {GemType.WHITE: 7}),
            (4, GemType.RED, {GemType.BLUE: 7}),
            (4, GemType.BLACK, {GemType.GREEN: 7}),
            (5, GemType.WHITE, {GemType.BLACK: 7, GemType.RED: 3}),
            (5, GemType.BLUE, {GemType.WHITE: 7, GemType.GREEN: 3}),
            (5, GemType.GREEN, {GemType.BLUE: 7, GemType.BLACK: 3}),
            (5, GemType.RED, {GemType.GREEN: 7, GemType.WHITE: 3}),
            (5, GemType.BLACK, {GemType.RED: 7, GemType.BLUE: 3}),
        ]
        for points, bonus, cost in tier3_configs:
            self.deck_tier3.append(Card(tier=3, points=points, gem_bonus=bonus, cost=cost))
        random.shuffle(self.deck_tier1)
        random.shuffle(self.deck_tier2)
        random.shuffle(self.deck_tier3)
    def _initialize_nobles(self):
        """Create noble tiles."""
        all_nobles = [
            Noble({GemType.WHITE: 3, GemType.BLUE: 3, GemType.GREEN: 3}),
            Noble({GemType.BLUE: 3, GemType.GREEN: 3, GemType.RED: 3}),
            Noble({GemType.GREEN: 3, GemType.RED: 3, GemType.BLACK: 3}),
            Noble({GemType.RED: 3, GemType.BLACK: 3, GemType.WHITE: 3}),
            Noble({GemType.BLACK: 3, GemType.WHITE: 3, GemType.BLUE: 3}),
            Noble({GemType.WHITE: 4, GemType.BLUE: 4}),
            Noble({GemType.BLUE: 4, GemType.GREEN: 4}),
            Noble({GemType.GREEN: 4, GemType.RED: 4}),
            Noble({GemType.RED: 4, GemType.BLACK: 4}),
            Noble({GemType.BLACK: 4, GemType.WHITE: 4}),
        ]
        random.shuffle(all_nobles)
        self.nobles = all_nobles[:self.num_players + 1]
    def _initialize_players(self):
        """Initialize player data structures."""
        for _ in range(self.num_players):
            self.player_gems.append(defaultdict(int))
            self.player_bonuses.append(defaultdict(int))
            self.player_cards.append([])
            self.player_reserved.append([])
            self.player_points.append(0)
            self.player_nobles.append([])
    def _deal_cards(self):
        """Deal initial visible cards."""
        for _ in range(4):
            if self.deck_tier1:
                self.visible_tier1.append(self.deck_tier1.pop())
            if self.deck_tier2:
                self.visible_tier2.append(self.deck_tier2.pop())
            if self.deck_tier3:
                self.visible_tier3.append(self.deck_tier3.pop())
    def get_legal_actions(self, player_id: int) -> List[Action]:
        """
        Generate all legal actions for the specified player.
        Returns:
```

```
            List of valid Action objects
        """
        actions = []
        player_gems = self.player_gems[player_id]
        player_bonuses = self.player_bonuses[player_id]
        player_reserved = self.player_reserved[player_id]
        total_gems = sum(player_gems.values())
        available_colors = [gem for gem in [GemType.WHITE, GemType.BLUE,
GemType.GREEN,
                                            GemType.RED, GemType.BLACK]
                            if self.gem_bank[gem] > 0]
        if len(available_colors) >= 3 and total_gems <= 7:
            from itertools import combinations
            for combo in combinations(available_colors, 3):
                actions.append(Action(TAKE_3_DIFFERENT, gems=list(combo)))
        for gem_type in available_colors:
            if self.gem_bank[gem_type] >= 4 and total_gems <= 8:
                actions.append(Action(TAKE_2_SAME, gem=gem_type))
        if len(player_reserved) < 3:
            all_visible_cards = self.visible_tier1 + self.visible_tier2 + self.visible_tier3
            if all_visible_cards:
                for card in all_visible_cards:
                    actions.append(Action(RESERVE_CARD, card=card))
        for card in self.visible_tier1 + self.visible_tier2 + self.visible_tier3:
            can_afford, gold_needed = card.can_afford(dict(player_gems), dict(player_bonuses))
            if can_afford:
                actions.append(Action(BUY_CARD, card=card))
        for card in player_reserved:
            can_afford, gold_needed = card.can_afford(dict(player_gems), dict(player_bonuses))
            if can_afford:
                actions.append(Action(BUY_CARD, card=card, from_reserved=True))
        if not actions:
            if len(available_colors) >= 2 and total_gems <= 8:
                from itertools import combinations
                for combo in combinations(available_colors, 2):
                    actions.append(Action(TAKE_3_DIFFERENT, gems=list(combo)))
            if len(available_colors) >= 1 and total_gems <= 9:
                for gem in available_colors:
                    actions.append(Action(TAKE_3_DIFFERENT, gems=[gem]))
        return actions
    def apply_action(self, action: Action) -> None:
        """
        Apply an action and update game state.
        Args:
            action: The action to execute
        """
        player_id = self.current_player
        if action.type == TAKE_3_DIFFERENT:
            for gem in action.data['gems']:
                self.gem_bank[gem] -= 1
                self.player_gems[player_id][gem] += 1
        elif action.type == TAKE_2_SAME:
            gem = action.data['gem']
            self.gem_bank[gem] -= 2
            self.player_gems[player_id][gem] += 2
        elif action.type == RESERVE_CARD:
            card = action.data['card']
            if card in self.visible_tier1:
```

```
            self.visible_tier1.remove(card)
            if self.deck_tier1:
                self.visible_tier1.append(self.deck_tier1.pop())
        elif card in self.visible_tier2:
            self.visible_tier2.remove(card)
            if self.deck_tier2:
                self.visible_tier2.append(self.deck_tier2.pop())
        elif card in self.visible_tier3:
            self.visible_tier3.remove(card)
            if self.deck_tier3:
                self.visible_tier3.append(self.deck_tier3.pop())
        self.player_reserved[player_id].append(card)
        if self.gem_bank[GemType.GOLD] > 0:
            self.gem_bank[GemType.GOLD] -= 1
            self.player_gems[player_id][GemType.GOLD] += 1
    elif action.type == BUY_CARD:
        card = action.data['card']
        from_reserved = action.data.get('from_reserved', False)
        can_afford, gold_needed = card.can_afford(
            dict(self.player_gems[player_id]),
            dict(self.player_bonuses[player_id])
        )
        if can_afford:
            for gem_type, required in card.cost.items():
                bonus = self.player_bonuses[player_id].get(gem_type, 0)
                gems_to_pay = max(0, required - bonus)
                actual_payment = min(gems_to_pay, self.player_gems[player_id].get(gem_type, 0))
                if actual_payment > 0:
                    self.player_gems[player_id][gem_type] -= actual_payment
                    self.gem_bank[gem_type] += actual_payment
            if gold_needed > 0 and self.player_gems[player_id].get(GemType.GOLD, 0) >=
gold_needed:
                self.player_gems[player_id][GemType.GOLD] -= gold_needed
                self.gem_bank[GemType.GOLD] += gold_needed
            if from_reserved:
                self.player_reserved[player_id].remove(card)
            else:
                if card in self.visible_tier1:
                    self.visible_tier1.remove(card)
                    if self.deck_tier1:
                        self.visible_tier1.append(self.deck_tier1.pop())
                elif card in self.visible_tier2:
                    self.visible_tier2.remove(card)
                    if self.deck_tier2:
                        self.visible_tier2.append(self.deck_tier2.pop())
                elif card in self.visible_tier3:
                    self.visible_tier3.remove(card)
                    if self.deck_tier3:
                        self.visible_tier3.append(self.deck_tier3.pop())
            self.player_cards[player_id].append(card)
            self.player_bonuses[player_id][card.gem_bonus] += 1
            self.player_points[player_id] += card.points
        self._check_nobles(player_id)
        self._enforce_gem_limit(player_id)
        self.current_player = (self.current_player + 1) % self.num_players
        if self.current_player == 0:
            self.turn_number += 1
        self.check_winner()
```

```
    def _check_nobles(self, player_id: int):
        """Check if player attracts any nobles."""
        player_bonuses = self.player_bonuses[player_id]
        for noble in self.nobles[:]:
            if noble.can_visit(player_bonuses):
                self.nobles.remove(noble)
                self.player_nobles[player_id].append(noble)
                self.player_points[player_id] += noble.points
                break
    def _enforce_gem_limit(self, player_id: int):
        """Ensure player has at most 10 gems."""
        total_gems = sum(self.player_gems[player_id].values())
        while total_gems > 10:
            for gem_type in [GemType.WHITE, GemType.BLUE, GemType.GREEN,
                             GemType.RED, GemType.BLACK, GemType.GOLD]:
                if self.player_gems[player_id].get(gem_type, 0) > 0:
                    self.player_gems[player_id][gem_type] -= 1
                    self.gem_bank[gem_type] += 1
                    total_gems -= 1
                    break
    def check_winner(self) -> Optional[int]:
        """
        Check if game is over and determine winner.
        Official 2-player rule: When a player reaches 15 points, finish the round
        so both players have had equal turns.
        Returns:
            Player ID of winner, or None if game continues
        """
        for i, pts in enumerate(self.player_points):
            if pts >= 15 and not self.end_game_triggered:
                self.end_game_triggered = True
                self.trigger_player = i
        if self.end_game_triggered and self.current_player == 0:
            max_points = max(self.player_points)
            winners = [i for i, pts in enumerate(self.player_points) if pts == max_points]
            if len(winners) > 1:
                card_counts = [(i, len(self.player_cards[i])) for i in winners]
                winner = min(card_counts, key=lambda x: x[1])[0]
            else:
                winner = winners[0]
            self.game_over = True
            self.winner = winner
            return winner
        return None
    def get_state_copy(self):
        """Return a deep copy of the game state for simulations."""
        return deepcopy(self)
    def get_player_score(self, player_id: int) -> int:
        """Get total score for a player."""
        return self.player_points[player_id]
    def get_hash(self) -> str:
        """
        Return a deterministic string representation of the game state.
        This is used for persistent learning to uniquely identify a board position.
        """
        parts = []
        bank_str = ",".join(f"{g.value}:{c}" for g, c in sorted(self.gem_bank.items(), key=lambda
x: x[0].value))
```

```
        parts.append(f"BANK[{bank_str}]")
        v1 = ",".join(sorted(c.get_hash() for c in self.visible_tier1))
        v2 = ",".join(sorted(c.get_hash() for c in self.visible_tier2))
        v3 = ",".join(sorted(c.get_hash() for c in self.visible_tier3))
        parts.append(f"V1[{v1}]V2[{v2}]V3[{v3}]")
        nobles_str = ",".join(sorted(n.get_hash() for n in self.nobles))
        parts.append(f"NOBLES[{nobles_str}]")
        shifted_indices = [(self.current_player + i) % self.num_players for i in
range(self.num_players)]
        for i, p_idx in enumerate(shifted_indices):
            gems_str = ",".join(f"{g.value}:{c}" for g, c in sorted(self.player_gems[p_idx].items(),
key=lambda x: x[0].value))
            bonuses_str = ",".join(f"{g.value}:{c}" for g, c in
sorted(self.player_bonuses[p_idx].items(), key=lambda x: x[0].value))
            res_str = ",".join(sorted(c.get_hash() for c in self.player_reserved[p_idx]))

parts.append(f"P{i}[G({gems_str})B({bonuses_str})R({res_str})Pts({self.player_points[p_idx]
})]")
        return "|".join(parts)
`
```

test_analytics_system.py

```
`python
"""
Quick test script to verify the comprehensive post-game analytics system.
This script:
1. Runs a short game (to save time)
2. Generates all 5 visualizations
3. Verifies files were created
"""
import sys
from pathlib import Path
def test_analytics_system():
    """Test the complete analytics pipeline."""
    print("="*60)
    print("POST-GAME ANALYTICS SYSTEM TEST")
    print("="*60)
    data_dir = Path('data')
    game_folders = sorted([f for f in data_dir.iterdir() if f.is_dir() and f.name.startswith('game_')],
                          key=lambda x: x.name, reverse=True)
    if not game_folders:
        print("❌ No game folders found in data/")
        print("   Please run a game first using main.py")
        return False
    latest_game = game_folders[0]
    print(f"\n📁 Testing on: {latest_game.name}")
    game_id = latest_game.name.replace('game_', '')
    log_path = Path('logs') / f'game_{game_id}.json'
    if not log  path.exists():
        print(f"❌ Game log not found: {log_path}")
        return False
    print(f"[OK] Game log found: {log_path}")
    try:
        from generate_post_game_analytics import PostGameAnalytics
        print(f"\n🔧 Generating analytics...")
```

```
        analytics = PostGameAnalytics(str(latest_game))
        analytics.generate_all_visualizations()
    except Exception as e:
        print(f"❌ Analytics generation failed: {e}")
        import traceback
        traceback.print_exc()
        return False
    print(f"\n📊 Verifying outputs...")
    expected_files = [
        'visual_1_battle_of_confidence.png',
        'visual_2_move_quality_strip_plot.png',
        'visual_3_strategy_dna.png',
        'visual_4_computational_intensity.png'
    ]
    all_present = True
    for filename in expected_files:
        filepath = latest_game / filename
        if filepath.exists():
            size_kb = filepath.stat().st_size / 1024
            print(f"  [OK] {filename} ({size_kb:.1f} KB)")
        else:
            print(f"  ❌ {filename} - NOT FOUND")
            all_present = False
    tree_files = list(latest_game.glob('mcts_brain_*.html'))
    if tree_files:
        print(f"  [OK] {tree_files[0].name} (Decision Tree)")
    else:
        print(f"  ℹ️ No decision tree (captured only during turns 10-15)")
    print("\n" + "="*60)
    if all_present:
        print("✅ ALL TESTS PASSED")
        print(f"\n📂 View results at: {latest_game}")
        print("\n🎓 You can now:")
        print("  1. Open PNG files to see visualizations")
        print("  2. Open HTML file in browser for interactive tree")
        print("  3. Use these in your thesis/presentation")
        return True
    else:
        print("❌ SOME TESTS FAILED")
        print("   Check error messages above")
        return False
if __name__ == '__main__':
    success = test_analytics_system()
    sys.exit(0 if success else 1)
```

`

```
# test_deep_dive.py

`python
"""
Test script to demonstrate the new Deep Dive Analytics features.
Run this to see the MCTS metadata and MCTS Critic in action.
"""
import json
from game_engine.state import GameState
from agents.basic import BasicAgent
from agents.advanced import MCTSAgent
```

```
def test_mcts_metadata():
    """Test MCTS agent returning metadata."""
    print("=" * 60)
    print("Test 1: MCTS Agent with Metadata")
    print("=" * 60)
    game_state = GameState(num_players=2)
    mcts_agent = MCTSAgent(player_id=1, time_limit=1.0)
    print("\nGetting MCTS move with metadata...")
    action, metadata = mcts_agent.get_move(game_state)
    print(f"\nAction: {action.type}")
    print(f"\nMetadata:")
    print(f"  Win Probability: {metadata['win_probability']:.3f}")
    print(f"  Search Depth: {metadata['search_depth']}")
    print(f"  Simulation Count: {metadata['simulation_count']}")
    print(f"  Visit Distribution (top 3):")
    for i, visit in enumerate(metadata['visit_distribution'], 1):
        print(f"    #{i}: {visit['visits']} visits, value={visit['value']:.3f}")
    print("\n[OK] Test 1 passed!\n")
def test_mcts_critic():
    """Test MCTS Critic evaluating a Heuristic Agent's move."""
    print("=" * 60)
    print("Test 2: MCTS Critic for Heuristic Agent")
    print("=" * 60)
    game_state = GameState(num_players=2)
    basic_agent = BasicAgent(player_id=0)
    mcts_agent = MCTSAgent(player_id=1, time_limit=1.0)
    print("\nHeuristic Agent (P1) makes a move...")
    action = basic_agent.select_move(game_state)
    print(f"Action: {action.type}")
    game_state.apply_action(action)
    print("\nMCTS Critic evaluating P1's position...")
    critic_result = mcts_agent.evaluate_state_as_critic(
        game_state,
        target_player=0,
        simulations=50
    )
    print(f"\nCritic Evaluation:")
    print(f"  Win Probability: {critic_result['win_probability']:.3f}")
    print(f"  Projected Score: {critic_result['projected_score']}")
    print(f"  Simulation Count: {critic_result['simulation_count']}")
    print("\n[OK] Test 2 passed!\n")
def test_game_log_structure():
    """Test game log data structure."""
    print("=" * 60)
    print("Test 3: Game Log Data Structure")
    print("=" * 60)
    sample_log_entry = {
        'turn_number': 5,
        'active_player': 1,
        'player_type': 'MCTS',
        'cards_owned_by_tier': [3, 1, 0],
        'nobles_owned': 0,
        'gems_returned': 2,
        'win_probability': 0.68,
        'search_depth': 12,
        'simulation_count': 847,
        'visit_distribution': [
            {'visits': 342, 'value': 0.68},
```

```
            {'visits': 289, 'value': 0.64},
            {'visits': 216, 'value': 0.59}
        ],
        'player_points': 5,
        'action_type': 'buy_card',
        'duration_ms': 2003.5
    }
    print("\nSample Game Log Entry:")
    print(json.dumps(sample_log_entry, indent=2))
    print("\n[OK] Test 3 passed!\n")
def test_comparison_data():
    """Demonstrate how to use the data for comparison."""
    print("=" * 60)
    print("Test 4: Data Comparison Example")
    print("=" * 60)
    sample_game_log = [
        {'turn_number': 1, 'active_player': 0, 'player_type': 'Basic',
         'win_probability': 0.48, 'cards_owned_by_tier': [1, 0, 0], 'player_points': 0},
        {'turn_number': 2, 'active_player': 1, 'player_type': 'MCTS',
         'win_probability': 0.52, 'cards_owned_by_tier': [1, 0, 0], 'player_points': 0},
        {'turn_number': 3, 'active_player': 0, 'player_type': 'Basic',
         'win_probability': 0.45, 'cards_owned_by_tier': [2, 0, 0], 'player_points': 0},
        {'turn_number': 4, 'active_player': 1, 'player_type': 'MCTS',
         'win_probability': 0.58, 'cards_owned_by_tier': [2, 1, 0], 'player_points': 2},
        {'turn_number': 5, 'active_player': 0, 'player_type': 'Basic',
         'win_probability': 0.42, 'cards_owned_by_tier': [3, 0, 0], 'player_points': 1},
    ]
    p1_turns = [t for t in sample_game_log if t['active_player'] == 0]
    p2_turns = [t for t in sample_game_log if t['active_player'] == 1]
    p1_avg_prob = sum(t['win_probability'] for t in p1_turns) / len(p1_turns)
    p2_avg_prob = sum(t['win_probability'] for t in p2_turns) / len(p2_turns)
    p1_total_cards = sum(sum(t['cards_owned_by_tier']) for t in p1_turns)
    p2_total_cards = sum(sum(t['cards_owned_by_tier']) for t in p2_turns)
    print("\nComparison Metrics:")
    print(f"\nPlayer 1 (Heuristic Agent):")
    print(f"  Avg Win Probability: {p1_avg_prob:.2%}")
    print(f"  Total Cards Acquired: {p1_total_cards}")
    print(f"  Final Points: {p1_turns[-1]['player_points']}")
    print(f"\nPlayer 2 (MCTS Agent):")
    print(f"  Avg Win Probability: {p2_avg_prob:.2%}")
    print(f"  Total Cards Acquired: {p2_total_cards}")
    print(f"  Final Points: {p2_turns[-1]['player_points']}")
    print(f"\nDifference:")
    print(f"  Win Prob Advantage: {(p2_avg_prob - p1_avg_prob):.2%} (MCTS)")
    print("\n[OK] Test 4 passed!\n")
if __name__ == "__main__":
    print("\n" + "=" * 60)
    print("DEEP DIVE ANALYTICS - TEST SUITE")
    print("=" * 60 + "\n")
    test_mcts_metadata()
    test_mcts_critic()
    test_game_log_structure()
    test_comparison_data()
    print("=" * 60)
    print("ALL TESTS PASSED!")
    print("=" * 60)
    print("\nYou can now run 'python main.py' to play a full game")
    print("with Deep Dive Analytics enabled.")
```

```
    print("\nData will be saved to logs/game_YYYYMMDD_HHMMSS.json")
    print("=" * 60 + "\n")
```

test_kb.py

```
`python
import os
from game_engine.state import GameState
from agents.advanced import MCTSAgent
def test_persistent_learning():
    print("Testing MCTS Knowledge Base integration...")
    state = GameState(num_players=2)
    agent = MCTSAgent(player_id=1, time_limit=1.0)
    print(f"Agent loaded KB with {len(agent.kb.memory)} states.")
    print("Agent thinking...")
    action, meta = agent.get_move(state)
    print(f"Agent chose action: {action}")
    print(f"Agent explored {meta['simulation_count']} simulations.")
    agent.kb.save()
    kb_path = "data/mcts_knowledge.json"
    if os.path.exists(kb_path):
        size = os.path.getsize(kb_path)
        print(f"SUCCESS: KB file created at {kb_path} (size: {size} bytes)")
    else:
        print("ERROR: KB file not created!")
if __name__ == "__main__":
    test_persistent_learning()
`
```

visualize_accuracy_graph.py

```
`python
"""
Visualize move accuracy with graphs for both AI players.
Creates a visual comparison chart.
"""
import json
import sys
from pathlib import Path
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
def visualize_accuracy(json_file: str):
    """
    Display move-by-move accuracy graph for both players.
    Args:
        json_file: Path to analysis JSON file
    """
    with open(json_file, 'r') as f:
        data = json.load(f)
    player_analyses = data.get('player_analyses', {})
    if not player_analyses:
        print("No player analysis data found.")
        return
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10))
    fig.suptitle('AI Player Move Quality Analysis', fontsize=16, fontweight='bold')
    colors = ['#3498db', '#e74c3c']
```

```
max_moves = 0
for player_id_str, analysis in player_analyses.items():
    player_id = int(player_id_str)
    move_scores = analysis.get('move_scores', [])
    if move_scores:
        max_moves = max(max_moves, len(move_scores))
        moves = list(range(1, len(move_scores) + 1))
        ax1.plot(moves, move_scores,
                 color=colors[player_id],
                 linewidth=2,
                 marker='o',
                 markersize=4,
                 label=f'Player {player_id + 1}',
                 alpha=0.7)
ax1.set_xlabel('Move Number', fontsize=12)
ax1.set_ylabel('Move Quality Score (0-100)', fontsize=12)
ax1.set_title('Move-by-Move Quality Comparison', fontsize=14, fontweight='bold')
ax1.grid(True, alpha=0.3)
ax1.legend(loc='best', fontsize=11)
ax1.set_ylim(0, 110)
for player_id_str, analysis in player_analyses.items():
    player_id = int(player_id_str)
    move_scores = analysis.get('move_scores', [])
    if move_scores:
        avg_score = sum(move_scores) / len(move_scores)
        ax1.axhline(y=avg_score, color=colors[player_id],
                    linestyle='--', alpha=0.5, linewidth=1.5)
player_names = []
quality_scores = []
vp_percentages = []
buy_percentages = []
for player_id_str, analysis in player_analyses.items():
    player_id = int(player_id_str)
    player_names.append(f'Player {player_id + 1}')
    quality_scores.append(analysis.get('quality_score', 0))
    vp_percentages.append(analysis.get('vp_move_percent', 0))
    buy_percentages.append(analysis.get('buy_move_percent', 0))
x_pos = range(len(player_names))
bar_width = 0.25
bars1 = ax2.bar([p - bar_width for p in x_pos], quality_scores,
                bar_width, label='Overall Quality', color='#2ecc71', alpha=0.8)
bars2 = ax2.bar(x_pos, vp_percentages,
                bar_width, label='VP Moves %', color='#f39c12', alpha=0.8)
bars3 = ax2.bar([p + bar_width for p in x_pos], buy_percentages,
                bar_width, label='Buy Moves %', color='#9b59b6', alpha=0.8)
ax2.set_xlabel('Player', fontsize=12)
ax2.set_ylabel('Score / Percentage', fontsize=12)
ax2.set_title('Overall Performance Metrics', fontsize=14, fontweight='bold')
ax2.set_xticks(x_pos)
ax2.set_xticklabels(player_names)
ax2.legend(loc='best', fontsize=11)
ax2.grid(True, axis='y', alpha=0.3)
ax2.set_ylim(0, 110)
for bars in [bars1, bars2, bars3]:
    for bar in bars:
        height = bar.get_height()
        ax2.text(bar.get_x() + bar.get_width()/2., height,
                 f'{height:.1f}',
```

```
                ha='center', va='bottom', fontsize=9)
    summary_text = "TOTAL ACCURACY SCORES\n" + "="*30 + "\n"
    for player_id_str, analysis in player_analyses.items():
        player_id = int(player_id_str)
        quality = analysis.get('quality_score', 0)
        total_moves = analysis.get('total_moves', 0)
        vp_pct = analysis.get('vp_move_percent', 0)
        summary_text += f"\nPlayer {player_id + 1}:\n"
        summary_text += f"  Quality Score: {quality:.1f}/100\n"
        summary_text += f"  Total Moves: {total_moves}\n"
        summary_text += f"  VP Moves: {vp_pct:.1f}%\n"
    fig.text(0.02, 0.02, summary_text,
             fontsize=10,
             family='monospace',
             bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
    plt.tight_layout(rect=[0, 0.12, 1, 0.96])
    output_path = Path(json_file).parent / f"accuracy_graph_{Path(json_file).stem}.png"
    plt.savefig(output_path, dpi=150, bbox_inches='tight')
    print(f"\n[OK] Graph saved to: {output_path}")
    plt.show()
    print("\n" + "="*60)
    print("TOTAL ACCURACY SUMMARY")
    print("="*60)
    for player_id_str, analysis in player_analyses.items():
        player_id = int(player_id_str)
        print(f"\nPlayer {player_id + 1}:")
        print(f"  Overall Quality Score: {analysis.get('quality_score', 0):.1f}/100")
        print(f"  Total Moves: {analysis.get('total_moves', 0)}")
        print(f"  VP-Scoring Moves: {analysis.get('vp_move_percent', 0):.1f}%")
        print(f"  Card Purchases: {analysis.get('buy_move_percent', 0):.1f}%")
        print(f"  Avg Move Time: {analysis.get('avg_time_ms', 0):.1f} ms")
    print("\n" + "="*60)
if __name__ == "__main__":
    try:
        import matplotlib
        matplotlib.use('TkAgg')
    except:
        pass
    if len(sys.argv) > 1:
        visualize_accuracy(sys.argv[1])
    else:
        data_dir = Path(__file__).parent.parent / "data"
        json_files = list(data_dir.glob("analysis_*.json"))
        if json_files:
            latest = max(json_files, key=lambda p: p.stat().st_mtime)
            print(f'Loading most recent file: {latest.name}\n")
            visualize_accuracy(str(latest))
        else:
            print("No analysis files found in data/ folder.")
            print("\nUsage: python analytics/visualize_accuracy_graph.py [path/to/analysis.json]")
`
```

visualize_basic_ai.py

`python

```
"""
Visualize Basic AI accuracy analysis.
Run this after a game to see detailed move-by-move accuracy.
"""
import json
import sys
from pathlib import Path
def analyze_basic_ai(json_file: str):
    """
    Display Basic AI accuracy analysis from saved game data.
    Args:
        json_file: Path to accuracy JSON file
    """
    with open(json_file, 'r') as f:
        data = json.load(f)
    print("="*70)
    print("BASIC AI ACCURACY ANALYSIS")
    print("="*70)
    print(f"\nGame ID: {data['game_id']}")
    print(f"Player Analyzed: Player {data['basic_player_id'] + 1} (Basic AI)")
    print("\n" + "="*70)
    print("\n 📊 OVERALL STATISTICS")
    print("-"*70)
    print(f"  Total Moves Analyzed:     {data.get('total_moves_analyzed', 0)}")
    print(f"  Accuracy:                 {data['accuracy_percent']:.1f}%")
    print(f"  Total Point Loss:         {data['total_point_loss']:.2f}")
    print(f"  Avg Point Loss per Move:  {data['average_point_loss']:.2f}")
    print("-"*70)
    comparisons = data.get('detailed_comparisons', [])
    if comparisons:
        print("\n 📋 MOVE-BY-MOVE BREAKDOWN")
        print("-"*70)
        print(f"{'Turn':<6} {'Basic AI Action':<35} {'Match':<10}")
        print("-"*70)
        for comp in comparisons:
            turn = comp['turn']
            action = comp['basic_action'][:32] + '...' if len(comp['basic_action']) > 32 else
comp['basic_action']
            match = "[OK] YES" if comp['match'] else "✗ NO"
            print(f"{turn:<6} {action:<35} {match:<10}")
        print("-"*70)
    print("\n" + "="*70)
    print("Analysis Complete!")
    print("="*70)
if __name__ == "__main__":
    if len(sys.argv) > 1:
        analyze_basic_ai(sys.argv[1])
    else:
        data_dir = Path(__file__).parent.parent / "data"
        json_files = list(data_dir.glob("accuracy_*.json"))
        if json_files:
            latest = max(json_files, key=lambda p: p.stat().st_mtime)
            print(f"Loading most recent file: {latest.name}\n")
            analyze_basic_ai(str(latest))
```

```
    else:
        print("No accuracy files found in data/ folder.")
        print("\nUsage: python analytics/visualize_basic_ai.py [path/to/accuracy.json]")
`
```

visualize_dashboard.py

`python

```
"""
Example Dashboard Visualizations for Deep Dive Analytics
Use this script to create comparison graphs from game_log data.
"""
import json
import matplotlib.pyplot as plt
import numpy as np
from pathlib import Path
def load_game_data(json_path):
    """Load game data from JSON file."""
    with open(json_path, 'r') as f:
        return json.load(f)
def plot_win_probability_comparison(game_log,
output_path='data/win_probability_comparison.png'):
    """
    Plot win probability over time for both players.
    This is the KEY metric - now fair for both agents thanks to MCTS Critic!
    """
    p1_data = [(t['turn_number'], t['win_probability'])
               for t in game_log if t['active_player'] == 0 and t['win_probability'] is not None]
    p2_data = [(t['turn_number'], t['win_probability'])
               for t in game_log if t['active_player'] == 1 and t['win_probability'] is not None]
    if not p1_data or not p2_data:
        print("No win probability data available")
        return
    p1_turns, p1_probs = zip(*p1_data)
    p2_turns, p2_probs = zip(*p2_data)
    fig, ax = plt.subplots(figsize=(14, 7))
    ax.plot(p1_turns, p1_probs, 'o-', label='Player 1 (Heuristic)',
            color='#3498db', linewidth=2, markersize=6, alpha=0.8)
    ax.plot(p2_turns, p2_probs, 's-', label='Player 2 (MCTS)',
            color='#e74c3c', linewidth=2, markersize=6, alpha=0.8)
    ax.axhline(y=0.5, color='gray', linestyle='--', linewidth=1, alpha=0.5, label='Even (50%)')
    ax.set_xlabel('Turn Number', fontsize=14, fontweight='bold')
    ax.set_ylabel('Win Probability', fontsize=14, fontweight='bold')
    ax.set_title('Win Probability Comparison (MCTS Evaluated)', fontsize=16, fontweight='bold')
    ax.grid(True, alpha=0.3)
    ax.legend(loc='best', fontsize=12)
    ax.set_ylim(0, 1)
    p1_avg = np.mean(p1_probs)
    p2_avg = np.mean(p2_probs)
    stats_text = (
        f"Average Win Probability:\n"
        f"  Heuristic: {p1_avg:.2%}\n"
        f"  MCTS: {p2_avg:.2%}\n"
        f"  Difference: {abs(p2_avg - p1_avg):.2%}"
    )
    ax.text(0.02, 0.98, stats_text, transform=ax.transAxes,
```

```
        fontsize=11, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.7),
        family='monospace')
    plt.tight_layout()
    plt.savefig(output_path, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"[OK] Win probability comparison saved: {output_path}")
def plot_card_acquisition_strategy(game_log,
output_path='data/card_strategy_comparison.png'):
    """Plot card acquisition by tier over time."""
    p1_data = [(t['turn_number'], t['cards_owned_by_tier'])
            for t in game_log if t['active_player'] == 0]
    p2_data = [(t['turn_number'], t['cards_owned_by_tier'])
            for t in game_log if t['active_player'] == 1]
    if not p1_data or not p2_data:
        print("No card data available")
        return
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
    p1_turns = [t[0] for t in p1_data]
    p1_tier1 = [t[1][0] for t in p1_data]
    p1_tier2 = [t[1][1] for t in p1_data]
    p1_tier3 = [t[1][2] for t in p1_data]
    ax1.stackplot(p1_turns, p1_tier1, p1_tier2, p1_tier3,
                labels=['Tier 1', 'Tier 2', 'Tier 3'],
                colors=['#95a5a6', '#3498db', '#9b59b6'], alpha=0.8)
    ax1.set_xlabel('Turn Number', fontsize=12, fontweight='bold')
    ax1.set_ylabel('Card Count', fontsize=12, fontweight='bold')
    ax1.set_title('Player 1 (Heuristic) - Card Strategy', fontsize=14, fontweight='bold')
    ax1.legend(loc='upper left', fontsize=11)
    ax1.grid(True, alpha=0.3)
    p2_turns = [t[0] for t in p2_data]
    p2_tier1 = [t[1][0] for t in p2_data]
    p2_tier2 = [t[1][1] for t in p2_data]
    p2_tier3 = [t[1][2] for t in p2_data]
    ax2.stackplot(p2_turns, p2_tier1, p2_tier2, p2_tier3,
                labels=['Tier 1', 'Tier 2', 'Tier 3'],
                colors=['#95a5a6', '#e74c3c', '#e67e22'], alpha=0.8)
    ax2.set_xlabel('Turn Number', fontsize=12, fontweight='bold')
    ax2.set_ylabel('Card Count', fontsize=12, fontweight='bold')
    ax2.set_title('Player 2 (MCTS) - Card Strategy', fontsize=14, fontweight='bold')
    ax2.legend(loc='upper left', fontsize=11)
    ax2.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(output_path, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"[OK] Card strategy comparison saved: {output_path}")
def plot_mcts_confidence(game_log, output_path='data/mcts_confidence.png'):
    """Plot MCTS decision confidence over time."""
    mcts_data = [(t['turn_number'], t['visit_distribution'])
                for t in game_log
                if t['player_type'] == 'MCTS' and t['visit_distribution']]
    if not mcts_data:
        print("No MCTS visit distribution data available")
        return
    turns = []
    confidences = []
    for turn_num, visit_dist in mcts_data:
        if len(visit_dist) >= 2:
```

```
            total_visits = sum(m['visits'] for m in visit_dist)
            top_visits = visit_dist[0]['visits']
            confidence = (top_visits / total_visits) * 100
            turns.append(turn_num)
            confidences.append(confidence)
    if not turns:
        print("Insufficient visit distribution data")
        return
    fig, ax = plt.subplots(figsize=(14, 7))
    ax.plot(turns, confidences, 'o-', color='#e74c3c',
            linewidth=2, markersize=8, alpha=0.8)
    ax.axhline(y=50, color='orange', linestyle='--',
              linewidth=2, alpha=0.7, label='Low Confidence (50%)')
    ax.axhline(y=70, color='green', linestyle='--',
              linewidth=2, alpha=0.7, label='High Confidence (70%)')
    ax.set_xlabel('Turn Number', fontsize=14, fontweight='bold')
    ax.set_ylabel('Confidence (%)', fontsize=14, fontweight='bold')
    ax.set_title('MCTS Decision Confidence (Top Move Visit %)', fontsize=16,
fontweight='bold')
    ax.grid(True, alpha=0.3)
    ax.legend(loc='best', fontsize=12)
    ax.set_ylim(0, 100)
    avg_confidence = np.mean(confidences)
    stats_text = (
        f"Average Confidence: {avg_confidence:.1f}%\n"
        f"Min: {min(confidences):.1f}%\n"
        f"Max: {max(confidences):.1f}%"
    )
    ax.text(0.02, 0.98, stats_text, transform=ax.transAxes,
            fontsize=11, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.7),
            family='monospace')
    plt.tight_layout()
    plt.savefig(output_path, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"[OK] MCTS confidence plot saved: {output_path}")
def plot_performance_metrics(game_log, output_path='data/performance_comparison.png'):
    """Plot simulation count and decision time comparison."""
    p1_data = [(t['turn_number'], t['simulation_count'], t['duration_ms'])
               for t in game_log if t['active_player'] == 0]
    p2_data = [(t['turn_number'], t['simulation_count'], t['duration_ms'])
               for t in game_log if t['active_player'] == 1]
    if not p1_data or not p2_data:
        print("No performance data available")
        return
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10))
    p1_turns = [t[0] for t in p1_data]
    p1_sims = [t[1] for t in p1_data]
    p2_turns = [t[0] for t in p2_data]
    p2_sims = [t[1] for t in p2_data]
    ax1.plot(p1_turns, p1_sims, 'o-', label='Player 1 (Heuristic + Critic)',
            color='#3498db', linewidth=2, markersize=6, alpha=0.8)
    ax1.plot(p2_turns, p2_sims, 's-', label='Player 2 (MCTS Full)',
            color='#e74c3c', linewidth=2, markersize=6, alpha=0.8)
    ax1.set_xlabel('Turn Number', fontsize=12, fontweight='bold')
    ax1.set_ylabel('Simulation Count', fontsize=12, fontweight='bold')
    ax1.set_title('MCTS Simulations per Turn', fontsize=14, fontweight='bold')
    ax1.legend(loc='best', fontsize=11)
```

```
    ax1.grid(True, alpha=0.3)
    p1_times = [t[2] for t in p1_data]
    p2_times = [t[2] for t in p2_data]
    ax2.plot(p1_turns, p1_times, 'o-', label='Player 1 (Heuristic)',
             color='#3498db', linewidth=2, markersize=6, alpha=0.8)
    ax2.plot(p2_turns, p2_times, 's-', label='Player 2 (MCTS)',
             color='#e74c3c', linewidth=2, markersize=6, alpha=0.8)
    ax2.set_xlabel('Turn Number', fontsize=12, fontweight='bold')
    ax2.set_ylabel('Decision Time (ms)', fontsize=12, fontweight='bold')
    ax2.set_title('Decision Time per Turn', fontsize=14, fontweight='bold')
    ax2.legend(loc='best', fontsize=11)
    ax2.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(output_path, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"[OK] Performance comparison saved: {output_path}")
def generate_all_dashboards(json_path):
    """Generate all dashboard visualizations from a game log."""
    print("\n" + "=" * 60)
    print("DEEP DIVE ANALYTICS - DASHBOARD GENERATOR")
    print("=" * 60 + "\n")
    print(f"Loading game data from: {json_path}")
    data = load_game_data(json_path)
    game_log = data.get('game_log', [])
    if not game_log:
        print("✗ No game_log found in JSON file!")
        print("  Make sure you're using a game played after implementing Deep Dive Analytics.")
        return
    print(f"[OK] Loaded {len(game_log)} turn entries\n")
    print("Generating visualizations...\n")
    plot_win_probability_comparison(game_log)
    plot_card_acquisition_strategy(game_log)
    plot_mcts_confidence(game_log)
    plot_performance_metrics(game_log)
    print("\n" + "=" * 60)
    print("ALL DASHBOARDS GENERATED!")
    print("=" * 60)
    print("\nCheck the 'data/' folder for output images:")
    print("  - win_probability_comparison.png")
    print("  - card_strategy_comparison.png")
    print("  - mcts_confidence.png")
    print("  - performance_comparison.png")
    print("\n" + "=" * 60 + "\n")
if __name__ == "__main__":
    import sys
    log_files = sorted(Path('logs').glob('game_*.json'), reverse=True)
    if len(sys.argv) > 1:
        json_path = sys.argv[1]
    elif log_files:
        json_path = str(log_files[0])
        print(f"Using most recent game log: {json_path}")
    else:
        print("No game logs found!")
        print("\nUsage:")
        print("  python visualize_dashboard.py [path/to/game_log.json]")
        print("\nOr run a game first:")
        print("  python main.py")
```

```
        sys.exit(1)
    generate_all_dashboards(json_path)
`
```

visualize_mcts_ai.py

```
`python
"""
Visualize MCTS AI performance analysis.
Run this after a game to see detailed move statistics.
"""
import json
import sys
from pathlib import Path
def analyze_mcts_ai(json_file: str):
    """
    Display MCTS AI performance analysis from saved game data.
    Args:
        json_file: Path to game log JSON file
    """
    with open(json_file, 'r') as f:
        data = json.load(f)
    metadata = data['metadata']
    moves = data['moves']
    print("="*70)
    print("MCTS AI PERFORMANCE ANALYSIS")
    print("="*70)
    print(f"\nGame ID: {metadata['game_id']}")
    print(f"Winner: Player {metadata['winner'] + 1}")
    print(f"Total Turns: {metadata['total_turns']}")
    print(f"Game Duration: {metadata.get('total_duration_seconds', 0):.1f} seconds")
    print("\n" + "="*70)
    mcts_moves = [m for m in moves if m['player_type'] == 'MCTS']
    basic_moves = [m for m in moves if m['player_type'] == 'Basic']
    if mcts_moves:
        print("\n🤖 MCTS AI STATISTICS")
        print("-"*70)
        print(f"  Total Moves:              {len(mcts_moves)}")
        total_time = sum(m['duration_ms'] for m in mcts_moves)
        avg_time = total_time / len(mcts_moves) if mcts_moves else 0
        print(f"  Avg Computation Time:     {avg_time:.0f} ms")
        print(f"  Total Thinking Time:      {total_time/1000:.1f} seconds")
        action_types = {}
        for move in mcts_moves:
            action_type = move['action_type']
            action_types[action_type] = action_types.get(action_type, 0) + 1
        print(f"\n  Action Breakdown:")
        for action, count in sorted(action_types.items()):
            pct = (count / len(mcts_moves) * 100)
            print(f"    - {action:<20} {count:>3} moves ({pct:>5.1f}%)")
        print("-"*70)
    if basic_moves:
        print("\n🎯 BASIC AI STATISTICS (Comparison)")
        print("-"*70)
        print(f"  Total Moves:              {len(basic_moves)}")
        total_time = sum(m['duration_ms'] for m in basic_moves)
```

```
        avg_time = total_time / len(basic_moves) if basic_moves else 0
        print(f"  Avg Computation Time:     {avg_time:.1f} ms")
        print(f"  Total Thinking Time:      {total_time/1000:.2f} seconds")
        action_types = {}
        for move in basic_moves:
            action_type = move['action_type']
            action_types[action_type] = action_types.get(action_type, 0) + 1
        print(f"\n  Action Breakdown:")
        for action, count in sorted(action_types.items()):
            pct = (count / len(basic_moves) * 100)
            print(f"    - {action:<20} {count:>3} moves ({pct:>5.1f}%)")
        print("-"*70)
    print("\n📋 MOVE TIMELINE")
    print("-"*70)
    print(f"{'Turn':<6} {'Player':<10} {'Action':<25} {'Time (ms)':<12}")
    print("-"*70)
    for move in moves[:20]:
        turn = move['turn_id']
        player = f"P{move['player_id'] + 1} ({move['player_type']})"
        action = move['action_type'][:22] + '...' if len(move['action_type']) > 22 else
move['action_type']
        time_ms = f"{move['duration_ms']:.0f}"
        print(f"{turn:<6} {player:<10} {action:<25} {time_ms:<12}")
    if len(moves) > 20:
        print(f"  ... ({len(moves) - 20} more moves)")
    print("-"*70)
    print("\n⏳ FINAL SCORES")
    print("-"*70)
    for player_id, score in metadata['final_scores'].items():
        player_num = int(player_id) + 1
        winner_mark = " 👑 WINNER" if int(player_id) == metadata['winner'] else ""
        print(f"  Player {player_num}: {score} points{winner_mark}")
    print("-"*70)
    print("\n" + "="*70)
    print("Analysis Complete!")
    print("="*70)
if __name__ == "__main__":
    if len(sys.argv) > 1:
        analyze_mcts_ai(sys.argv[1])
    else:
        logs_dir = Path(__file__).parent.parent / "logs"
        json_files = list(logs_dir.glob("game_*.json"))
        if json_files:
            latest = max(json_files, key=lambda p: p.stat().st_mtime)
            print(f"Loading most recent file: {latest.name}\n")
            analyze_mcts_ai(str(latest))
        else:
            print("No game log files found in logs/ folder.")
            print("\nUsage: python analytics/visualize_mcts_ai.py [path/to/game.json]")
`
```

visualize_tree.py

`python

```
"""
Vertical Tree Visualization for MCTS Agent
Generates a compact, easy-to-explain hierarchical tree visualization.
"""
import time
import os
from datetime import datetime
from pathlib import Path
from pyvis.network import Network
from game_engine.state import GameState
from agents.advanced import MCTSAgent, MCTSNode
def run_visualization():
    """
    Run MCTS agent and generate vertical tree visualization.
    """
    print("=" * 60)
    print("MCTS Vertical Tree Visualization")
    print("=" * 60)
    print("\n1. Initializing game state...")
    game_state = GameState(num_players=2)
    print("2. Creating MCTS agent...")
    mcts_agent = MCTSAgent(player_id=0, time_limit=3.0)
    print("3. Running MCTS for 3 seconds...")
    root = MCTSNode(game_state.get_state_copy())
    start_time = time.time()
    simulations = 0
    while time.time() - start_time < 3.0:
        node = mcts_agent._select(root, mcts_agent.player_id)
        result = mcts_agent._simulate(node.state)
        node.backpropagate(result)
        simulations += 1
    elapsed = time.time() - start_time
    print(f"   [OK] Completed {simulations} simulations in {elapsed:.2f}s")
    print("4. Exporting tree structure...")
    tree_data = mcts_agent.export_tree_structure(
        root_node=root,
        max_depth=3,
        top_k=3
    )
    print(f"   [OK] Nodes: {len(tree_data['nodes'])}")
    print(f"   [OK] Edges: {len(tree_data['edges'])}")
    print("5. Creating visualization...")
    net = Network(
        height='900px',
        width='100%',
        bgcolor='#222222',
        font_color='white',
        directed=True
    )
    net.set_options("""
    var options = {
      "layout": {
        "hierarchical": {
          "enabled": true,
          "direction": "UD",
```

```
          "sortMethod": "directed",
          "levelSeparation": 250,
          "nodeSpacing": 10,
          "treeSpacing": 10,
          "blockShifting": false,
          "edgeMinimization": false,
          "parentCentralization": true
        }
      },
      "physics": {
        "enabled": false,
        "hierarchicalRepulsion": {
          "nodeDistance": 0,
          "centralGravity": 0.0,
          "springLength": 0,
          "damping": 1
        }
      },
      "edges": {
        "smooth": {
          "type": "cubicBezier",
          "forceDirection": "vertical",
          "roundness": 0.5
        },
        "color": {
          "color": "#848484",
          "highlight": "#848484"
        }
      }
    }
    """)
    for node in tree_data['nodes']:
        net.add_node(
            node['id'],
            label=node['label'],
            color=node['color'],
            size=node['size'],
            title=f"Visits: {node['visits']}<br>Win Rate: {node['win_rate']*100:.1f}%"
        )
    for edge in tree_data['edges']:
        net.add_edge(edge['from'], edge['to'])
    print("6. Saving visualization...")
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    data_dir = Path("data")
    data_dir.mkdir(exist_ok=True)
    game_folder = data_dir / f"game_{timestamp}"
    game_folder.mkdir(exist_ok=True)
    html_filename = f"mcts_tree_vertical.html"
    html_filepath = game_folder / html_filename
    net.save_graph(str(html_filepath))
    print(f"\n[OK] Visualization saved to: {html_filepath}")
    print("\n" + "=" * 60)
    print("DONE!")
    print("=" * 60)
    print(f"\nOpen the file in your browser to view the interactive tree:")
    print(f"  {html_filepath.absolute()}")
    return str(html_filepath)
if __name__ == "__main__":
```

```
    run_visualization()
`
```

window.py

```
`python
"""
Main game window with pygame.
"""
import pygame
import sys
from typing import Optional
from game_engine.state import GameState
from game_engine.components import GemType
from gui.renderer import (
    draw_card, draw_noble, draw_gem_bank, draw_player_area,
    BG_COLOR, TEXT_COLOR
)
from gui.logger_widget import LoggerWidget
class GameWindow:
    """
    Main pygame window for Splendor visualization.
    Layout:
    - Left side: Game board (gems, cards, nobles)
    - Right side: Scrollable move log
    - Bottom: Player areas
    - Controls: Start, Next Turn, Auto-Play toggle, Speed slider
    """
    def __init__(self, width: int = 1400, height: int = 900):
        """
        Initialize the game window.
        Args:
            width, height: Window dimensions
        """
        pygame.init()
        self.width = width
        self.height = height
        self.screen = pygame.display.set_mode((width, height))
        pygame.display.set_caption("Splendor - AI Battle")
        self.clock = pygame.time.Clock()
        self.running = True
        self.game_state: Optional[GameState] = None
        self.logger = LoggerWidget(950, 20, 430, 400)
        self.auto_play = False
        self.game_speed = 1.0
        self.paused = False
        self.font_large = pygame.font.Font(None, 32)
        self.font_medium = pygame.font.Font(None, 24)
        self.font_small = pygame.font.Font(None, 18)
        self.btn_start = (50, self.height - 60, 120, 40)
        self.btn_next = (190, self.height - 60, 120, 40)
        self.btn_auto = (330, self.height - 60, 120, 40)
        self.slider_area = (500, self.height - 50, 300, 20)
    def start_game(self, game_state: GameState):
        """
        Start a new game.
        Args:
```

```
        game_state: Initial game state
    """
    self.game_state = game_state
    self.logger.clear()
    self.logger.add_log("Game started!")
    self.logger.add_log(f"{game_state.num_players} players")
def log_move(self, player_id: int, action_desc: str):
    """
    Log a move to the widget.
    Args:
        player_id: Player who made the move
        action_desc: Description of the action
    """
    self.logger.add_log(f"P{player_id + 1}: {action_desc}")
def draw_board(self):
    """Draw the game board (cards, gems, nobles)."""
    if not self.game_state:
        return
    noble_x = 50
    noble_y = 50
    self.font_medium.render("Nobles:", True, TEXT_COLOR)
    for noble in self.game_state.nobles:
        draw_noble(self.screen, noble, noble_x, noble_y)
        noble_x += 85
    tier_y = 150
    tier_label = self.font_medium.render("Tier 3", True, TEXT_COLOR)
    self.screen.blit(tier_label, (50, tier_y))
    card_x = 150
    for card in self.game_state.visible_tier3:
        draw_card(self.screen, card, card_x, tier_y)
        card_x += 95
    tier_y = 270
    tier_label = self.font_medium.render("Tier 2", True, TEXT_COLOR)
    self.screen.blit(tier_label, (50, tier_y))
    card_x = 150
    for card in self.game_state.visible_tier2:
        draw_card(self.screen, card, card_x, tier_y)
        card_x += 95
    tier_y = 390
    tier_label = self.font_medium.render("Tier 1", True, TEXT_COLOR)
    self.screen.blit(tier_label, (50, tier_y))
    card_x = 150
    for card in self.game_state.visible_tier1:
        draw_card(self.screen, card, card_x, tier_y)
        card_x += 95
    draw_gem_bank(self.screen, self.game_state.gem_bank, 540, 150)
def draw_players(self):
    """Draw player areas."""
    if not self.game_state:
        return
    player_area_y = 520
    player_area_height = 140
    for i in range(self.game_state.num_players):
        is_current = (i == self.game_state.current_player)
        draw_player_area(
            self.screen,
            player_id=i,
            gems=dict(self.game_state.player_gems[i]),
```

```
                bonuses=dict(self.game_state.player_bonuses[i]),
                points=self.game_state.player_points[i],
                cards=self.game_state.player_cards[i],
                nobles=self.game_state.player_nobles[i],
                x=50,
                y=player_area_y + i * (player_area_height + 10),
                width=880,
                height=player_area_height,
                is_current=is_current
            )
    def draw_controls(self):
        """Draw control buttons and speed slider."""
        btn_color = (100, 200, 100) if self.game_state is None else (150, 150, 150)
        pygame.draw.rect(self.screen, btn_color, self.btn_start, border_radius=5)
        pygame.draw.rect(self.screen, TEXT_COLOR, self.btn_start, 2, border_radius=5)
        start_text = self.font_small.render("Start Game", True, TEXT_COLOR)
        self.screen.blit(start_text, (self.btn_start[0] + 15, self.btn_start[1] + 12))
        btn_color = (100, 150, 200) if self.game_state and not self.auto_play else (150, 150, 150)
        pygame.draw.rect(self.screen, btn_color, self.btn_next, border_radius=5)
        pygame.draw.rect(self.screen, TEXT_COLOR, self.btn_next, 2, border_radius=5)
        next_text = self.font_small.render("Next Turn", True, TEXT_COLOR)
        self.screen.blit(next_text, (self.btn_next[0] + 20, self.btn_next[1] + 12))
        btn_color = (200, 100, 100) if self.auto_play else (100, 100, 200)
        pygame.draw.rect(self.screen, btn_color, self.btn_auto, border_radius=5)
        pygame.draw.rect(self.screen, TEXT_COLOR, self.btn_auto, 2, border_radius=5)
        auto_text = self.font_small.render("Auto: ON" if self.auto_play else "Auto: OFF",
                                           True, TEXT_COLOR)
        self.screen.blit(auto_text, (self.btn_auto[0] + 25, self.btn_auto[1] + 12))
        slider_x, slider_y, slider_w, slider_h = self.slider_area
        pygame.draw.rect(self.screen, (200, 200, 200), self.slider_area, border_radius=3)
        pygame.draw.rect(self.screen, TEXT_COLOR, self.slider_area, 2, border_radius=3)
        handle_pos = slider_x + int((self.game_speed / 3.0) * slider_w)
        pygame.draw.circle(self.screen, (100, 100, 200), (handle_pos, slider_y + slider_h // 2), 10)
        speed_label = self.font_small.render(f"Speed: {self.game_speed:.1f}s", True,
TEXT_COLOR)
        self.screen.blit(speed_label, (slider_x + slider_w + 20, slider_y))
    def draw(self):
        """Draw the entire window."""
        self.screen.fill(BG_COLOR)
        if self.game_state:
            self.draw_board()
            self.draw_players()
        else:
            welcome = self.font_large.render("Splendor AI Battle", True, TEXT_COLOR)
            welcome_rect = welcome.get_rect(center=(self.width // 2, self.height // 2 - 50))
            self.screen.blit(welcome, welcome_rect)
            instruction = self.font_medium.render("Click 'Start Game' to begin", True,
TEXT_COLOR)
            inst_rect = instruction.get_rect(center=(self.width // 2, self.height // 2 + 10))
            self.screen.blit(instruction, inst_rect)
        self.logger.draw(self.screen)
        self.draw_controls()
        pygame.display.flip()
    def handle_click(self, pos):
        """
        Handle mouse click events.
        Args:
            pos: Mouse position (x, y)
```

```
        Returns:
            Event type string or None
        """
        x, y = pos
        if self._point_in_rect(pos, self.btn_start):
            return "start_game"
        elif self._point_in_rect(pos, self.btn_next):
            if self.game_state and not self.auto_play:
                return "next_turn"
        elif self._point_in_rect(pos, self.btn_auto):
            self.auto_play = not self.auto_play
            return "toggle_auto"
        elif self._point_in_rect(pos, self.slider_area):
            slider_x = self.slider_area[0]
            slider_w = self.slider_area[2]
            ratio = (x - slider_x) / slider_w
            self.game_speed = max(0.1, min(3.0, ratio * 3.0))
            return "adjust_speed"
        return None
    def _point_in_rect(self, point, rect):
        """Check if point is inside rectangle."""
        x, y = point
        rx, ry, rw, rh = rect
        return rx <= x <= rx + rw and ry <= y <= ry + rh
    def handle_events(self):
        """
        Handle pygame events.
        Returns:
            Event type string or None
        """
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                self.running = False
                return "quit"
            elif event.type == pygame.MOUSEBUTTONDOWN:
                if event.button == 1:
                    return self.handle_click(event.pos)
                elif event.button == 4:
                    self.logger.scroll_up()
                elif event.button == 5:
                    self.logger.scroll_down()
        return None
    def update(self):
        """Update window (called each frame)."""
        self.clock.tick(60)
    def quit(self):
        """Clean up and quit."""
        pygame.quit()
        sys.exit()
```